

基于 Vivado 的 MicroBlaze 基础教程

版本号：V1.0.1
2020-09-26 16:43:07



版权声明:

Copyright ©2012-2018 芯驿电子科技（上海）有限公司

公司网址:

[Http://www.alinx.com.cn](http://www.alinx.com.cn)

技术论坛：

<http://www.heijin.org>

官方旗舰店：

<http://alinx.jd.com>

邮箱:

avic@alinx.com.cn

电话:

021-67676997

传真：

021-37737073

ALINX 微信公众号：



文档修订记录:

版本	时间	描述
1.0.1	2018/11/6	初始版本

我们承诺本教程并非一劳永逸，固守不变的文档。我们会根据大家的反馈意见，以及实际的开发实践经验积累不断的修正和优化教程。

前言

首先感谢大家购买芯驿电子科技（上海）有限公司出品的开发板！您对我们和我们产品的支持和信任,给我们增添了勇往直前的信心和勇气。

“播下一粒种子，收获一片森林”，更是芯驿电子科技（上海）有限公司的美好愿望，同时我们会在黑金动力社区 <http://www.heijin.org> 和大家一起讨论，一起学习，一起进步，一起成长。

目录

前言	4
目录	5
准备工程及注意事项	7
软件环境	7
第一章 简单 MicroBlaze 系统搭建	8
1.1 建立 VIVADO 工程	8
1.1.1 新建工程	8
1.1.2 创建 Block 设计	11
1.1.3 添加软核	12
1.1.4 添加时钟	13
1.1.5 添加外设	17
1.1.6 修改端口名称	22
1.1.7 保存并检查错误	23
1.1.8 分配 IO	26
1.1.9 开始编译	30
1.1.10 bitstream 设置	32
1.1.11 导出硬件	39
1.2 SDK 开发“HelloWorld”	41
1.3 烧写程序到 Flash	51
第二章 SDK 编程开发	54
2.1 GPIO 控制点亮 LED	54
2.2 SDK 软件开发技巧	58
2.2.1 获取帮助文档	58
2.2.2 查看 API 函数原型	59
2.2.3 堆栈大小设置	60
2.2.4 优化等级和调试等级设置	61
2.2.5 查找 IP 核寄存器设计说明	64
第三章 带 mig(DDR3)的 MicroBlaze 系统	66
3.1 建立 VIVADO 工程	66
3.1.1 新建工程	66
3.1.2 创建 Block 设计	69
3.1.3 添加软核	70
3.1.4 添加 mig 核	71
3.1.5 添加外设	90

3.1.6 保存并检查错误	101
3.1.7 添加 xdc 约束文件	103
3.1.8 生成 Bitstream 文件	106
3.1.9 导出硬件	107
3.2 SDK 开发“HelloWorld”	109
3.3 烧写程序到 Flash	117
3.3.1 BootLoader 工程	117
3.3.2 修改 bootloader bsp	121
3.3.3 烧写 Bitstream 和 BootLoader	123
3.3.4 烧写应用程序	124

准备工程及注意事项

软件环境

软件开发环境基于 Vivado 2017.4



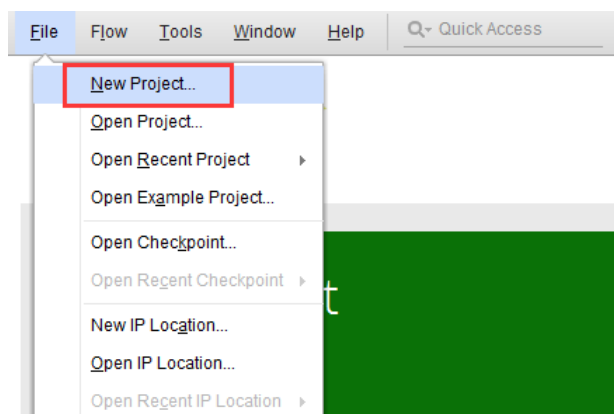
第一章 简单 MicroBlaze 系统搭建

本章如何使用 Vivado 软件建立一个 MicroBlaze 最小系统，使用 FPGA 内部 Block RAM 作为 CPU 运行 RAM，主要外设设有 UART 和 GPIO。

1.1 建立 VIVADO 工程

1.1.1 新建工程

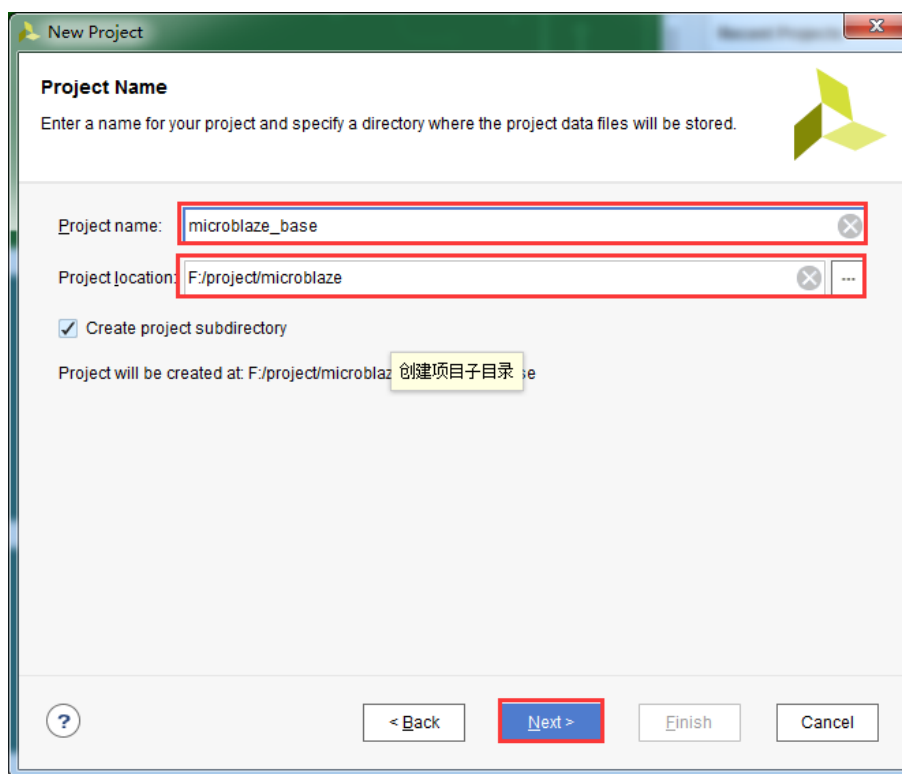
1. 打开 Vivado 软件，新建工程



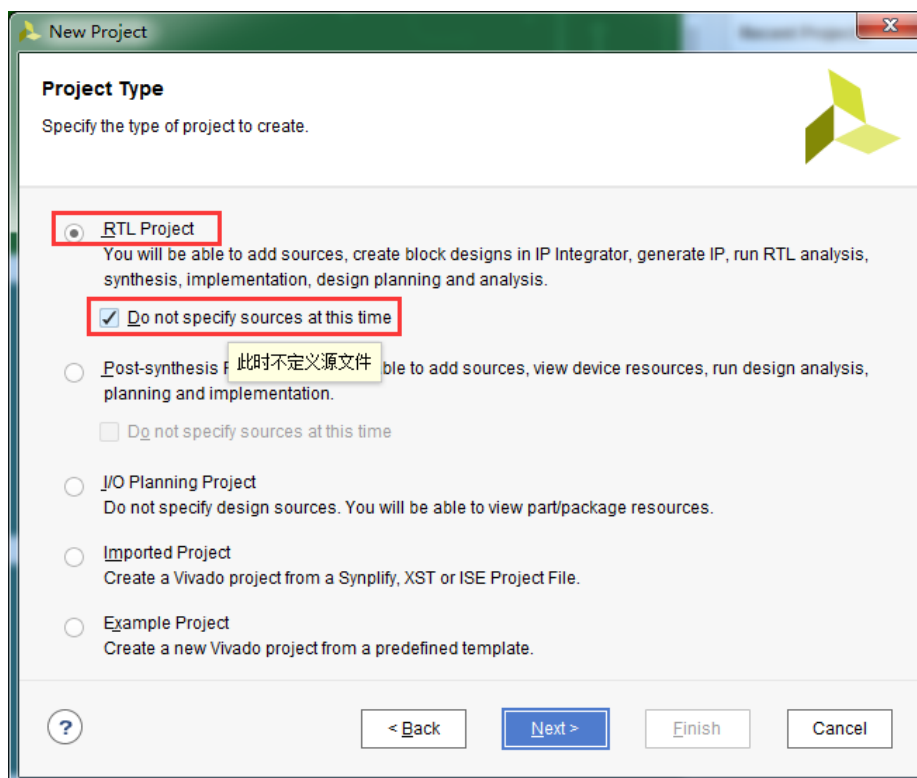
2. 点击“Next”创建一个新的 Vivado 工程



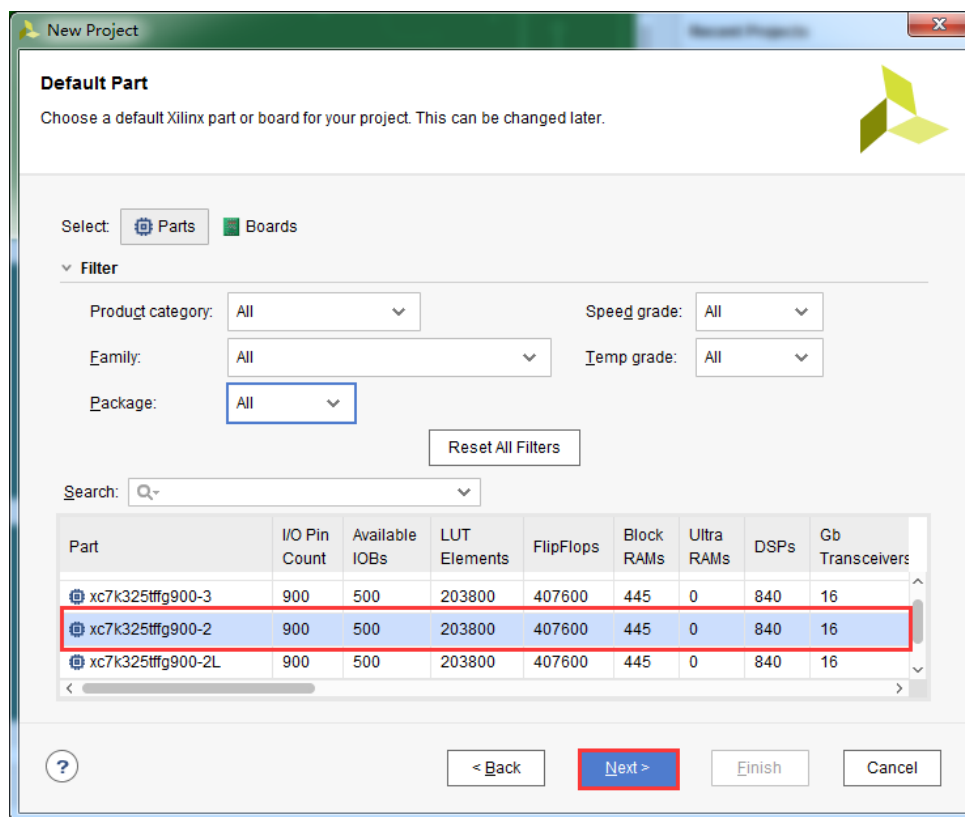
3. 工程名填写 “microblaze_base”，选择工程路径，并保持默认创建项目子目录选项，工程名和工程路径不要有中文空格，工程路径要尽可能短，否则可能会出现编译错误。点击 “Next”



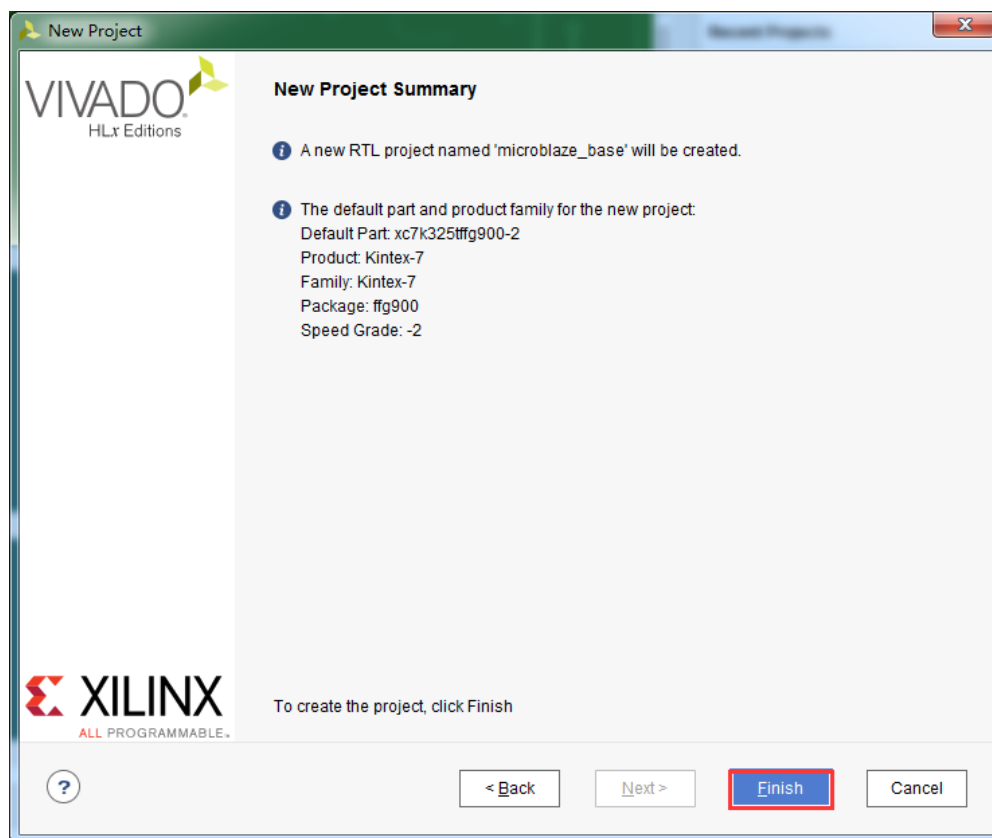
4. 选择创建一个 RTL 工程，并勾选 “Do not specify sources at this time”



5. 根据自己开发板选择 FPGA 型号 (这里以 ax7325 开发板为例, 如果是其它开发板, 需要选择对应的 FPGA 芯片), 点击“Next”

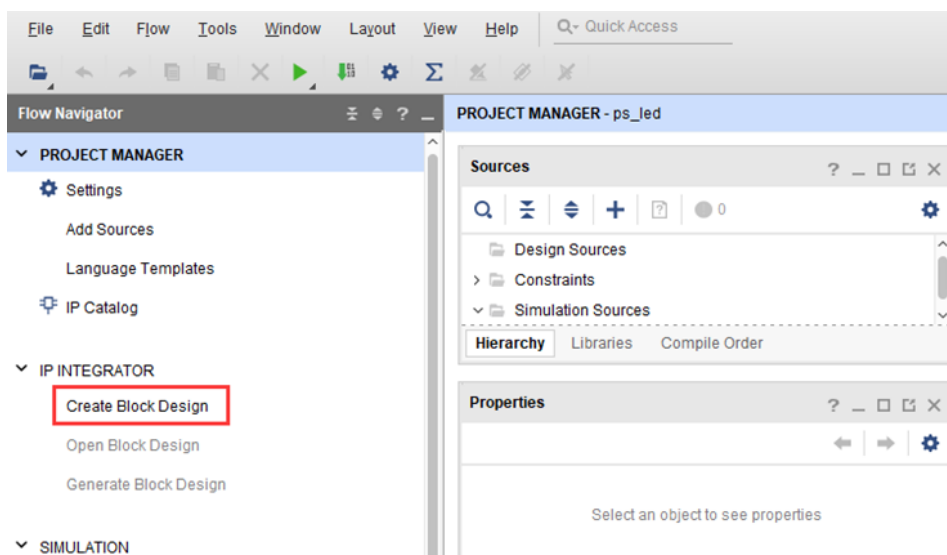


6. 点击“Finish”完成完成工程创建向导

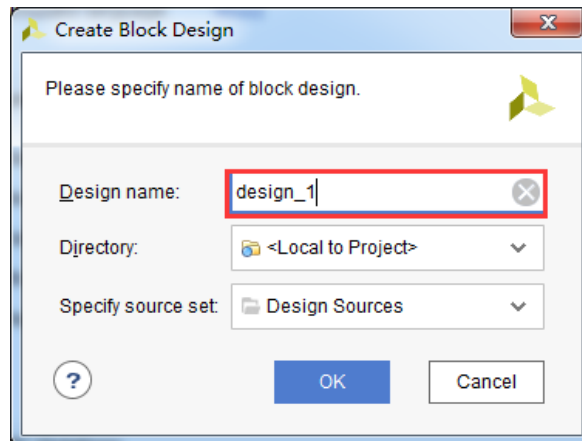


1.1.2 创建 Block 设计

7. 点击 Create Block Design

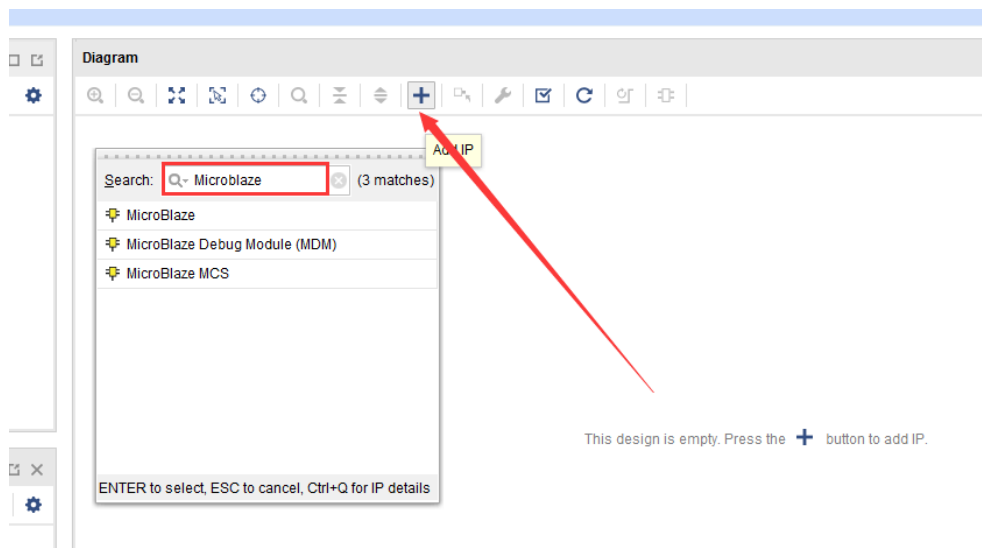


8. 保持设计名称为默认值，点击“OK”

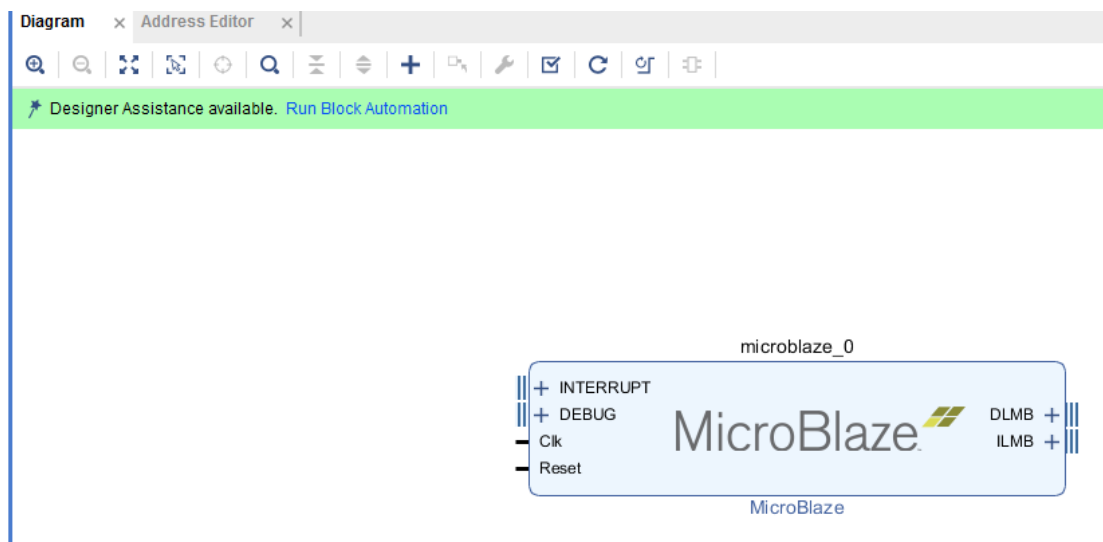


1.1.3 添加软核

9. 点击添加 IP 快捷按钮，搜索 “Microblaze”，双击下拉列表中 “Microblaze”

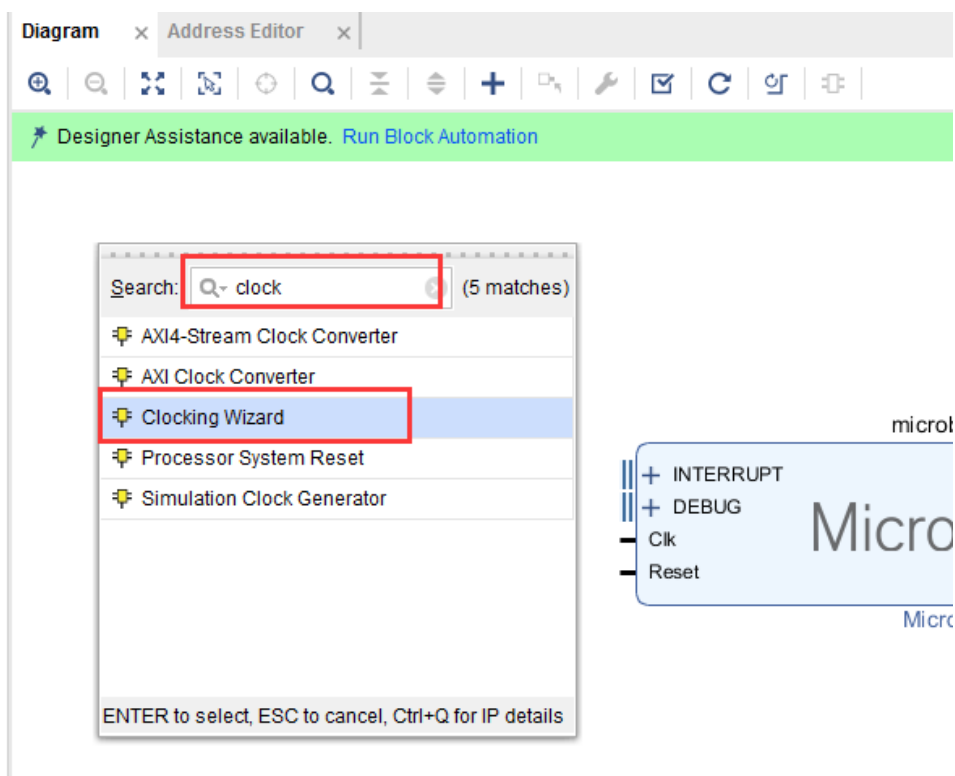


10. 一个 Microblaze 核添加到 Block 设计中



1.1.4 添加时钟

11. 搜索“clock”，双击“Clocking Wizard”添加一个时钟向导 IP 核（PLL）

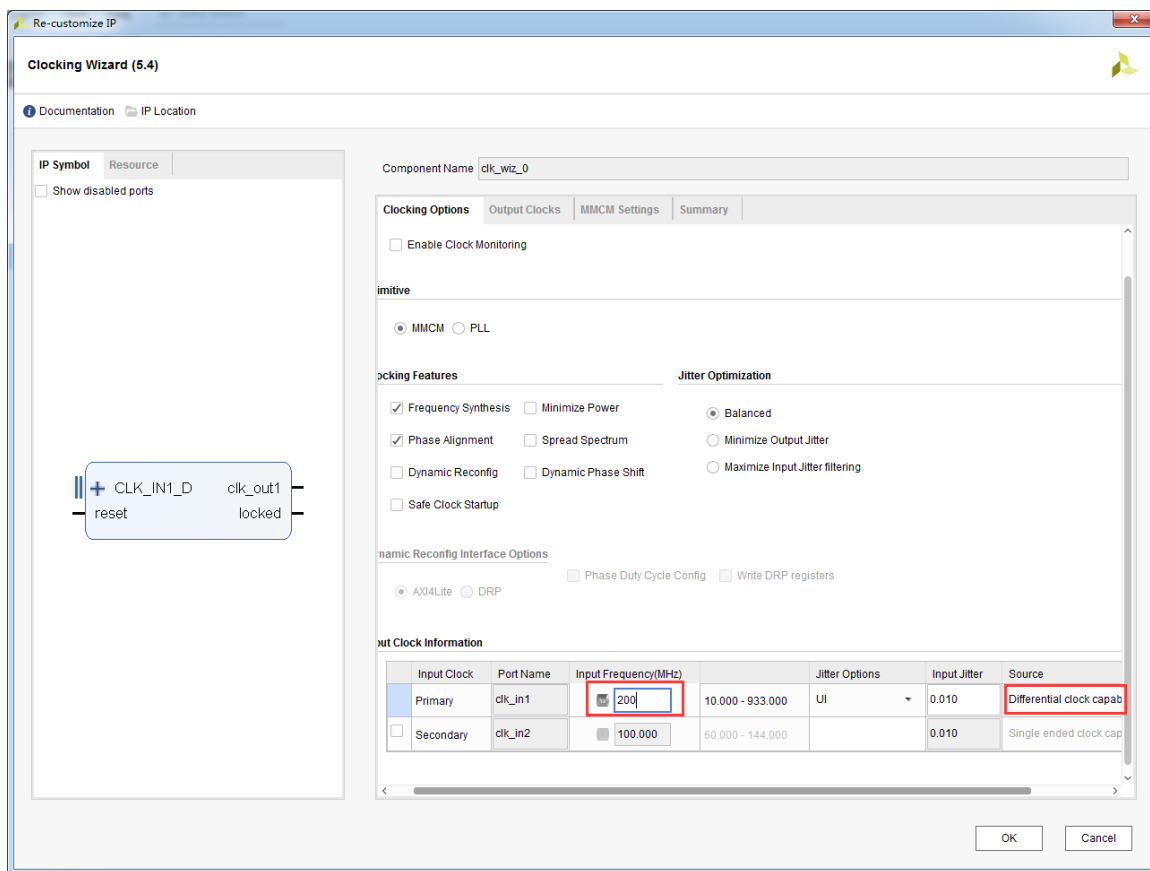


12. 双击刚添加的时钟向导 IP 核，进行参数设置



13. 在“Clocking Options”选项卡中，clk_in1 频率填写 200，Source 选择差分时钟（以 AX7325

为列)，这里的设置要根据开发板具体情况。



*备注：如果是 AX7035 的开发板，时钟输入是 50Mhz，单端输入。

14. clk_out1 作为 Microblaze 运行时钟，这里保持 100Mh 不修改，复位类型选择“Active Low”，低电平复位，这是因为开发板的按键按下时是低电平。

Clocking Options		Output Clocks	MMCM Settings	Summary				
☒	clk_out1	clk_out1	100.000	100.000	0.000	0.000	50.000	50
☐	clk_out2	clk_out2	100.000	N/A	0.000	N/A	50.000	N/A
☐	clk_out3	clk_out3	100.000	N/A	0.000	N/A	50.000	N/A
☐	clk_out4	clk_out4	100.000	N/A	0.000	N/A	50.000	N/A
☐	clk_out5	clk_out5	100.000	N/A	0.000	N/A	50.000	N/A
☐	clk_out6	clk_out6	100.000	N/A	0.000	N/A	50.000	N/A
☐	clk_out7	clk_out7	100.000	N/A	0.000	N/A	50.000	N/A

☐ USE CLOCK SEQUENCING

Clocking Feedback

Output Clock	Sequence Number
clk_out1	1
clk_out2	1
clk_out3	1
clk_out4	1
clk_out5	1
clk_out6	1
clk_out7	1

Source

☒ Automatic Control On-Chip
☐ Automatic Control Off-Chip
☐ User-Controlled On-Chip
☐ User-Controlled Off-Chip

Signaling

☒ Single-ended
☐ Differential

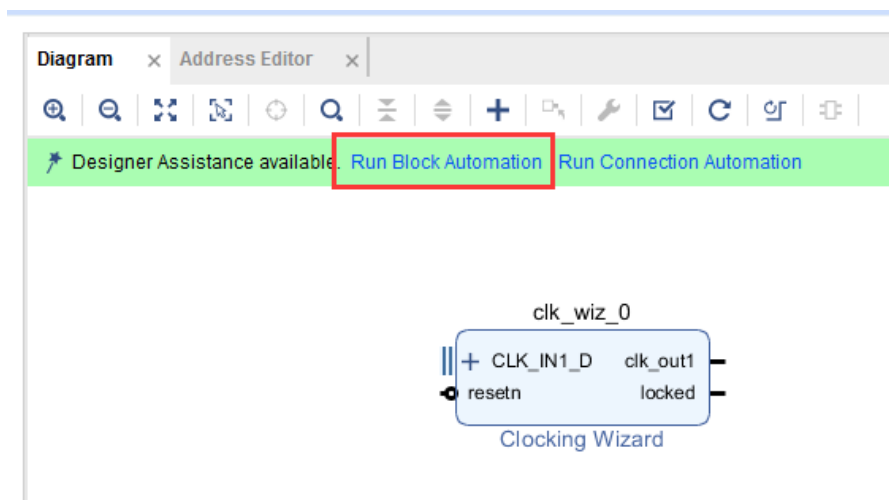
Enable Optional Inputs / Outputs for MMCM/PLL

☒ reset ☐ power_down ☐ input_clk_stopped
☒ locked ☐ clkfbstopped

Reset Type

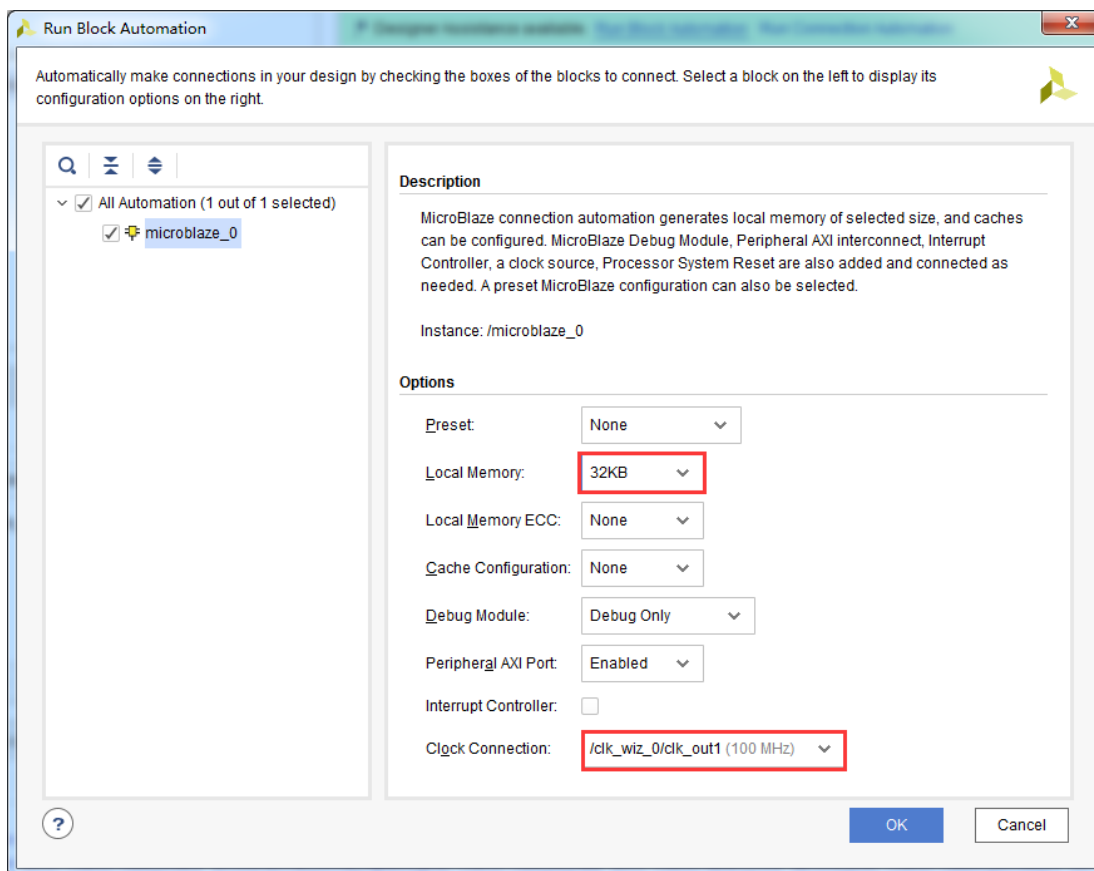
☐ Active High ☒ Active Low

15. 点击“Run Block Automation”完成一些自动设置，需要注意，这个选项不会一直都有，一旦设置完成，可能不会再有机会通过这种方式进行一些自动设置。



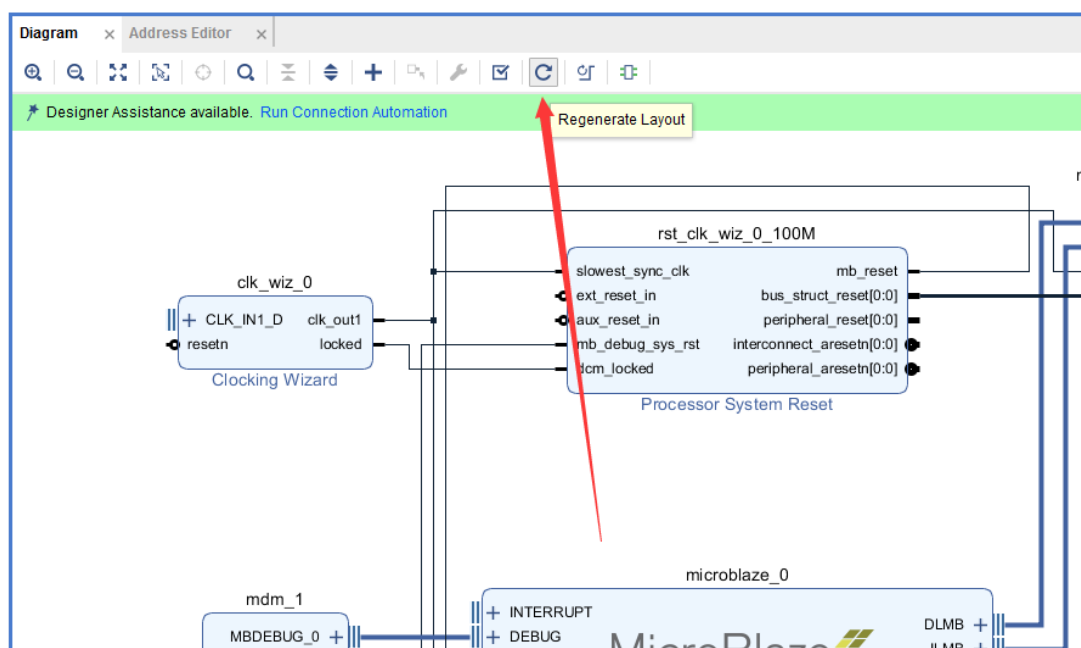
16. 弹出窗口里完成一些 microblaze 参数设置，在“Preset”选项中保持默认“None”，这个选项相

当于预设了一些 microblaze 的配置参数，根据应用选择。“Local Memory” 选择 32KB，因为没有外部存储器，这里不能选择太小，否则程序无法运行。“Clock Connection” 时钟选择时钟向导输出的 100Mhz 时钟作为 CPU 时钟。



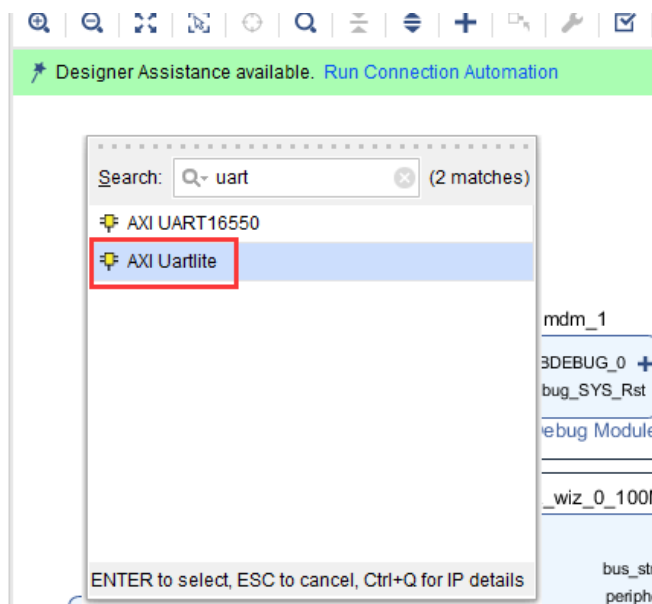
17. 点击 OK 完成配置后自动生成了一些模块并完成了部分连线

18. 点击重新布局按钮可以完成各个模块的重新布局

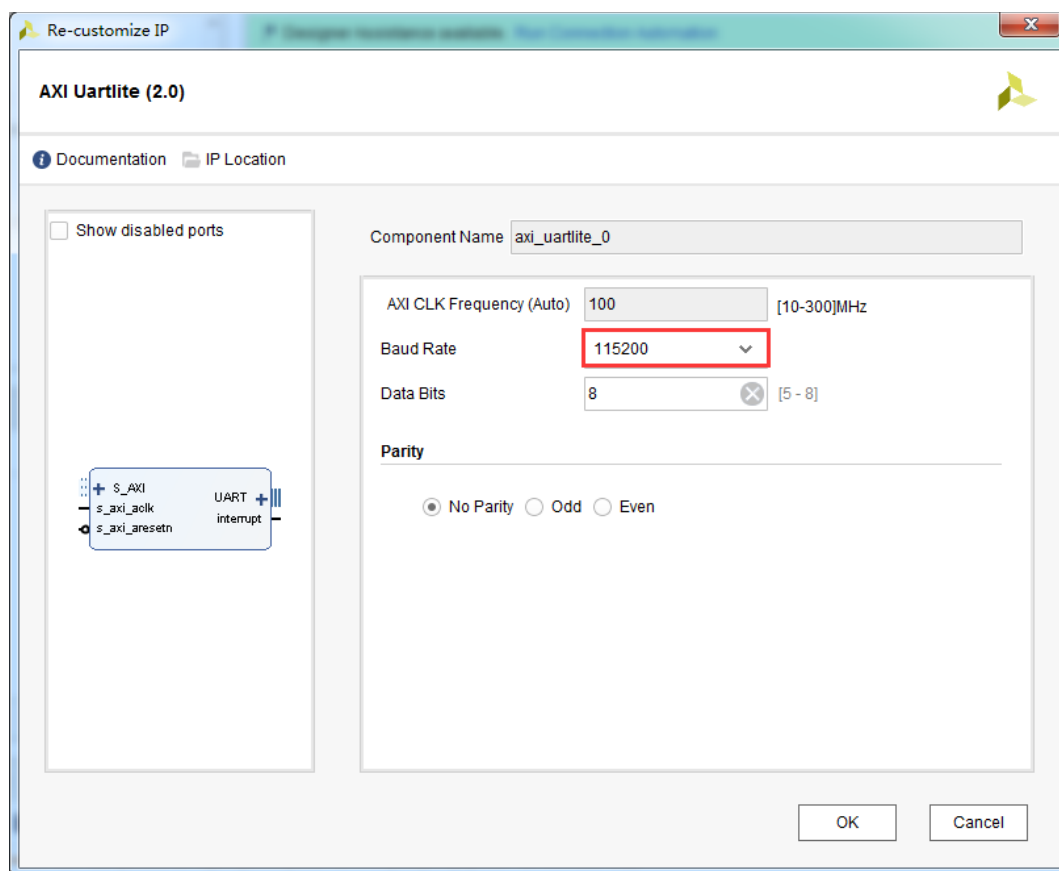


1.1.5 添加外设

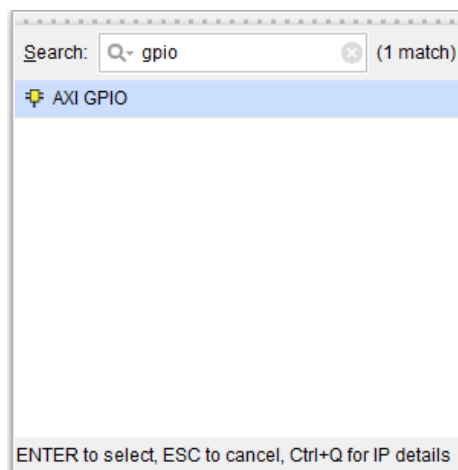
19. 搜索 “uart”，双击 “AXI Uartlite” 添加一个 Uart 串口 IP 核



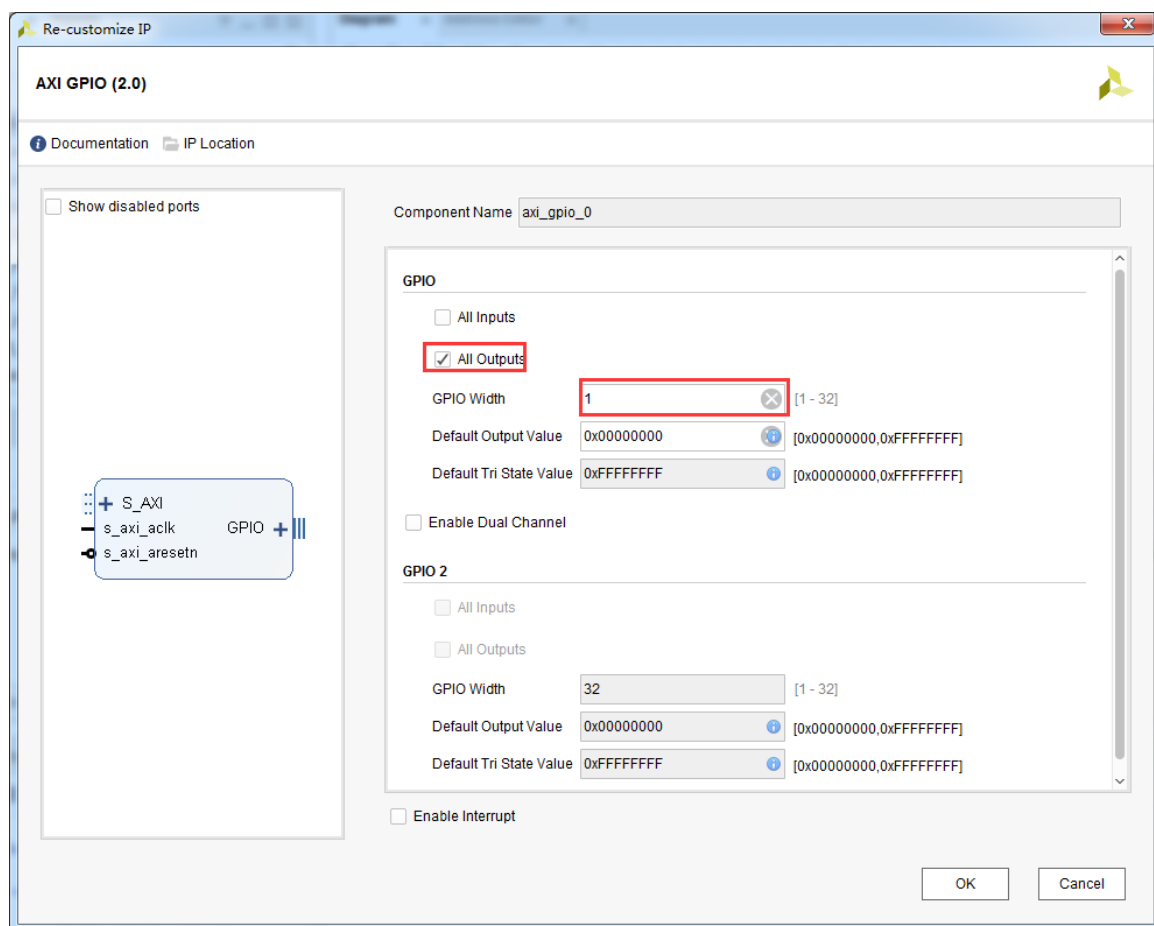
20. 双击刚才添加的 Uartlite 核，修改波特率为 115200，这是嵌入式系统比较常用的波特率，这里波特率是固定值，无法在程序中动态修改，点击 OK 完成配置



21. 搜索 gpio , 双击 “AXI GPIO” 添加一个 AXI GPIO , 用于控制 LED 灯。

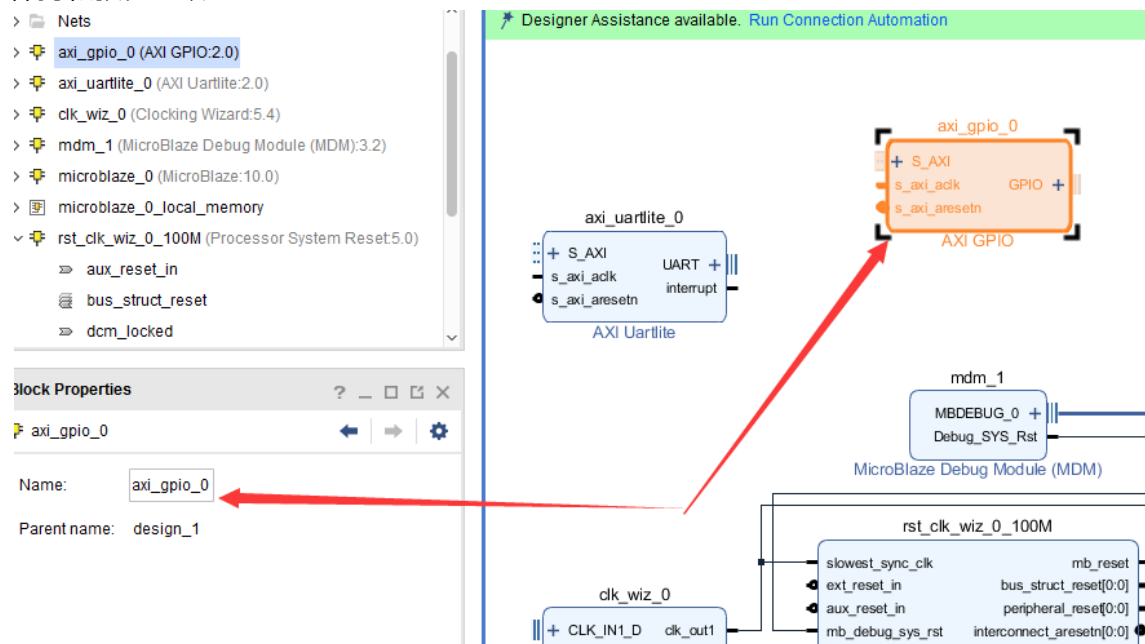


22. 双击刚添加的 AXI GPIO 修改参数, 勾选 “All Outputs”, 设置为全输出, “GPIO Width” 填写 1, 这里只控制 1 位 LED, 点击 “OK” 完成配置

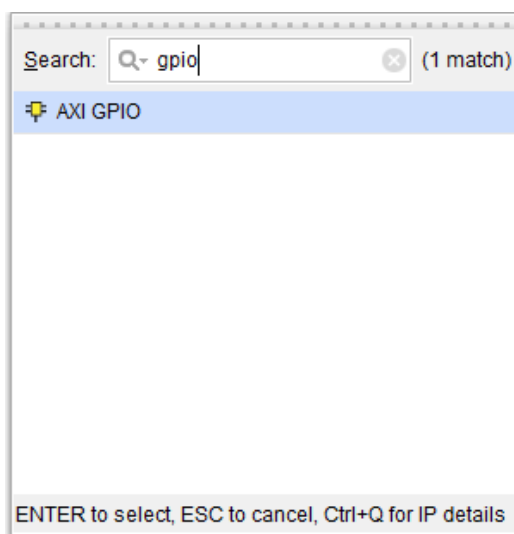


23. 选择刚添加的 AXI GPIO , 在 Block Properties 界面可以修改模块名称, 这里改为 “axi_led”, 需要注意, 如果名字和例程不同, 软件编程时可能要修改例程的 C 语言代码, 建议初学者

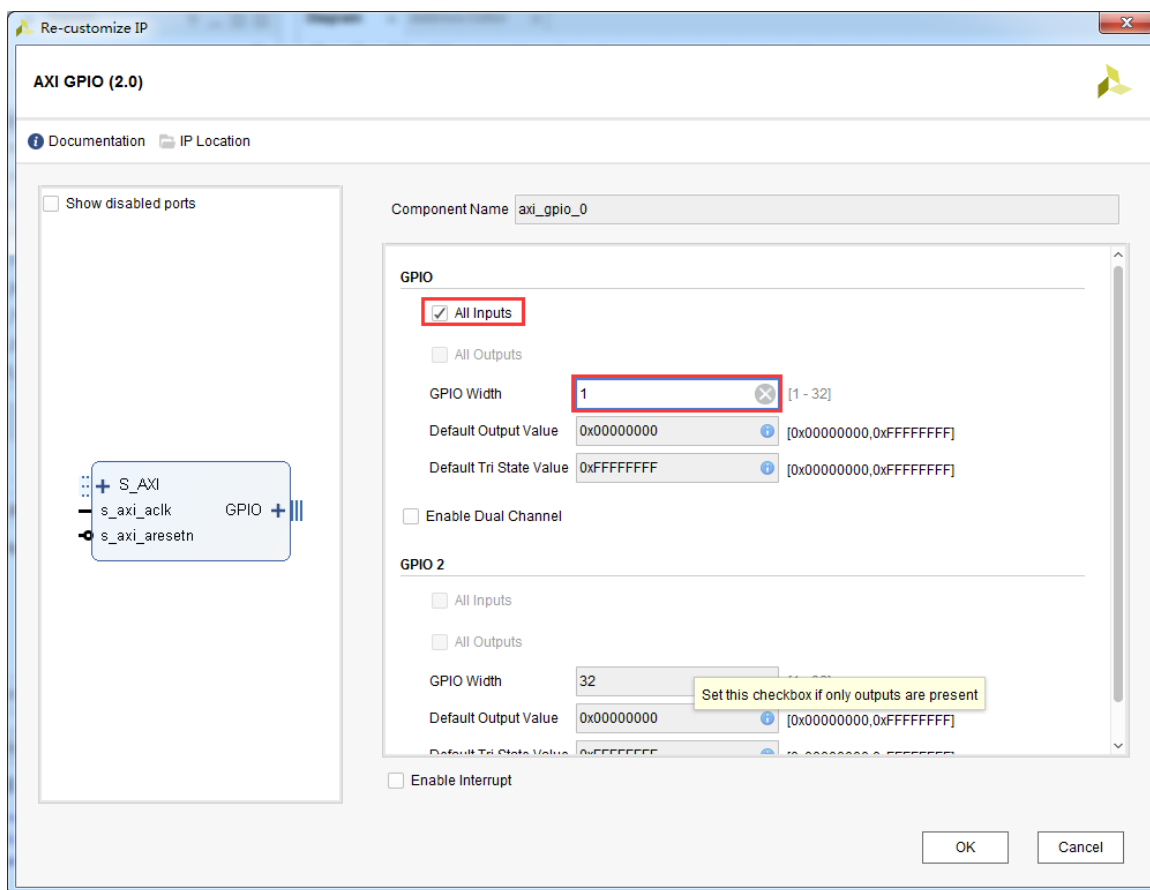
保持和教程一致。



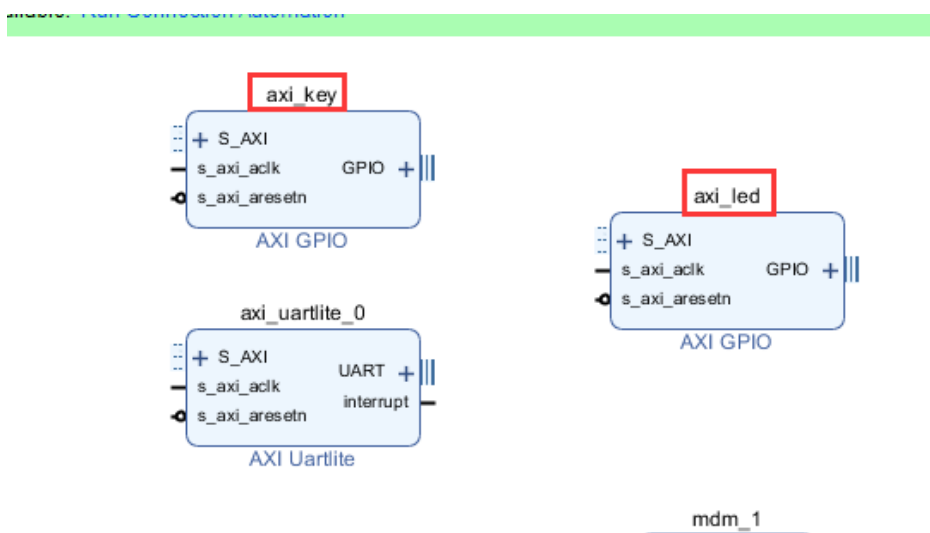
24. 添加一个 AXI GPIO，用于按键输入



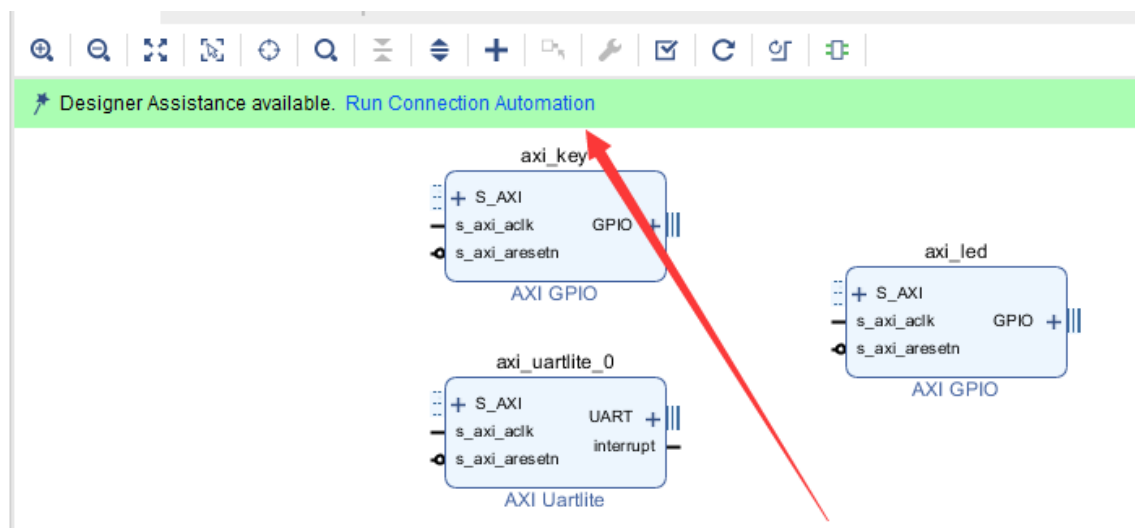
25. 修改 AXI GPIO 参数，勾选 “All Inputs”，“GPIO Width” 填写 1，这里只控制 1 位按键，点击 “OK” 完成配置



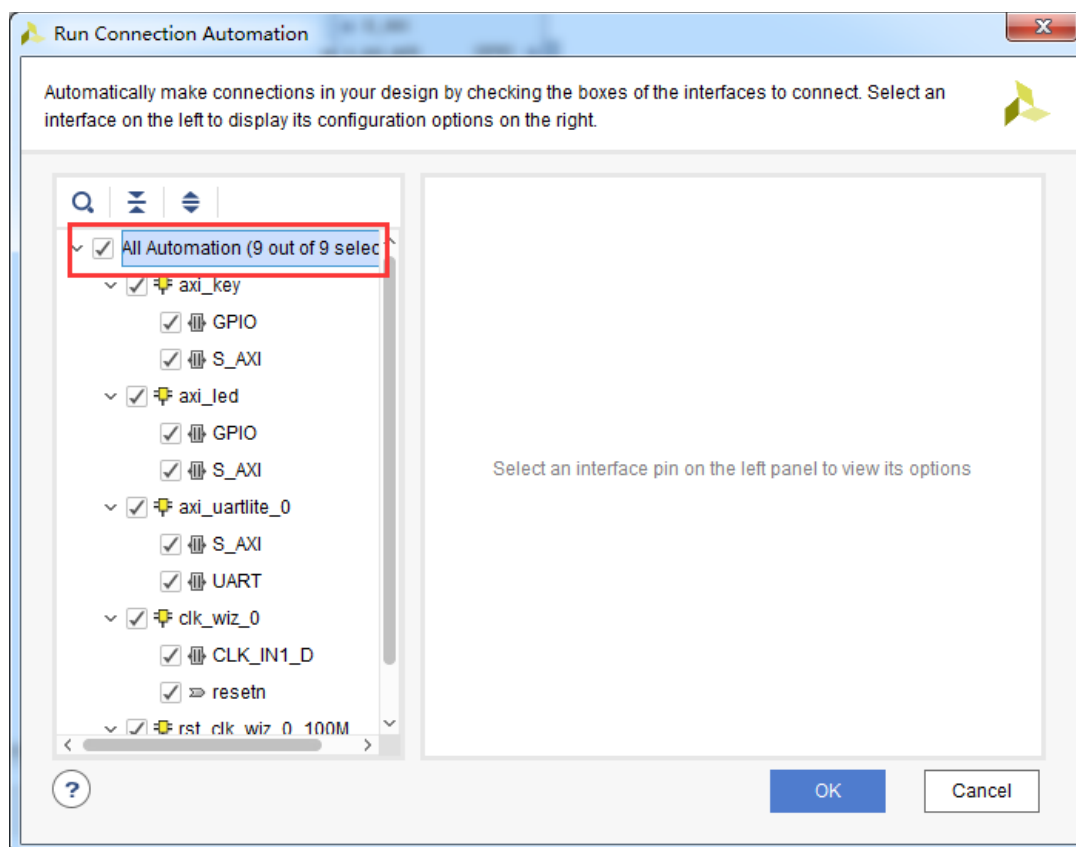
26. 选择刚添加的 AXI GPIO，在 Block Properties 界面可以修改模块名称，这里改为“axi_key”，需要注意，如果名字和例程不同，软件编程时可能要修改例程的 C 语言代码，建议初学者保持和教程一致。



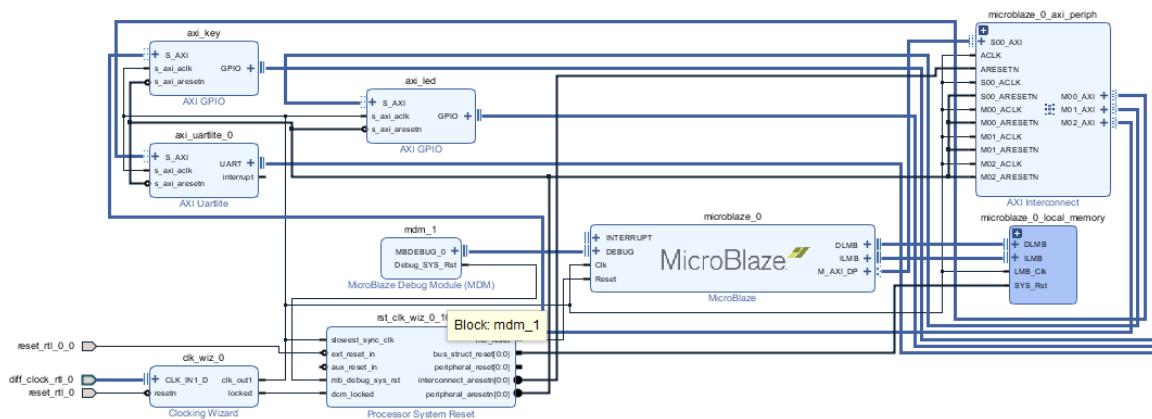
27. 点击“Run Connection Automation”，来自动连线



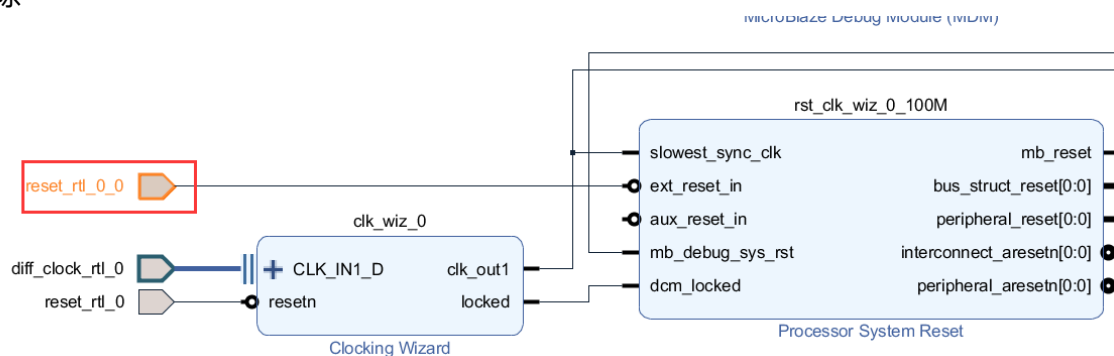
28. 选择 “All Automation”, 自动连接所有的线, 点击 “OK” 开始自动连线



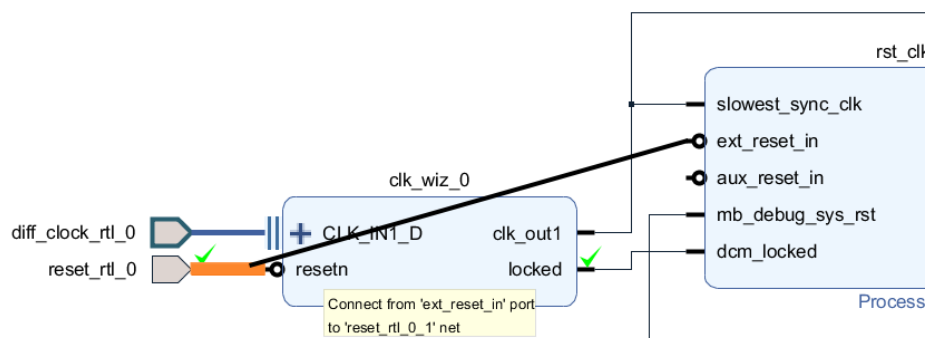
29. 自动连线完成了刚添加的 Uartlite 和 AXI GPIO 的自动连线, 并自动添加了一个 AXI 总线交叉模块, AXI 交叉模块可以完成多主多从的 AXI 设备交叉互联。



30. 删除 “rst_clk_wiz_0_100M” 模块上的端口 “reset_rtl_0_0”，选中，按 “delete” 按键可以删除

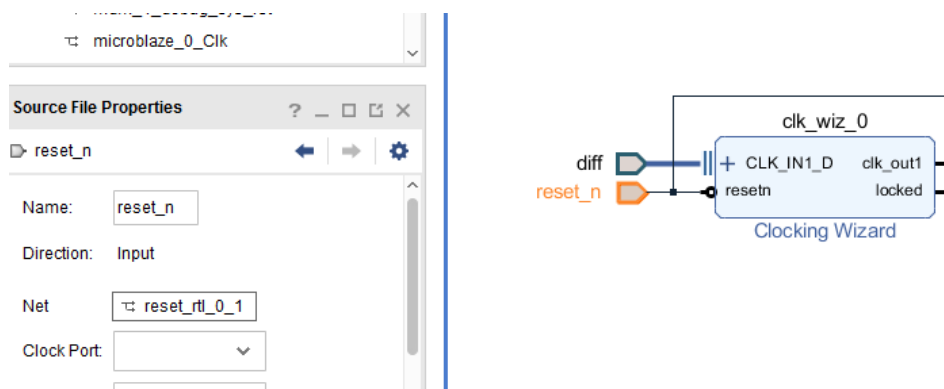


31. 连接 “rst_clk_wiz_0_100M” 模块上的 “ext_reset_in” 到 “clk_wiz_0” 的 “resetsn”，让 2 个模块共用一个复位，需要注意，这 2 个模块都是低电平复位有效。

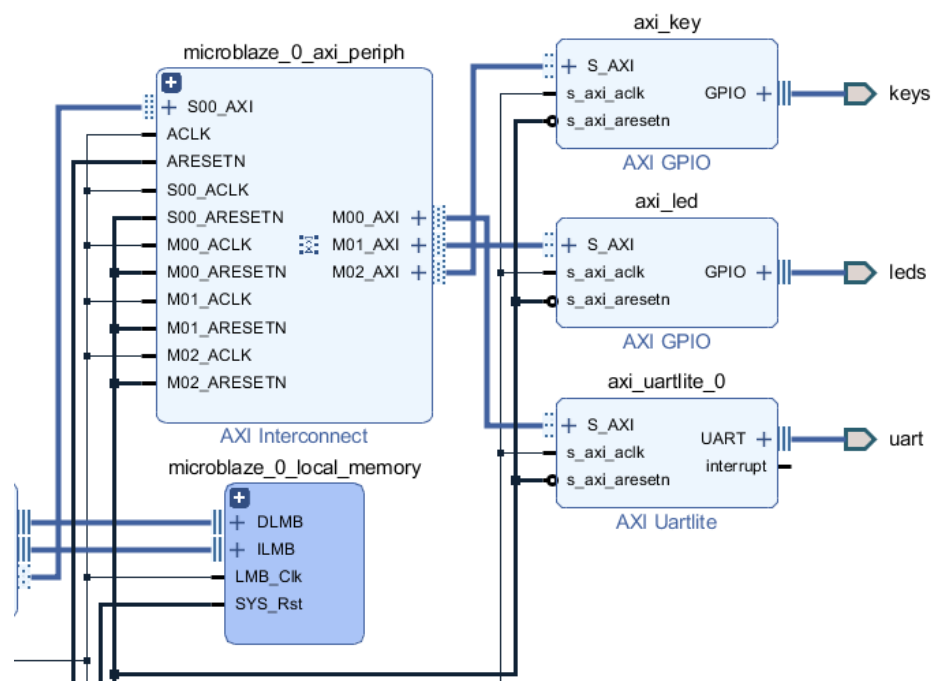


1.1.6 修改端口名称

32. 修改各端口名称，默认端口名称比较长，不好记，这一步也是很多初学者容易忘记的步骤，如果这一步不修改，会导致和例程 xdc 文件端口名不一致，编译会报错
33. “diff_clock_rtl_0” 修改为 “diff”，“reset_rtl_0” 修改为 “reset_n”

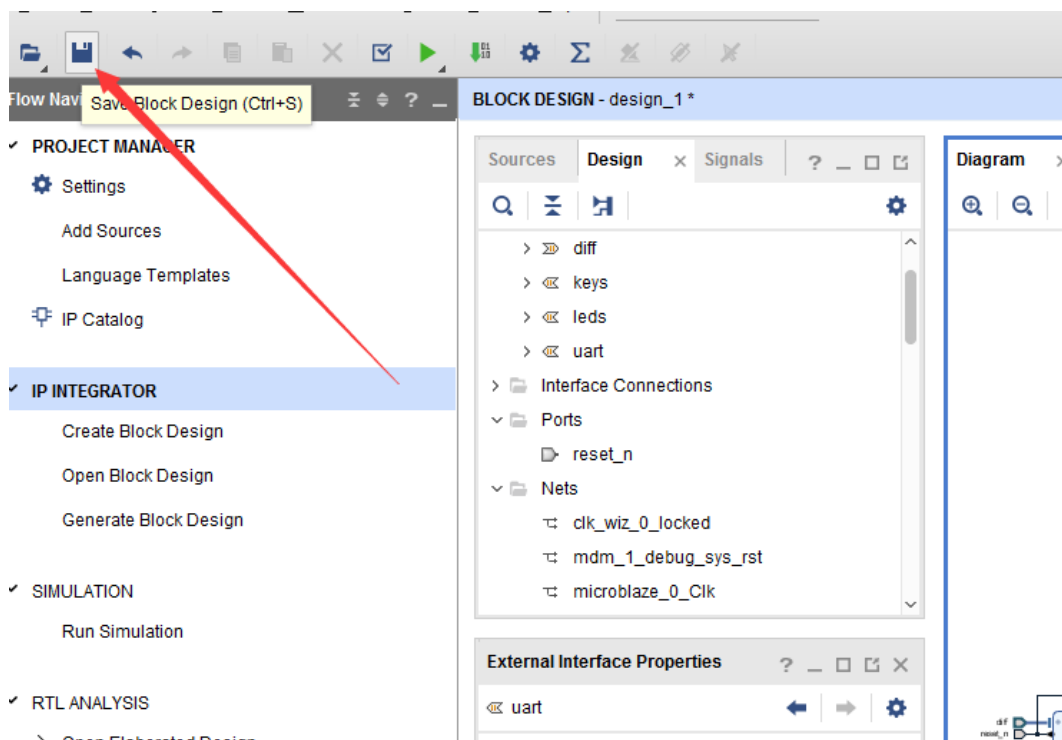


34. "gpio_rtl_1" 修改为 "keys", "gpio_rtl_0" 修改为 "leds", "uart_rtl_0" 修改为 "uart"

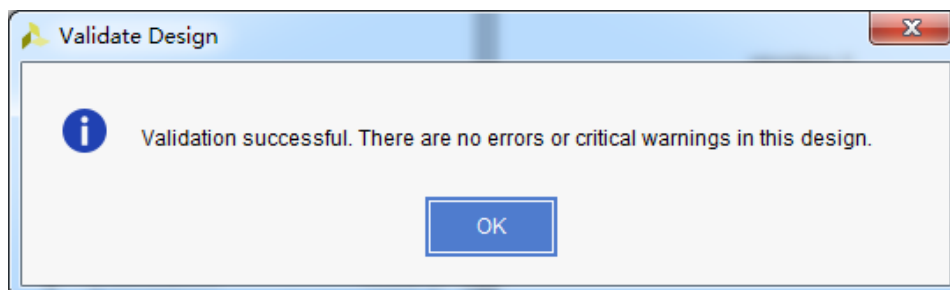


1.1.7 保存并检查错误

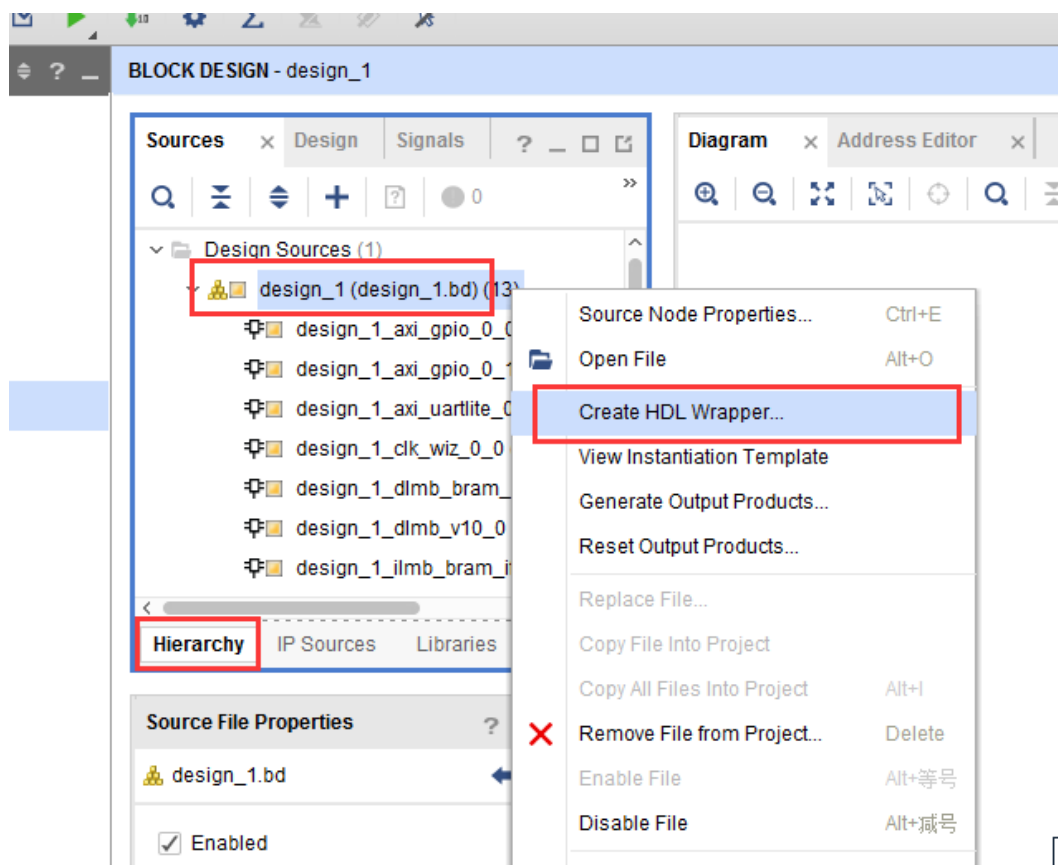
35. 保存设计，然后按“F6”检查设计



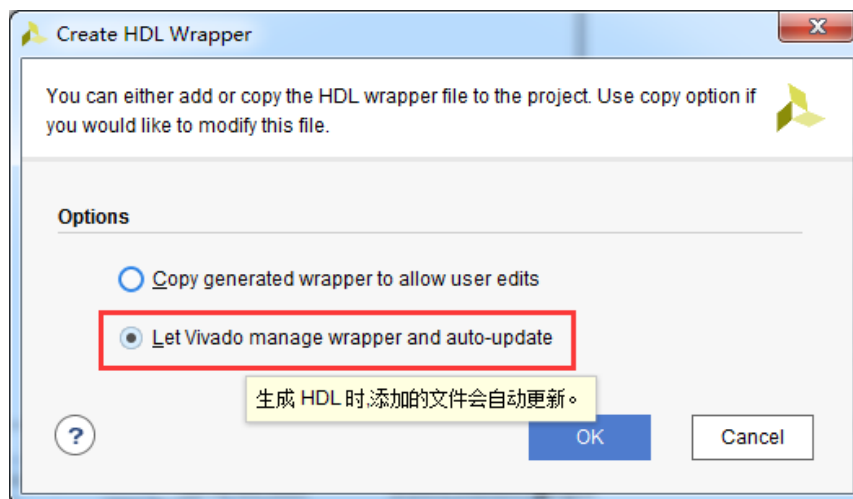
36. 设计中没有错误和警告



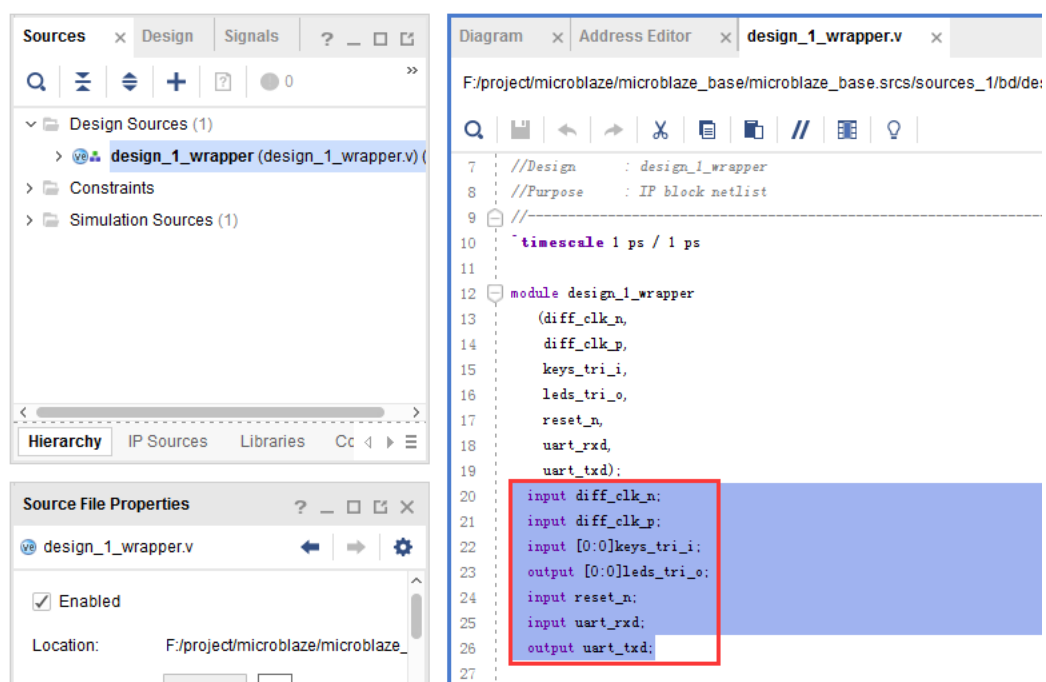
37. 选择 "design_1.bd" 文件，右键选择 "Create HDL Wrapper..." 创建一个 Verilog 或 VHDL 顶层文件



38. 选择 “Let Vivado manage wrapper and auto-update”, 让 Vivado 自动更新 Verilog 或 VHDL 文件

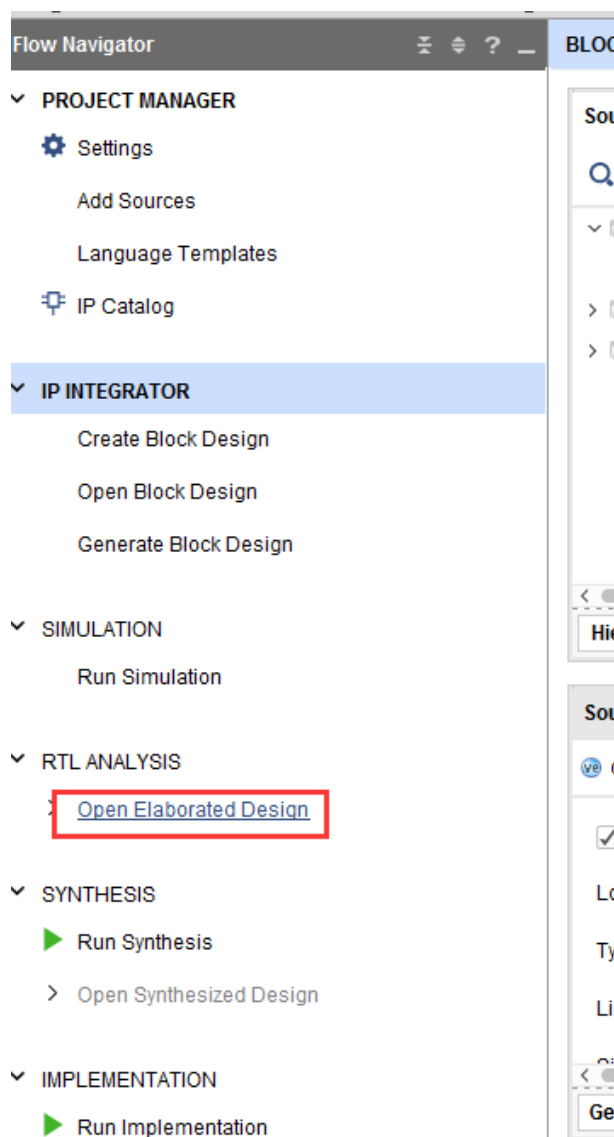


39. 双击打开生成的 “design_1_wrapper.v”, 可以看到一些端口, 下面为这些端口分配 IO

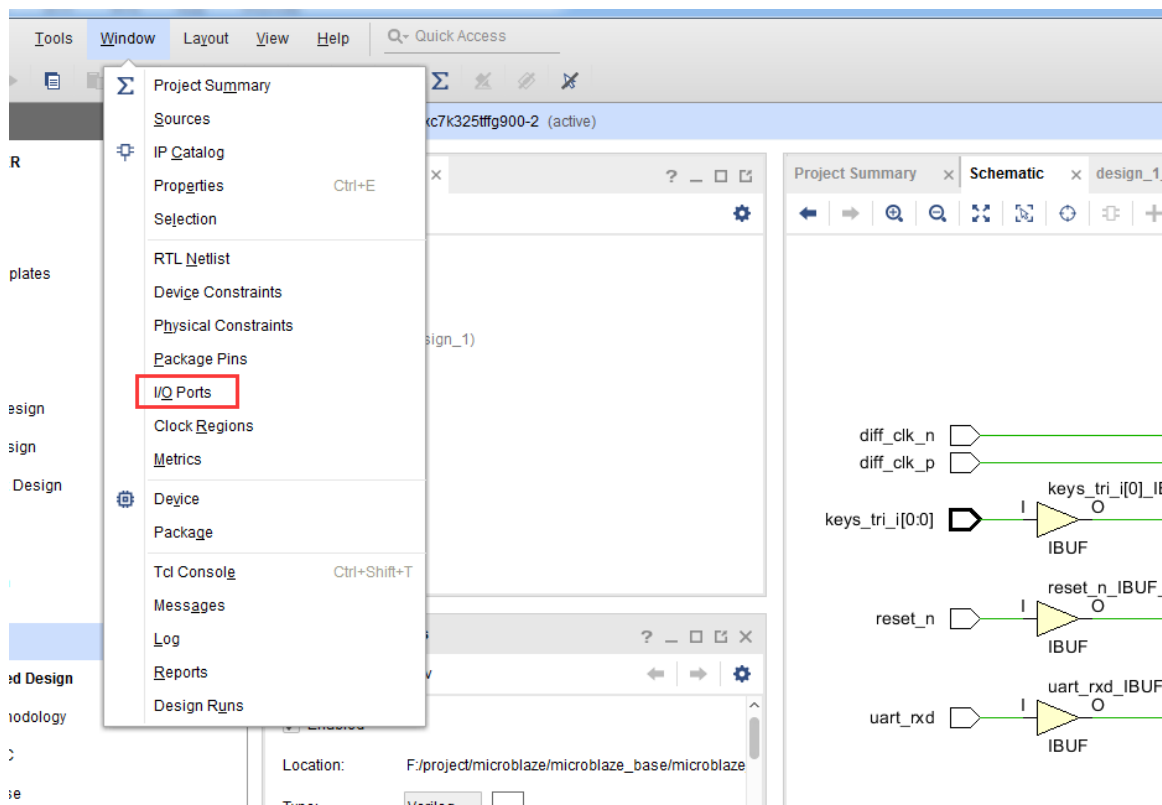


1.1.8 分配 IO

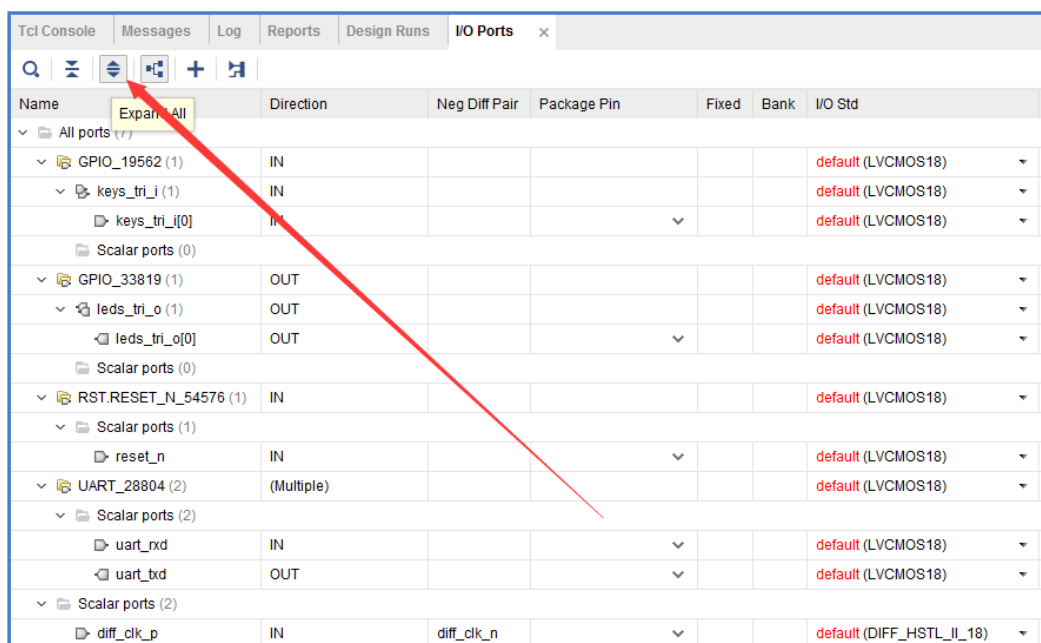
40. 点击 “Open Elaborated Design”，在弹出窗口中点击 “OK”



41. 点击 “Windows -> I/O Ports”



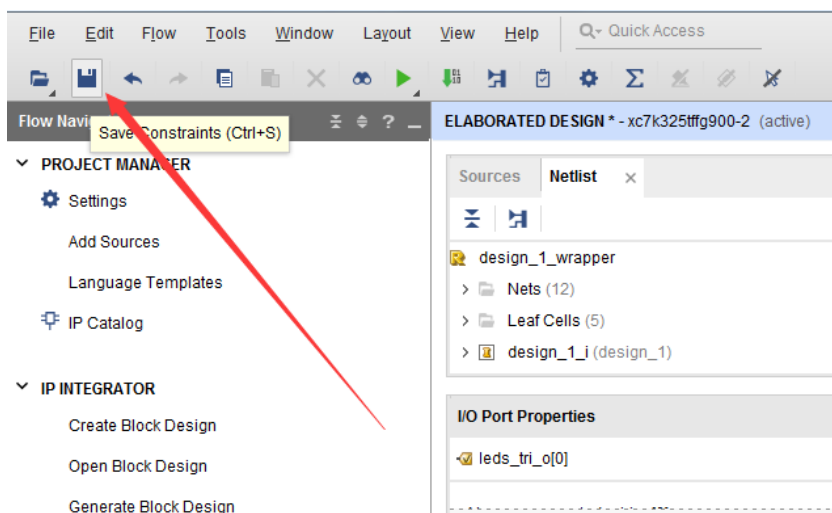
42. 点击“Expand All”，可以看到 IO 都没有分配，在“Package Pin”列分配 IO，在“I/O Std”列分配电平标准。



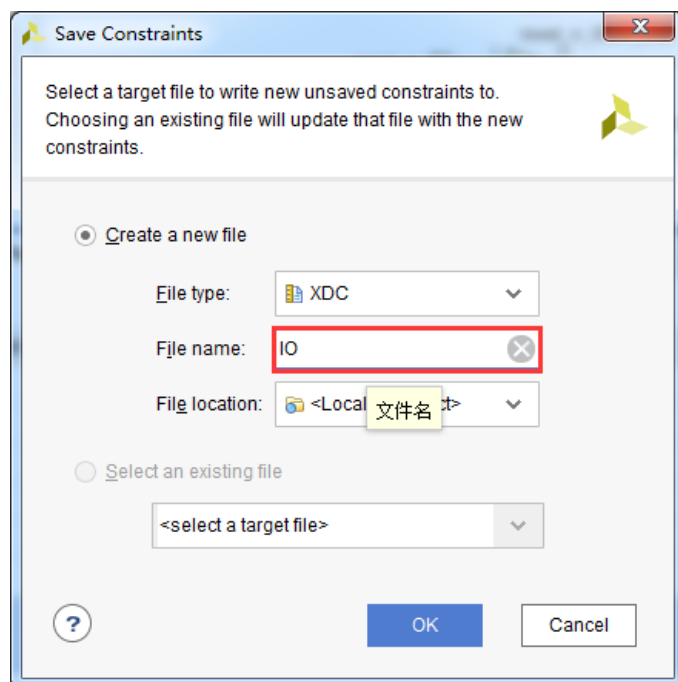
43. 分配好以后的情况，需要注意，IO 分配一定要根据实际开发板来分配（下图以 AX7325 开发板为例）

Tcl Console Messages Log Reports Design Runs I/O Ports x						
Name	Direction	Neg Diff Pair	Package Pin	Fixed	Bank	I/O Std
All ports (7)						
GPIO_19562 (1)	IN			✓	12	LVCMS33*
keys_tri_i (1)	IN			✓	12	LVCMS33*
keys_tri_i[0]	IN		AC24	✓	12	LVCMS33*
Scalar ports (0)						
GPIO_33819 (1)	OUT			✓	13	LVCMS33*
leds_tri_o (1)	OUT			✓	13	LVCMS33*
leds_tri_o[0]	OUT		Y28	✓	13	LVCMS33*
Scalar ports (0)						
RST.RESET_N_54576 (1)	IN			✓	12	LVCMS33*
Scalar ports (1)						
reset_n	IN		AD24	✓	12	LVCMS33*
UART_28804 (2)						
Scalar ports (2)						
uart_rxd	IN		AA27	✓	13	LVCMS33*
uart_txd	OUT		W28	✓	13	LVCMS33*
Scalar ports (2)						
diff_clk_p	IN	diff_clk_n	AE10	✓	33	DIFF_SSTL15*

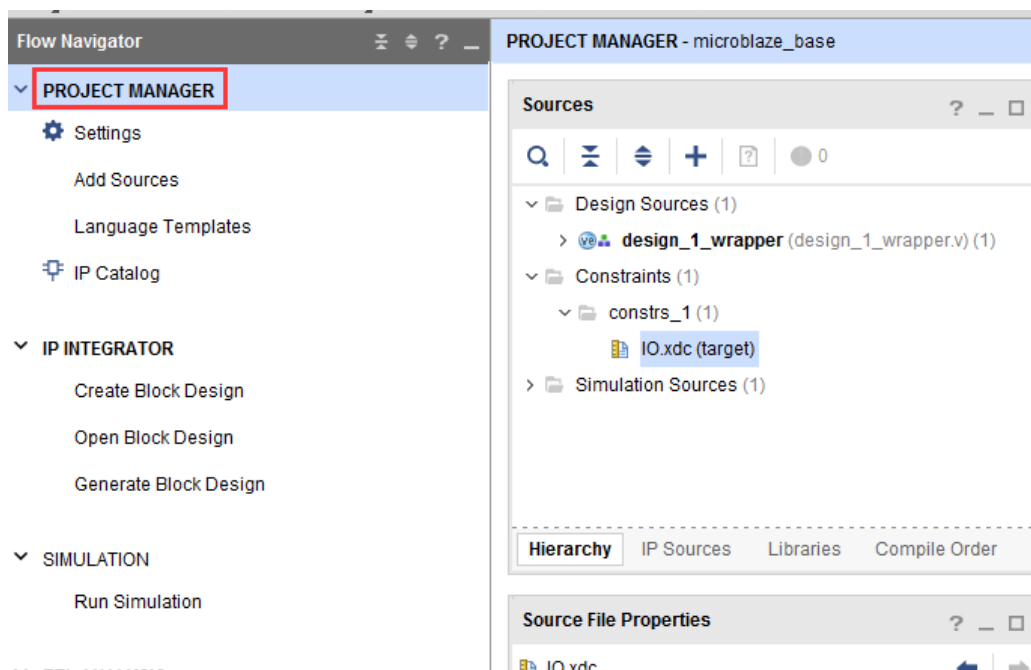
44. 点击保存按钮



45. 提示创建一个新的 xdc 文件，名称这里填写 IO，点击 OK

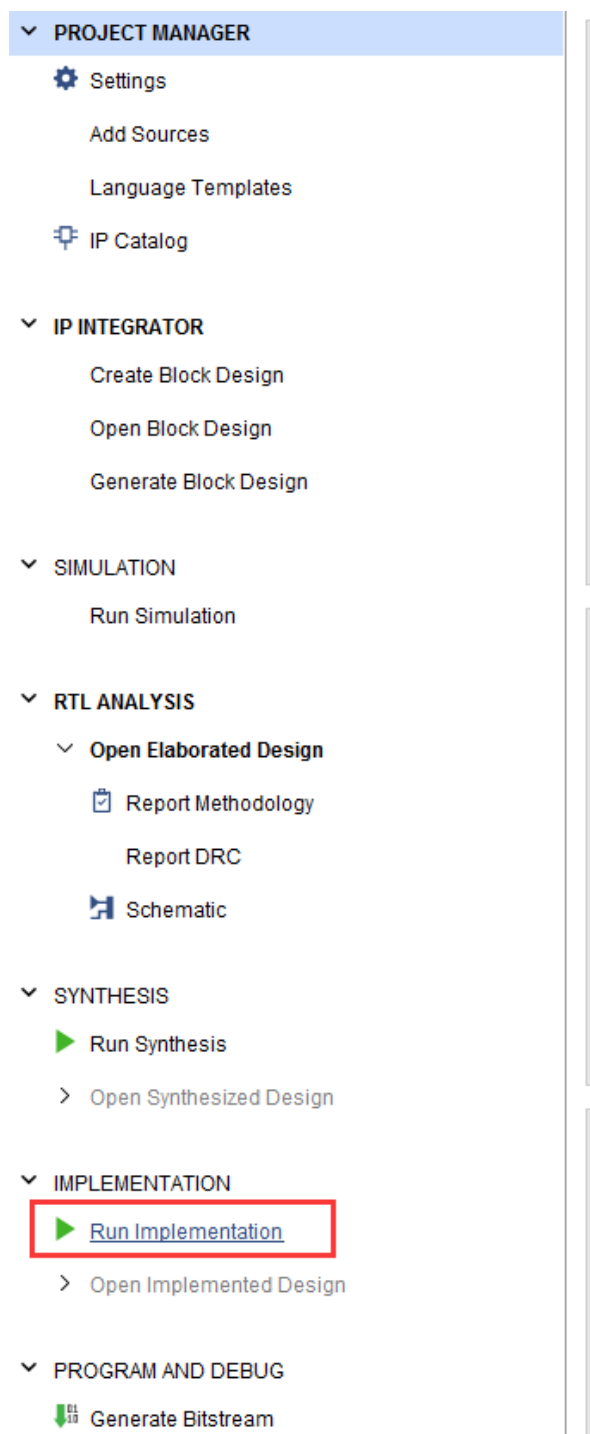


46. 点击“PROJECT MANAGER”回到工程管理窗口，双击“IO.xdc”文件可以查看内容

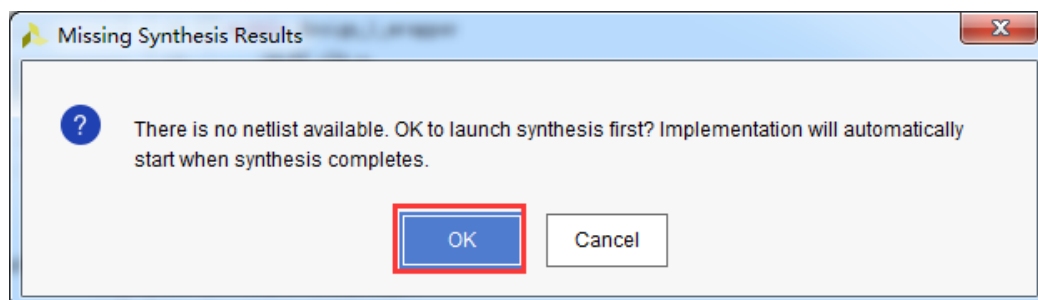


1.1.9 开始编译

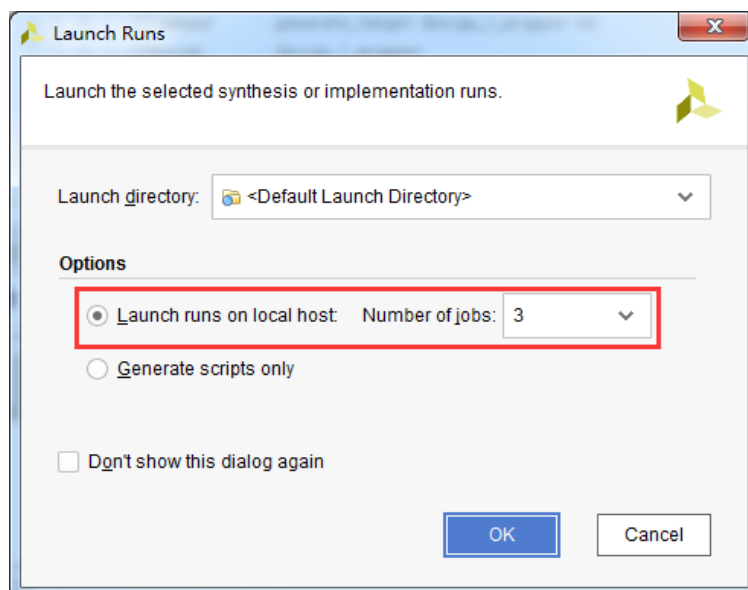
47. 点击“Run Implementation”



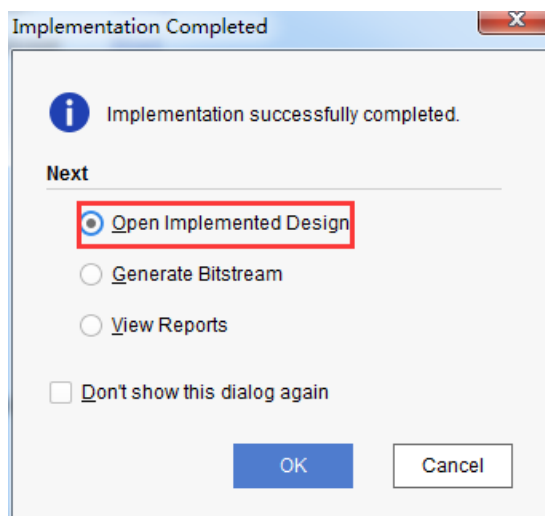
48. 如果出现下面窗口，点击“OK”即可



49. 在弹出窗口中有运行路径选择，这里默认，“Number of jobs” 线程数量可以根据自己 CPU 选择，数字越大，编译越快。

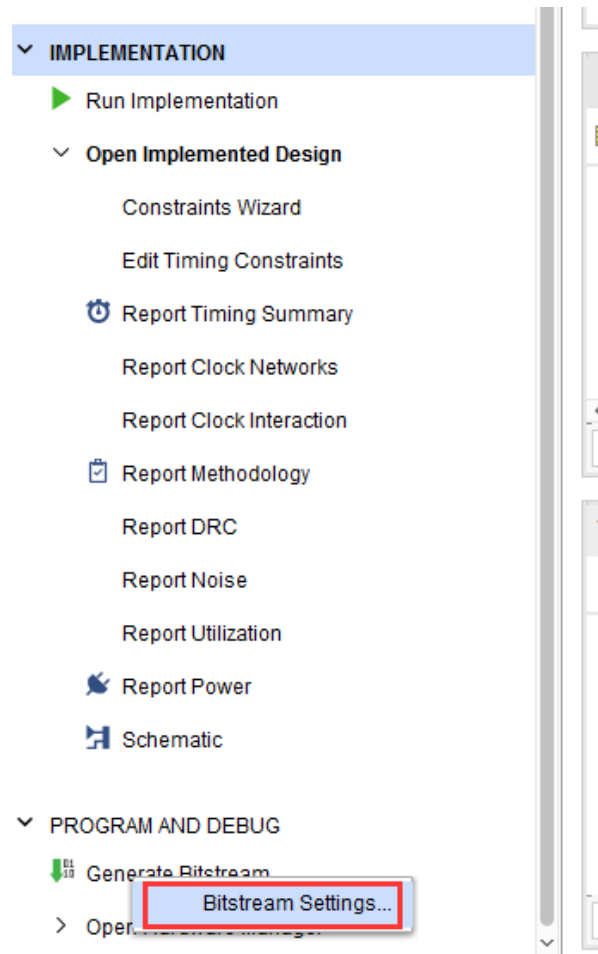


50. 编译完成以后选择 “Open Implemented Design”

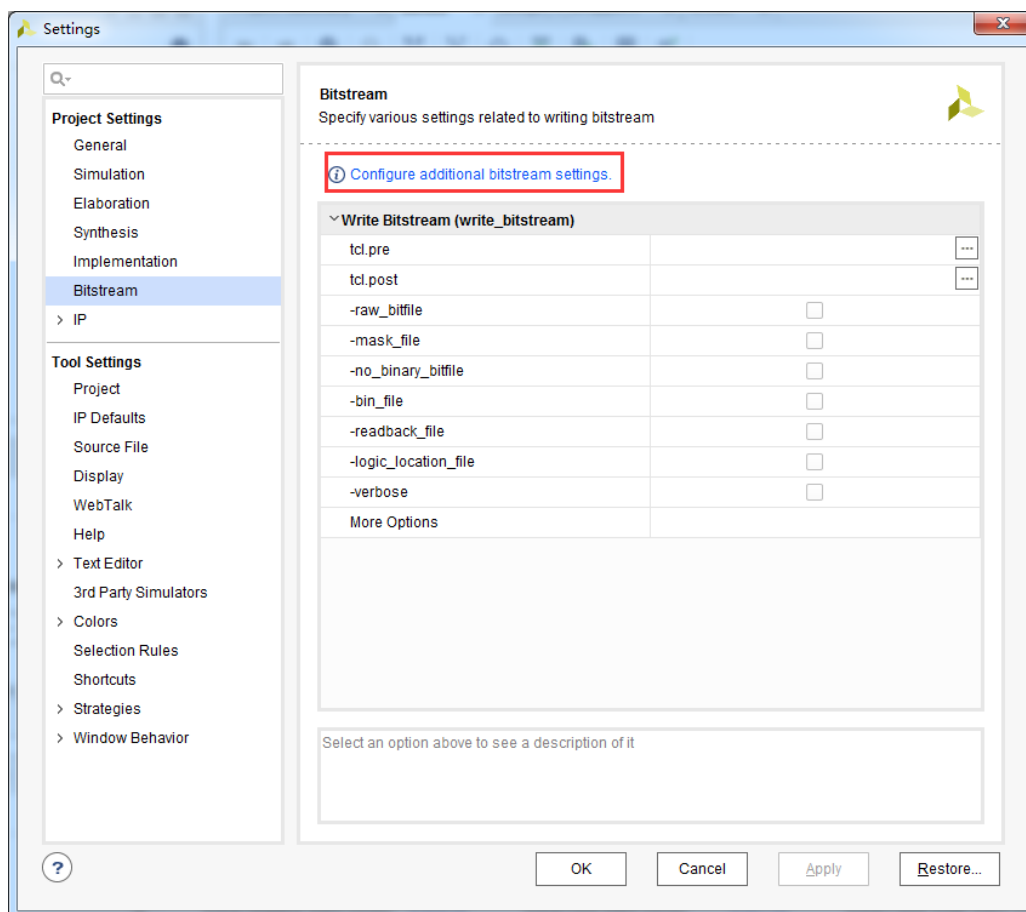


1.1.10 bitstream 设置

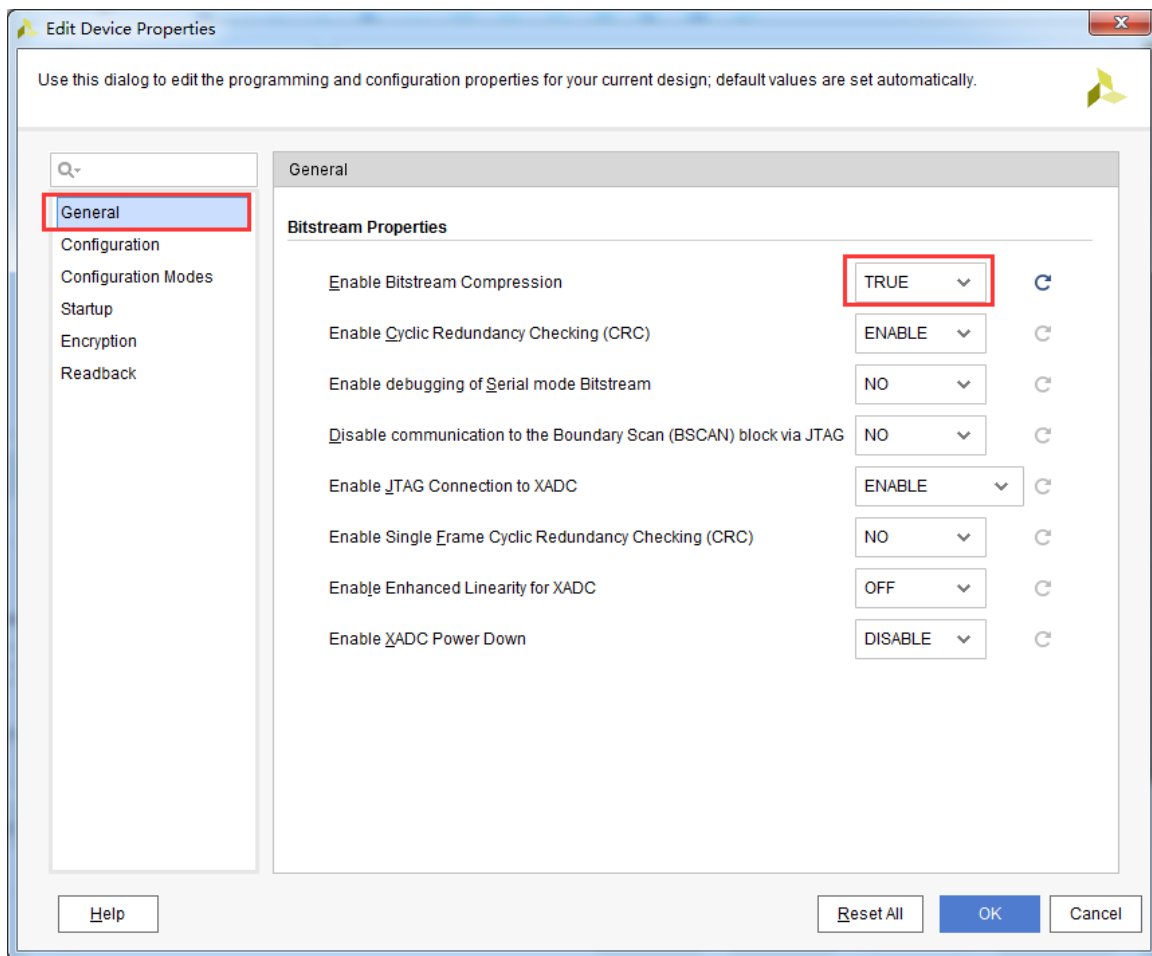
在 “PROGRAM AND DEBUG” 选项，右键选择 “Bitstream Setting...”



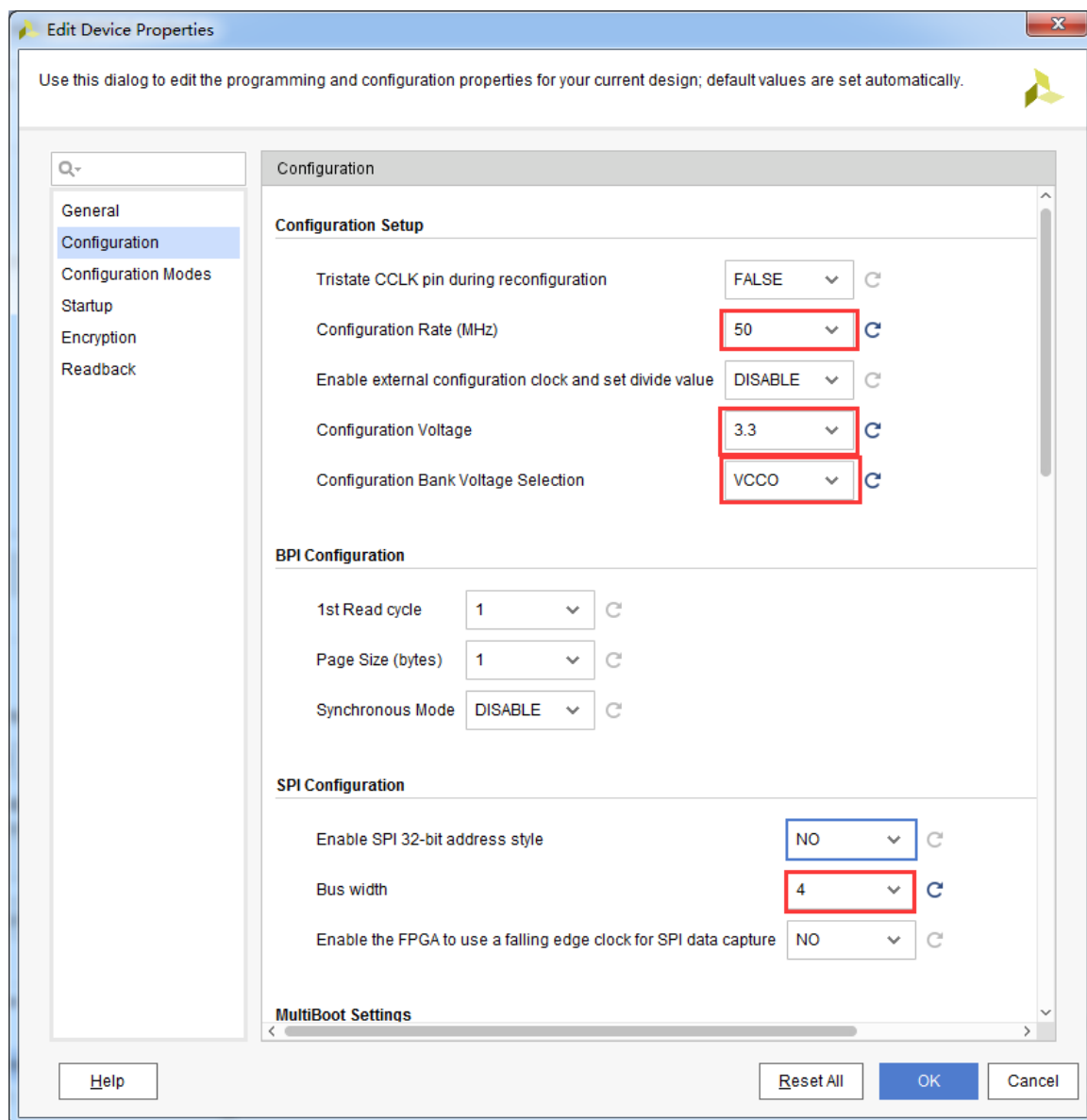
51. 在 “Setting” 窗口点击 “Configure additional bitstream settings”



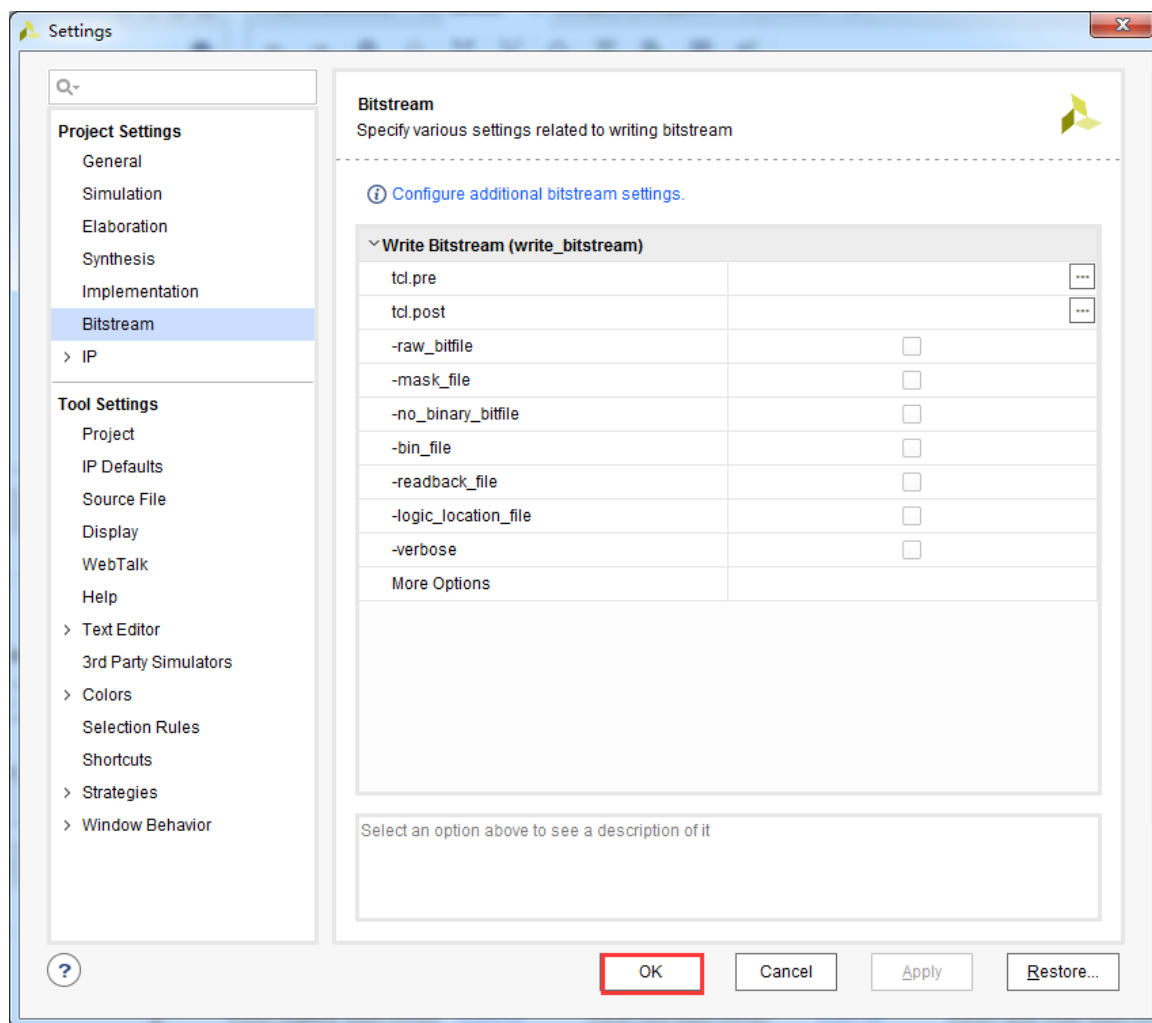
52. 在 “General” 选项卡中，“Enable Bitstream Compression” 选择 “TRUE”，压缩 Bitstream 文件，让 Bitstream 文件体积变小



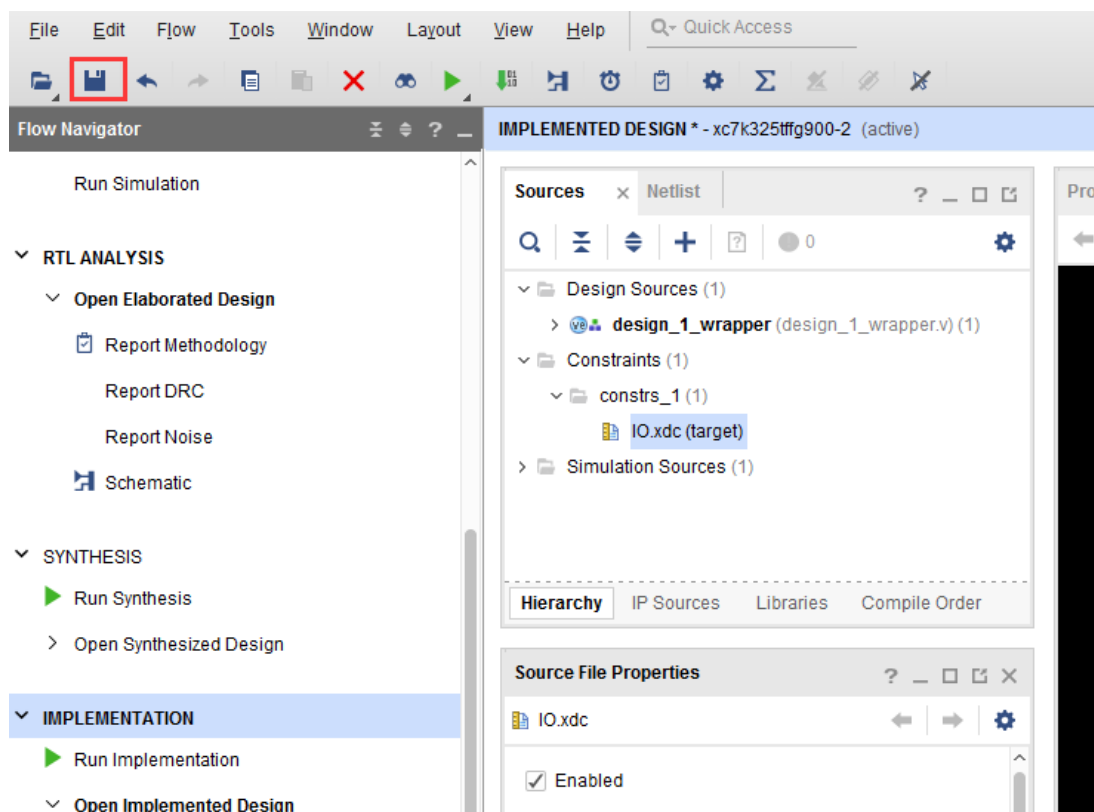
53. 在“Configuration”选项卡，修改“Configuration Rate (MHz)”为 50，数值太大可能无法正确加载配置文件，数值太小可能加载时间过长。“Configuration Voltage”配置电压选择 3.3，“Configuration Bank Voltage Selection”选择 VCCO，这些配置是硬件设计决定，这里不做解释。在“SPI Configuration”选择“Bus width”为 4，其他保持默认，点击“OK”



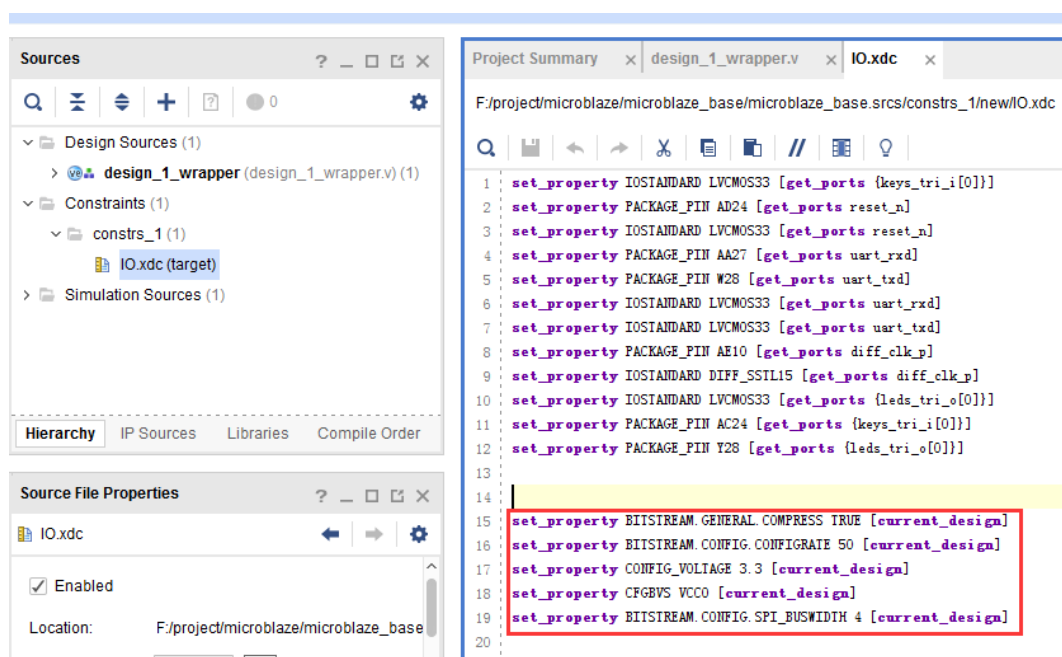
54. 点击“OK”完成 Bitstream 设置



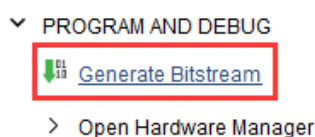
55. 点击保存按钮，将配置信息保存到 xdc 文件中



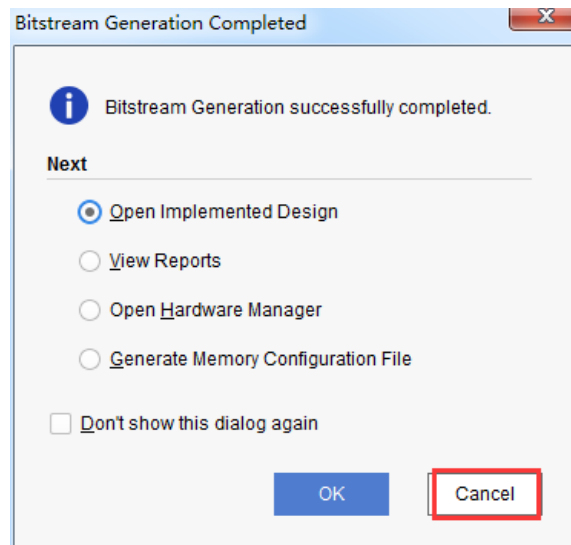
56. 打开 xdc 文件可以发现多出一段 Bitstream 配置相关代码



57. 点击 “Generate Bitstream” 生成 bit 流文件

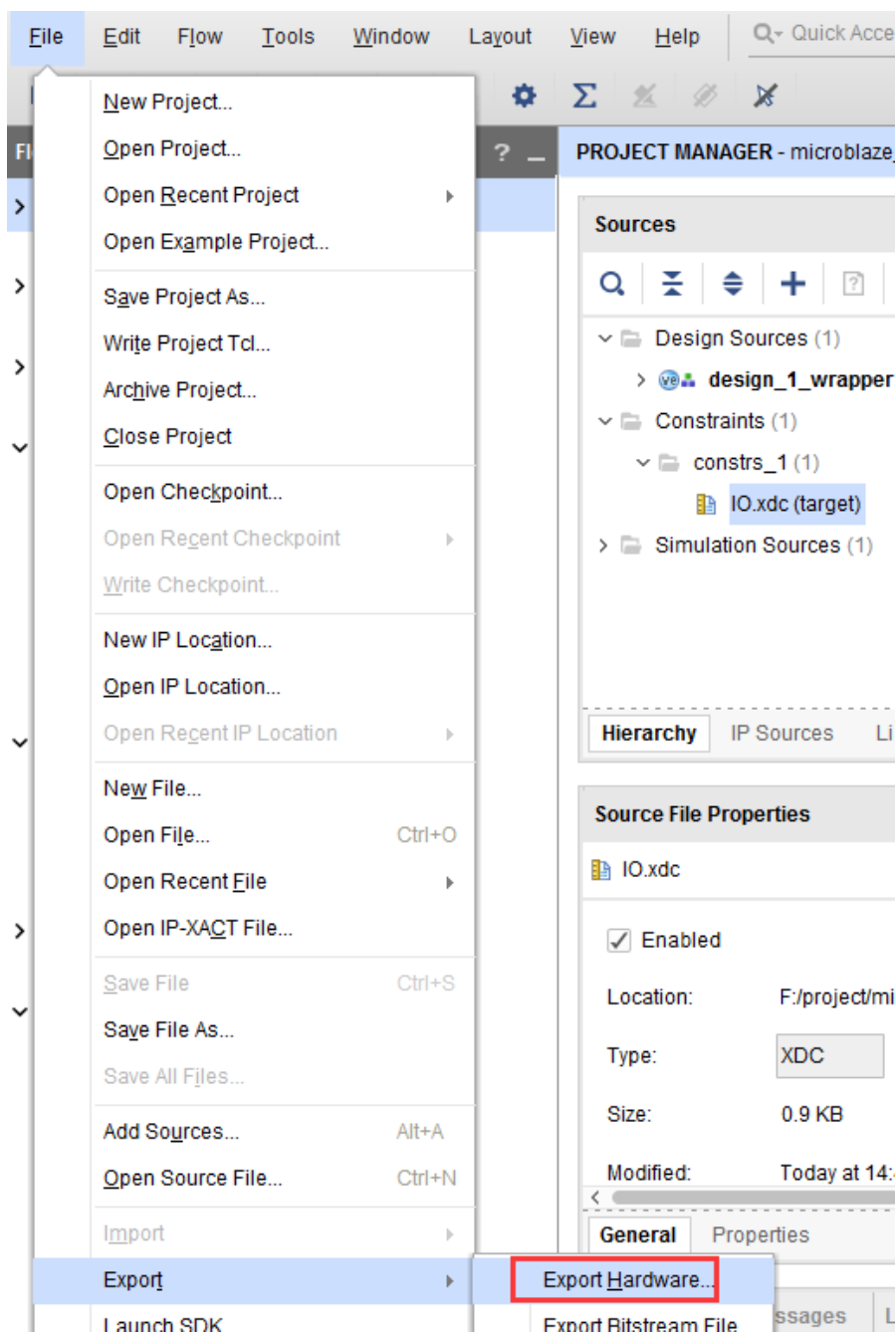


58. 生成完成以后，点击 “Cancel”，不做任何操作

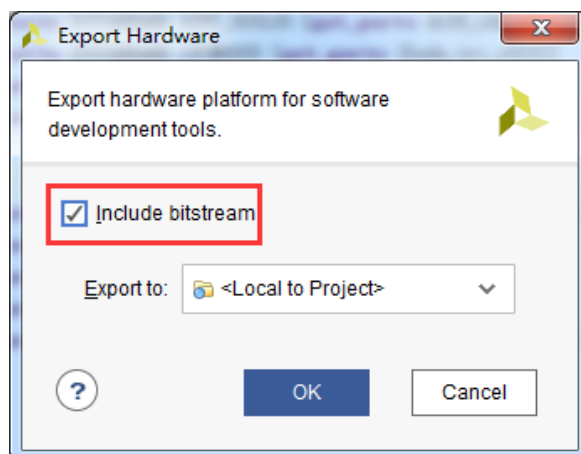


1.1.11 导出硬件

59. 在 “File -> Export -> Export Hardware...” 导出硬件

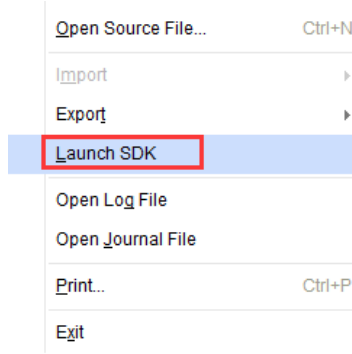


60. 在弹出窗口中选择 “Include bitstream” 包含 Bitstream 文件

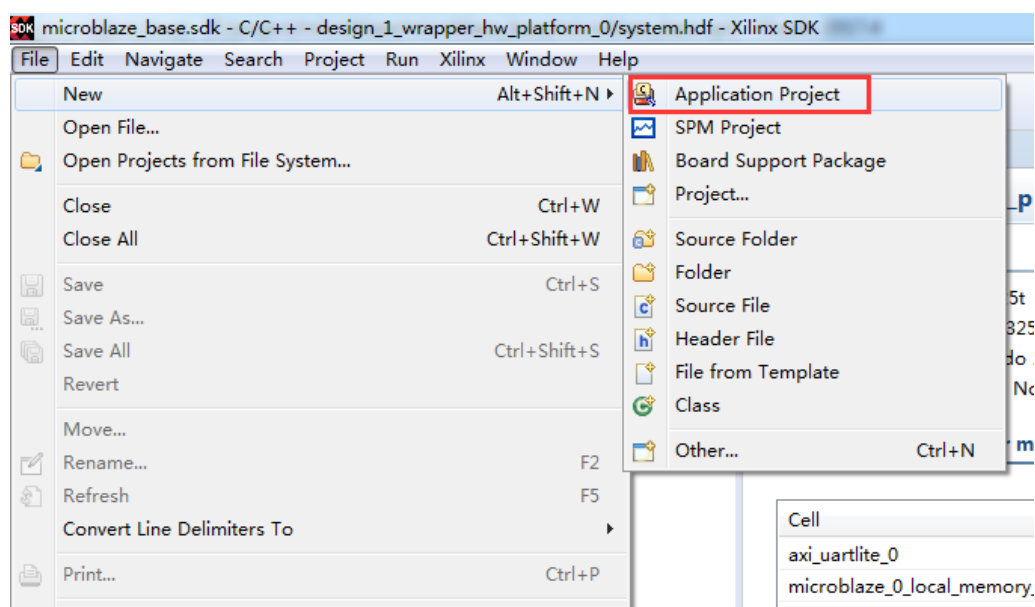


1.2 SDK 开发 “HelloWorld”

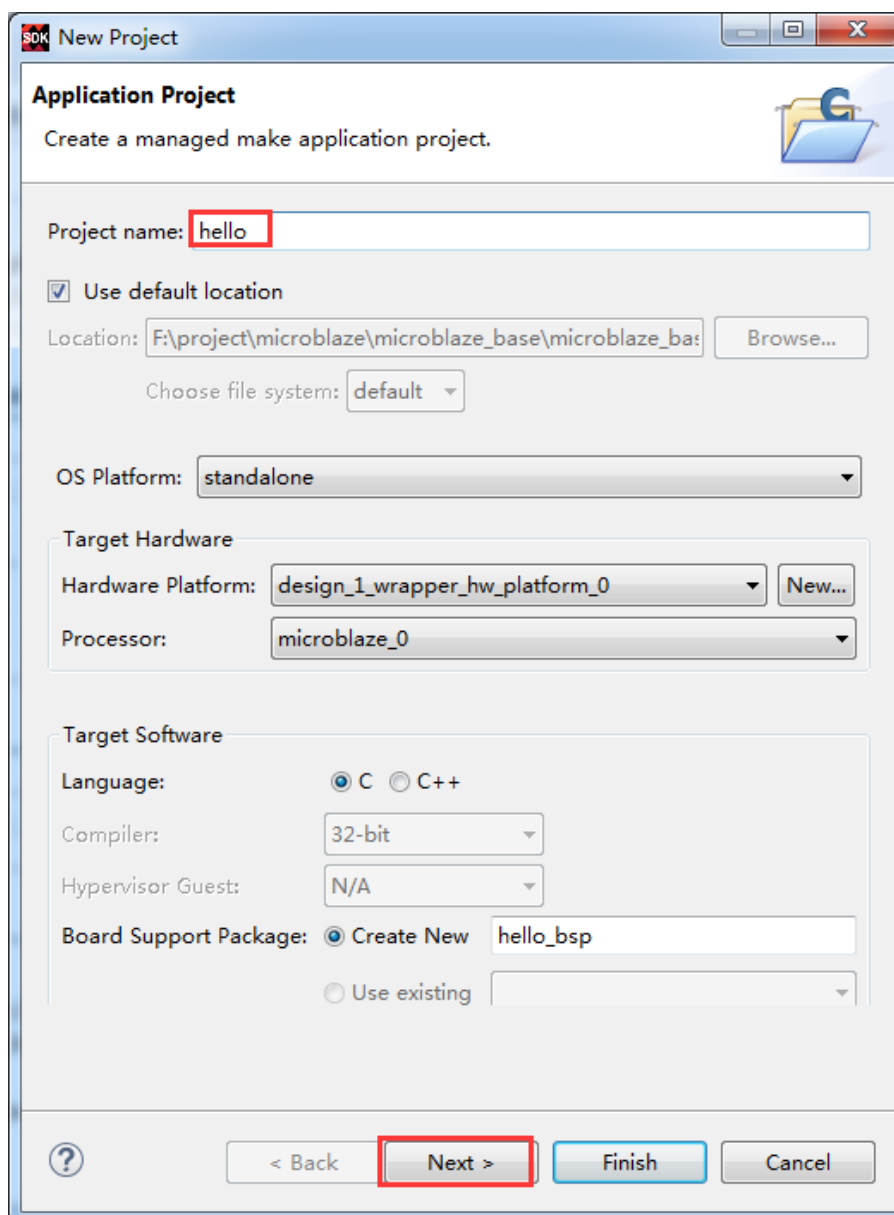
1. 点击 File→Launch SDK 进入 SDK 操作



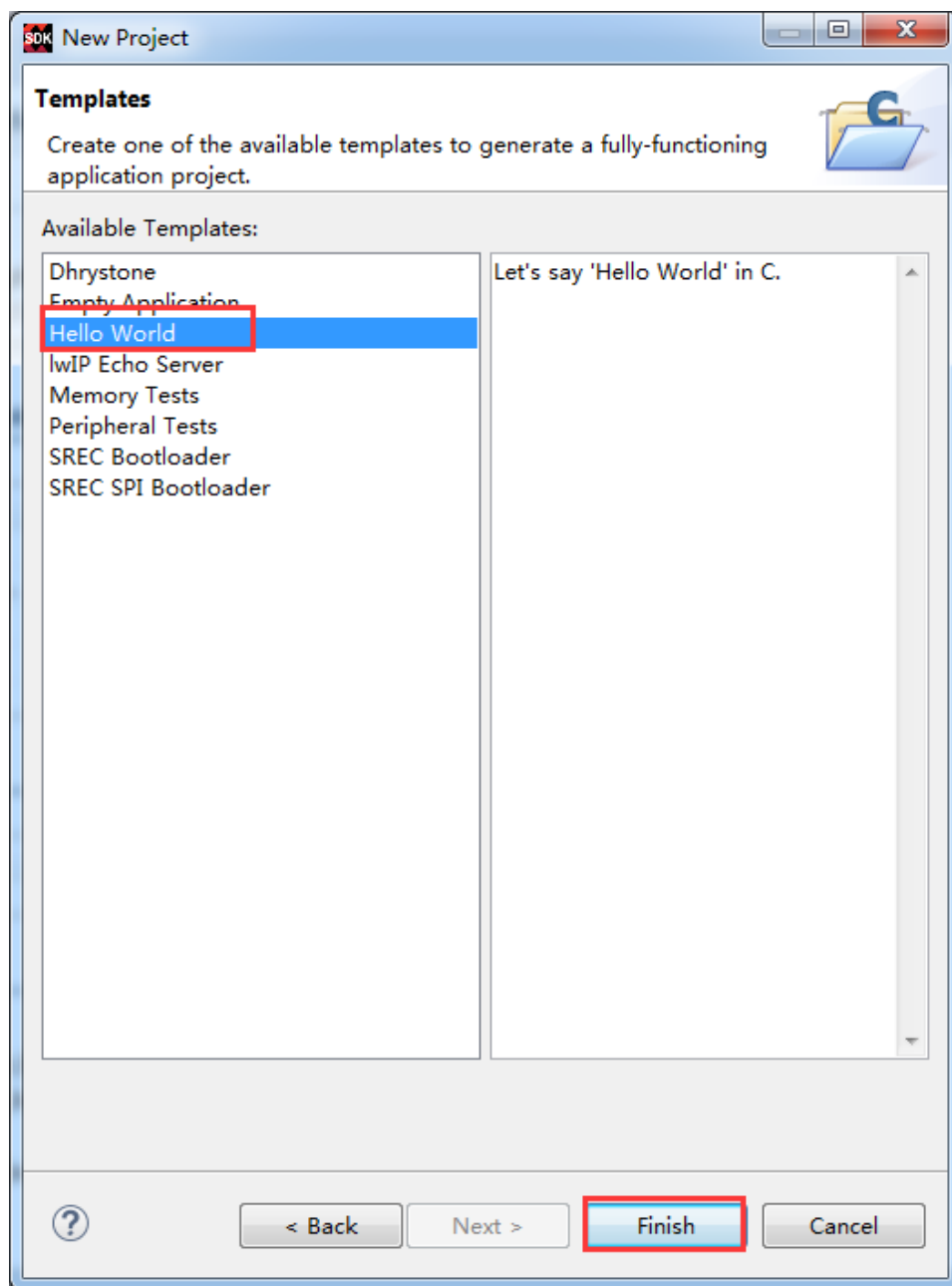
2. 新建工程



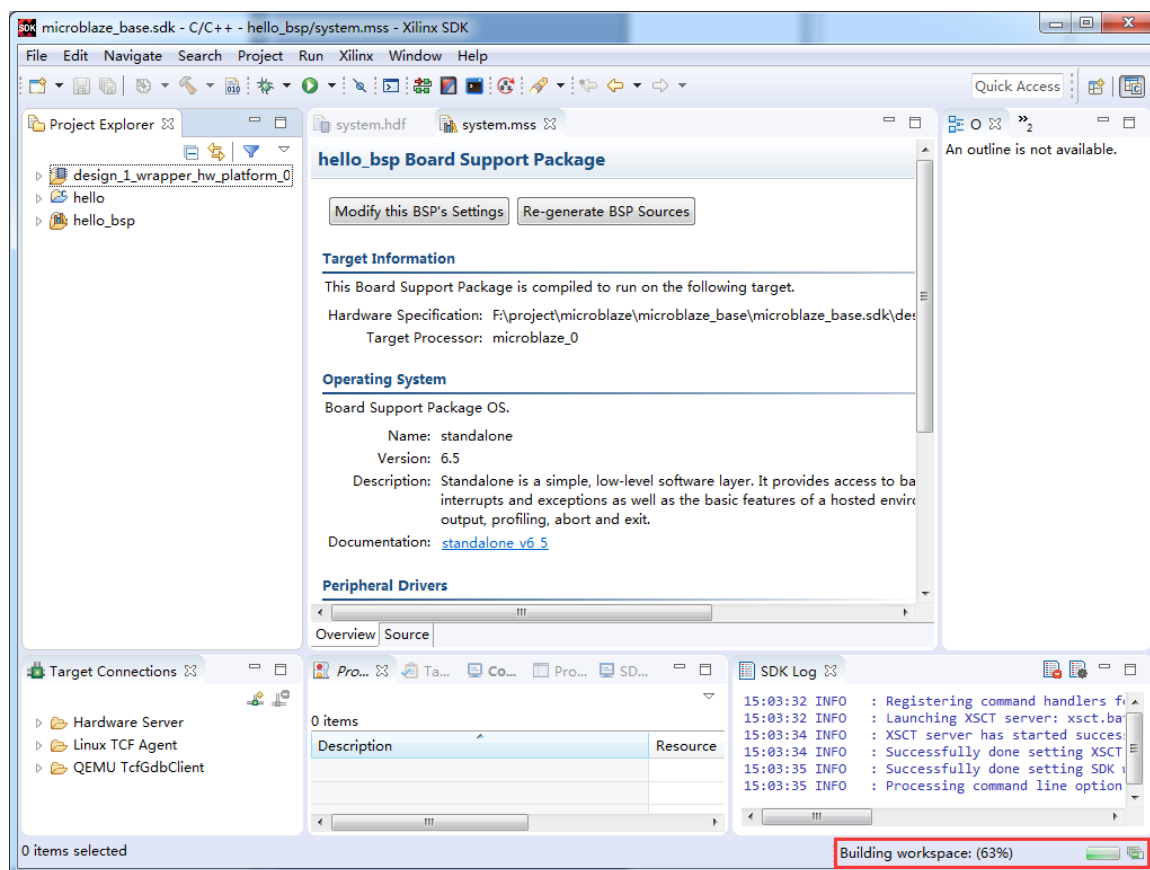
3. 设置工程名为 “hello”，点击 “Next”



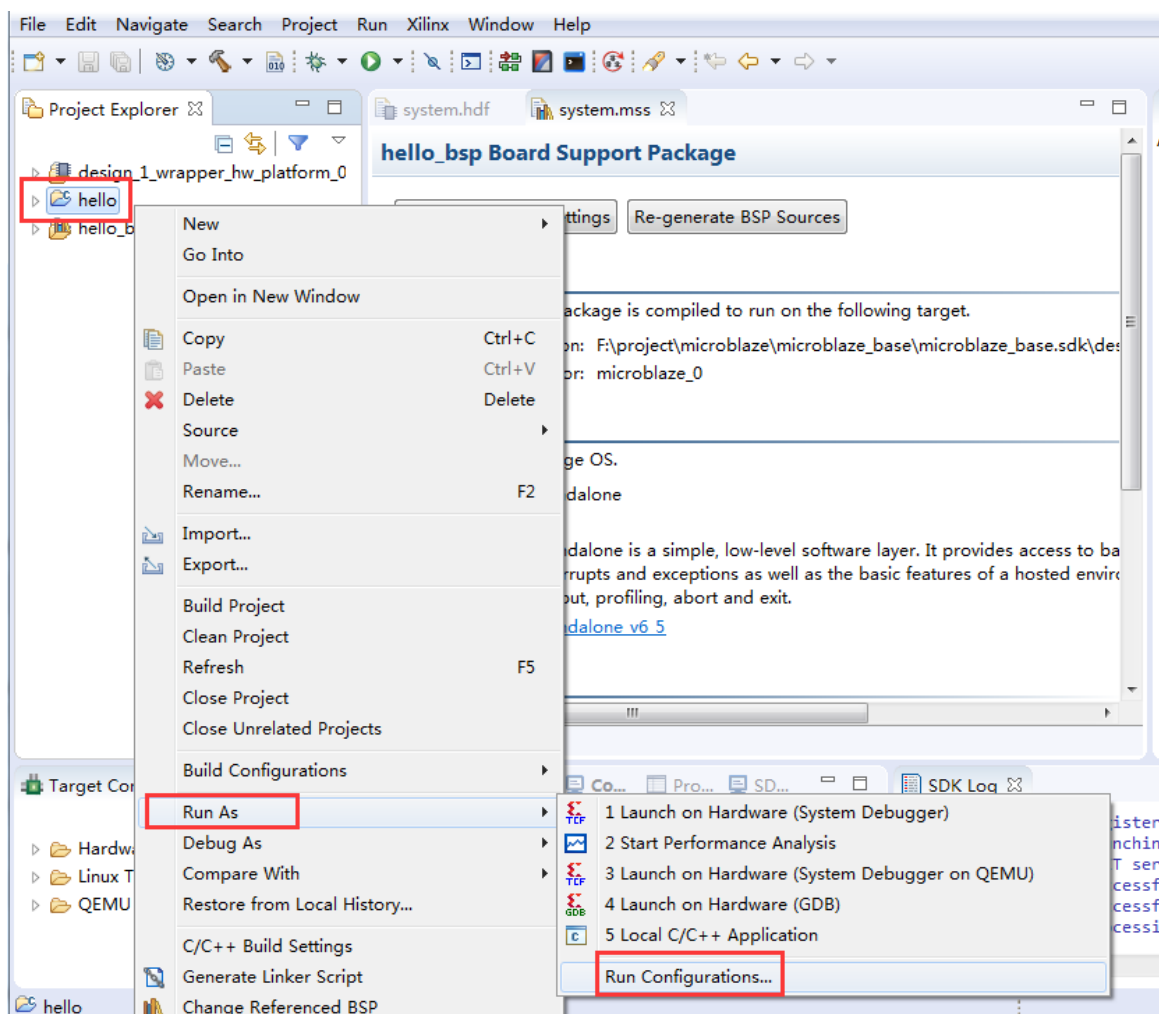
4. 模板选择 “Hello World”，点击 “Finish”



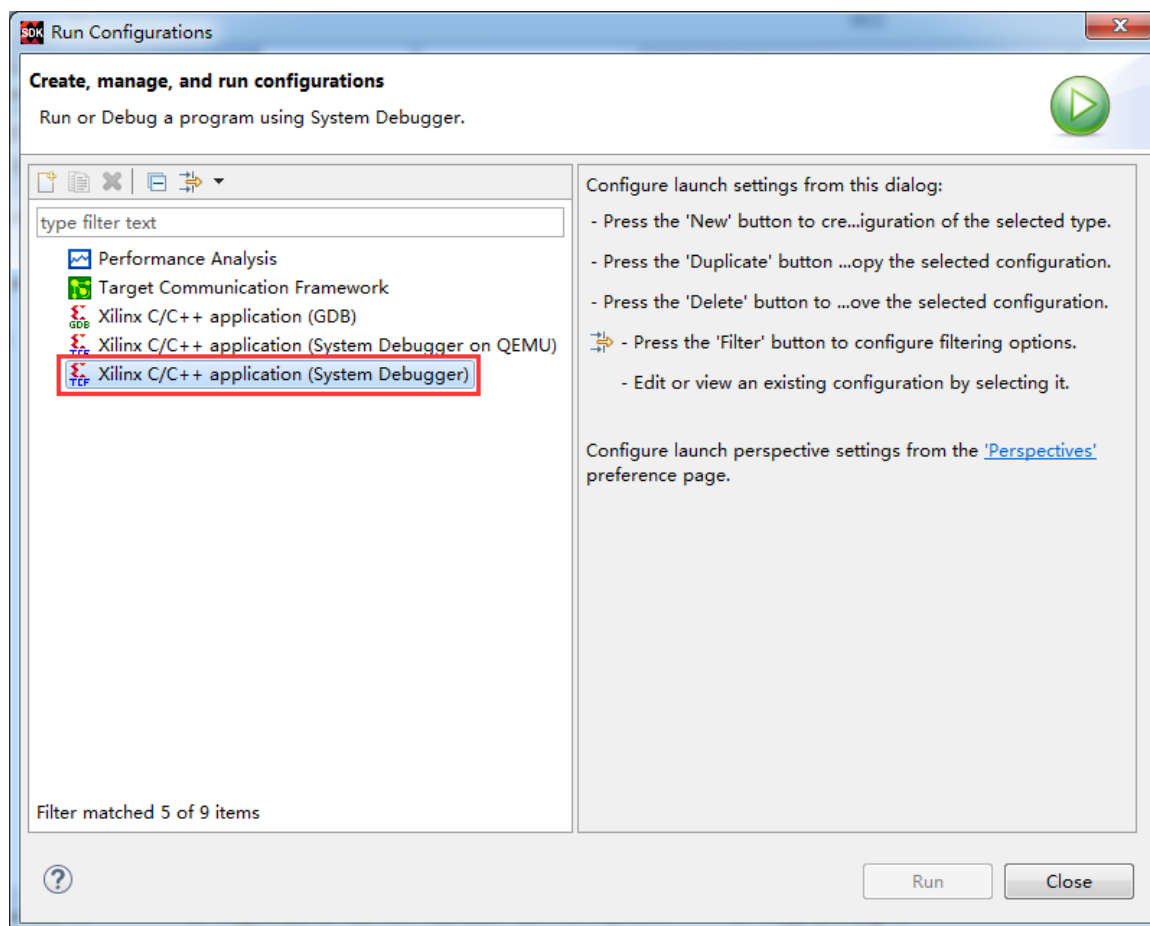
5. 工程建立完成以后就会自动编译，等待编译完成就可以调试运行。（这时开发板要上电，并且 JTAG 连接）



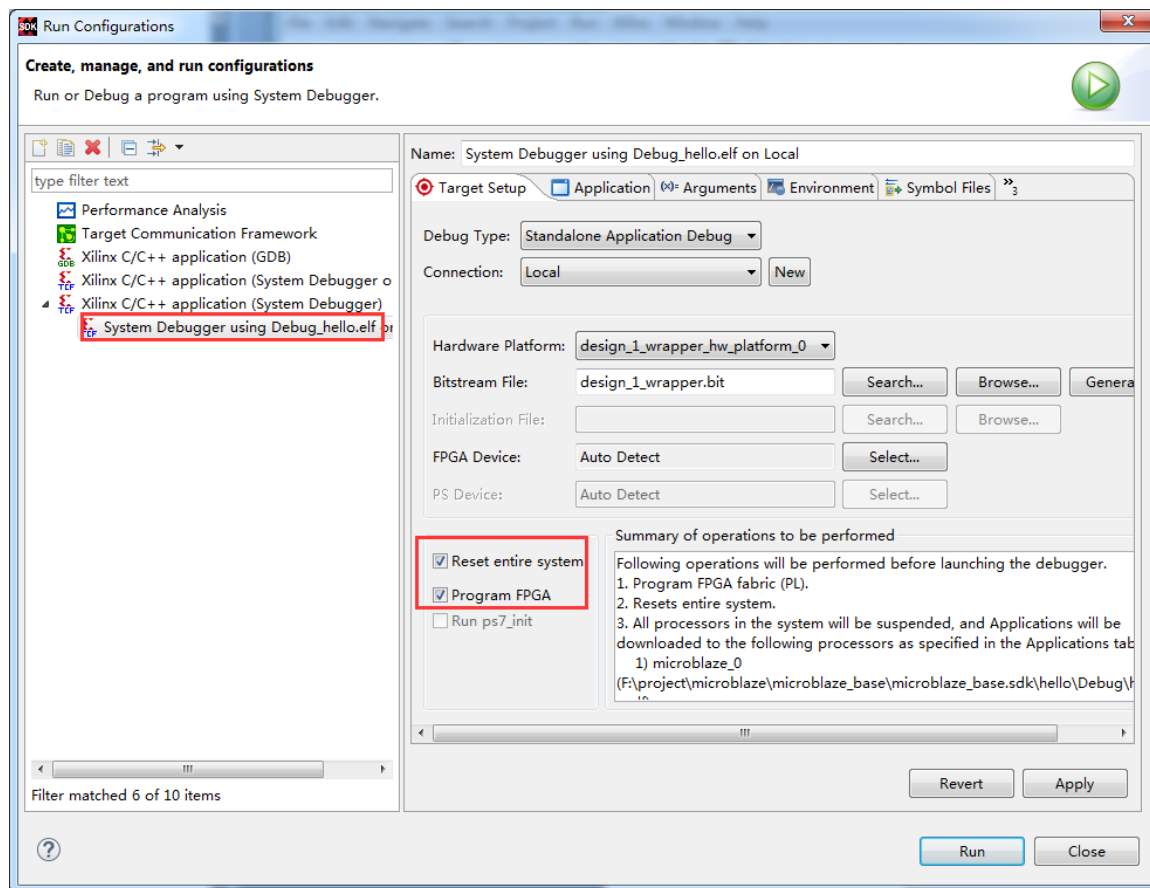
选择“Hello”工程，右键选择“Run As -> Run Configuration...”



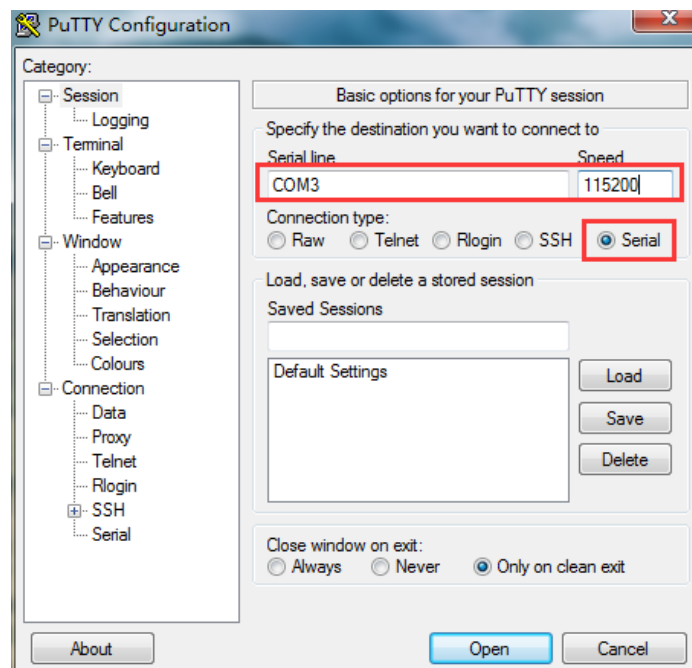
6. 双击 “Xilinx C/C++ application(System Debugger)”



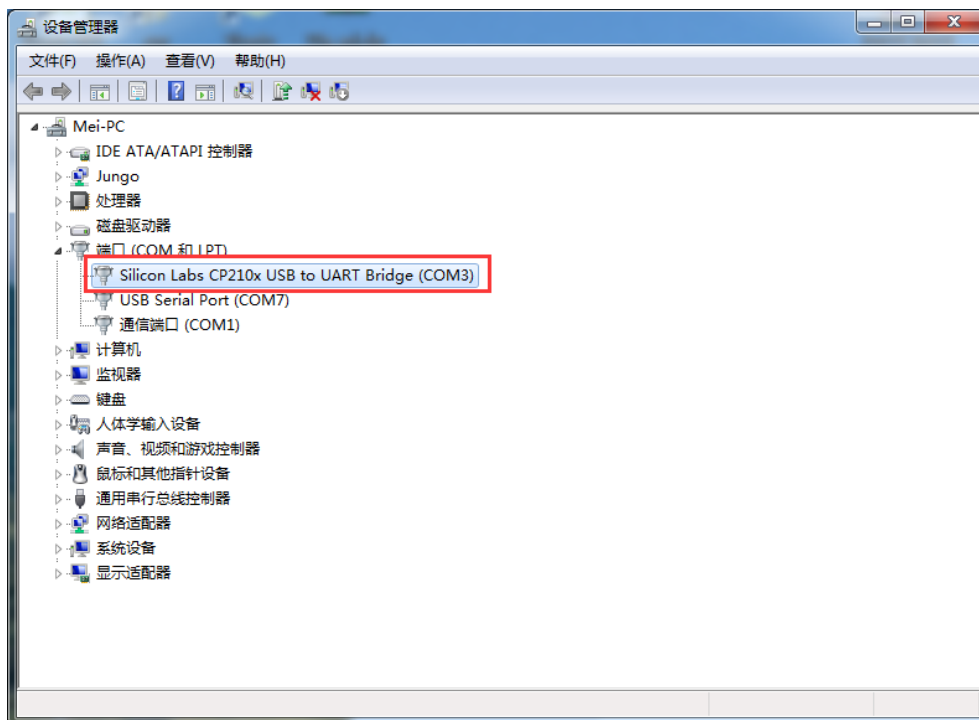
7. 勾选 “Reset entire system” 和 “Program FPGA” 复位整个系统并下载 FPGA



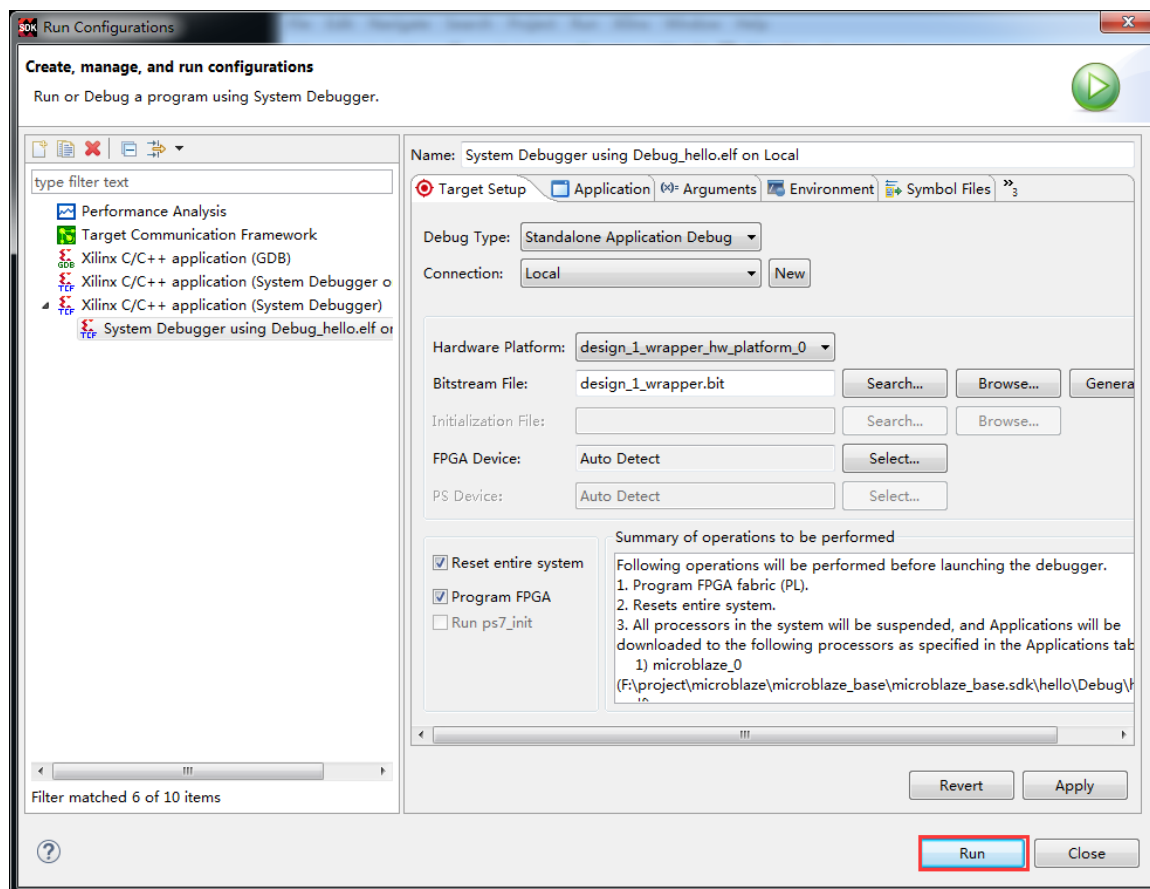
8. 连接 JTAG 线到开发板、UART 的 USB 线到 PC
9. 使用 PuTTY 软件做为串口终端调试工具，PuTTY 是一个免安装的小软件



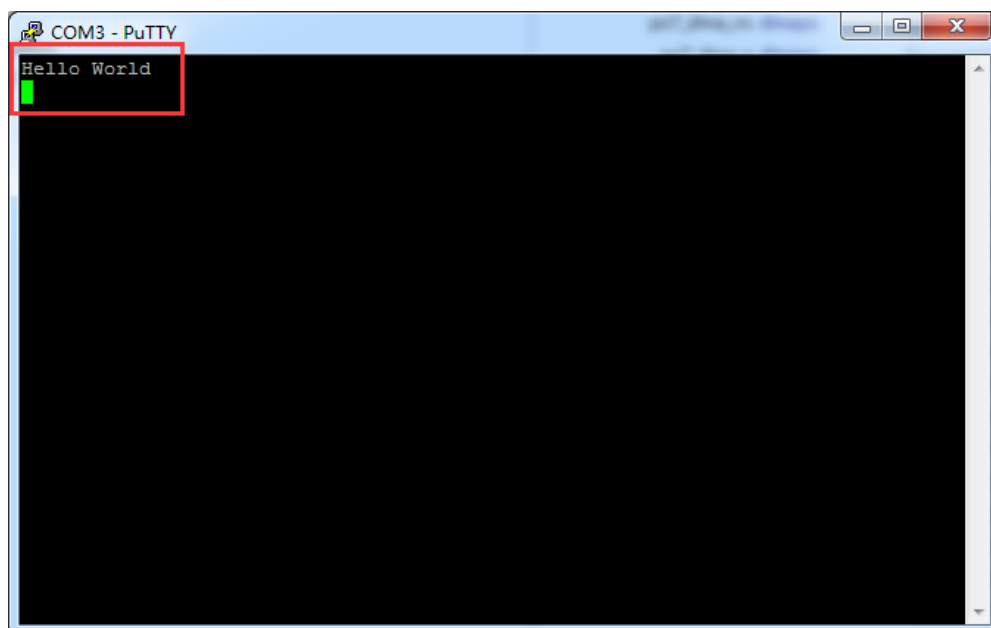
10. 选择 Serial，Serial line 填写 COM3，Speed 填写 115200，COM3 串口号根据设备管理器里显示的填写，点击“Open”



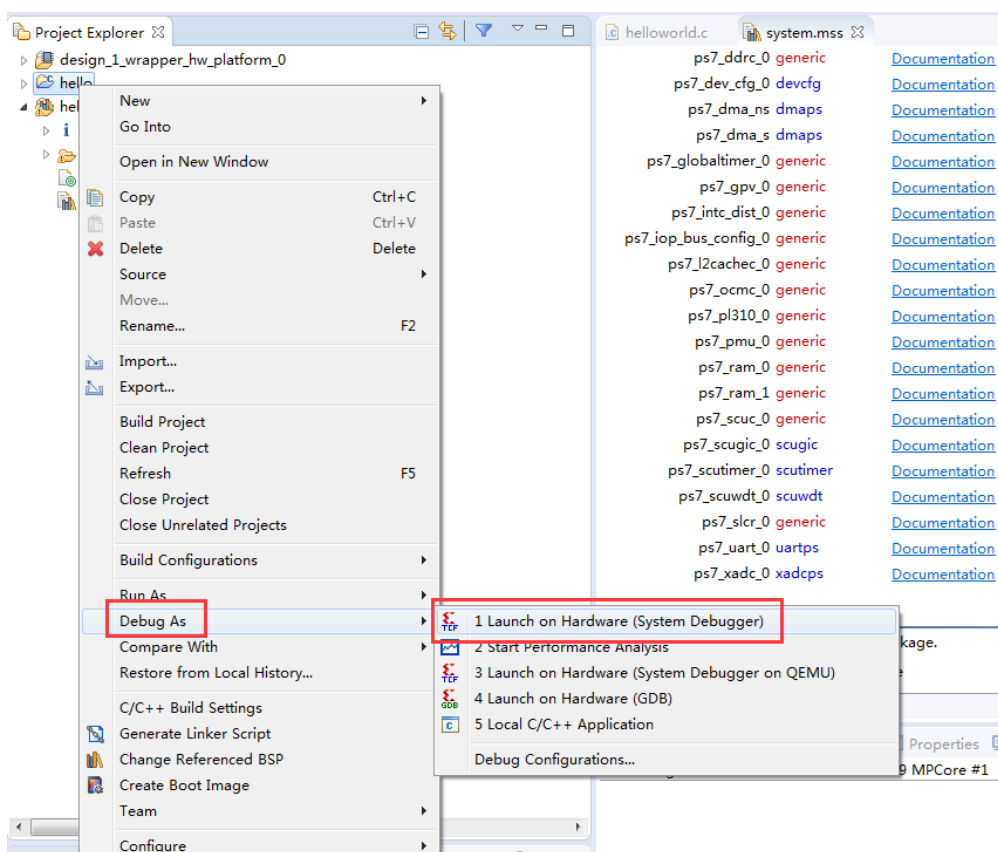
11. 点击 “Run”



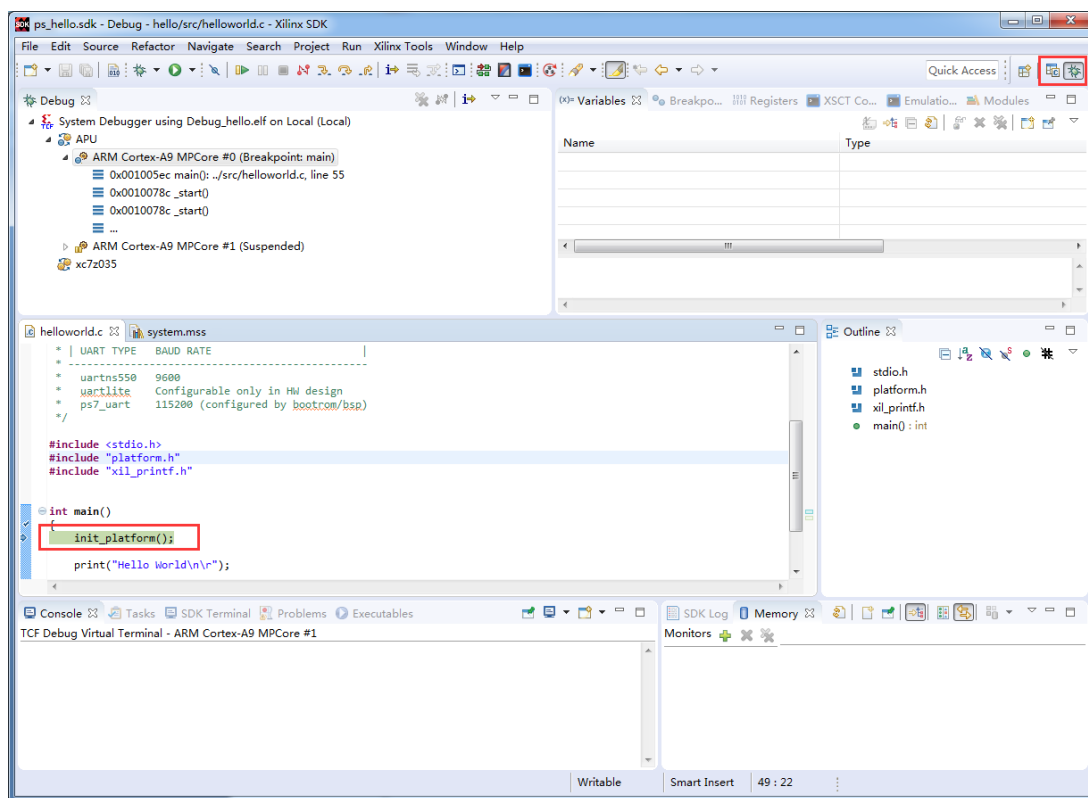
12. “Hello World” 显示出来了



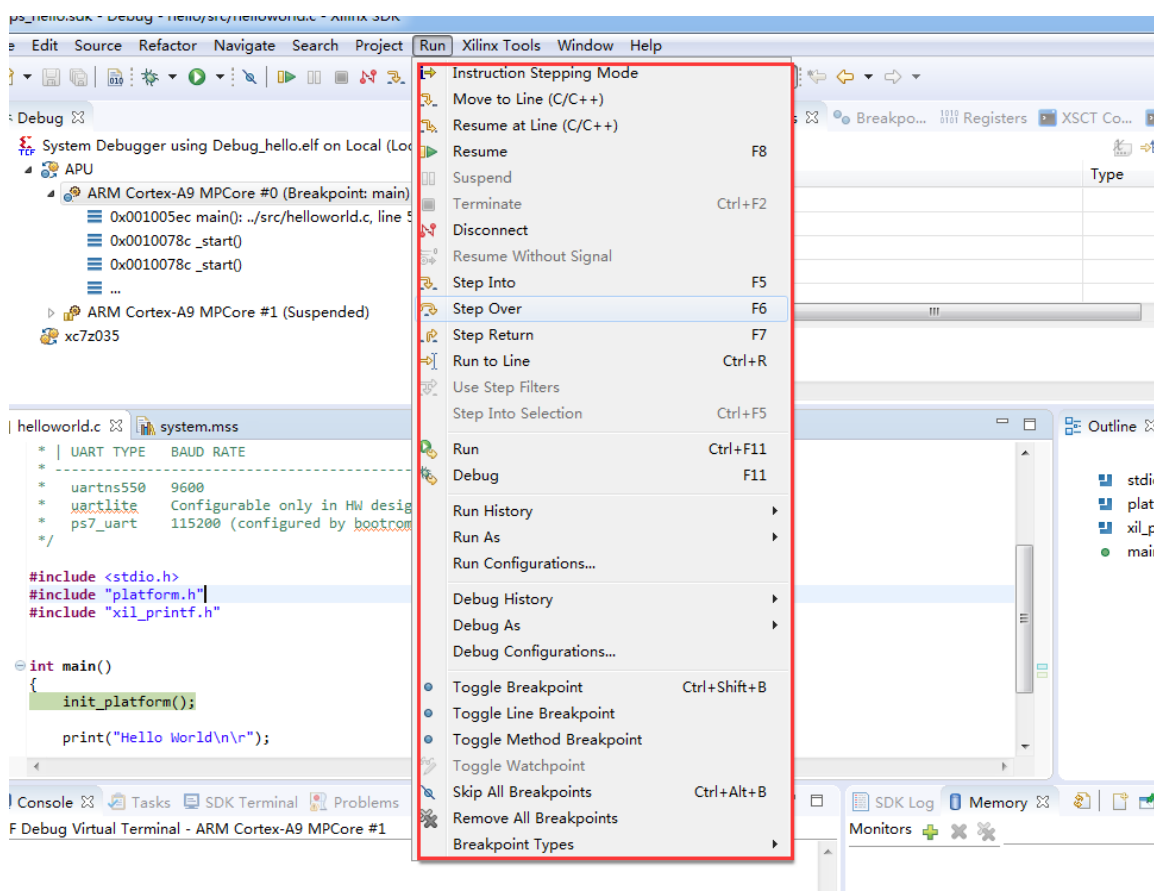
13. 除了 “Run As”, 还可以 “Debug As”, 这样可以设置断点, 单步运行



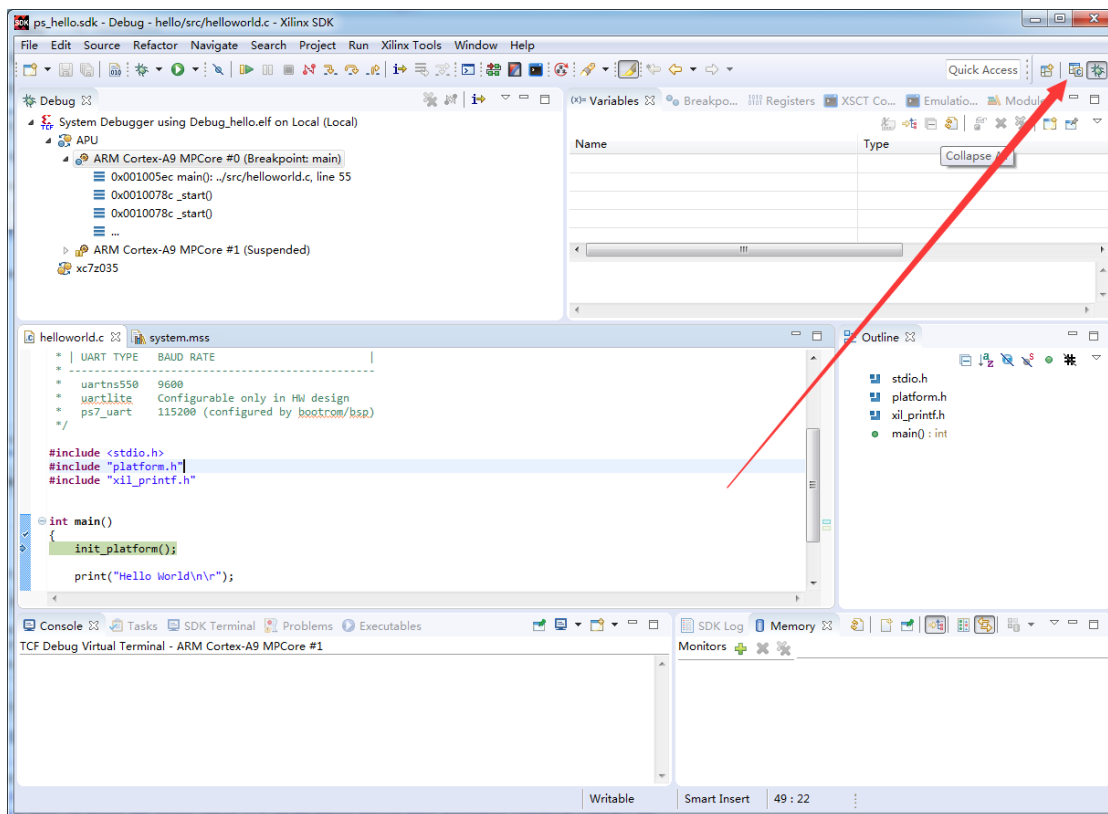
14. 进入 Debug 模式



15. 和其他 C 语言开发 IDE 一样，可以逐步运行、设置断点等

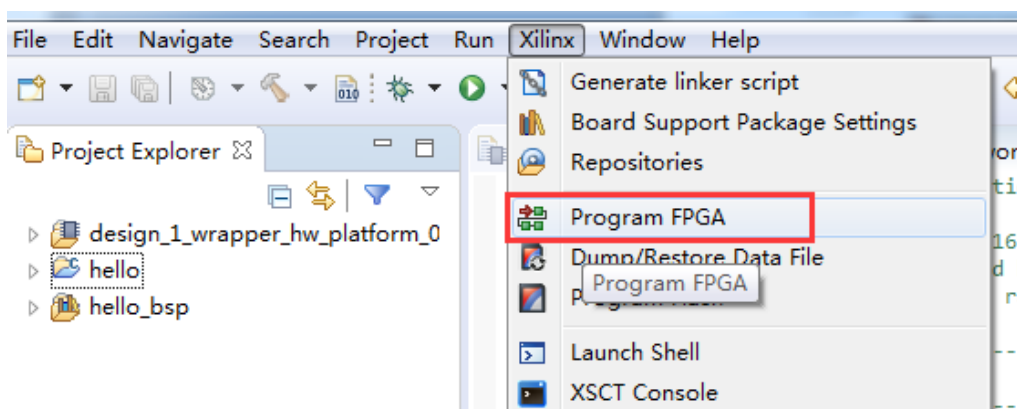


16. 右上角可以切换 IDE 模式

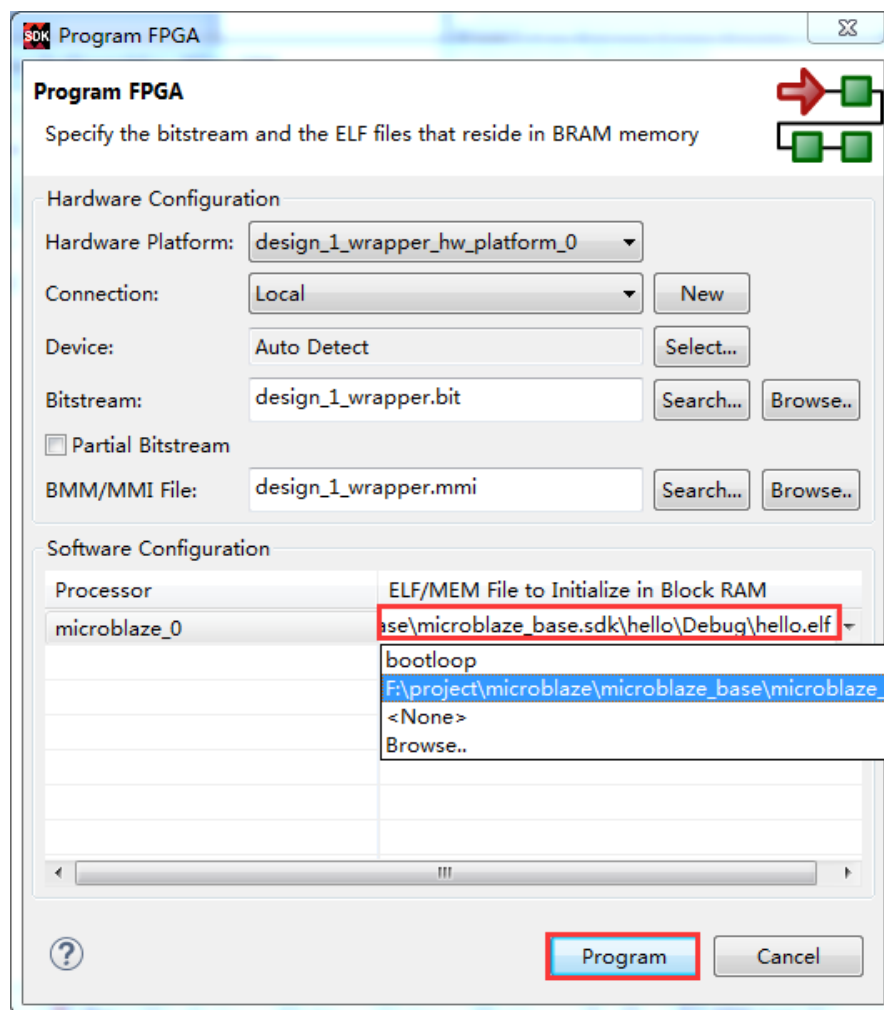


1.3 烧写程序到 Flash

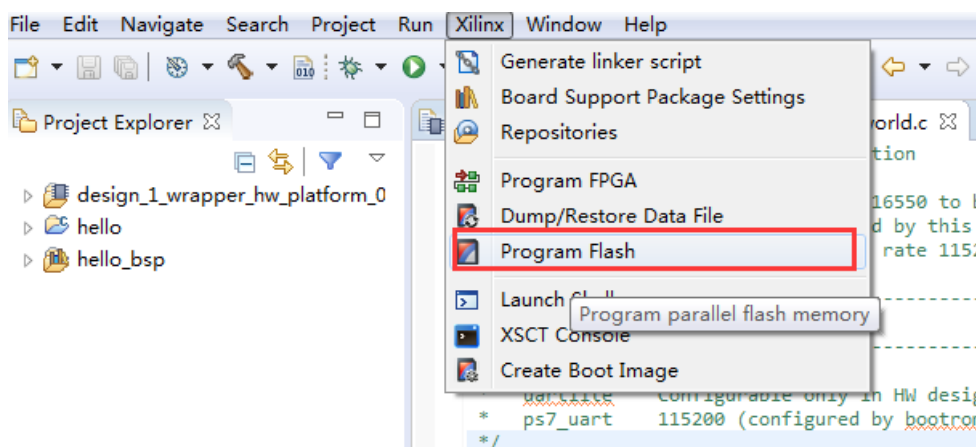
1. 选择 “Xilinx -> Program FPGA”



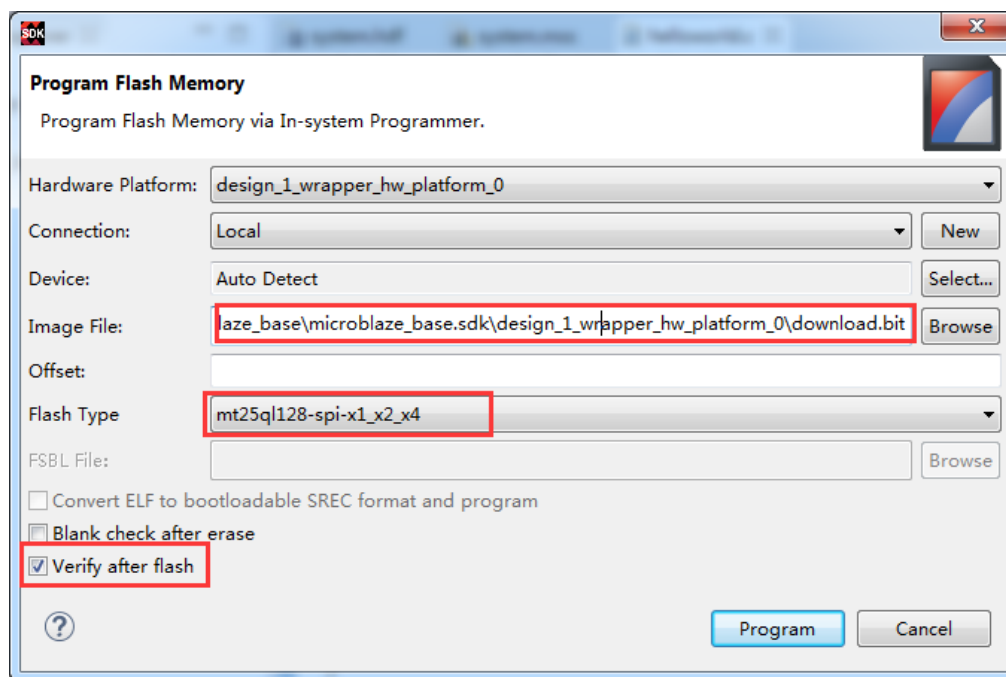
2. 在弹出窗口的 “Software Configuration” 选项的 “microblaze_0” 选择工程
“\microblaze_base.sdk\hello\Debug\hello.elf” 的 elf 文件，把 elf 文件和 bit 文件合并，点击
“Program”



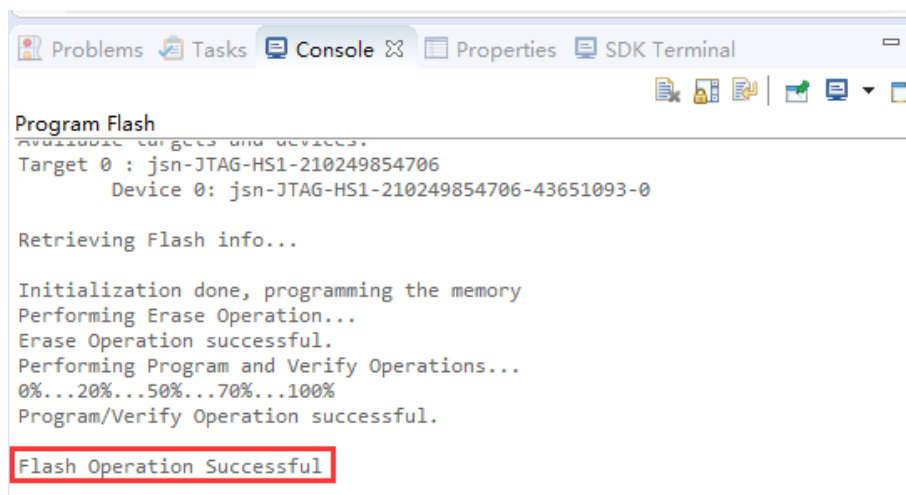
3. 点击 “Xilinx -> Program Flash”



4. “Image File” 选择上一步生成的 “download.bit” 文件，位于目录 “microblaze_base.sdk/design_1_wrapper_hw_platform_0” 中，“Flash Type” 选择 “mt25ql128-spi-x1_x2_x4”，勾选 “Verify after flash”，点击 “Program” 开始烧写 Flash



5. 烧写完成以后，最好拔掉 JTAG 连接线，再重启验证是否烧写成功



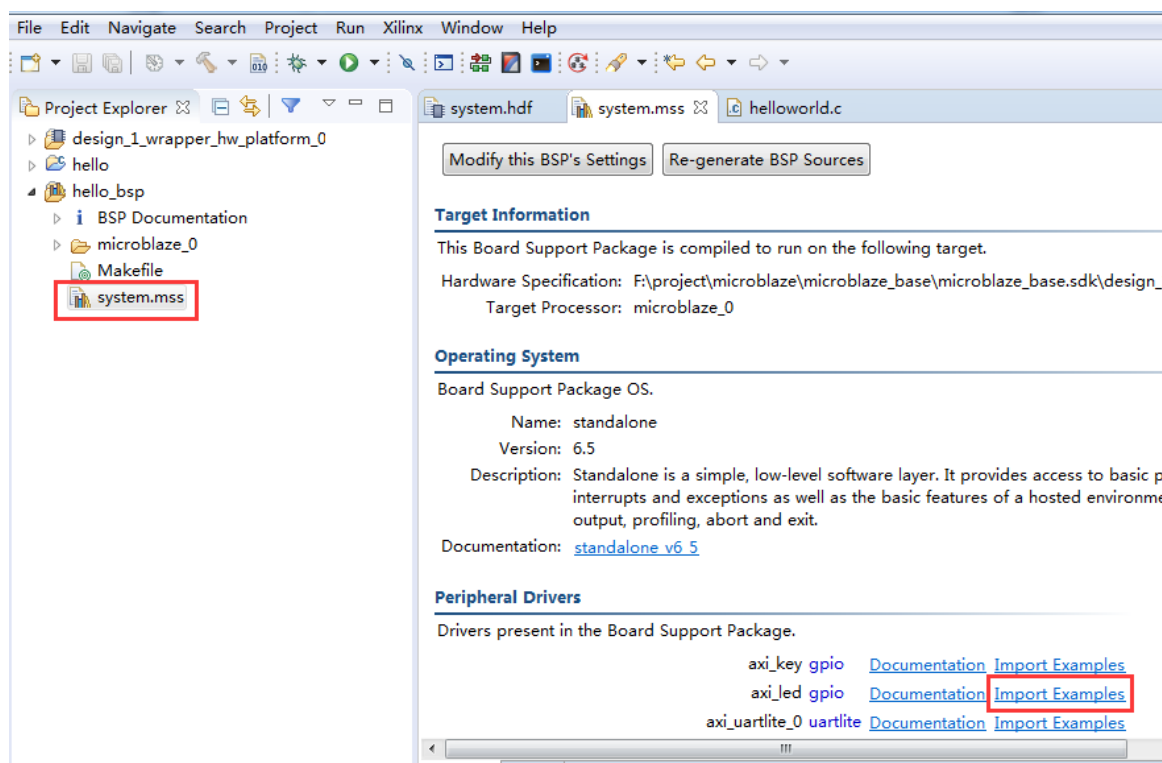
第二章 SDK 编程开发

前面章节介绍了如何搭建一个简单的 Microblaze 硬件系统，并用 Helloworld 程序进行了简单测试，本章学习如何使用 Xilinx 的 BSP 编写 SDK 程序，如何查找相关文档，但是如果没有很好的 C 语言基础，也不能很好掌握本章内容，建议学习者熟练掌握 C 语言再来学习。

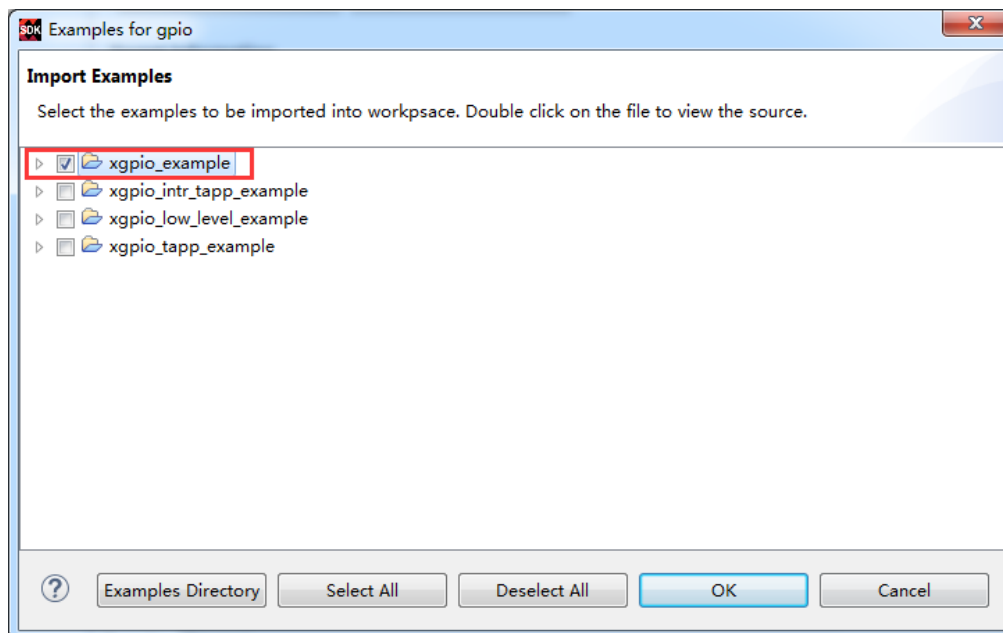
2.1 GPIO 控制点亮 LED

如果让我们编写一个程序，使用 GPIO 去驱动一个 LED，学过单片机可能都会想到库函数，或者想到使用寄存器直接控制。下面让我们看看使用 SDK 如何实现？

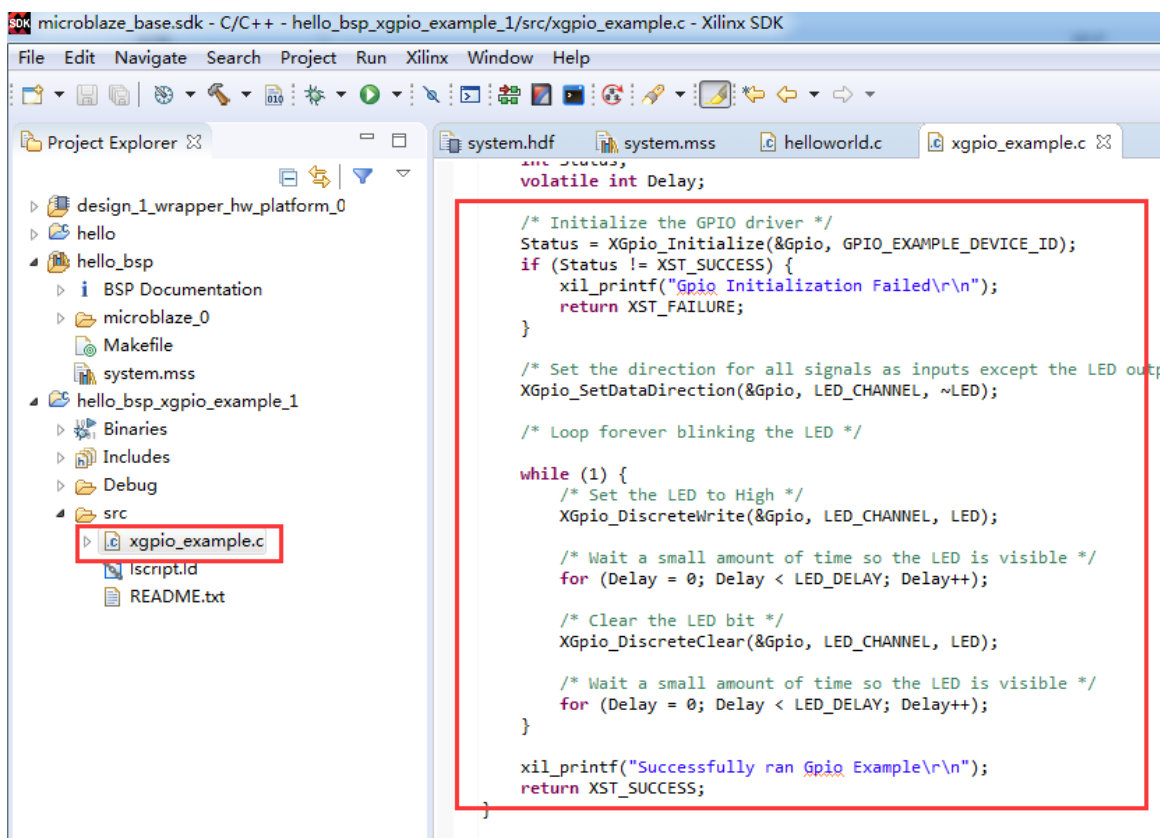
1. 双击 BSP 目录里的“system.mss”文件，点击 axi_led 旁的 “Import Examples”



2. 选择 “xgpio_example”



3. 生成了一个“hello_bsp_xgpio_example_1”工程，双击“xgpio_example.c”，可以查看源代码，通过阅读代码可以判断是否需要继续学习 C 语言，本例程通俗易懂，通过函数名可以直接判断函数作用。



4. 代码需要修改才能运行，代码通过定义“#define GPIO_EXAMPLE_DEVICE_ID XPAR_GPIO_0_DEVICE_ID”来确定哪个是 led 的 GPIO，哪个是 key 的 GPIO，在头文件“xparameters.h”中定义了这些 ID，

```

/***** Constant Definitions *****/

#define LED 0x01 /* Assumes bit 0 of GPIO is connected to an LED */

/*
 * The following constants map to the XPAR parameters created in the
 * xparameters.h file. They are defined here such that a user can easily
 * change all the needed parameters in one place.
 */
#define GPIO_EXAMPLE_DEVICE_ID XPAR_GPIO_0_DEVICE_ID

/*
 * The following constant is used to wait after an LED is turned on to make
 * sure that it is visible to the human eye. This constant might need to be
 * tuned for faster or slower processor speeds.
 */
#define LED_DELAY 1000000

/*
 * The following constant is used to determine which channel of the GPIO is
 * used for the LED if there are 2 channels supported.
 */

```

5. 光标移动到 “xparameters.h” 按下 “F3” 按键，可以快速打开这个头文件，同样的方法可以找到变量的定义

```

*
* </pre>
*****
/***** Include Files *****/

#include "xparameters.h"
#include "xgpio.h"
#include "xil_printf.h"

/***** Constant Definition *****/

#define LED 0x01 /* Assumes bit 0 of GPIO is
 */
 * The following constants map to the XPAR para
 * xparameters.h file. They are defined here su

```

6. 在 “xparameters.h” 文件找到 LED 的 ID 宏定义为 “XPAR_AXI_LED_DEVICE_ID”

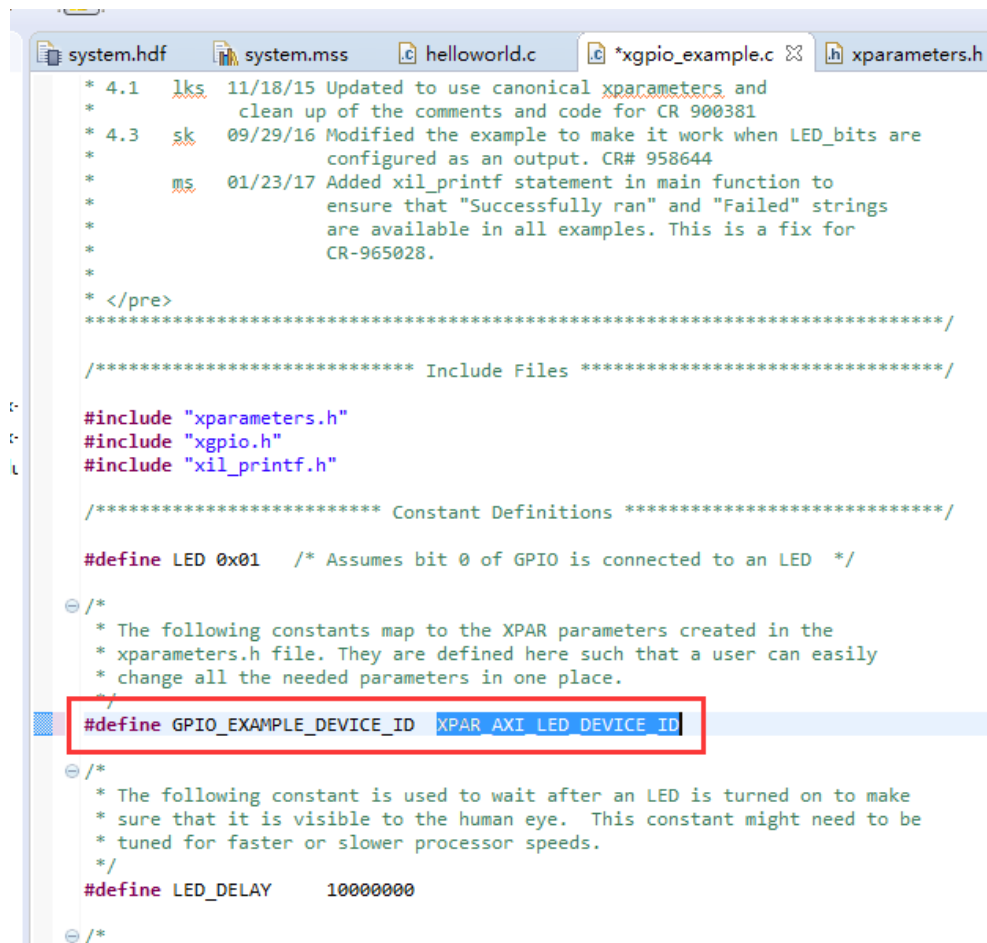
```

/* Canonical definitions for peripheral AXI_KEY */
#define XPAR_GPIO_0_BASEADDR 0x40010000
#define XPAR_GPIO_0_HIGHADDR 0x4001FFFF
#define XPAR_GPIO_0_DEVICE_ID XPAR_AXI_KEY_DEVICE_ID
#define XPAR_GPIO_0_INTERRUPT_PRESENT 0
#define XPAR_GPIO_0_IS_DUAL 0

/* Canonical definitions for peripheral AXI_LED */
#define XPAR_GPIO_1_BASEADDR 0x40000000
#define XPAR_GPIO_1_HIGHADDR 0x4000FFFF
#define XPAR_GPIO_1_DEVICE_ID XPAR_AXI_LED_DEVICE_ID
#define XPAR_GPIO_1_INTERRUPT_PRESENT 0
#define XPAR_GPIO_1_IS_DUAL 0

```

7. 修改 xgpio_example.c 文件中 GPIO 宏定义为 “#define GPIO_EXAMPLE_DEVICE_ID XPAR_AXI_LED_DEVICE_ID”



```
system.hdf system.mss helloworld.c *xgpio_example.c xparameters.h
* 4.1 lks 11/18/15 Updated to use canonical xparameters and
*         clean up of the comments and code for CR 900381
* 4.3 sk 09/29/16 Modified the example to make it work when LED_bits are
*         configured as an output. CR# 958644
* ms 01/23/17 Added xil_printf statement in main function to
*         ensure that "Successfully ran" and "Failed" strings
*         are available in all examples. This is a fix for
*         CR-965028.
*
* </pre>
*****/

/***** Include Files *****/

#include "xparameters.h"
#include "xgpio.h"
#include "xil_printf.h"

/***** Constant Definitions *****/

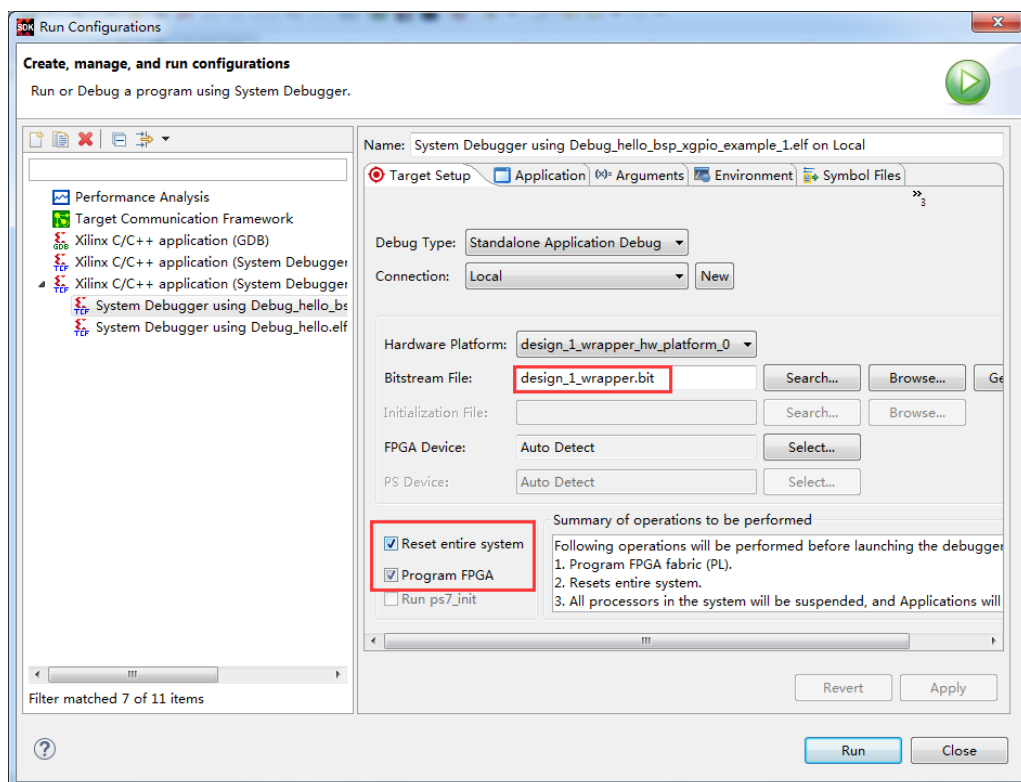
#define LED 0x01 /* Assumes bit 0 of GPIO is connected to an LED */

/*
 * The following constants map to the XPAR parameters created in the
 * xparameters.h file. They are defined here such that a user can easily
 * change all the needed parameters in one place.
 */
#define GPIO_EXAMPLE_DEVICE_ID XPAR_AXI_LED_DEVICE_ID

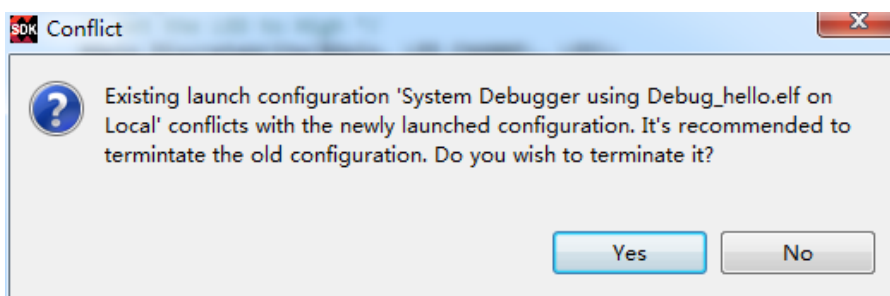
/*
 * The following constant is used to wait after an LED is turned on to make
 * sure that it is visible to the human eye. This constant might need to be
 * tuned for faster or slower processor speeds.
 */
#define LED_DELAY 1000000

/*
```

8. 运行配置，需要注意 Bitstream File 千万不要选错，默认可能是前面生成的 download.bit，这个文件是不正确的，要选择 design_1_wrapper.bit 文件



9. 如果出现下面窗口提示，选择 “Yes”

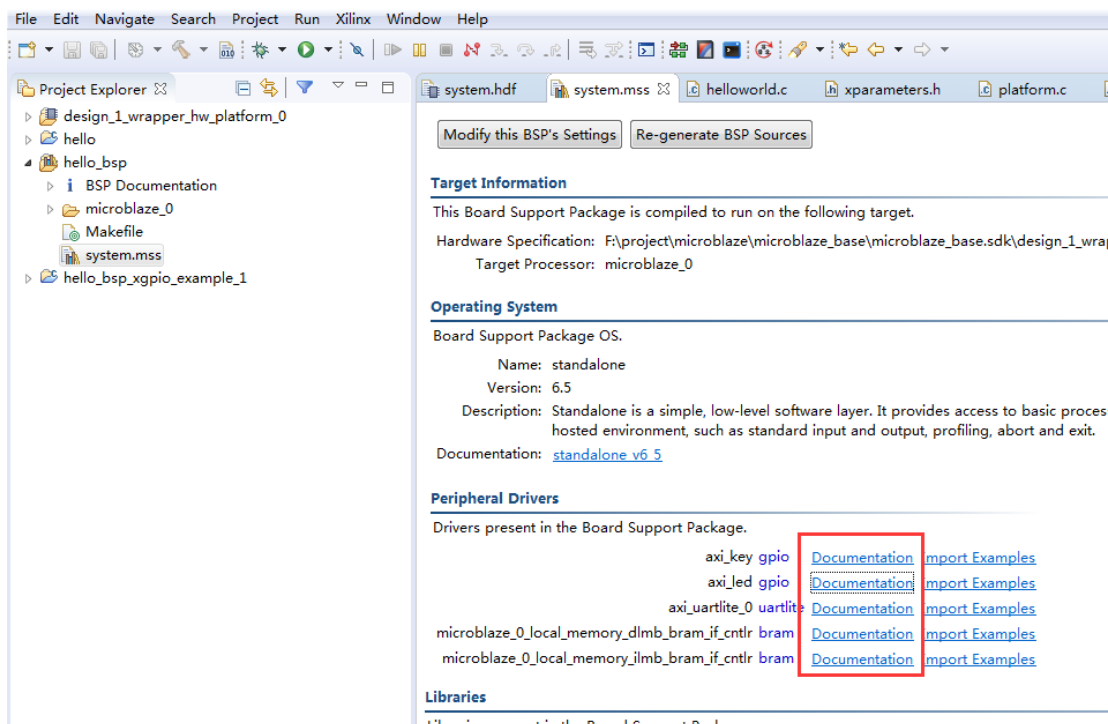


10. 运行后可以看到 LED 在不断闪烁

2.2 SDK 软件开发技巧

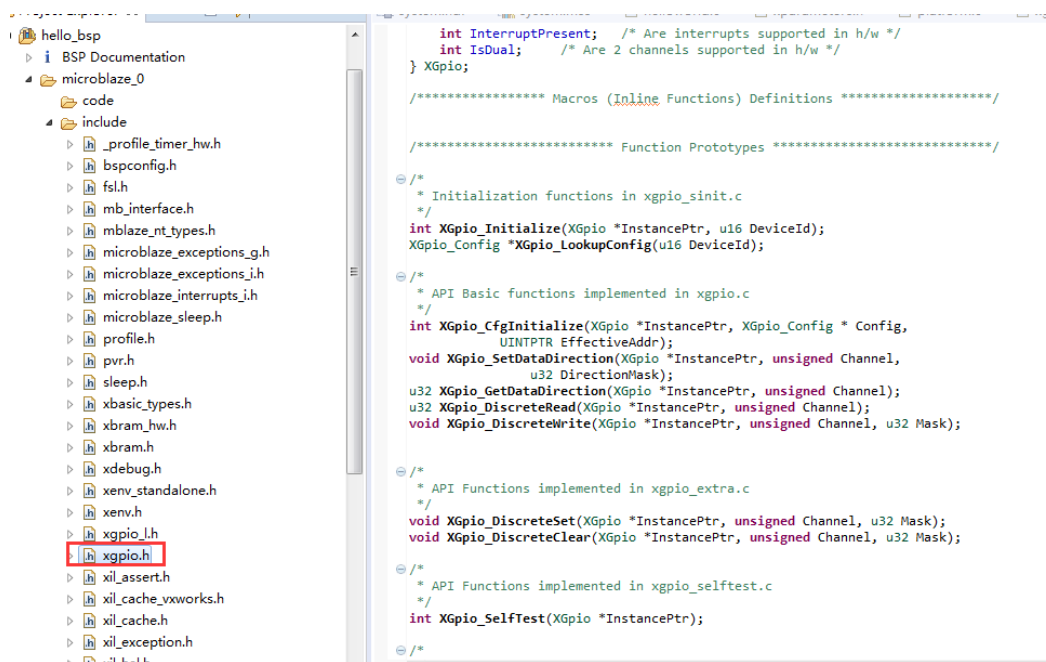
2.2.1 获取帮助文档

1. 双击 bsp 目录的 system.mss 文件，可以选择需要阅读的 IP 核文档

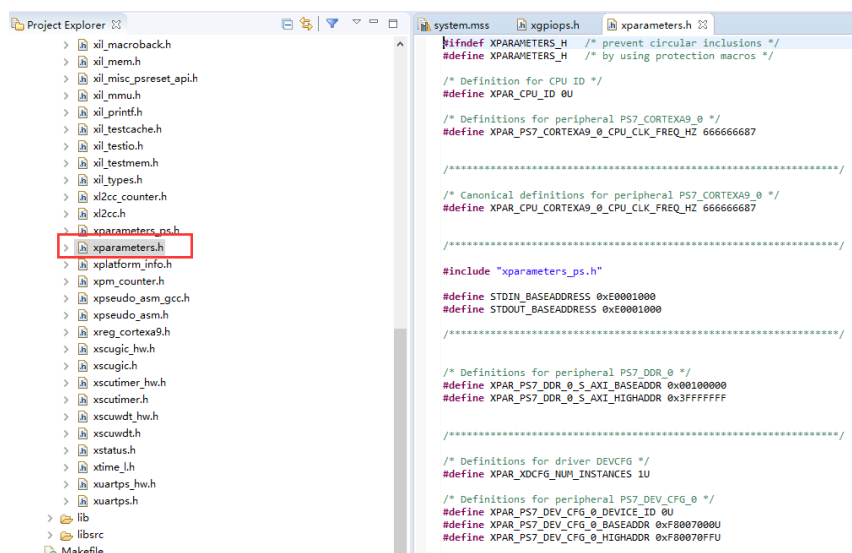


2.2.2 查看 API 函数原型

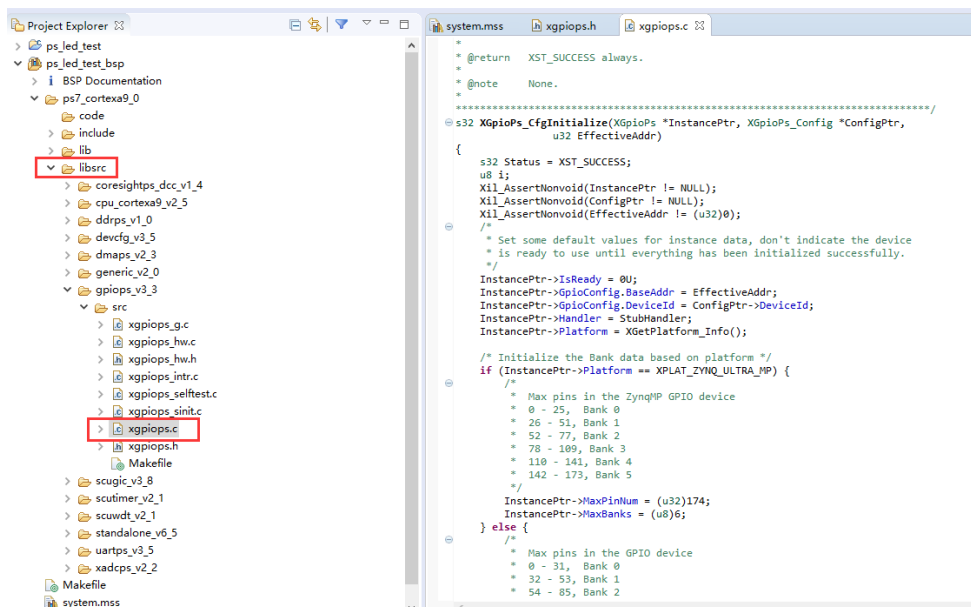
- 在 bsp 的 include 文件夹下包含了 xilinx 的各种头文件，如本章用到的 GPIO，用到了 xgpio.h，在此文件中可以看到各种宏定义，在调用 GPIO 函数时可以使用这些宏定义，提高可读性。同时也包含外设自带的函数声明



- 在 xparameters.h 头文件中定义了各个外设的基地址，器件 ID，中断等

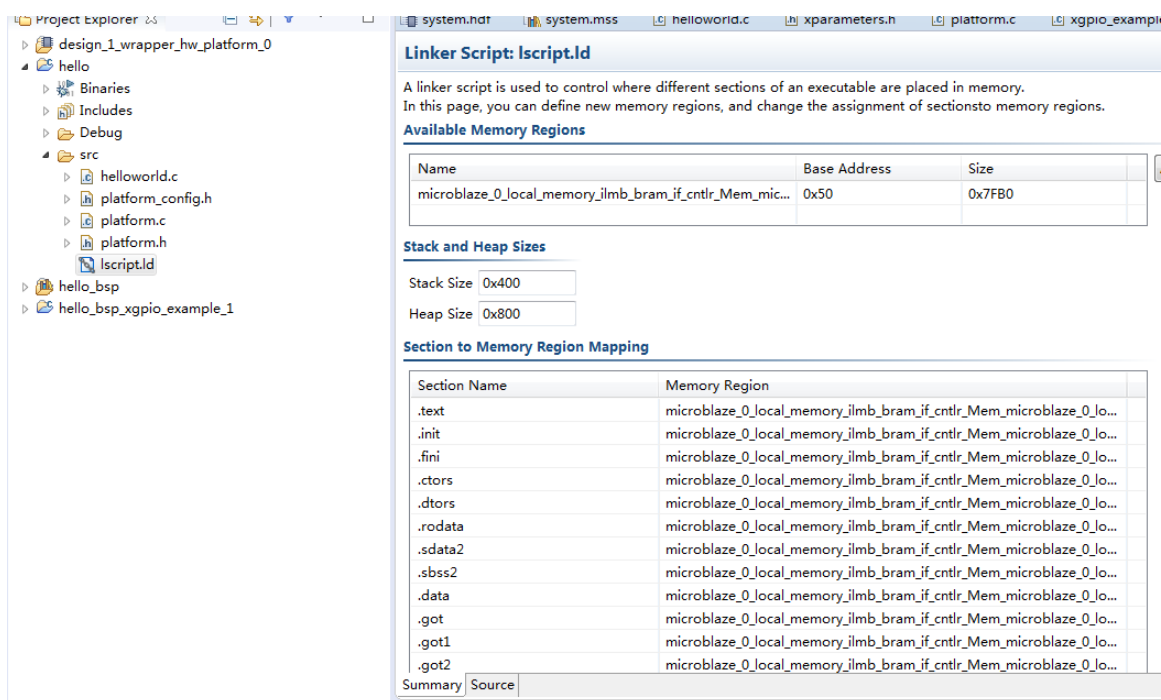


4. 在 libsrc 文件夹中，包含外设函数的定义，使用说明



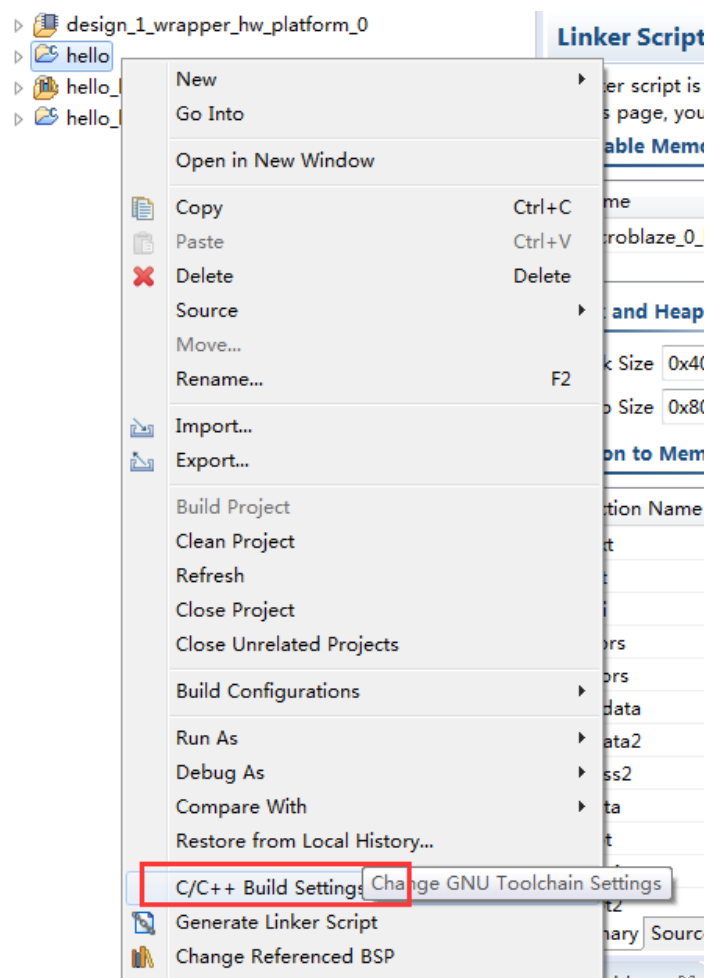
2.2.3 堆栈大小设置

5. 在 src 文件夹下的 Iscript.ld 中，定义了可用 memory 空间，栈和堆空间大小等，可根据需要修改。

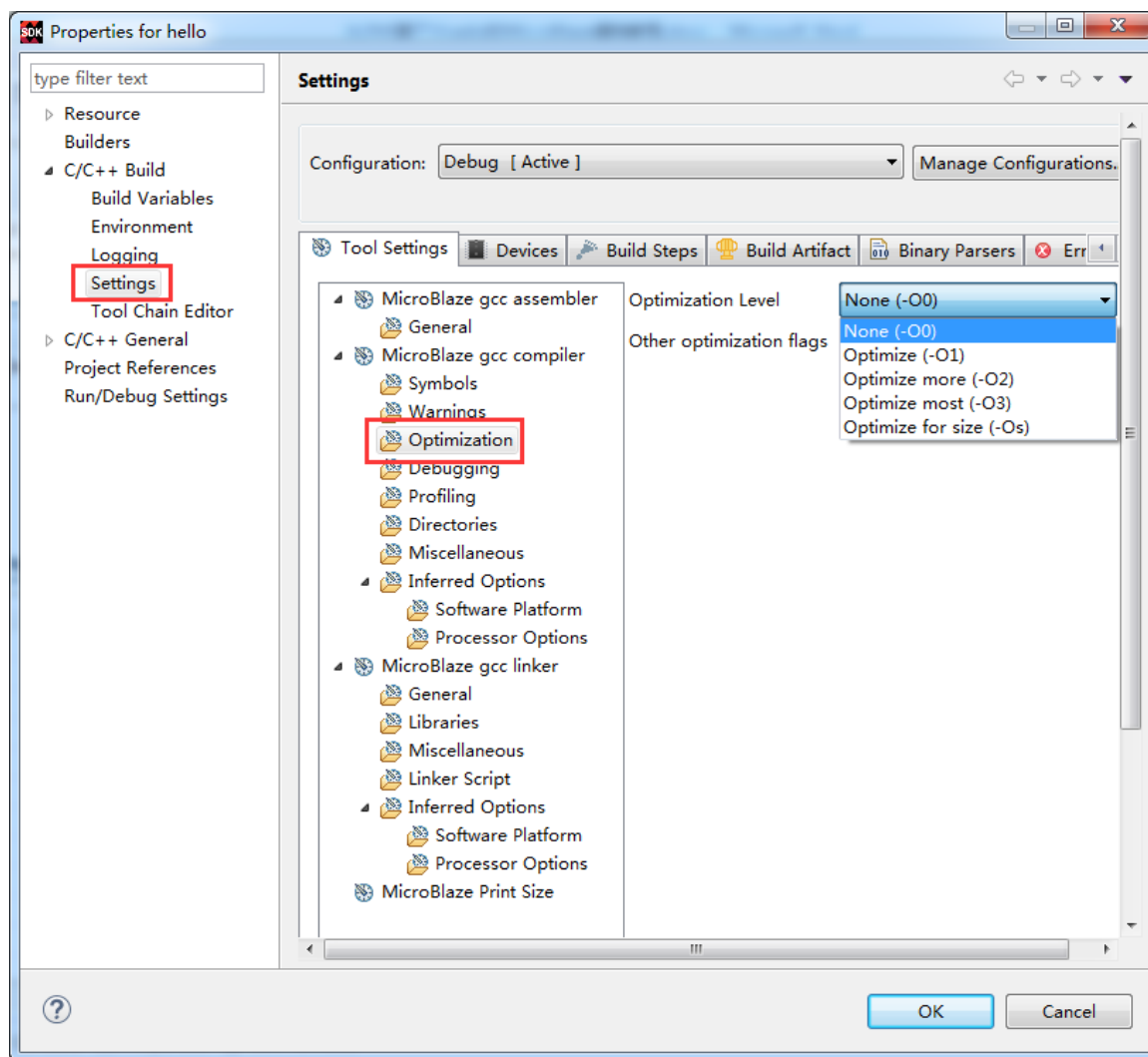


2.2.4 优化等级和调试等级设置

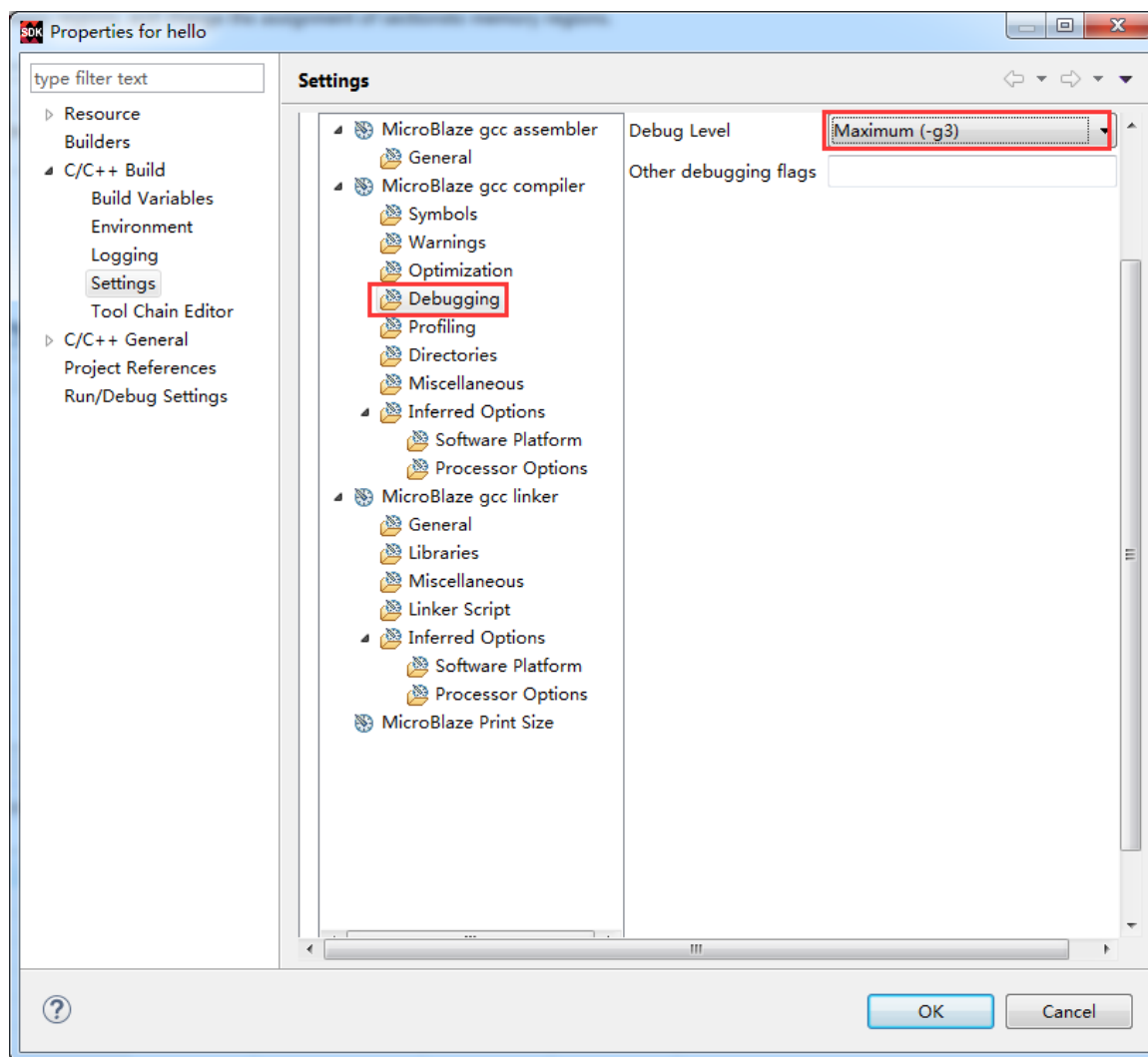
6. 选择工程右键，点击“C/C++ Build Settings”



7. 在 “Optimization” 选项中设置 “Optimization Level”，选择 None(-O0)表示不优化，Optimize for size(-Os)编译出来的代码最小。

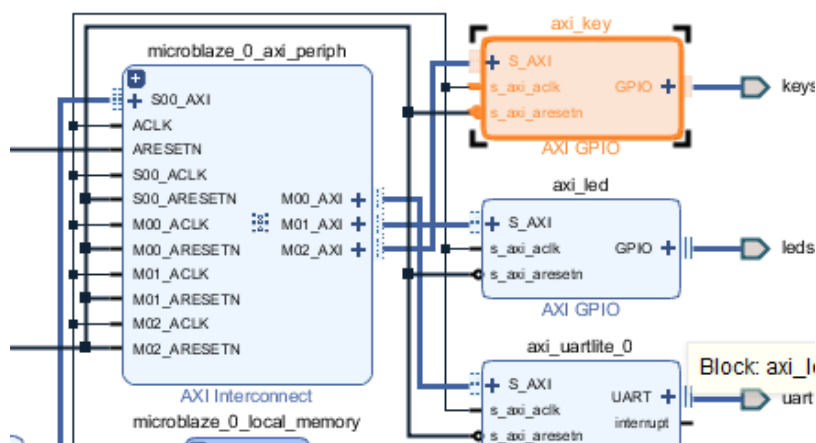


8. 在 “Debugging” 设置中可以设置调试等级

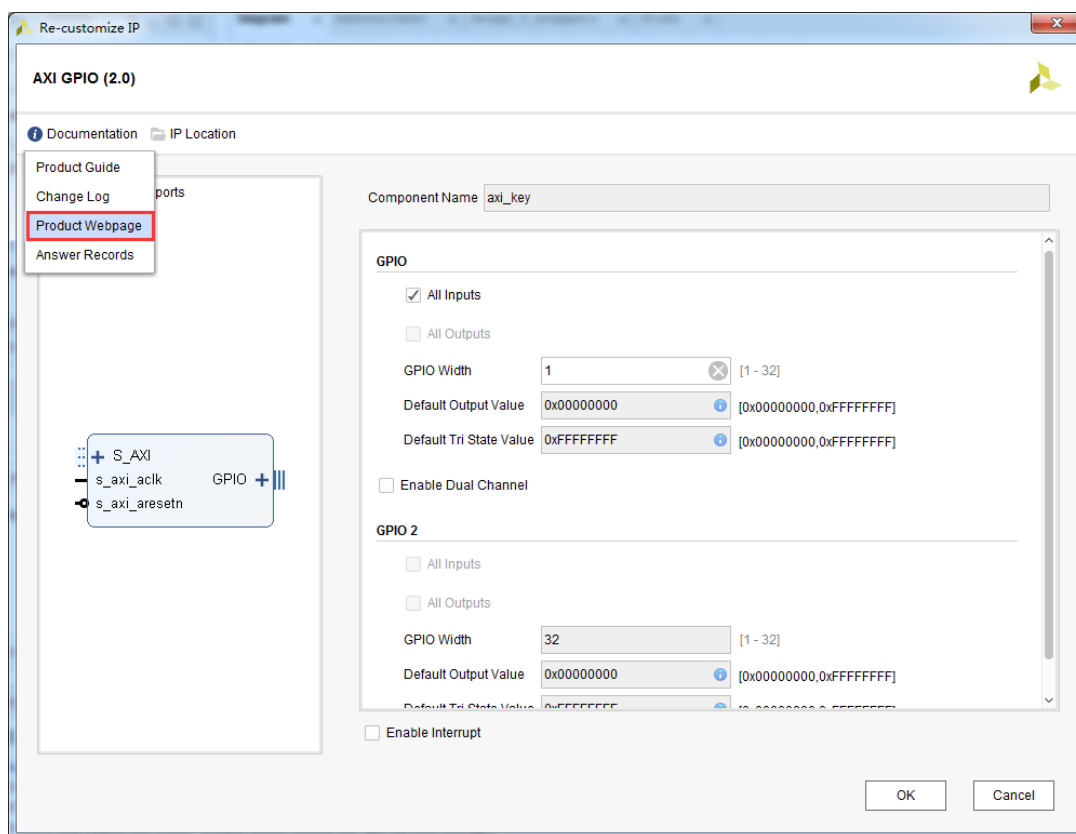


2.2.5 查找 IP 核寄存器设计说明

9. 在 vivado 中双击 IP 核



10. 点击 “Documentation -> Product Webpage” 打开 IP 核相关网页，可以找到 IP 核详细资料



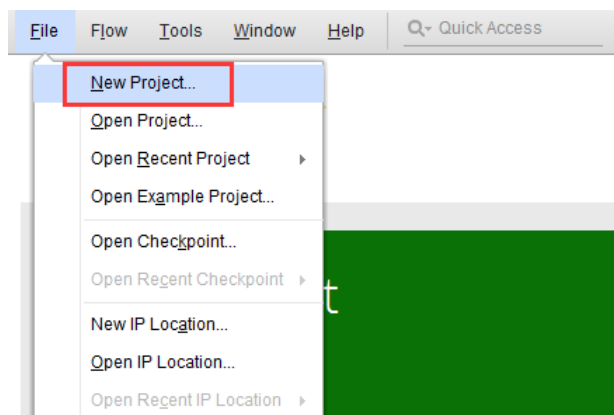
第三章 带 mig(ddr3)的 MicroBlaze 系统

本章如何使用 Vivado 软件建立一个 MicroBlaze 最小系统，使用 FPGA 内部 Block RAM 作为 CPU 运行 RAM，主要外设有 UART 和 GPIO。为了兼顾那些不按教程顺序学习者，本章叙述略显啰嗦。

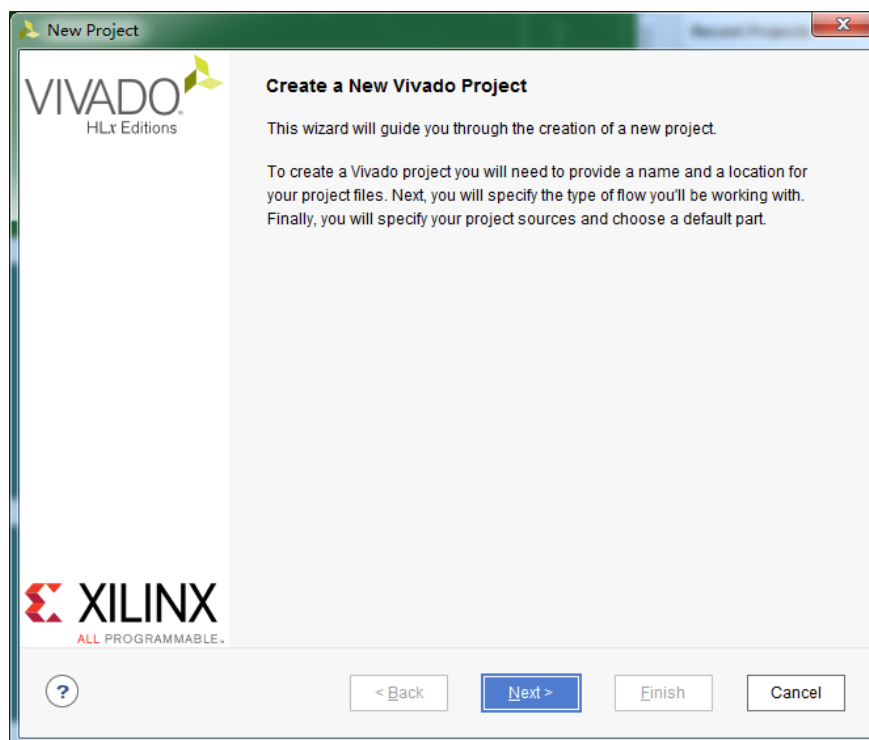
3.1 建立 VIVADO 工程

3.1.1 新建工程

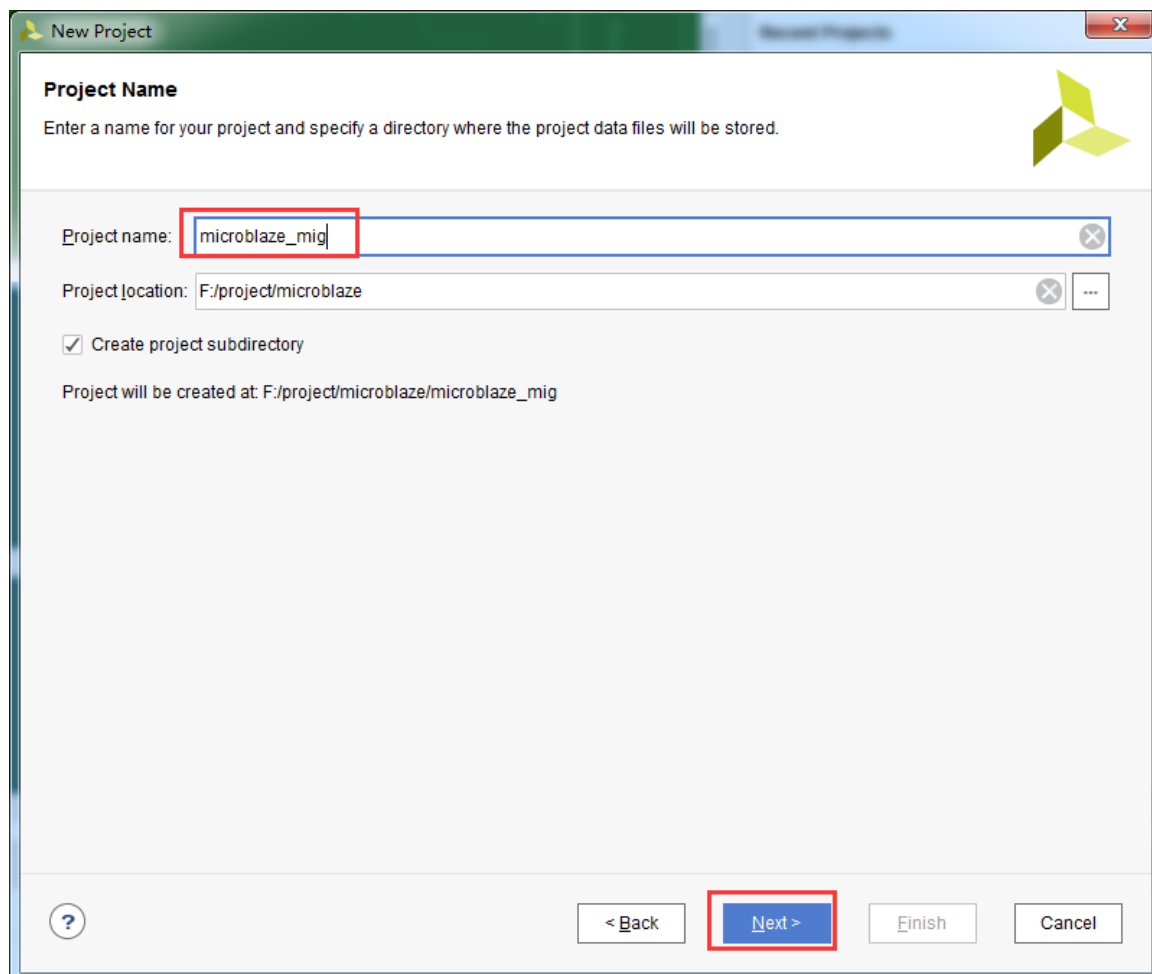
1. 打开 Vivado 软件，新建工程



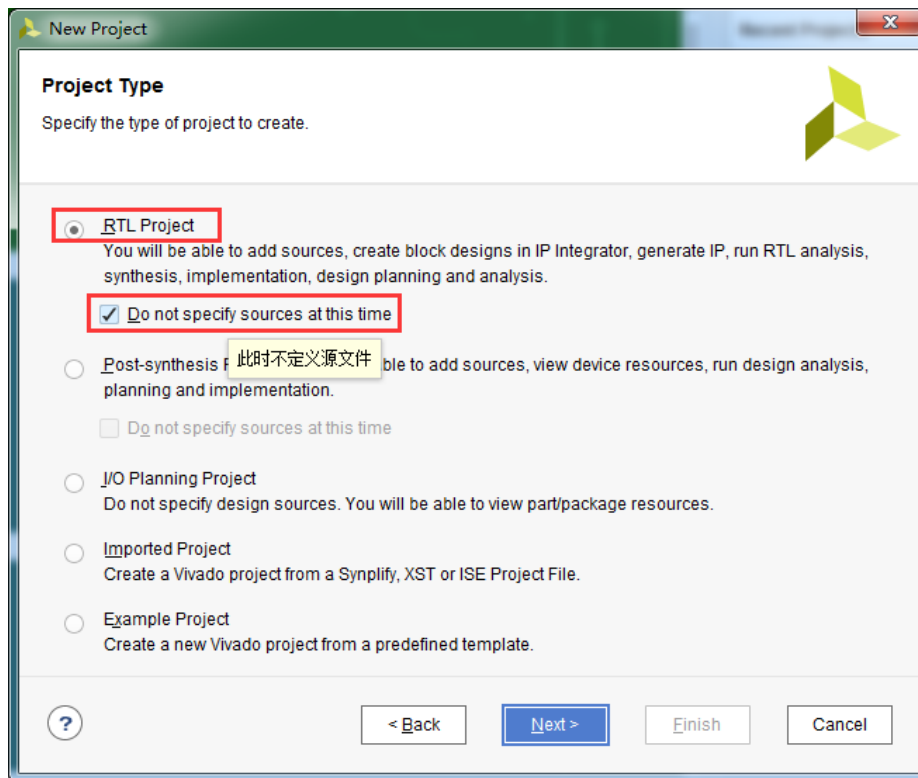
2. 点击 “Next” 创建一个新的 Vivado 工程



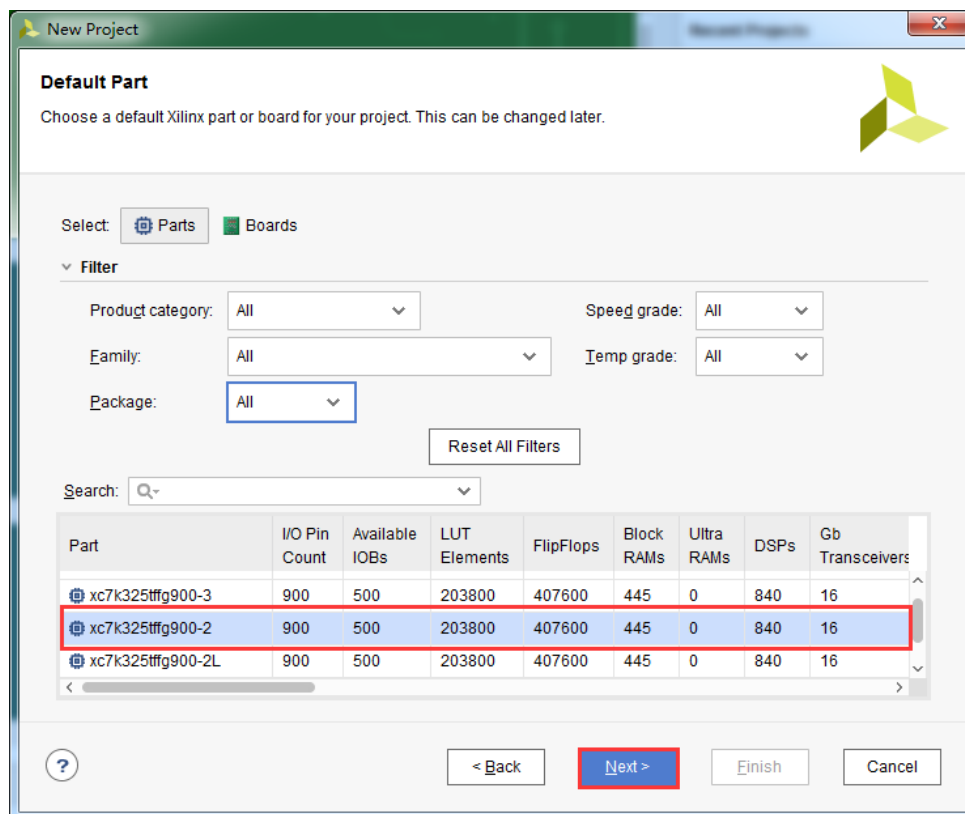
3. 工程名填写 “microblaze_mig”，选择工程路径，并保持默认创建项目子目录选项，工程名和工程路径不要有中文空格，工程路径要尽可能短，否则可能会出现编译错误。点击 “Next”



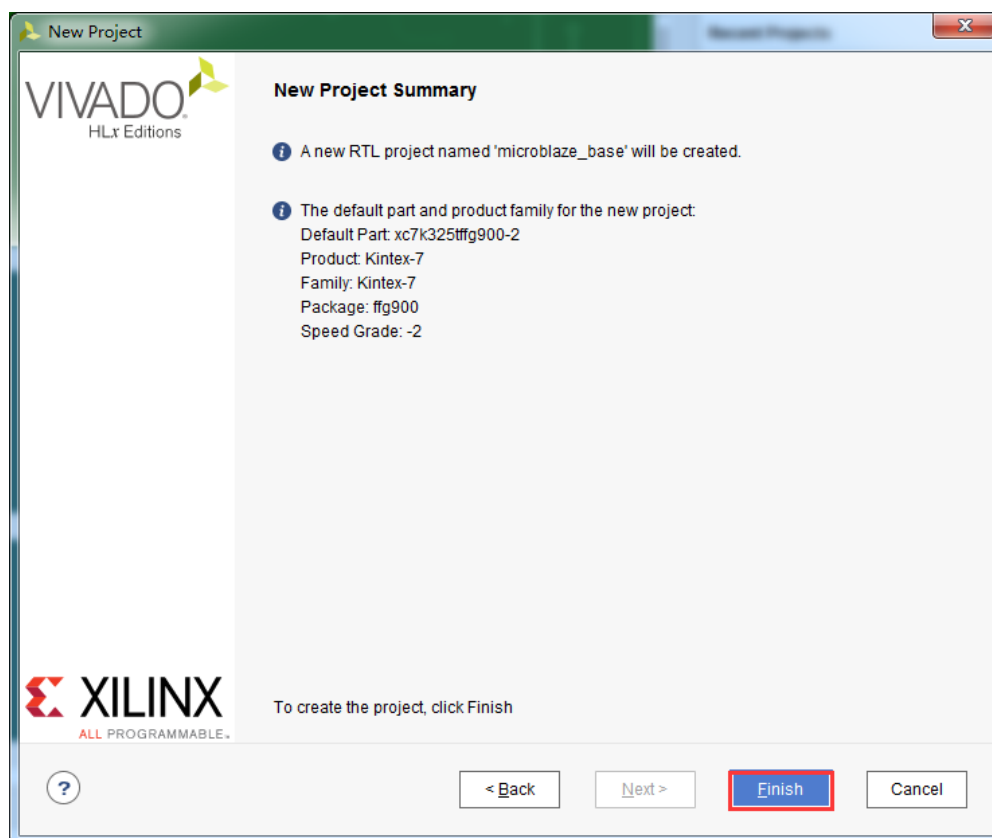
4. 选择创建一个 RTL 工程，并勾选 “Do not specify sources at this time”



5. 根据自己开发板选择 FPGA 型号，点击“Next”

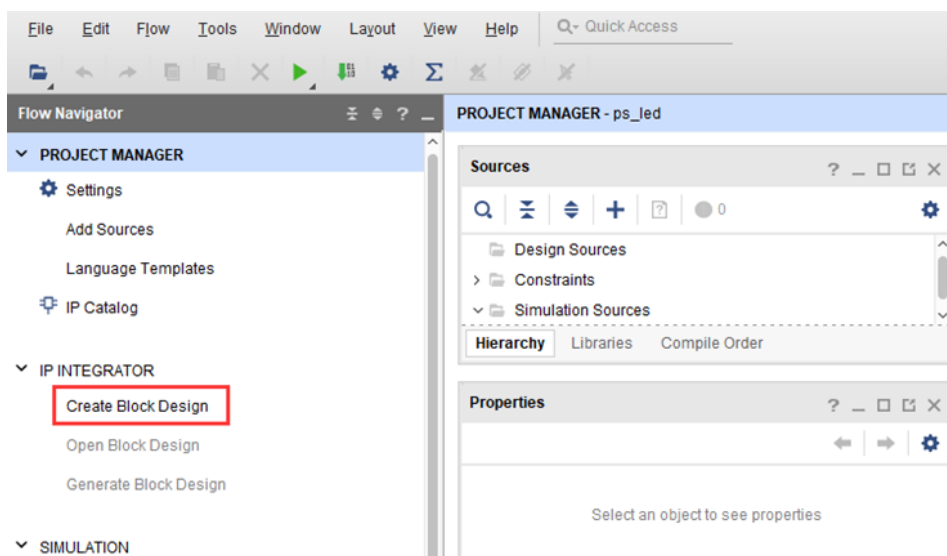


6. 点击“Finish”完成完成工程创建向导

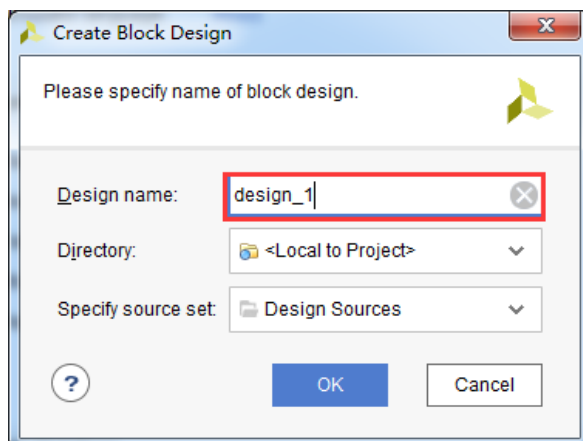


3.1.2 创建 Block 设计

7. 点击 Create Block Design

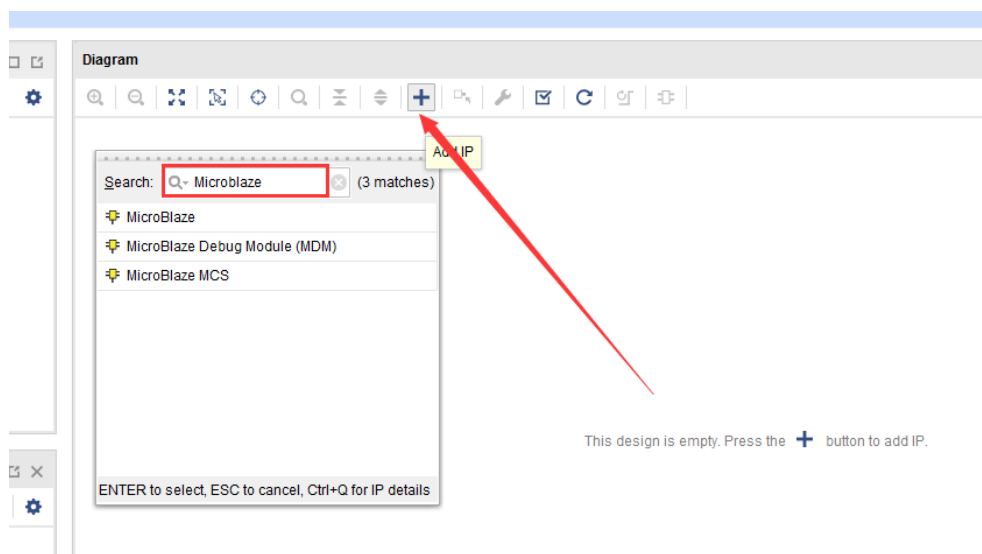


8. 保持设计名称为默认值，点击“OK”

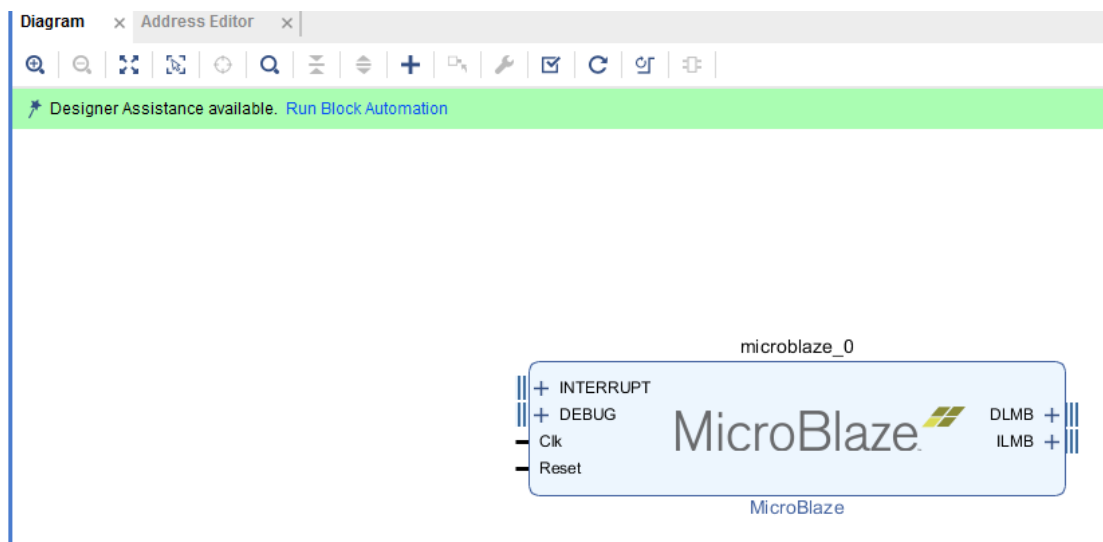


3.1.3 添加软核

9. 点击添加 IP 快捷按钮，搜索“Microblaze”，双击下拉列表中“Microblaze”

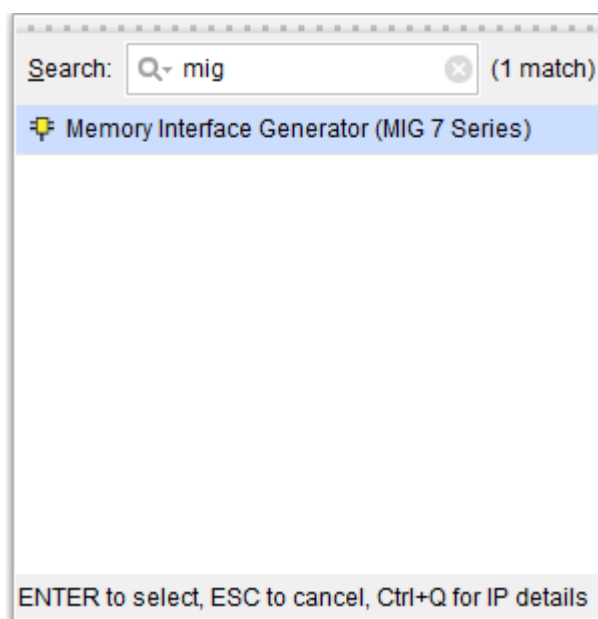


10. 一个 Microblaze 核添加到 Block 设计中

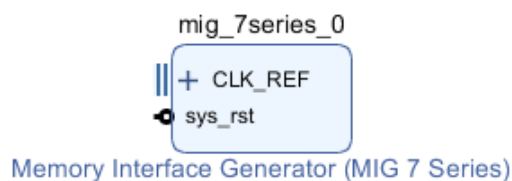


3.1.4 添加 mig 核

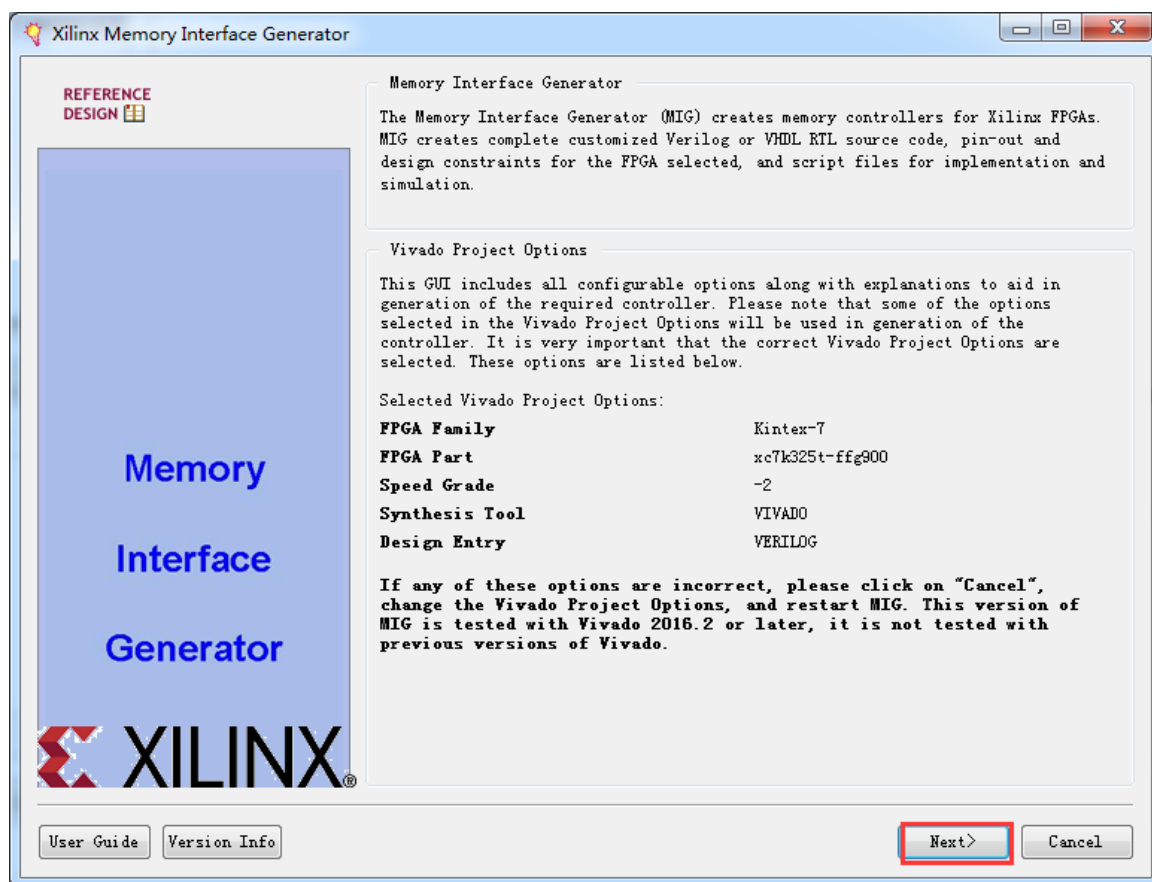
11. 搜索“mig”，双击“Memory Interface Generator(MIG 7 Series)” 添加一个 mig 核



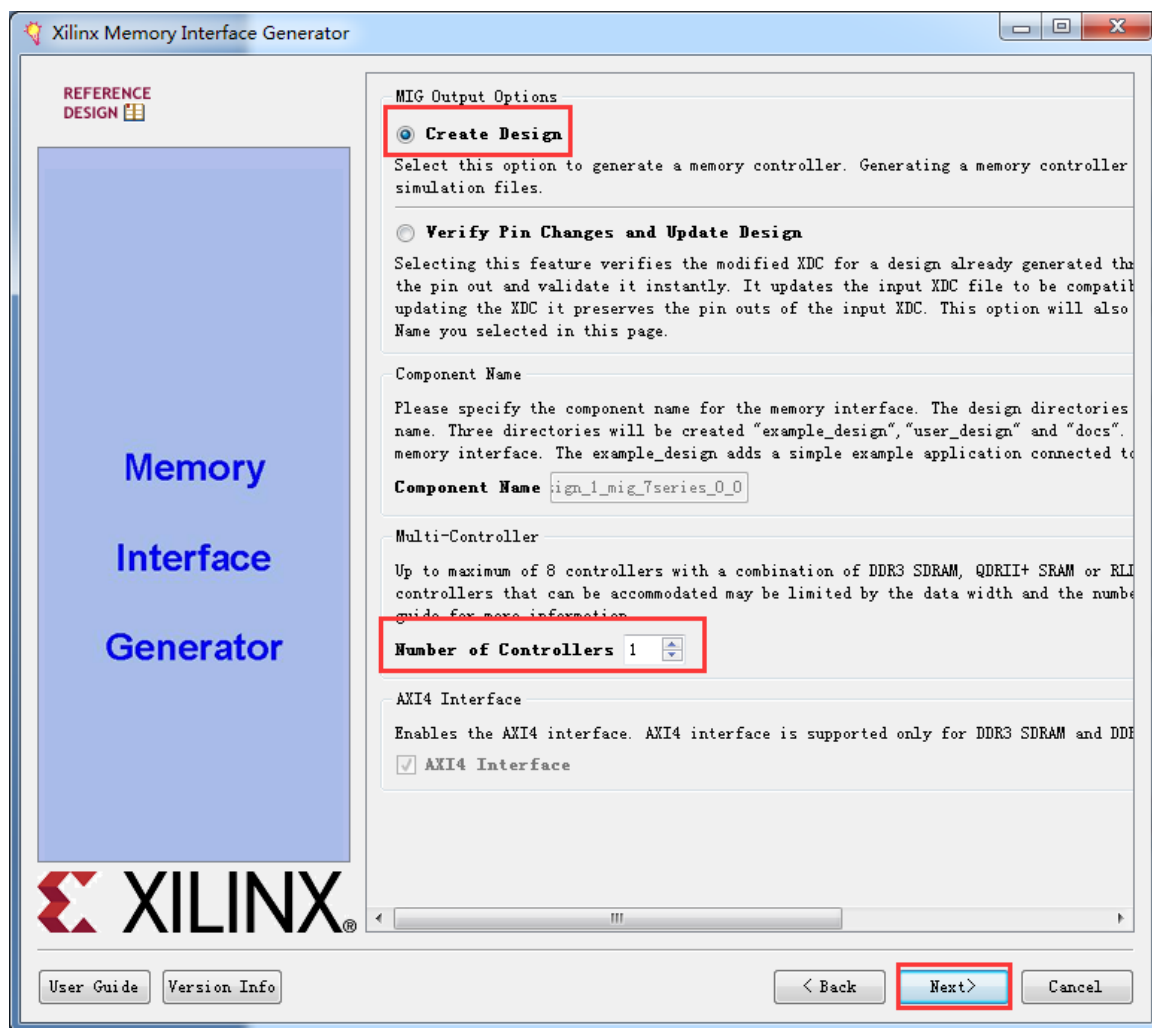
12. 双击刚添加的 mig 核进行配置



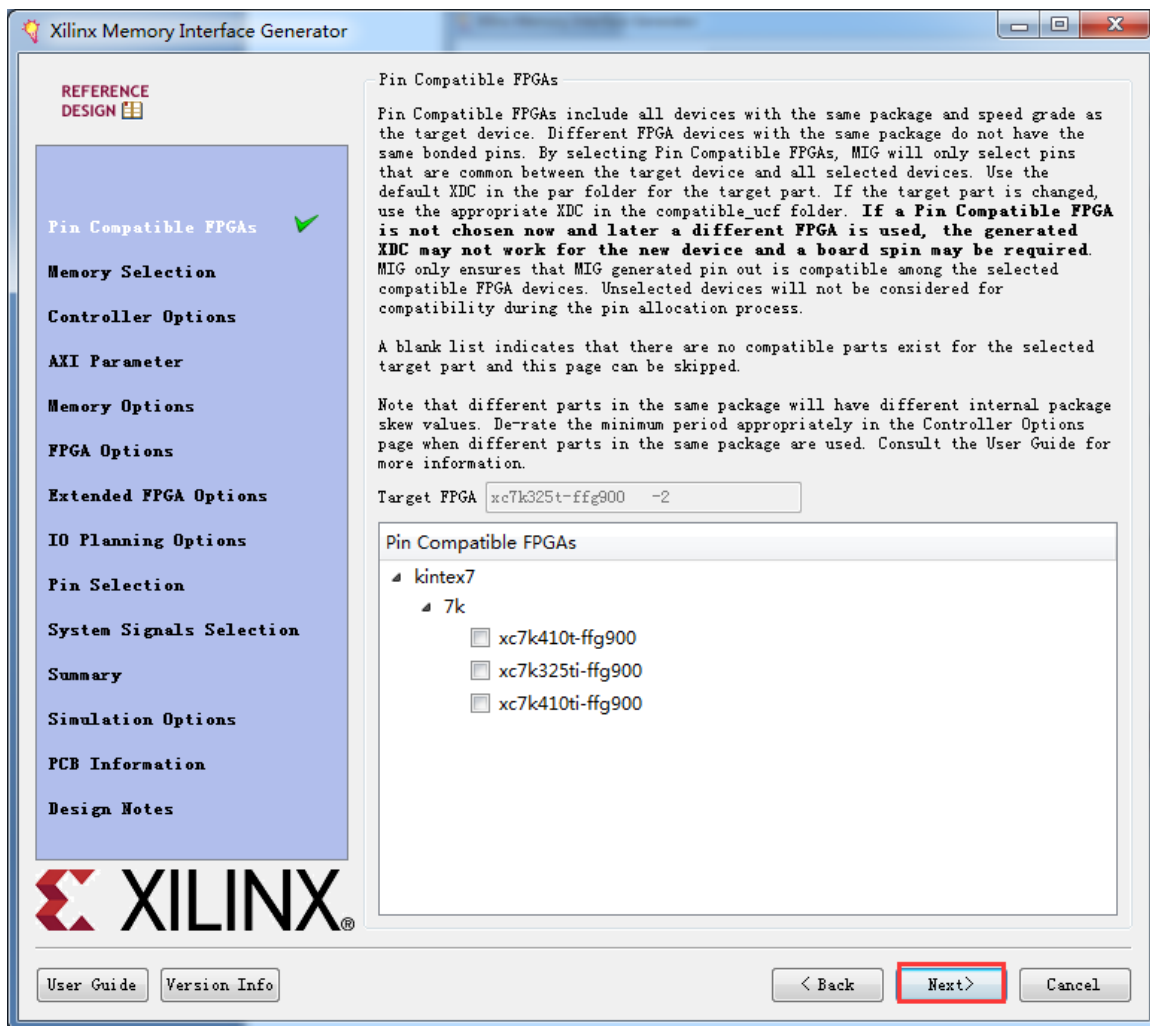
13. 点击 “Next”



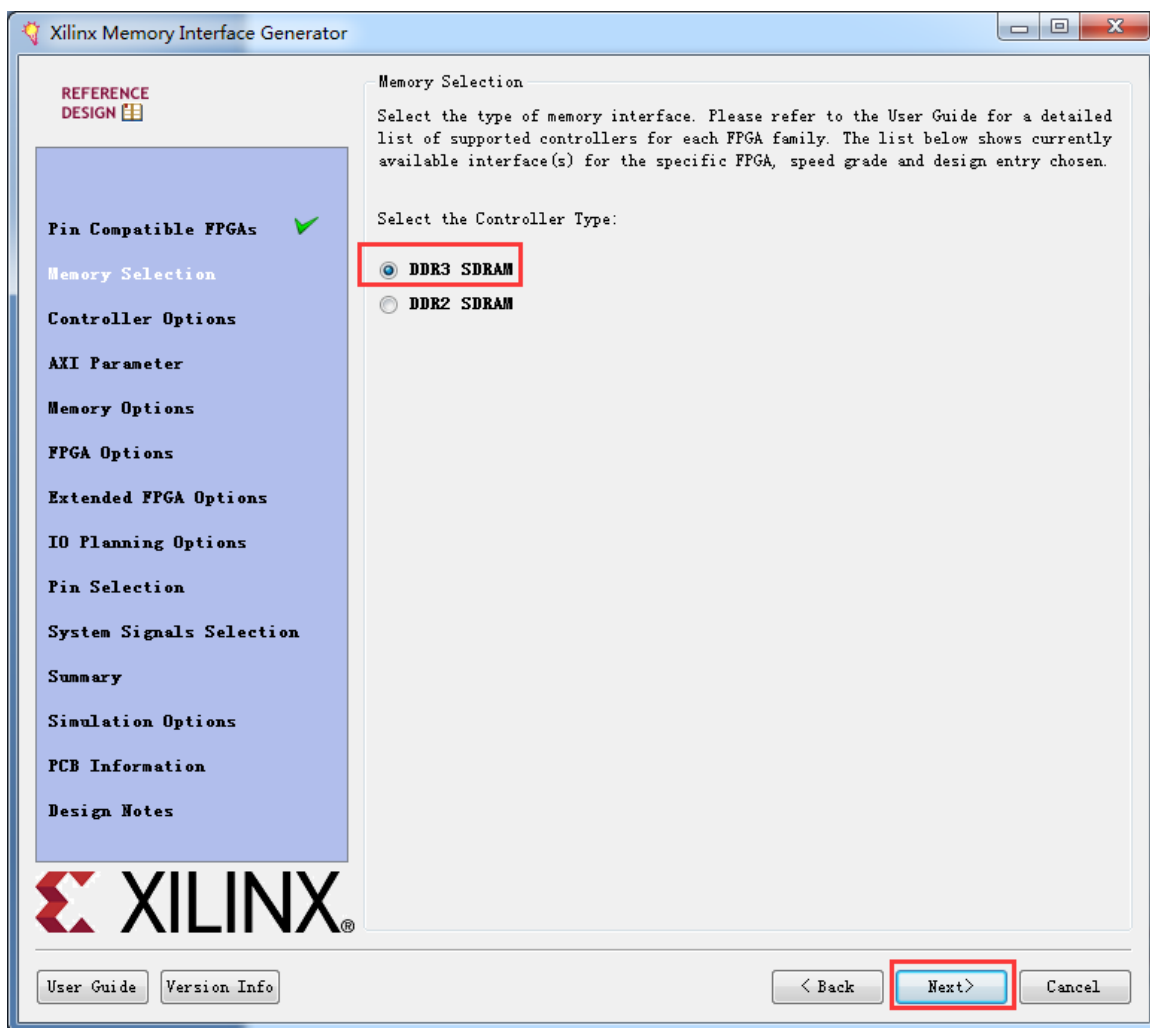
14. 保持默认值点击 “Next”



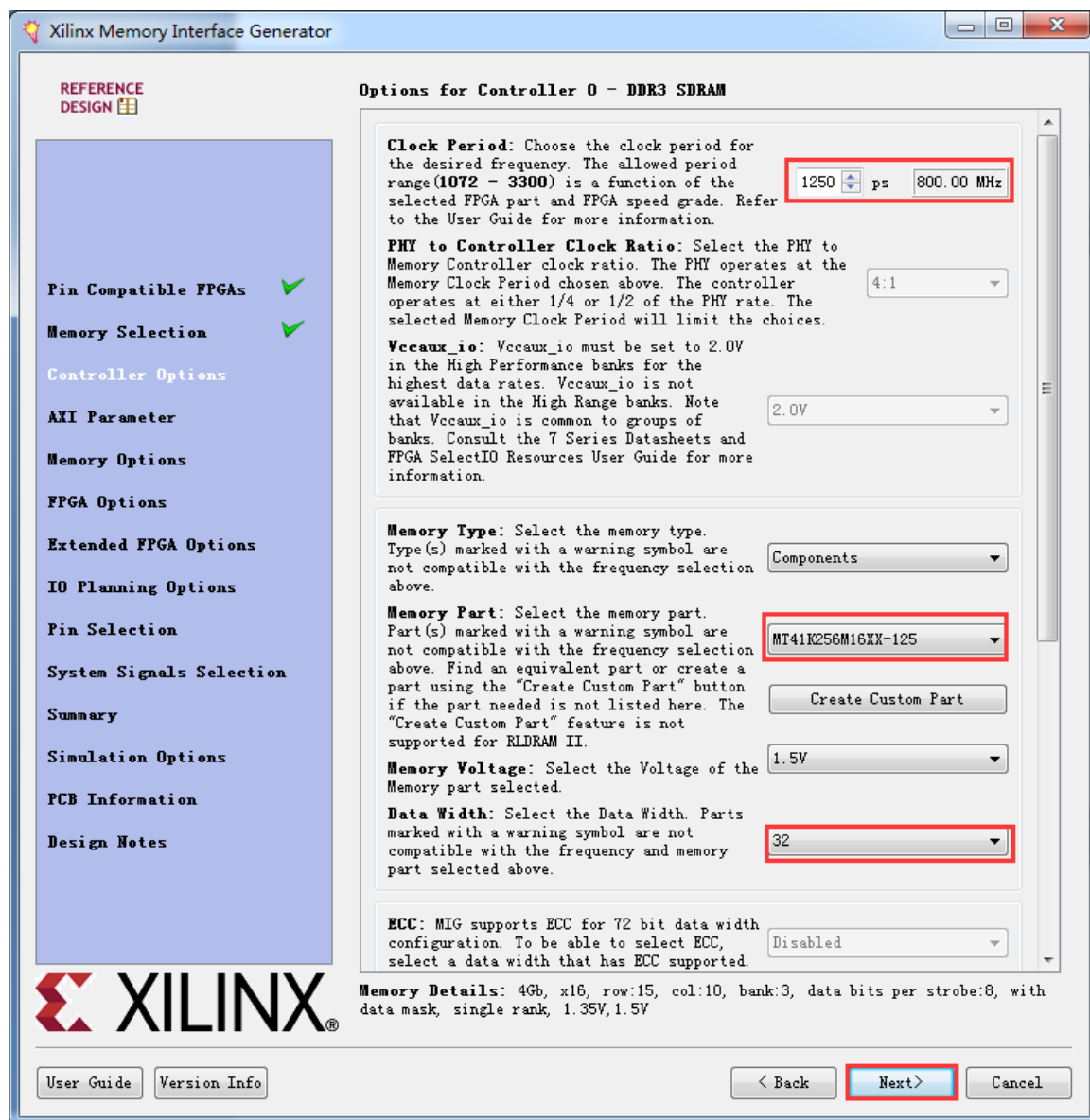
15. 点击 “Next”



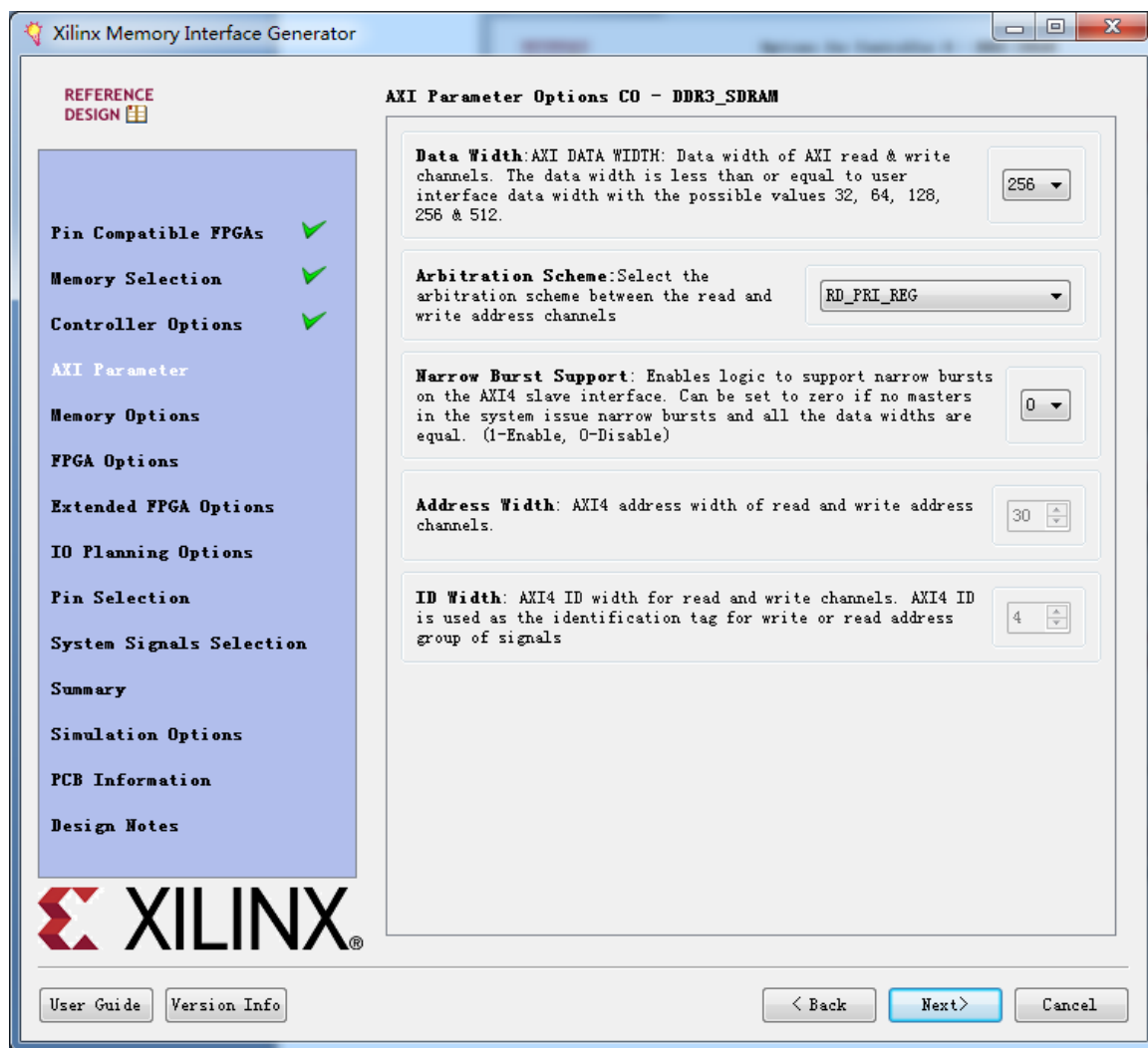
16. 点击 “Next”



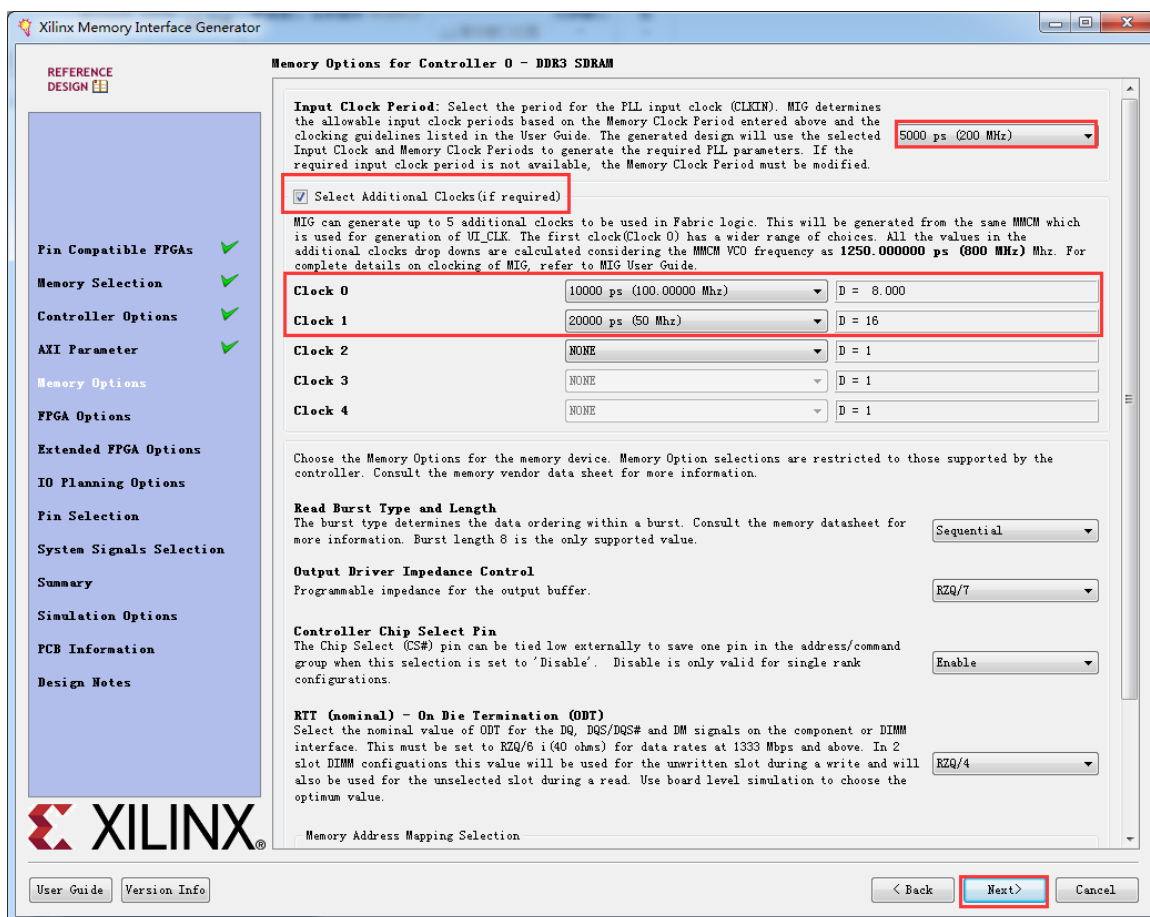
17. 配置 Clock Period 为 1250ps，也就是 ddr3 时钟为 800Mhz，Memory Part 选择 “MT41K256M16XX-125”，Data Width 选择 32 位，以上设置都要根据具体开发板进行调整（具体参考 verilog 例程里的 DDR 配置）。点击 “Next”



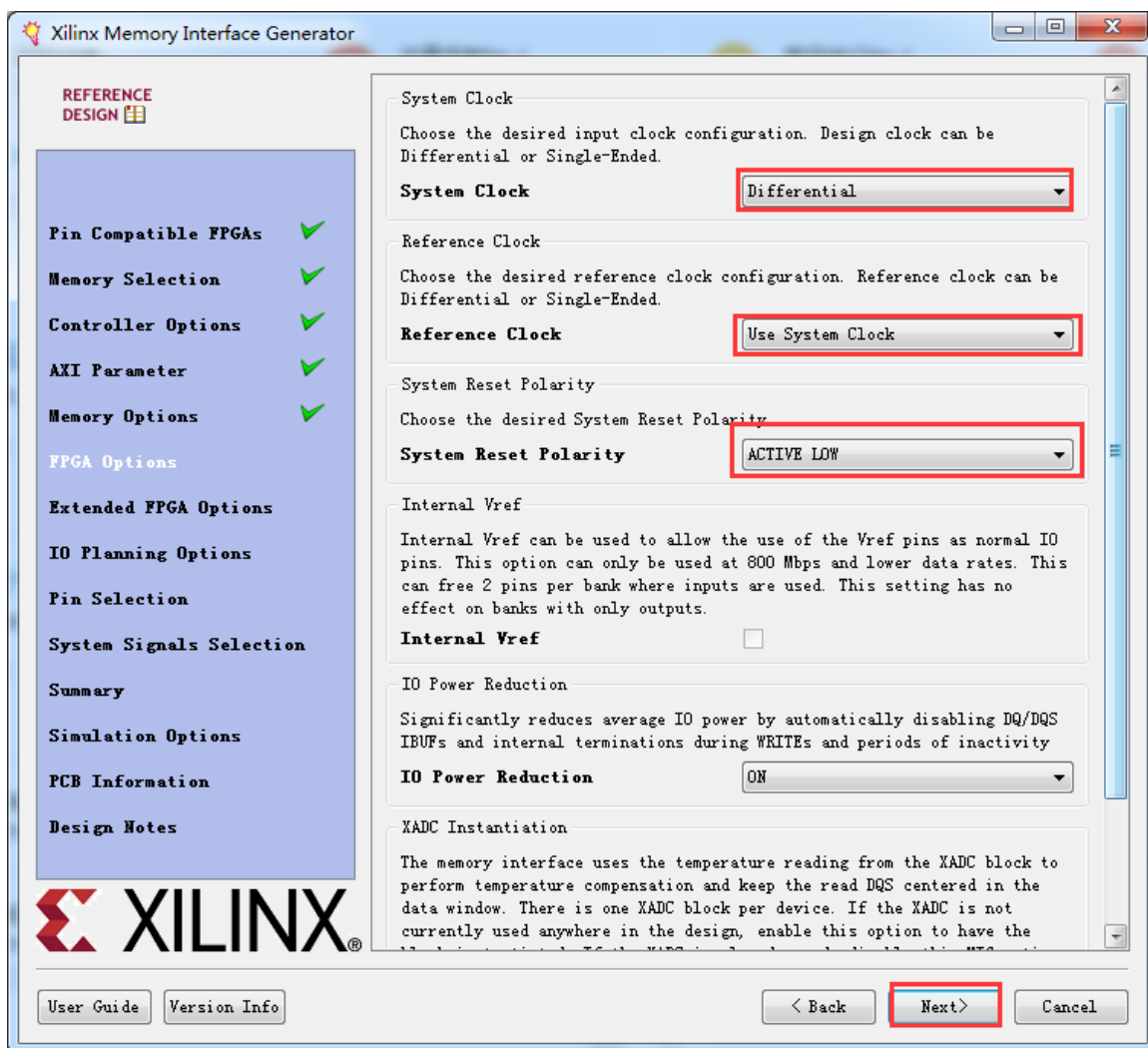
18. 点击 "Next"



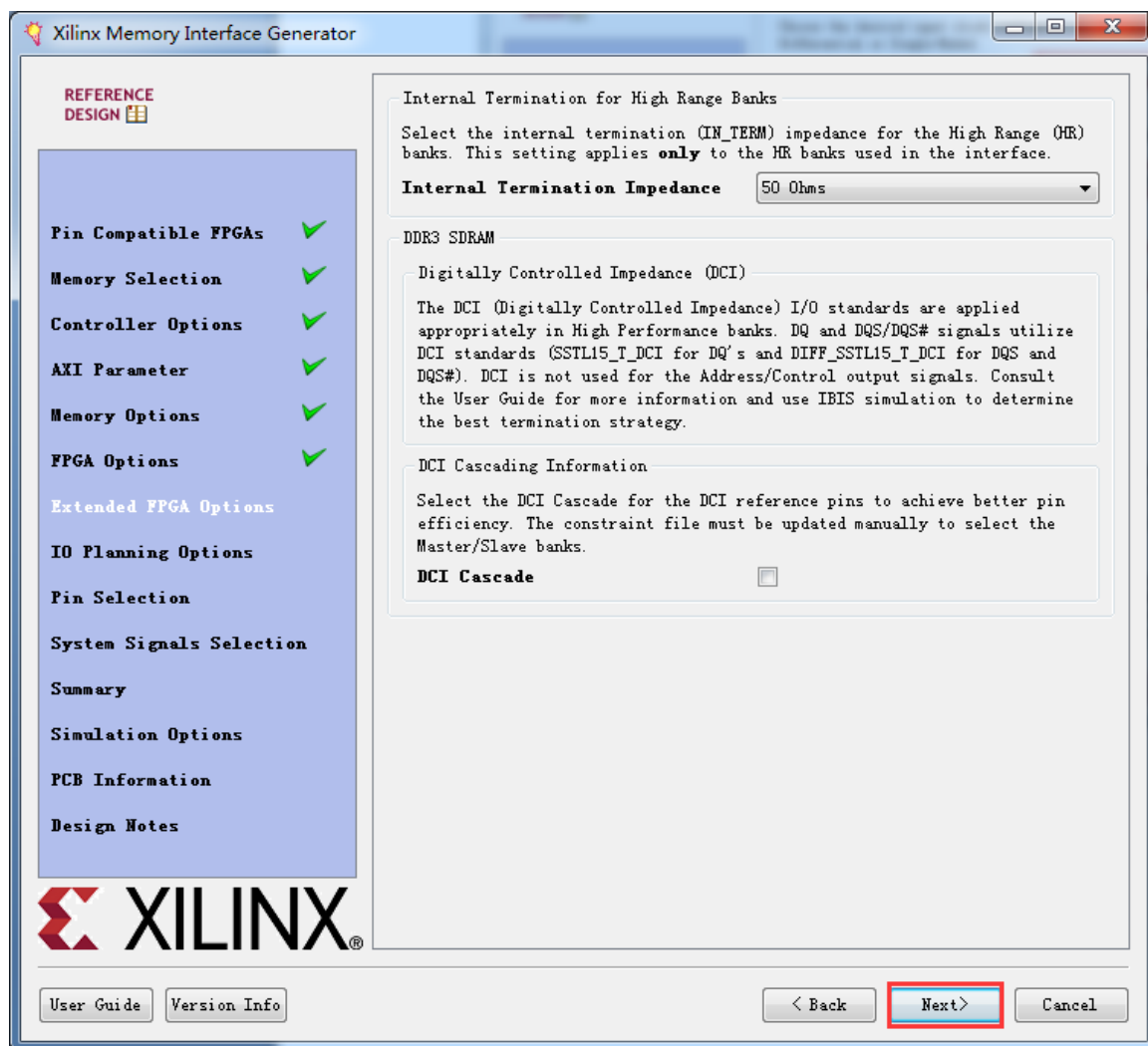
19. Input Clock Period 选择 5000ps，也就是外部输入 200Mhz 作为系统时钟（如果 ax7035 开发板，这里选择 50Mhz），勾选 “Select Additional Clocks(if required)”，选择一个额外的输出时钟给 Microblaze，这样可以节省一个 PLL，Clock 0 选择 100Mhz，Clock 1 选择 50Mhz。



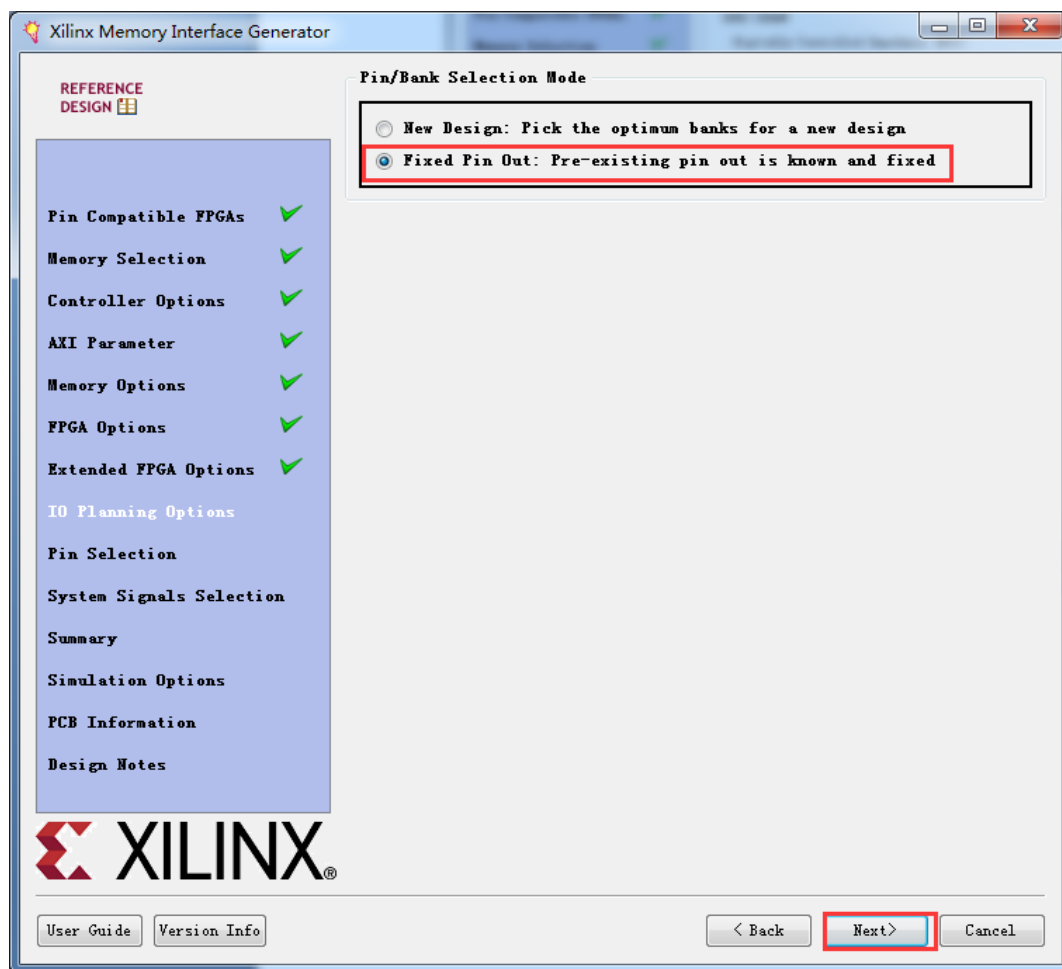
20. 系统时钟选择“Differential”差分类型(如果 ax7035 开发板, 这里选择 Single-End), “Reference Clock” 参考时钟必须是 200Mhz, 因为系统时钟刚好是 200Mhz, 这里可以选择 “Use System Clock”, 复位极性保持低电平复位。点击 “Next”



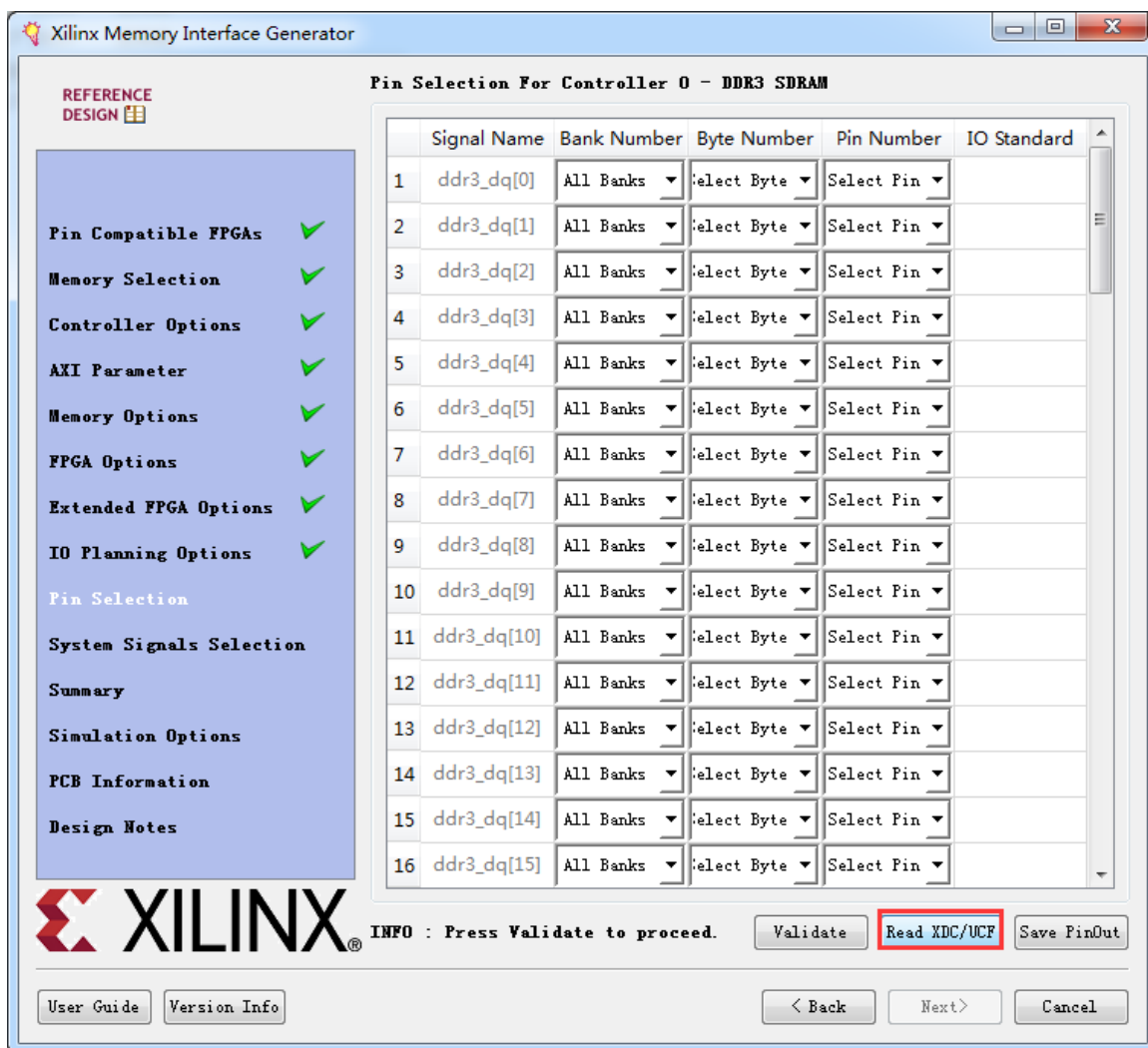
21. 点击 “Next”



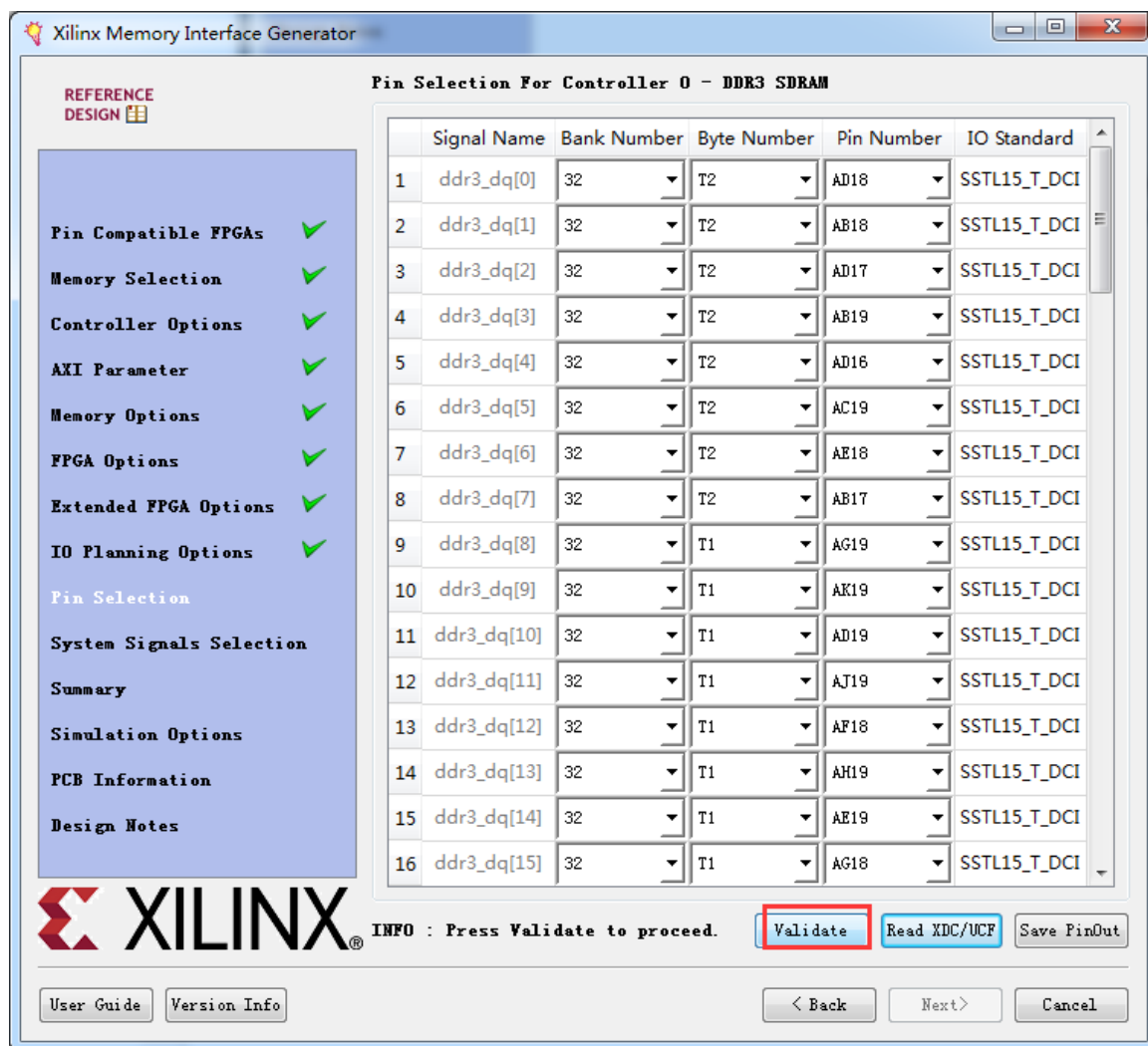
22. 选择 “Fixed Pin Out: Pre-existing pin out is known and fixed”，因为是开发板，ddr 管脚已经确定。



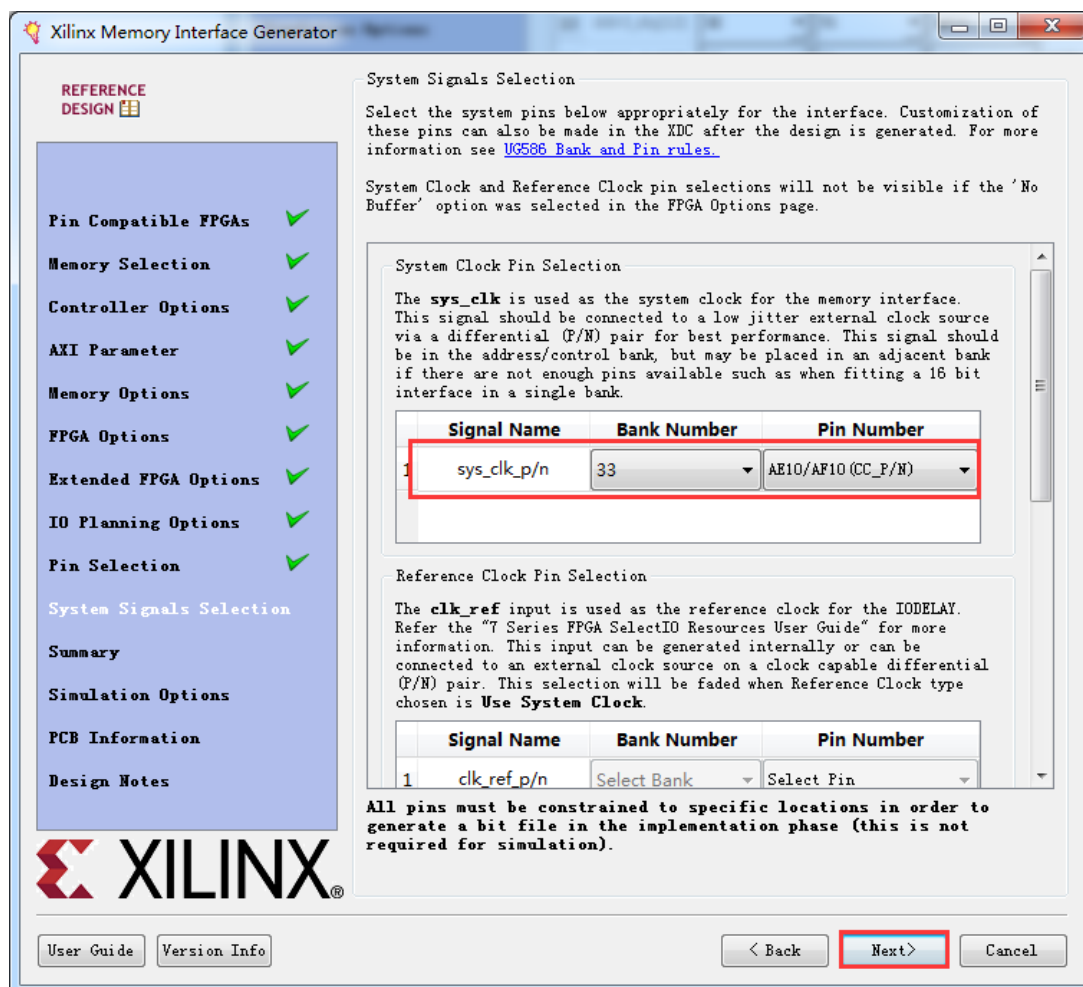
23. 点击 “Read XDC/UCF” , 选择开发板提供的 ddr3.ucf 文件



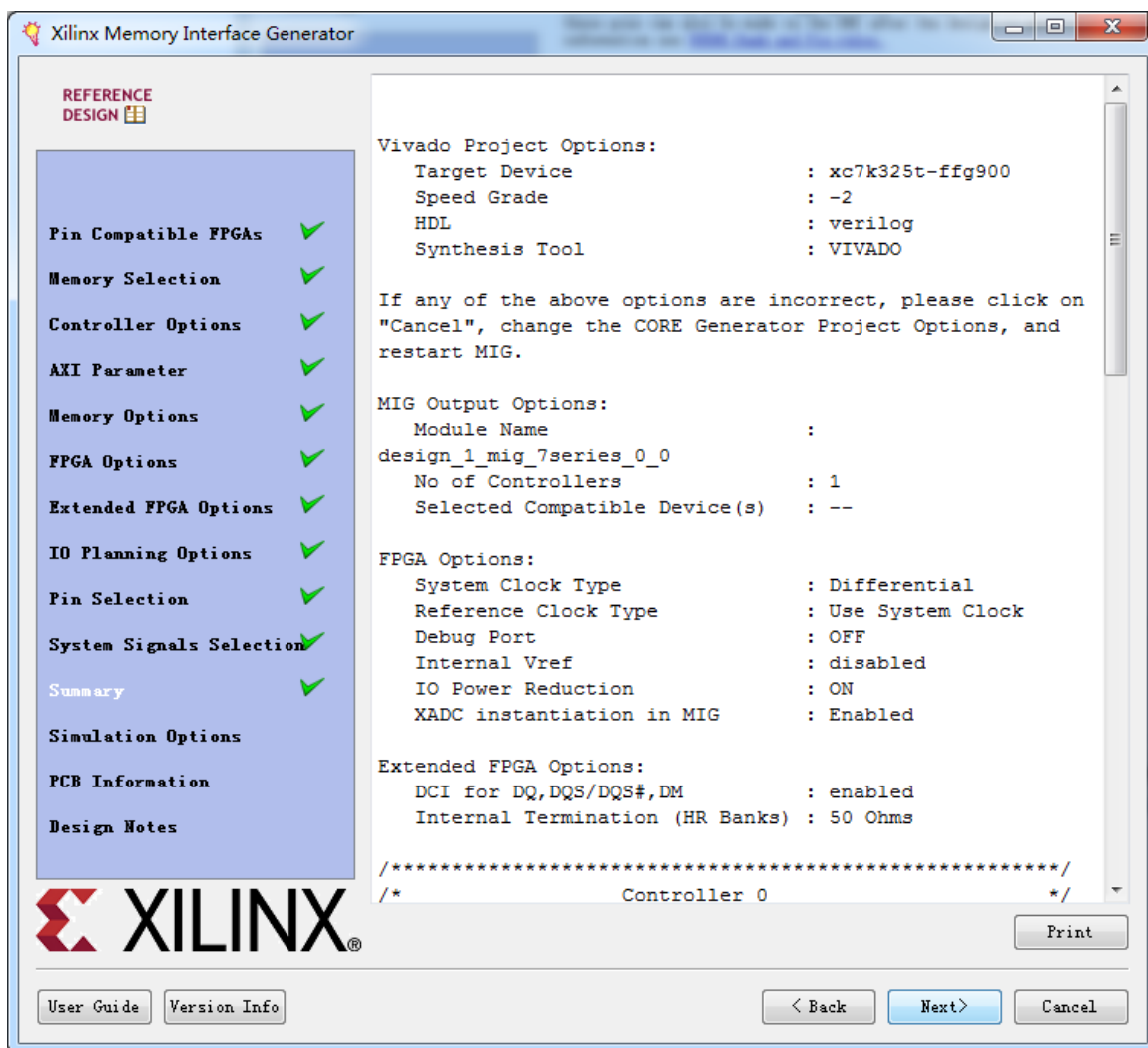
24. 点击“Validate”按钮，检验有没有错误，没有错误后点击“Next”



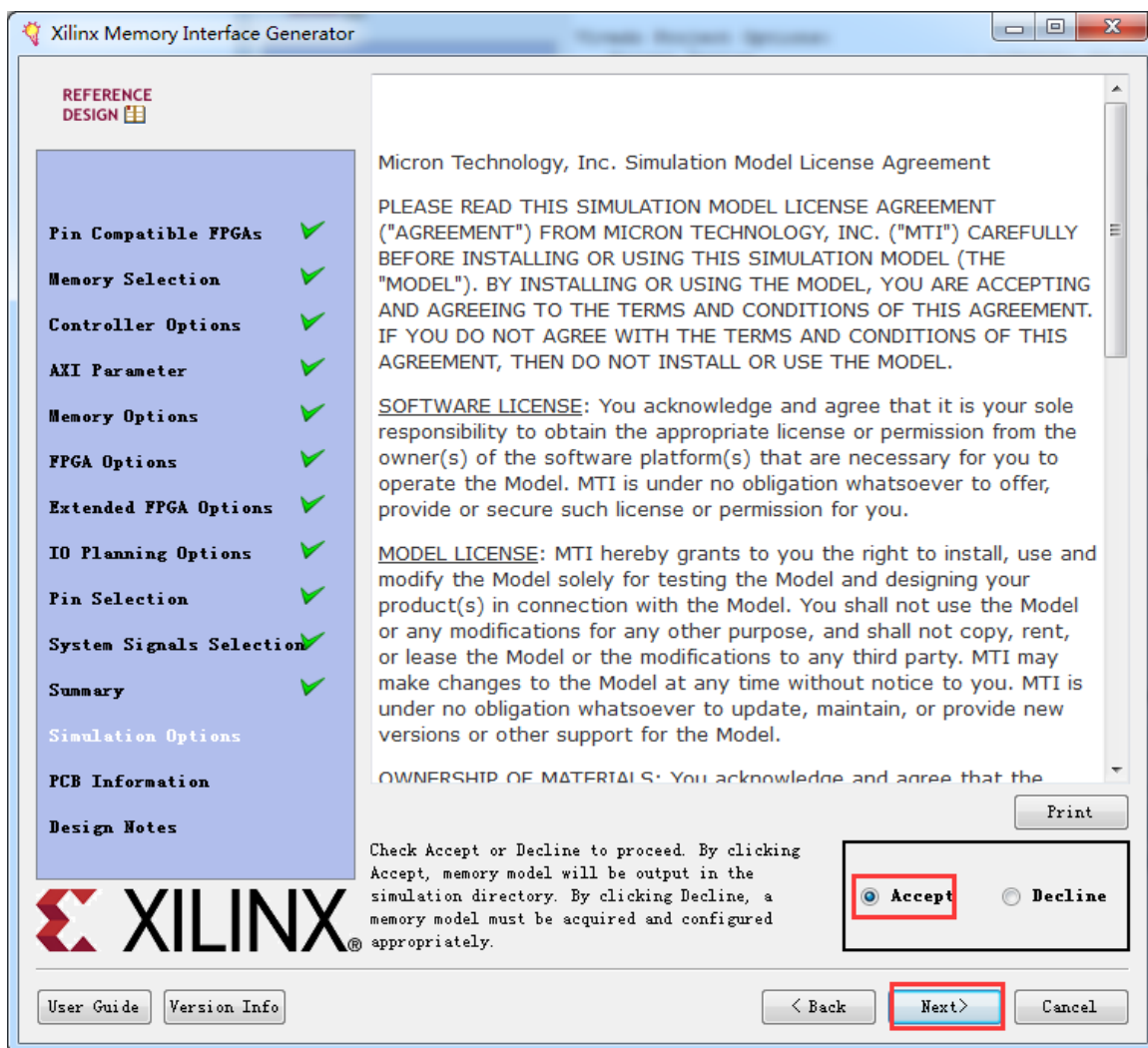
25. 系统时钟管脚分配，按照开发板具体情况选择，点击“Next”



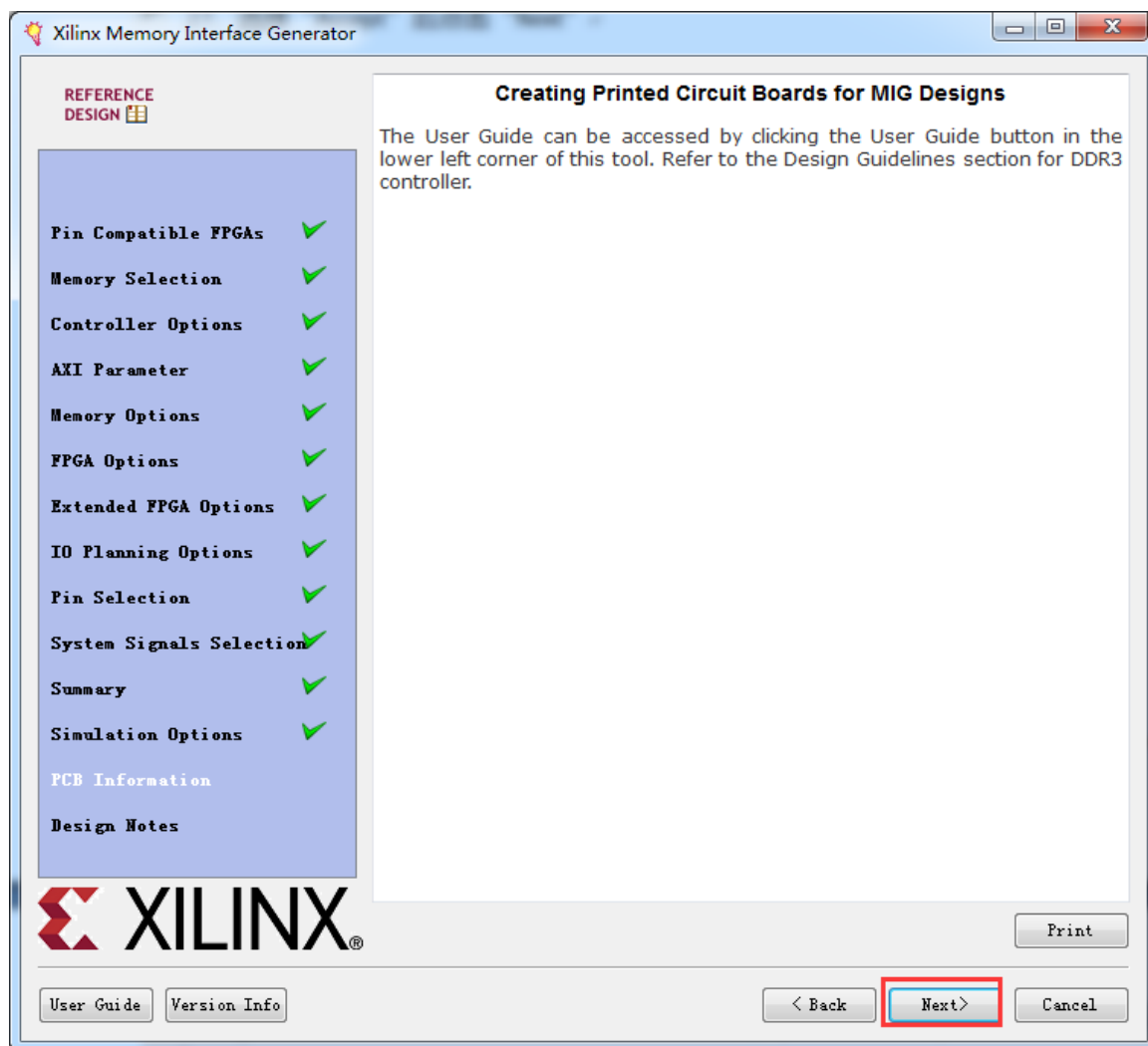
26. 点击 "Next"



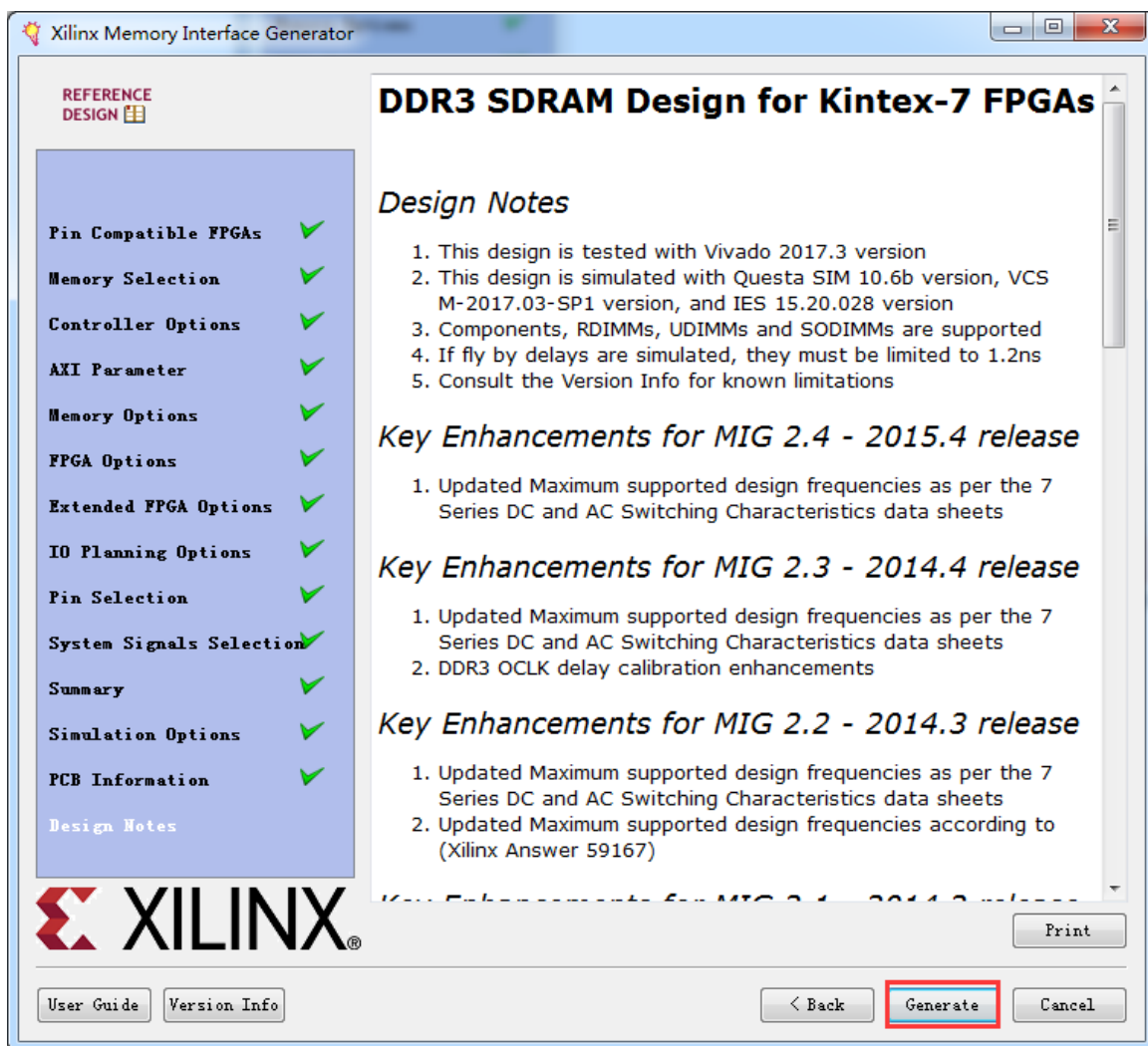
27. 选择 "Accept" 后点击 "Next"



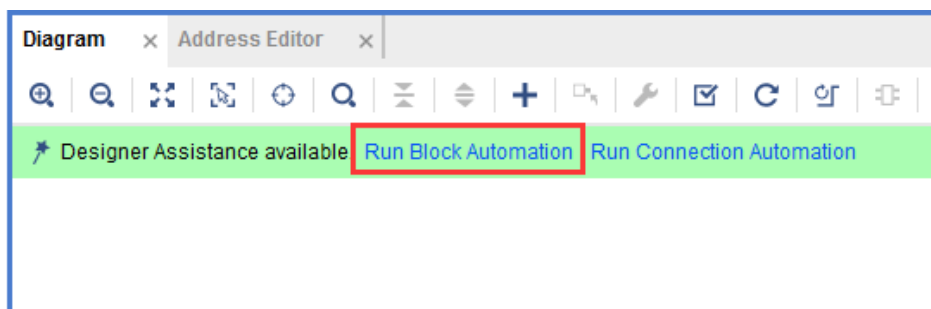
28. 点击 “Next”



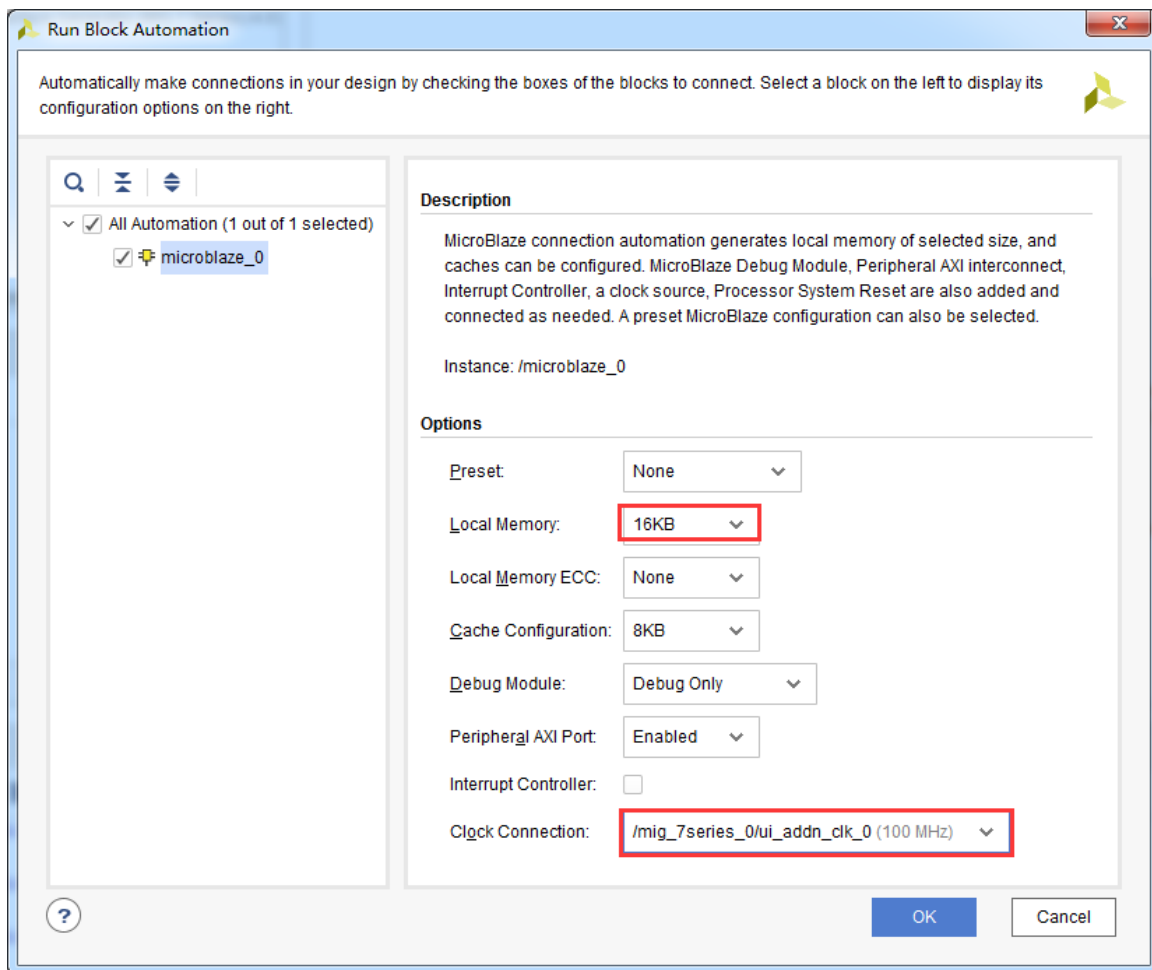
29. 点击 “Generate” 生成 mig IP



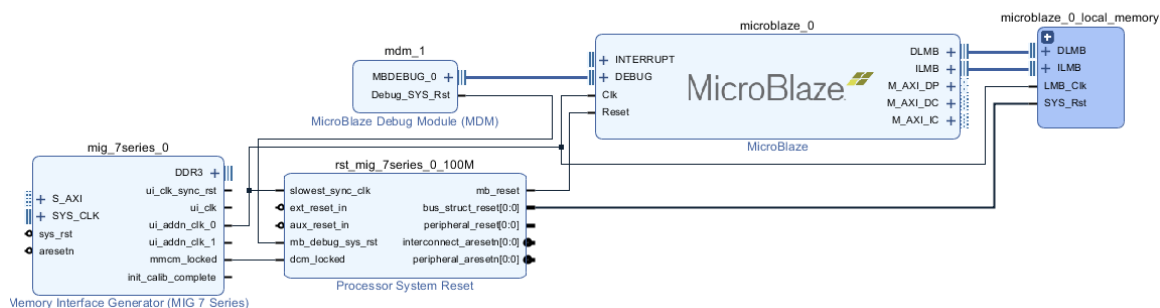
30. 点击“Run Block Automation”完成一些自动设置，需要注意，这个选项不会一直都有，一旦设置完成，可能不会再有机会通过这种方式进行一些自动设置。



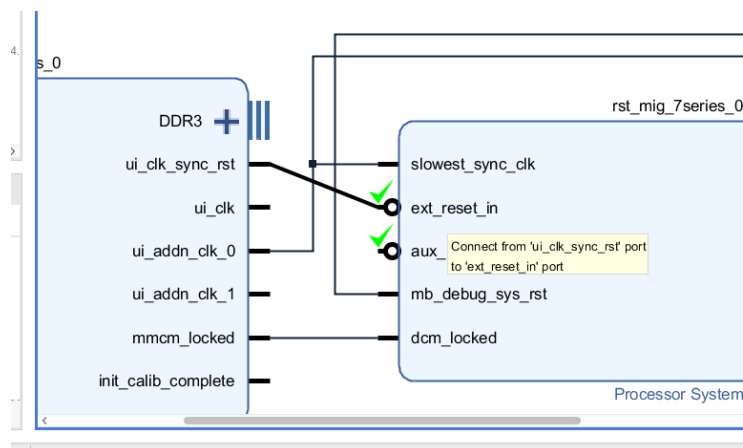
31. 弹出窗口里完成一些 microblaze 参数设置，在“Preset”选项中保持默认“None”，这个选项相当于预设了一些 microblaze 的配置参数，根据应用选择。“Local Memory”选择 16KB “Clock Connection”时钟选择 mig 核输出的输出的 100Mhz 时钟作为 CPU 时钟，时钟频率太高可能会导致时序问题，100Mhz 能满足大部分需求。



32. 自动添加了一些模块和连线

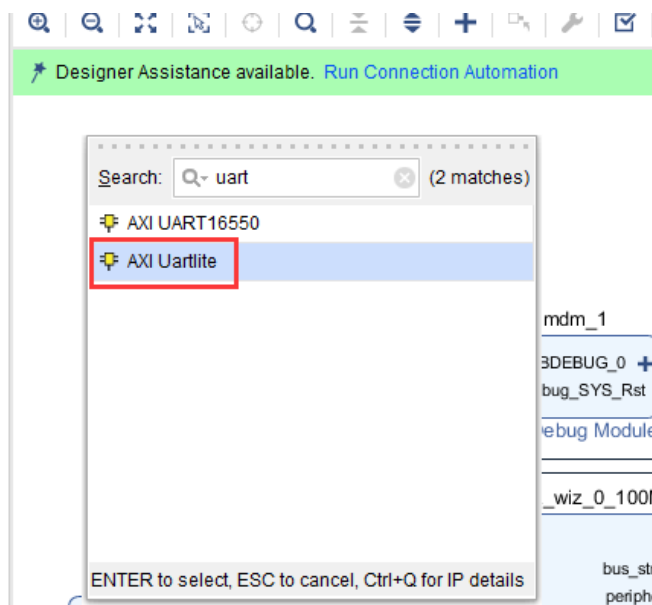


33. 连接 mig_7series_0 的 ui_clk_sync_rst 到 rst_mig_7series_0_100M 的 ext_reset_in

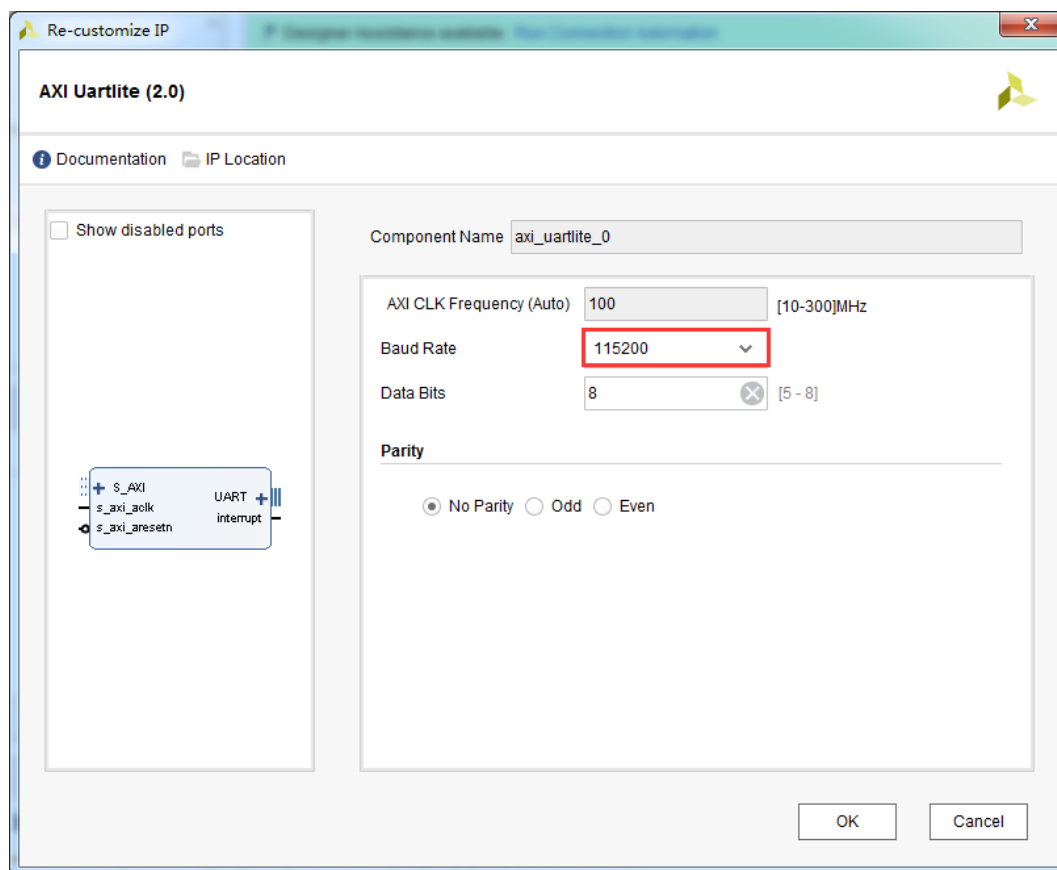


3.1.5 添加外设

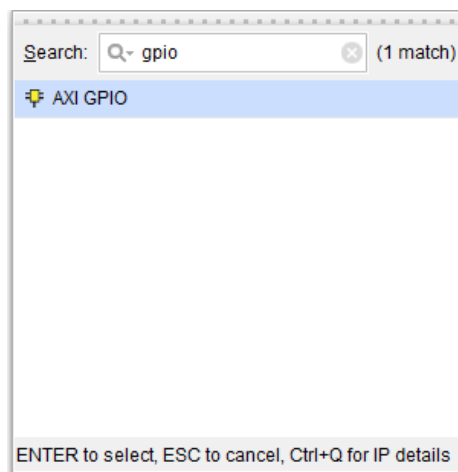
34. 搜索 “uart”，双击 “AXI Uartlite” 添加一个 Uart 串口 IP 核



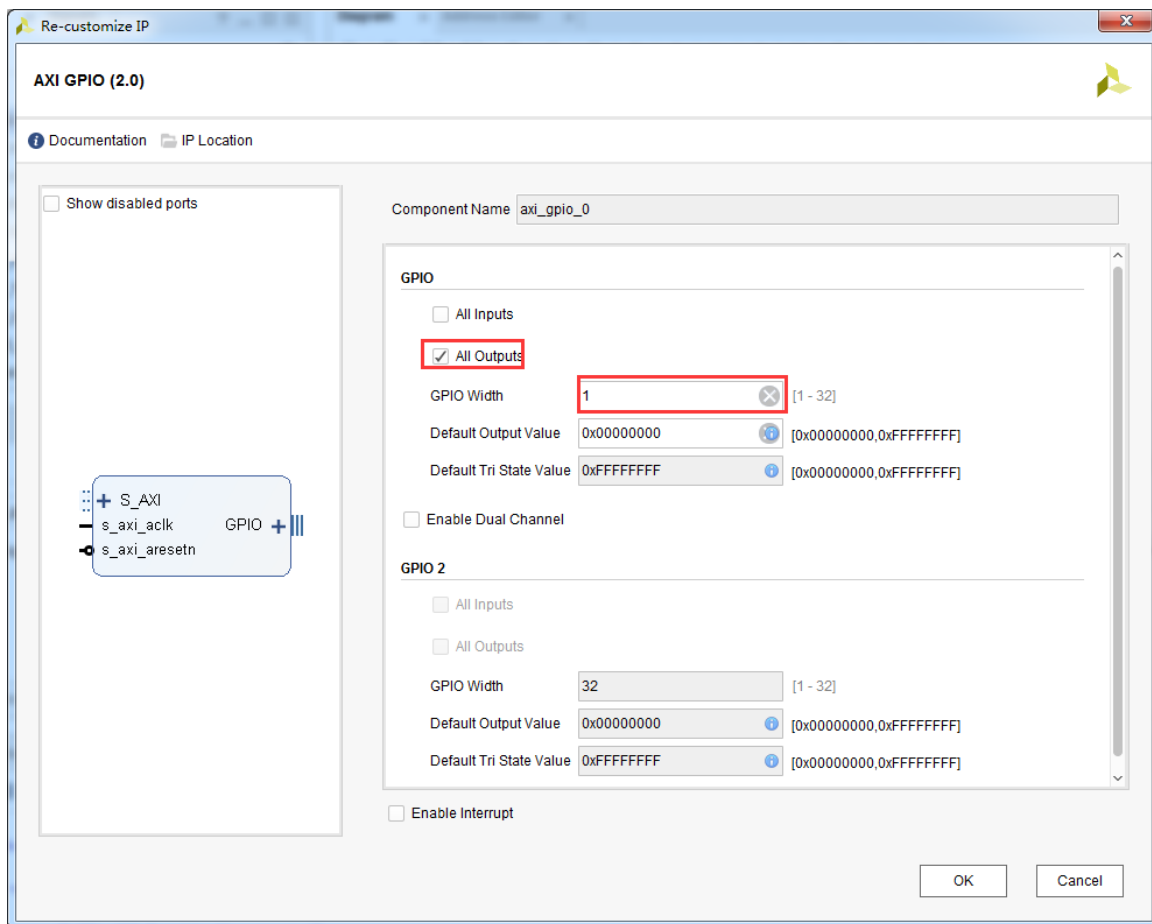
35. 双击刚才添加的 Uartlite 核，修改波特率为 115200，这是嵌入式系统比较常用的波特率，这里波特率是固定值，无法在程序中动态修改，点击 OK 完成配置



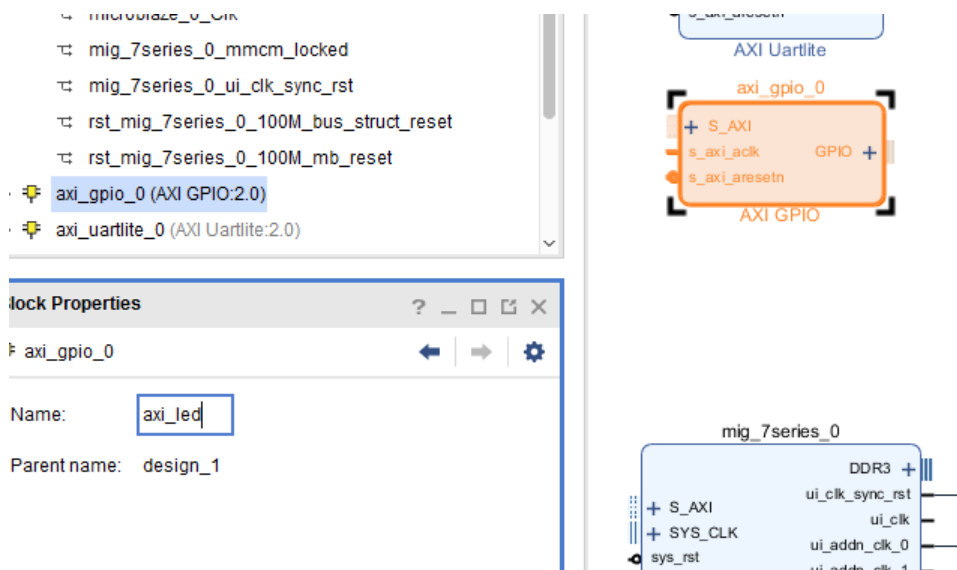
36. 搜索 gpio，双击“AXI GPIO”添加一个 AXI GPIO，用于控制 LED 灯。



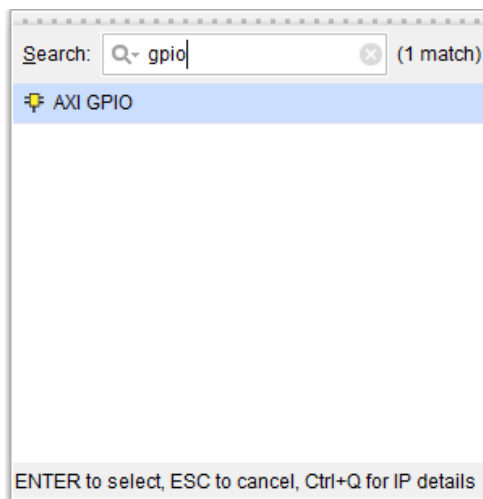
37. 双击刚添加的 AXI GPIO 修改参数，勾选“All Outputs”，设置为全输出，“GPIO Width”填写 1，这里只控制 1 位 LED，点击“OK”完成配置



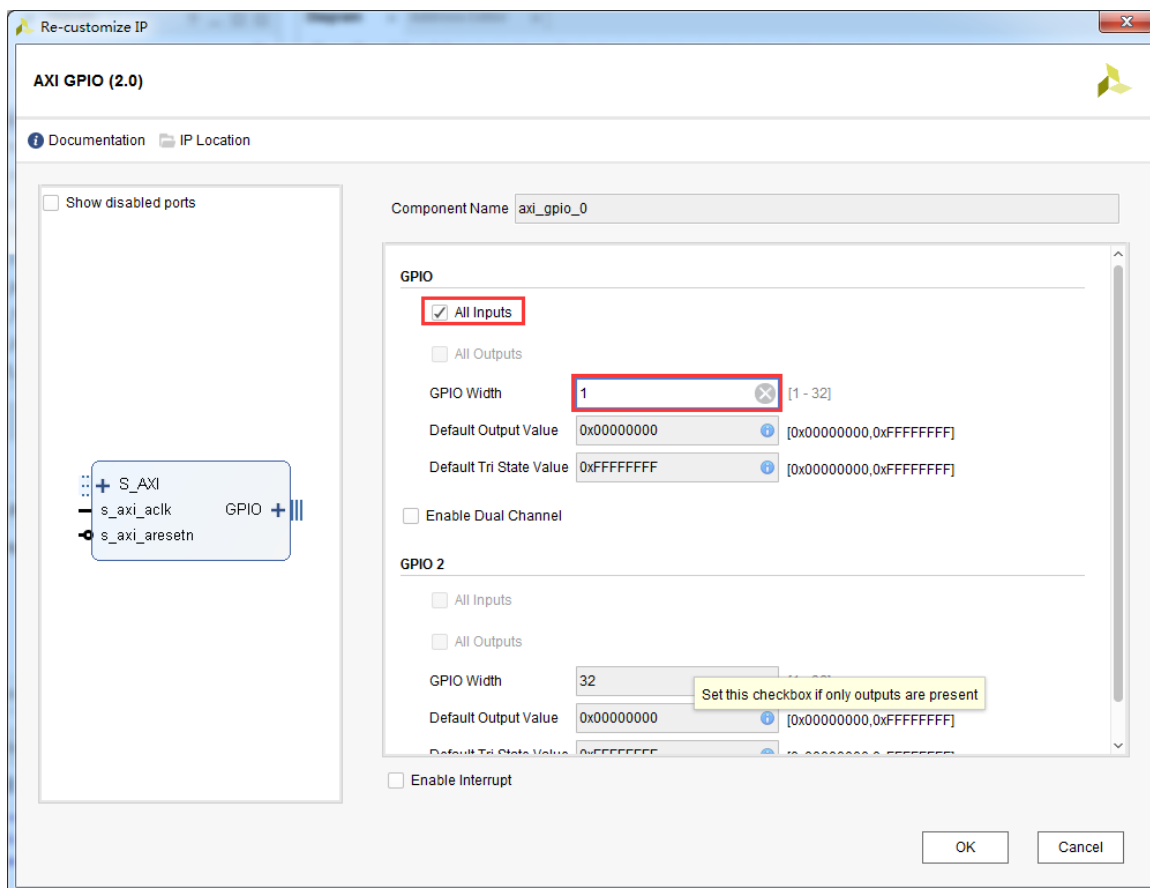
38. 选择刚添加的 AXI GPIO，在 Block Properties 界面可以修改模块名称，这里改为“axi_led”，需要注意，如果名字和例程不同，软件编程时可能要修改例程的 C 语言代码，建议初学者保持和教程一致。



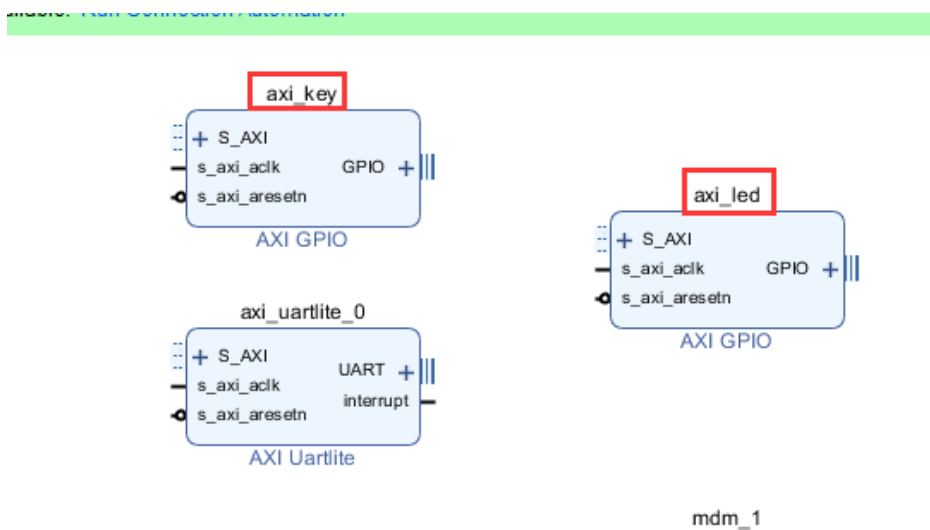
39. 添加一个 AXI GPIO，用于按键输入



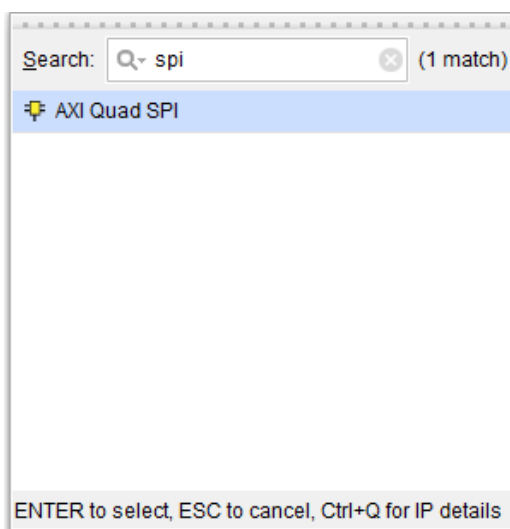
40. 修改 AXI GPIO 参数，勾选 “All Inputs”，“GPIO Width” 填写 1，这里只控制 1 位按键，点击 “OK” 完成配置



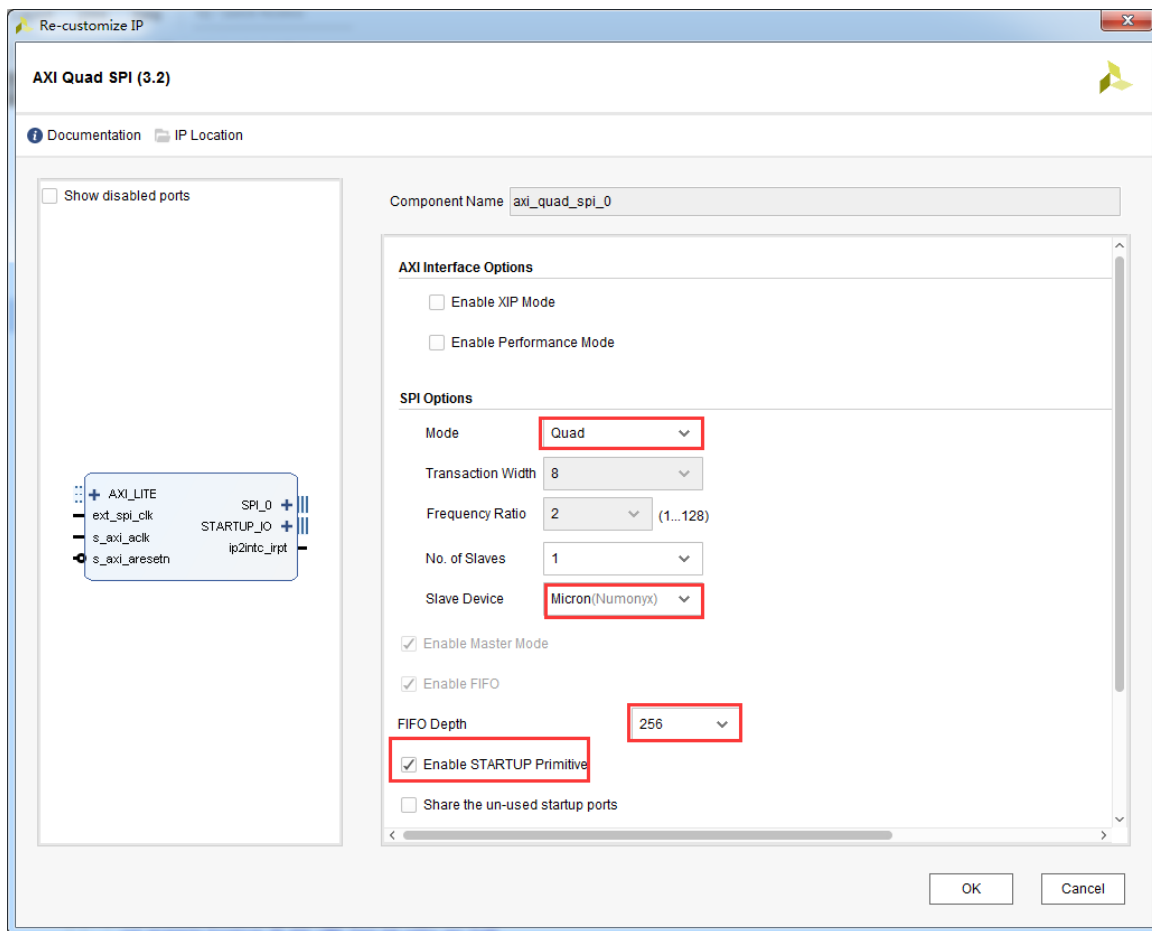
41. 选择刚添加的 AXI GPIO，在 Block Properties 界面可以修改模块名称，这里改为 “axi_key”，需要注意，如果名字和例程不同，软件编程时可能要修改例程的 C 语言代码，建议初学者保持和教程一致。



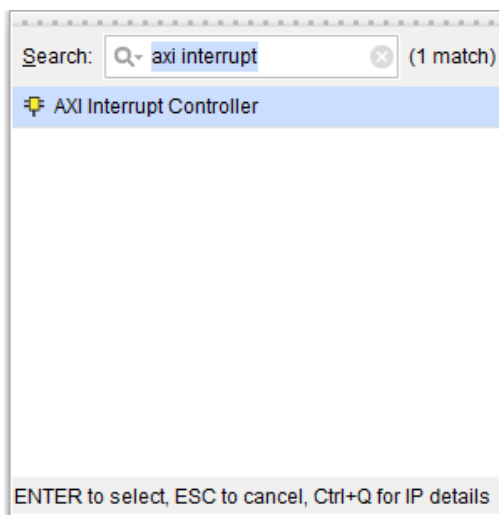
42. 搜索 spi，添加“AXI Quad SPI”核，用于控制板上的 QSPI Flash



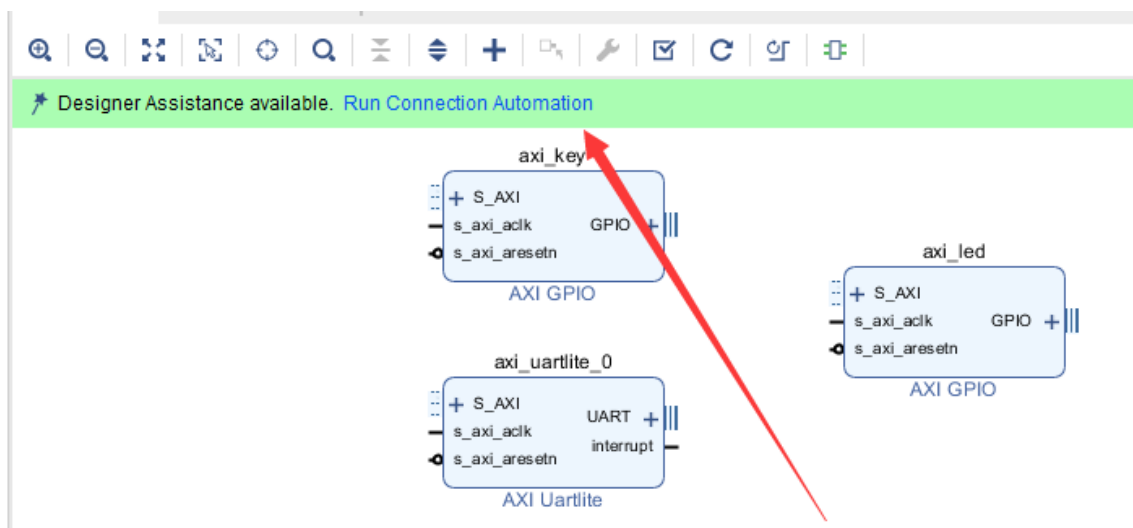
43. 双击刚才添加的 IP 核进行参数配置，“Mode”选择“Quad”，“Slave Device”选择“Micron”，“FIFO Depth”选择“256”，勾选“Enable STARTUP Primitive”



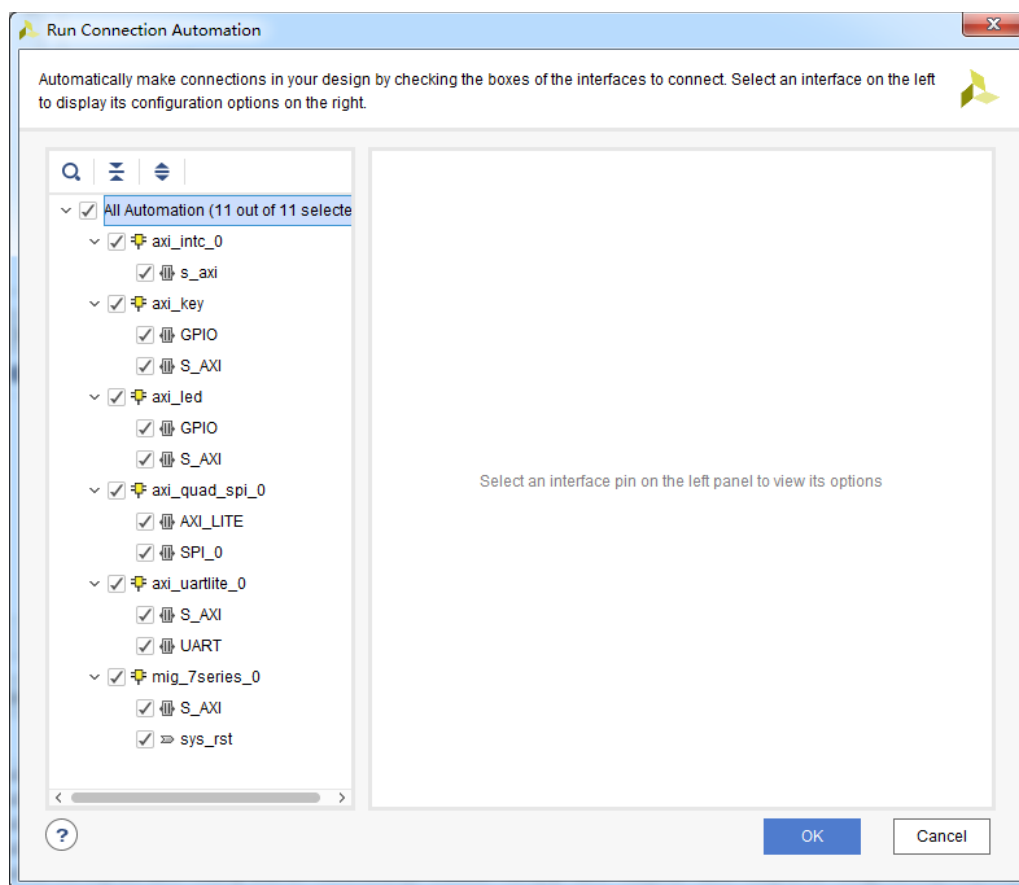
44. 搜索 “axi interrupt” 添加 “AXI Interrupt Controller”



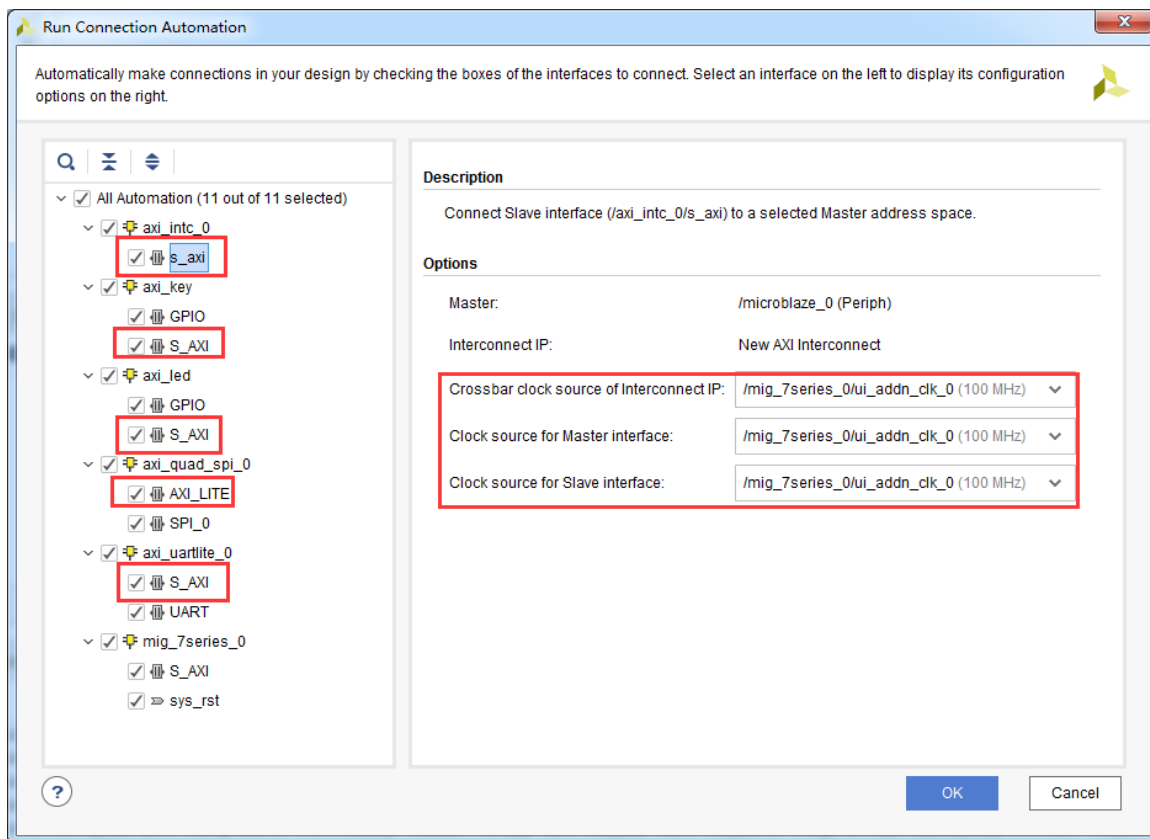
45. 点击 “Run Connection Automation” , 来自动连线



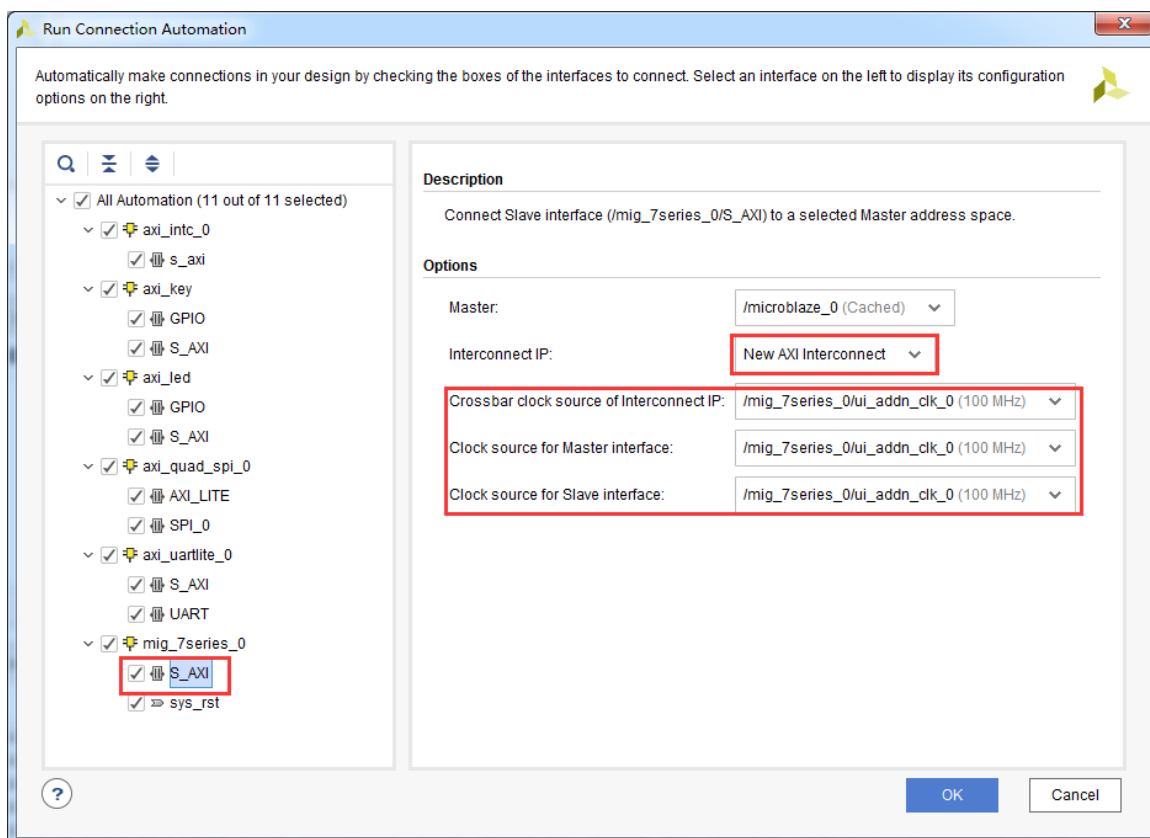
46. 选择 “All Automation”，自动连接所有的线，点击 “OK” 开始自动连线



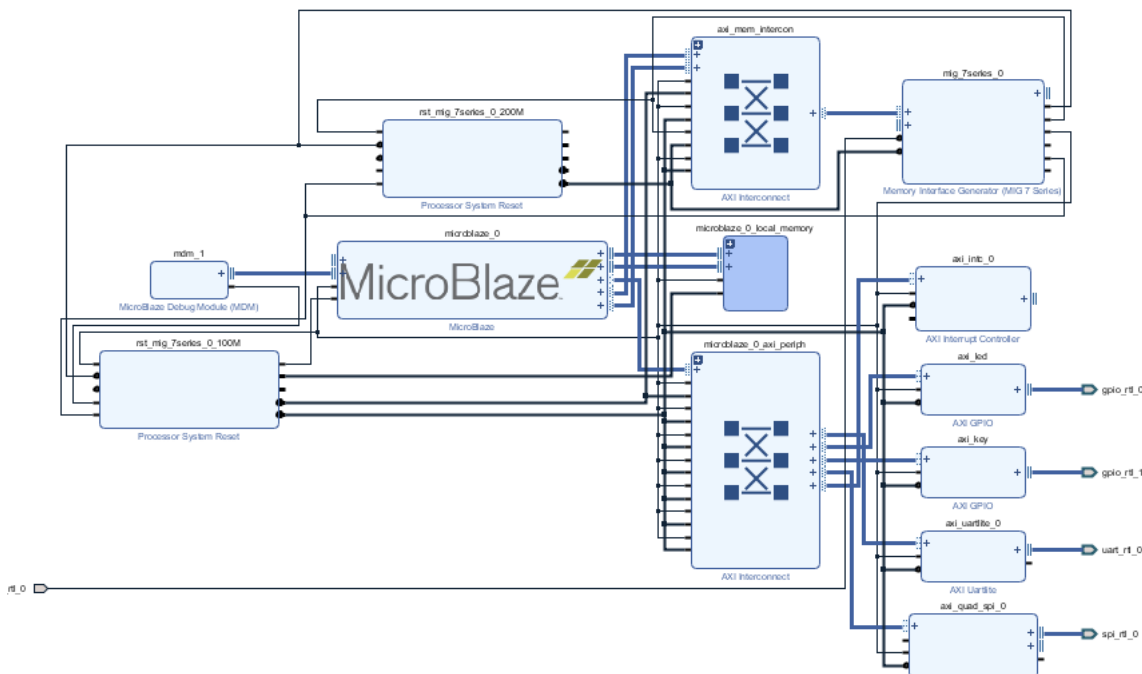
47. 手动选择时钟，“axi_intc_0”、“axi_key”等普通 AXI 接口外设时钟 “Crossbar clock source of interconnect IP”、“Clock source for Master interface” 和 “Clock source for Slave interface” 时钟都选择 mig 额外输出的 100Mh 时钟



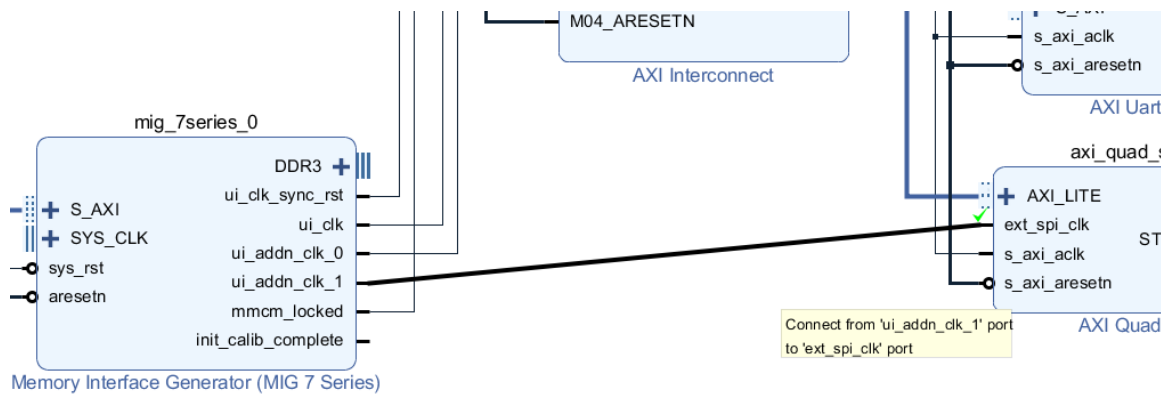
48. “mig_7series_0”的 AXI 接口时钟“Crossbar clock source of interconnect IP”、“Clock source for Master interface”和“Clock source for Slave interface”时钟都选择 mig 额外输出的 100Mh 时钟，“Interconnect IP”选择“New AXI Interconnect”



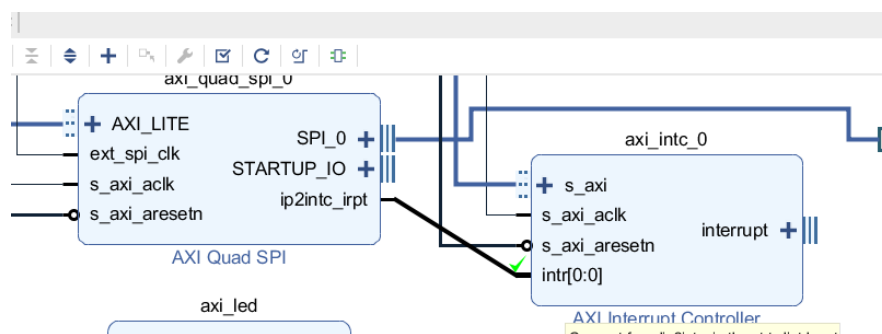
49. 自动连接完成



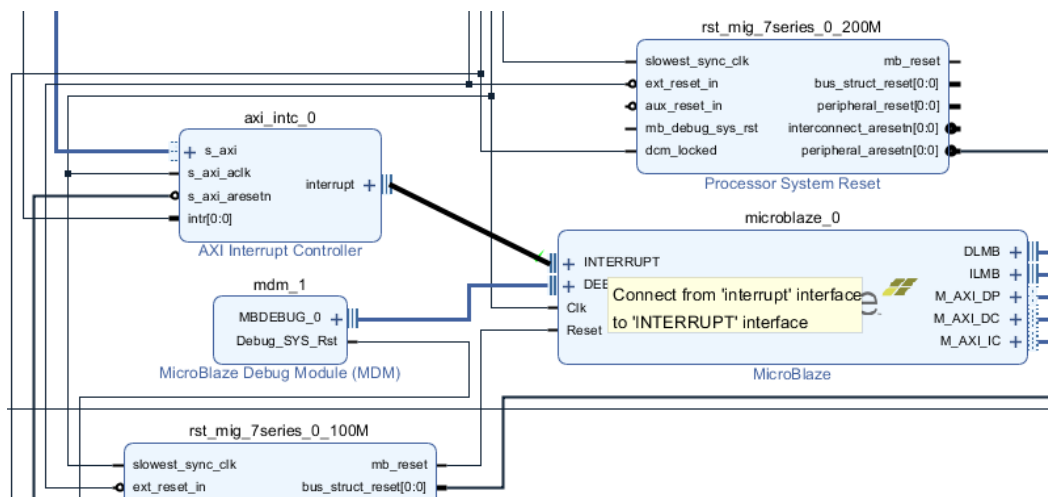
50. 连接 mig_7series_0 的 ui_addn_clk_1 到 axi_quad_spi_0 的 ext_spi_clk



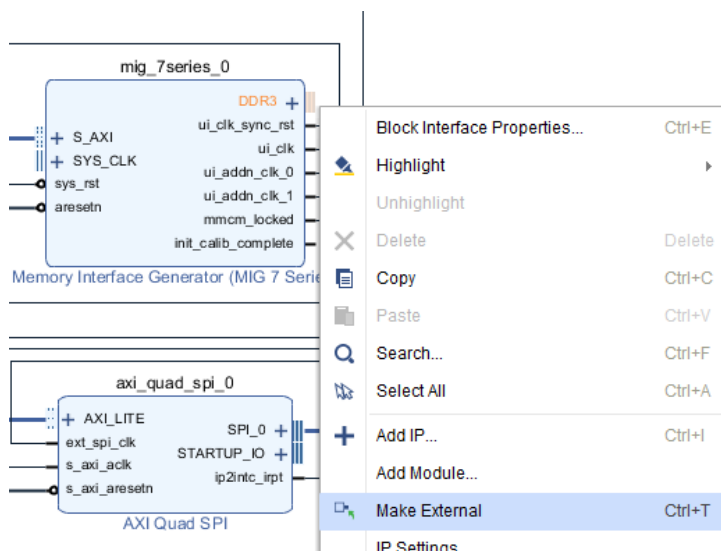
51. 连接 axi_quad_spi_0 的 ip2intc_irpt 到 axi_intc_0 的 intr



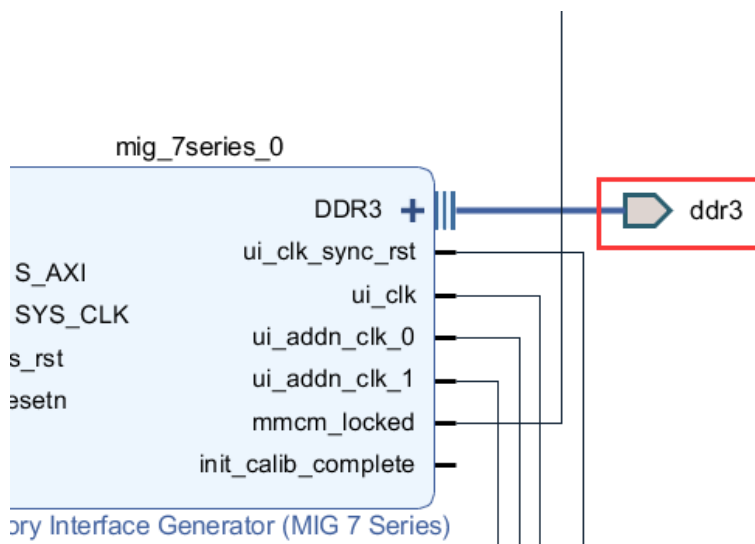
52. 连接 axi_intc_0 的 interrupt 到 microblaze_0 的 INTERRUPT



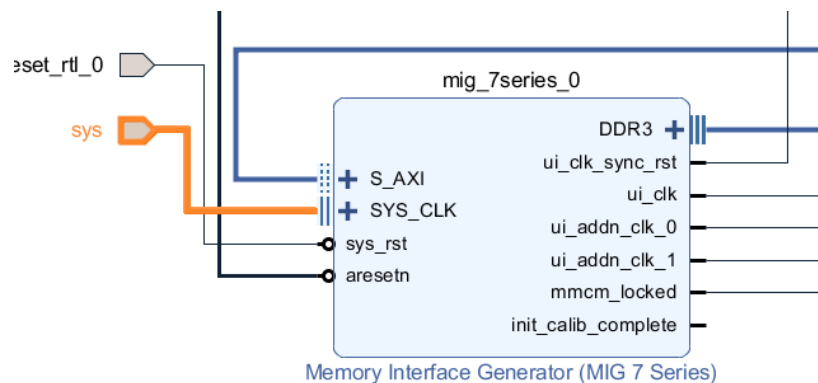
53. 导出 mig_7series_0 的 DDR3 端口



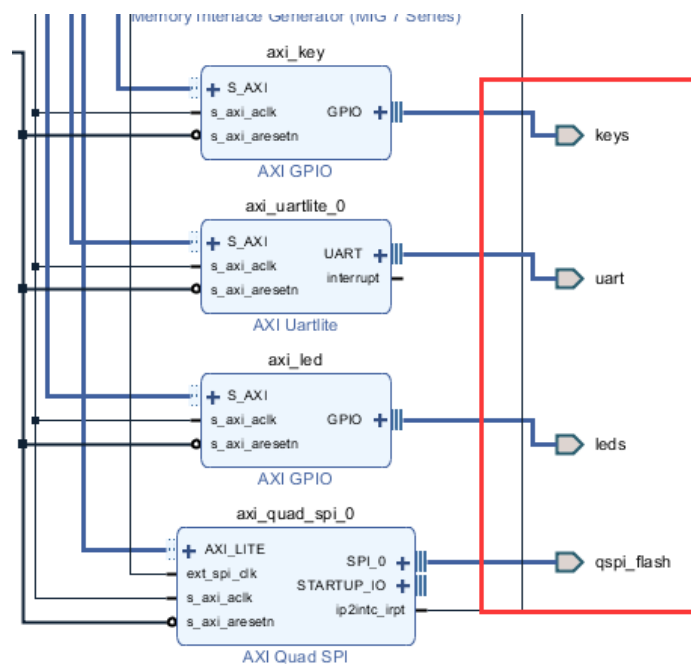
54. 端口名称改为小写 “ddr3”，改成其他可能会无法编译



55. 导出 mig_7series_0 的 SYS_CLK，并修改名称为“sys”，其他名称可能无法编译

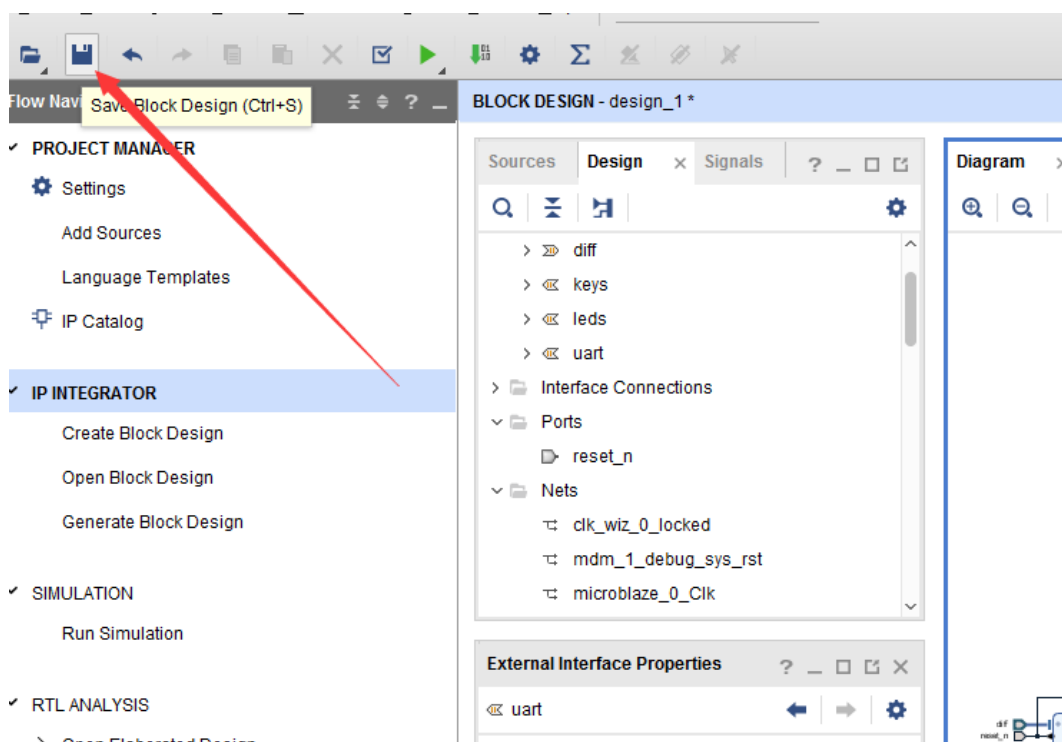


56. 修改其他端口名称，“reset_rtl_0”修改为“reset_n”，“gpio_rtl_1”修改为“keys”，“gpio_rtl_0”修改为“leds”，“uart_rtl_0”修改为“uart”，“spi_rtl_0”改为“qspi_flash”

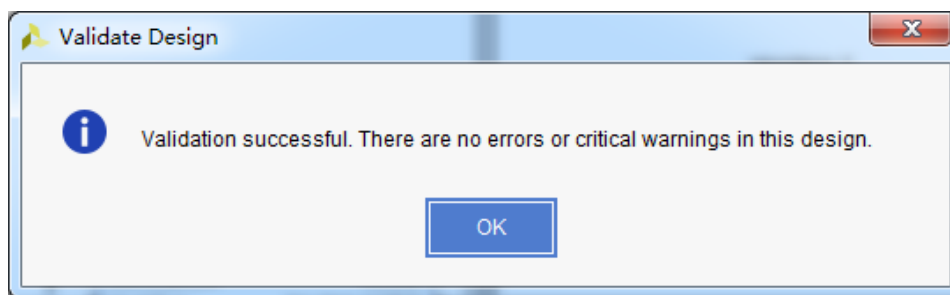


3.1.6 保存并检查错误

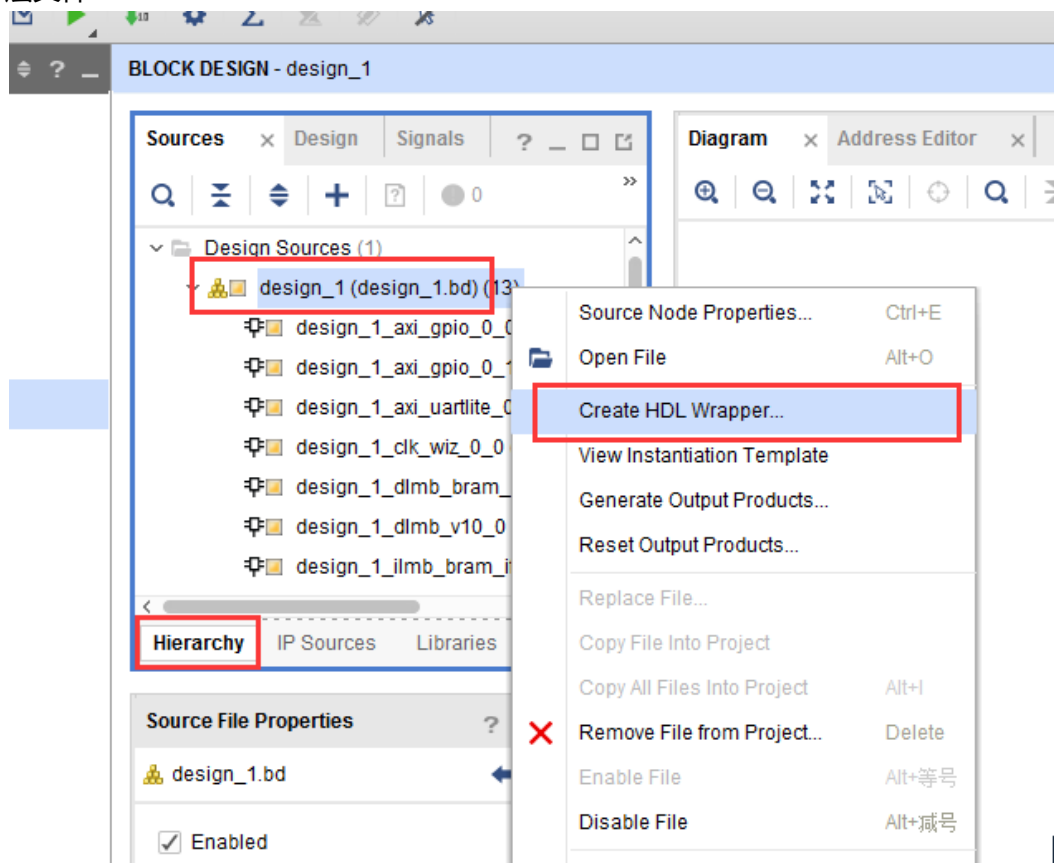
57. 保存设计，然后按“F6”检查设计



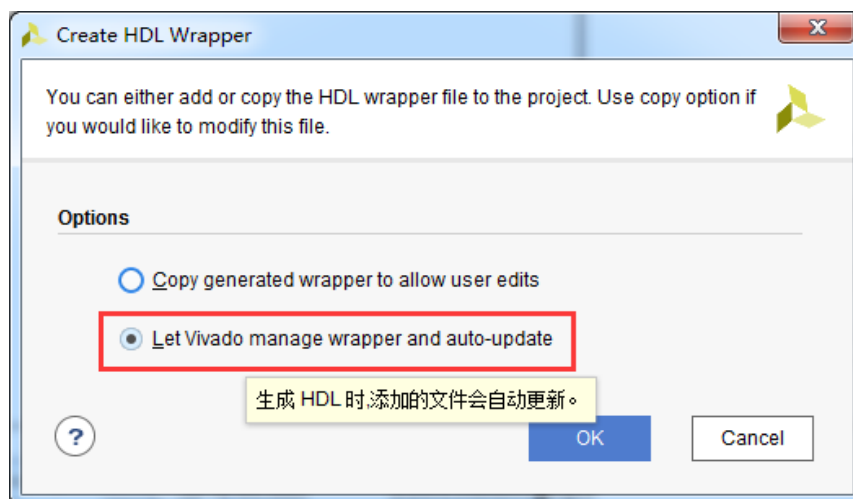
58. 设计中没有错误和警告



59. 选择 “design_1.bd” 文件，右键选择 “Create HDL Wrapper...” 创建一个 Verilog 或 VHDL 顶层文件



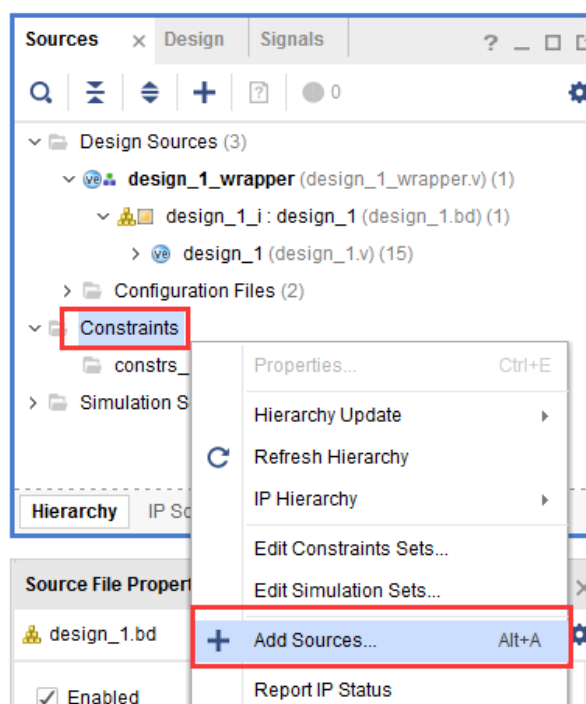
60. 选择 “Let Vivado manage wrapper and auto-update”，让 Vivado 自动更新 Verilog 或 VHDL 文件

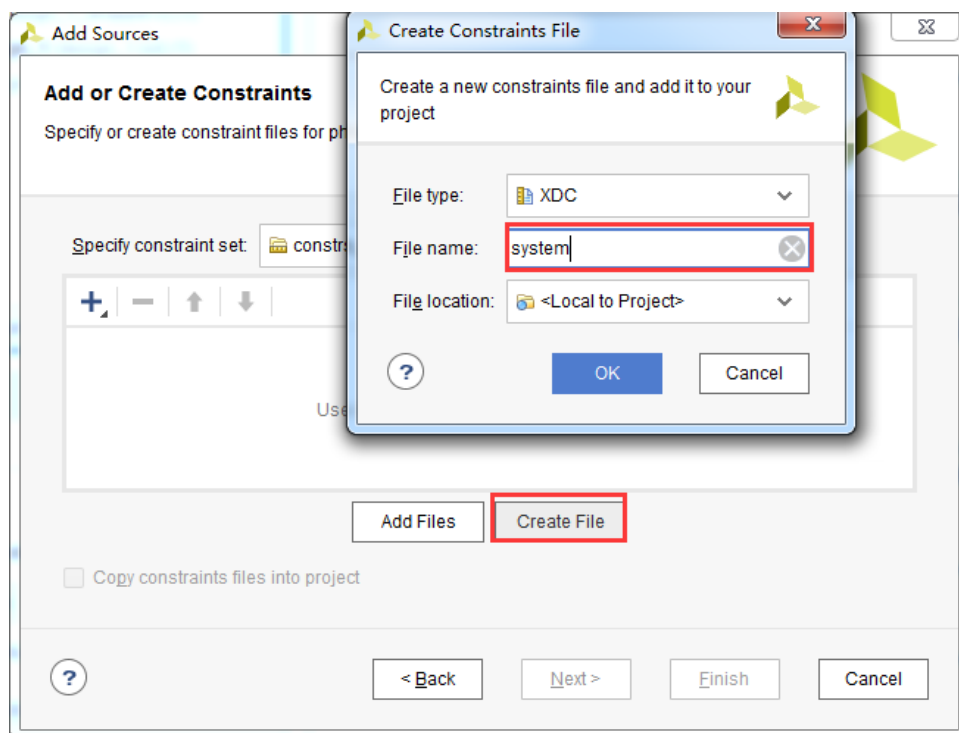
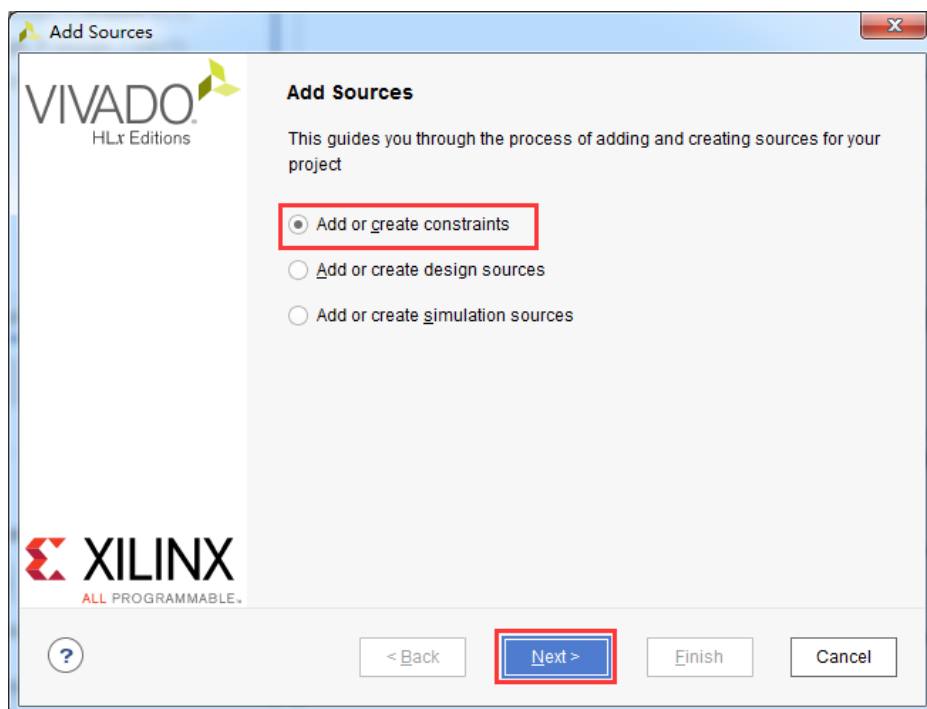


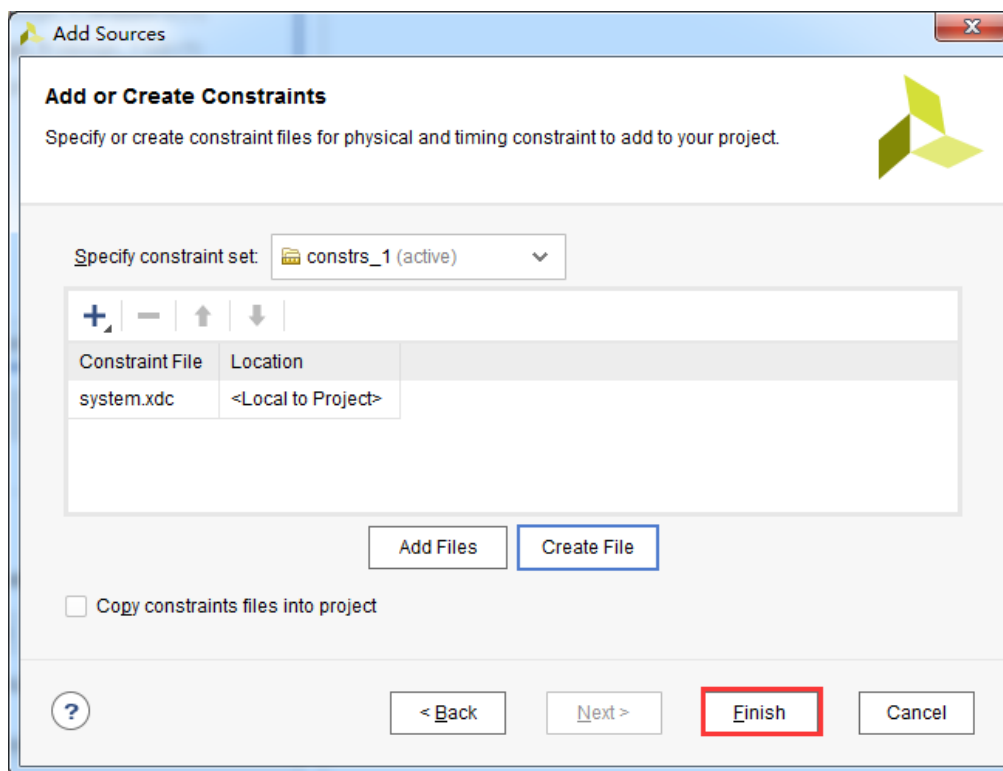
61. 双击打开生成的 “design_1_wrapper.v”, 可以看到一些端口, 下面为这些端口分配 IO

3.1.7 添加 xdc 约束文件

62. 选择 “Constraints”, 右键选择 “Add Sources...”







63. 修改 xdc 文件内容，请仔细检查 xdc 内的端口名称和 “design_1_wrapper.v” 内端口是否一致

system.xdc

```

1 set_property BITSTREAM.GENERAL.COMPRESS TRUE [current_design]
2 set_property BITSTREAM.CONFIG.CONFIGRATE 50 [current_design]
3 set_property CONFIG_VOLTAGE 3.3 [current_design]
4 set_property CFGBVS VCC0 [current_design]
5 set_property BITSTREAM.CONFIG.SPI_BUSWIDTH 4 [current_design]
6
7 set_property IOSTANDARD LVCMOS33 [get_ports reset_n]
8 set_property PACKAGE_PIN AD24 [get_ports reset_n]
9
10 set_property IOSTANDARD LVCMOS33 [get_ports uart_rxd]
11 set_property IOSTANDARD LVCMOS33 [get_ports uart_txd]
12 set_property PACKAGE_PIN AA27 [get_ports uart_rxd]
13 set_property PACKAGE_PIN W28 [get_ports uart_txd]
14
15 set_property IOSTANDARD LVCMOS33 [get_ports {keys_tri_i[0]}]
16 set_property PACKAGE_PIN AC24 [get_ports {keys_tri_i[0]}]
17 set_property IOSTANDARD LVCMOS33 [get_ports {leds_tri_o[0]}]
18 set_property PACKAGE_PIN Y28 [get_ports {leds_tri_o[0]}]
19
20 set_property IOSTANDARD LVCMOS33 [get_ports {qspi_flash_ss_io[0]}]
21 set_property IOSTANDARD LVCMOS33 [get_ports qspi_flash_io0_io]
22 set_property IOSTANDARD LVCMOS33 [get_ports qspi_flash_io1_io]
23 set_property IOSTANDARD LVCMOS33 [get_ports qspi_flash_io2_io]
24 set_property IOSTANDARD LVCMOS33 [get_ports qspi_flash_io3_io]
25 set_property PACKAGE_PIN U19 [get_ports {qspi_flash_ss_io[0]}]
26 set_property PACKAGE_PIN P24 [get_ports qspi_flash_io0_io]
27 set_property PACKAGE_PIN R25 [get_ports qspi_flash_io1_io]
28 set_property PACKAGE_PIN R20 [get_ports qspi_flash_io2_io]
29 set_property PACKAGE_PIN R21 [get_ports qspi_flash_io3_io]

```

External Port Properties

reset_n

Name: reset_n

Direction: Input

Net: reset_rtl_0_1

Clock Port:

Polarity: Active Low

```

set_property BITSTREAM.GENERAL.COMPRESS TRUE [current_design]
set_property BITSTREAM.CONFIG.CONFIGRATE 50 [current_design]
set_property CONFIG_VOLTAGE 3.3 [current_design]
set_property CFGBVS VCCO [current_design]
set_property BITSTREAM.CONFIG.SPI_BUSWIDTH 4 [current_design]

set_property IOSTANDARD LVCMOS33 [get_ports reset_n]
set_property PACKAGE_PIN AD24 [get_ports reset_n]

set_property IOSTANDARD LVCMOS33 [get_ports uart_rxd]
set_property IOSTANDARD LVCMOS33 [get_ports uart_txd]
set_property PACKAGE_PIN AA27 [get_ports uart_rxd]
set_property PACKAGE_PIN W28 [get_ports uart_txd]

set_property IOSTANDARD LVCMOS33 [get_ports {keys_tri_i[0]}]
set_property PACKAGE_PIN AC24 [get_ports {keys_tri_i[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {leds_tri_o[0]}]
set_property PACKAGE_PIN Y28 [get_ports {leds_tri_o[0]}]

set_property IOSTANDARD LVCMOS33 [get_ports {qspi_flash_ss_io[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports qspi_flash_io0_io]
set_property IOSTANDARD LVCMOS33 [get_ports qspi_flash_io1_io]
set_property IOSTANDARD LVCMOS33 [get_ports qspi_flash_io2_io]
set_property IOSTANDARD LVCMOS33 [get_ports qspi_flash_io3_io]
set_property PACKAGE_PIN U19 [get_ports {qspi_flash_ss_io[0]}]
set_property PACKAGE_PIN P24 [get_ports qspi_flash_io0_io]
set_property PACKAGE_PIN R25 [get_ports qspi_flash_io1_io]
set_property PACKAGE_PIN R20 [get_ports qspi_flash_io2_io]
set_property PACKAGE_PIN R21 [get_ports qspi_flash_io3_io]

```

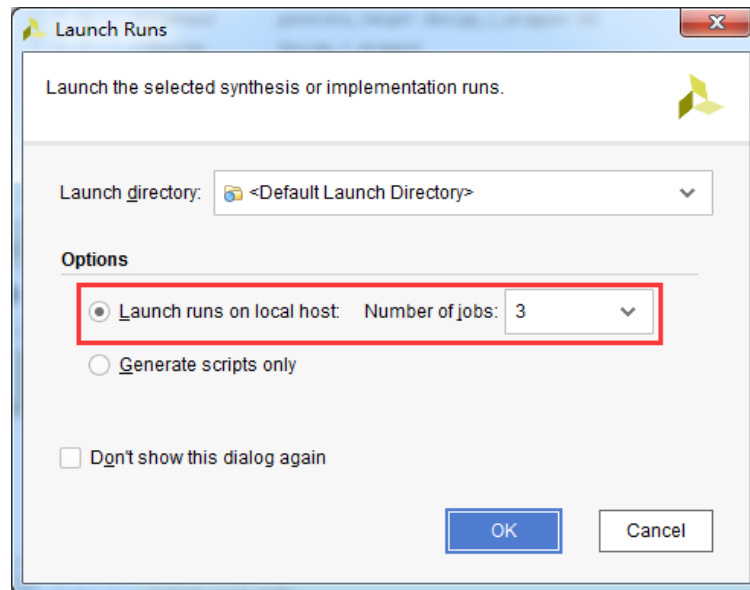
*注因为 DDR 的管脚和时钟管脚已经在 IP 里分配好了，这里不需要再分配。

3.1.8 生成 Bitstream 文件

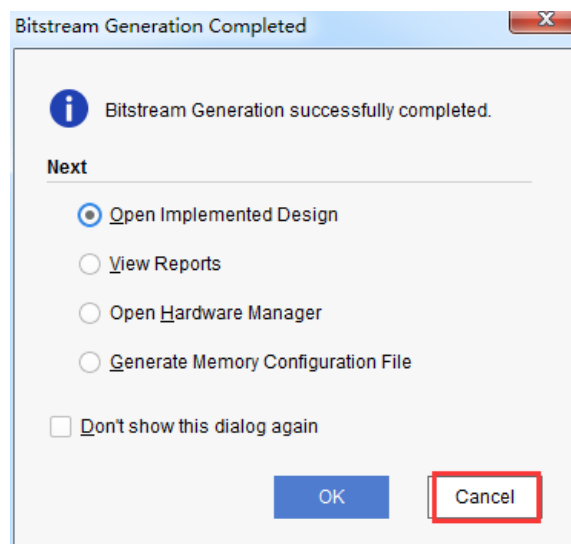
64. 点击 “Generate Bitstream” 生成 bit 流文件



65. 在弹出窗口中有运行路径选择，这里默认，“Number of jobs” 线程数量可以根据自己 CPU 选择，数字越大，编译越快。

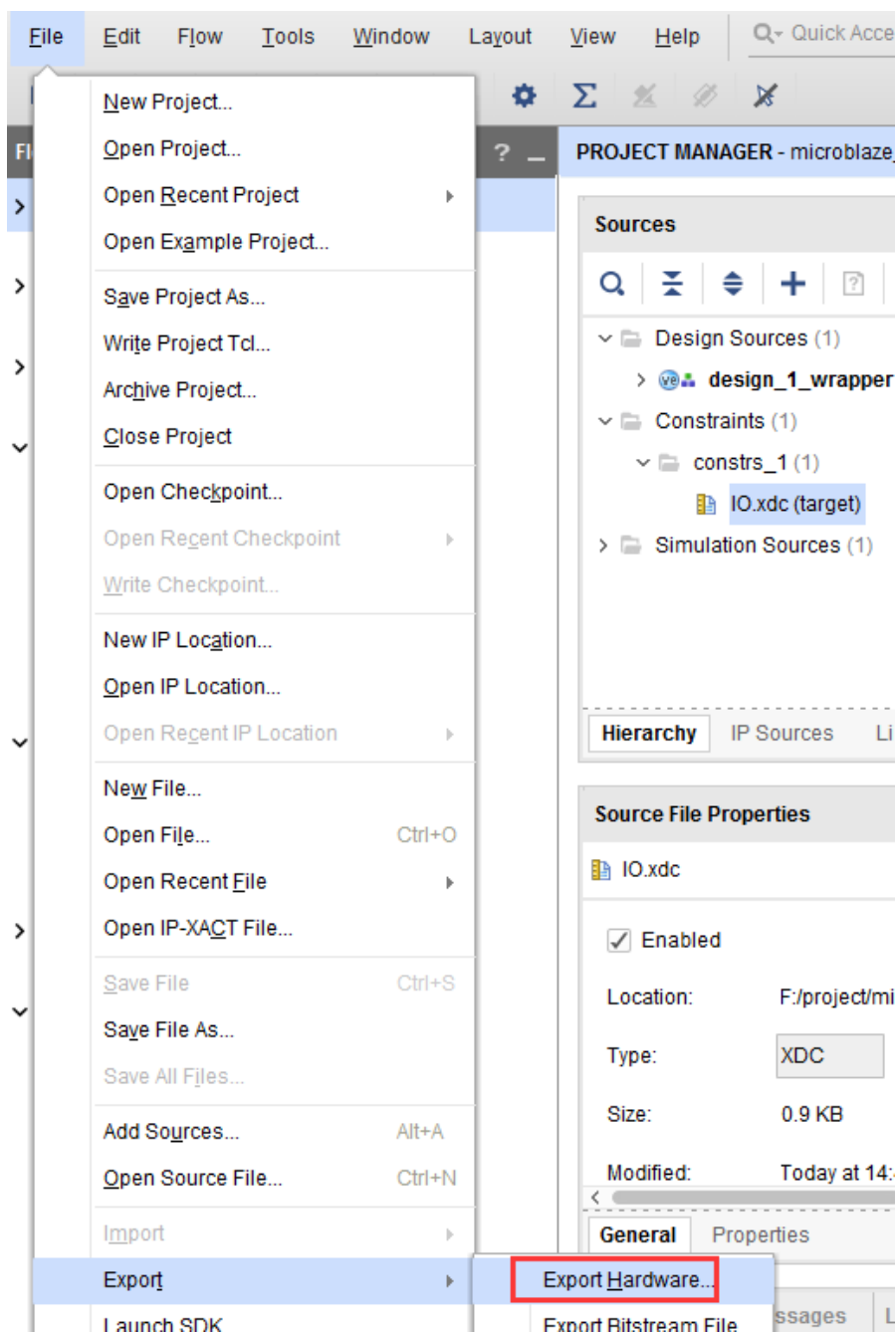


66. 生成完成以后，点击“Cancel”，不做任何操作

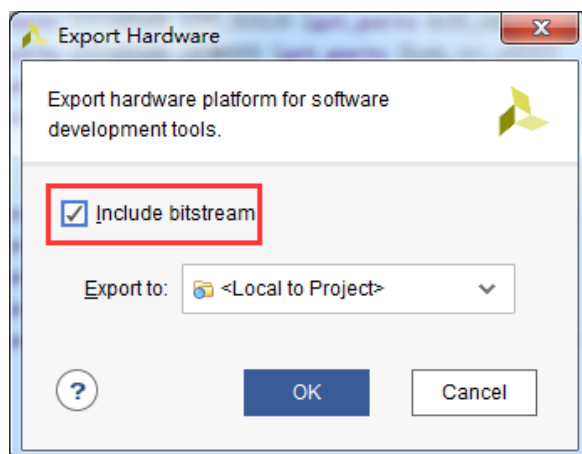


3.1.9 导出硬件

67. 在“File -> Export -> Export Hardware...” 导出硬件

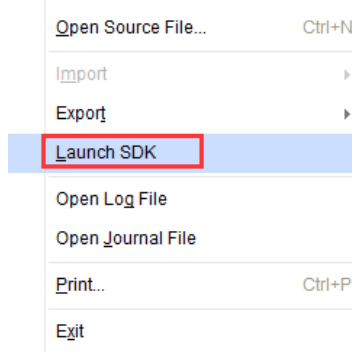


68. 在弹出窗口中选择 “Include bitstream” 包含 Bitstream 文件

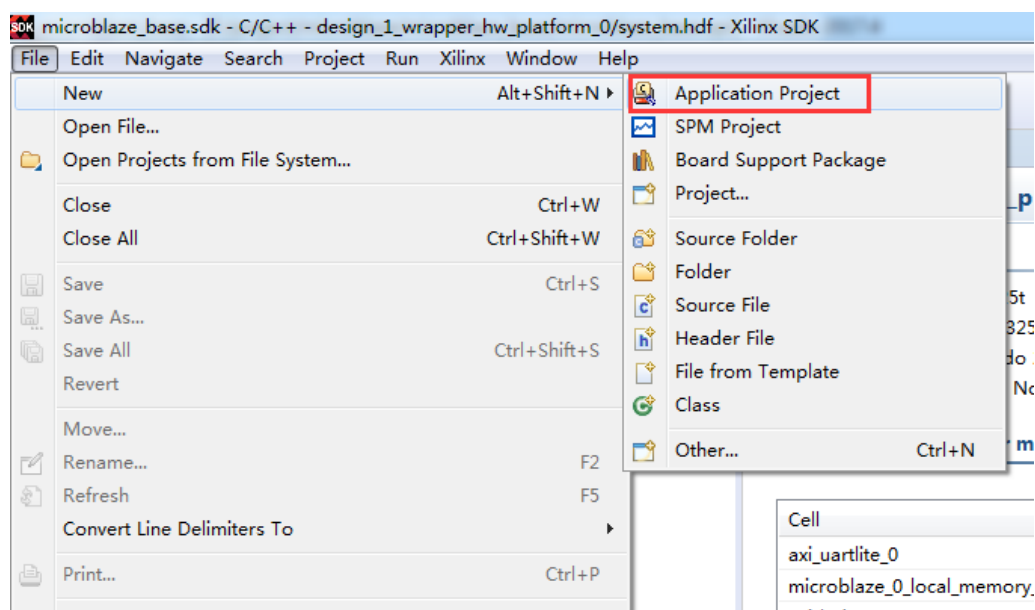


3.2 SDK 开发 “HelloWorld”

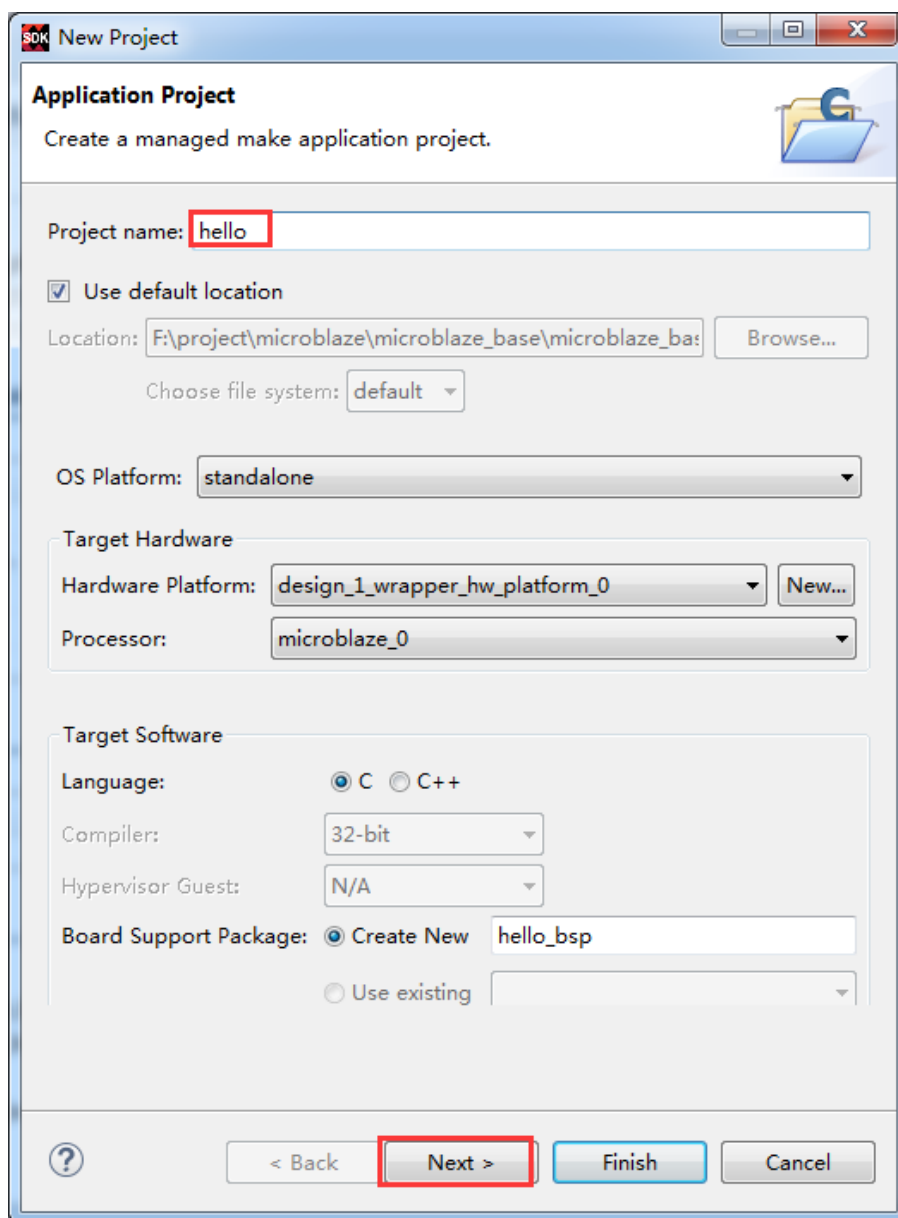
1. 点击 File→Launch SDK 进入 SDK 操作



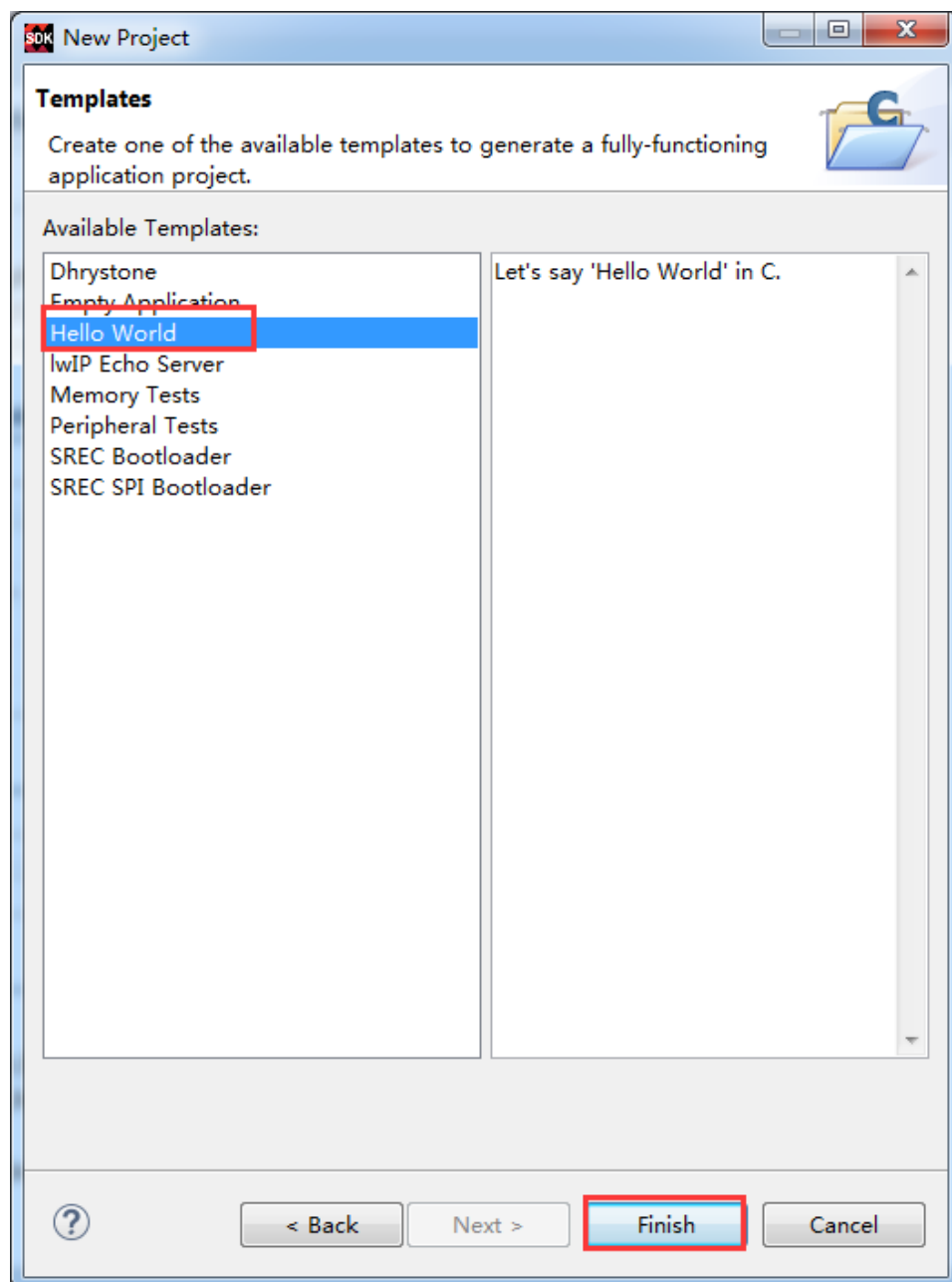
2. 新建工程



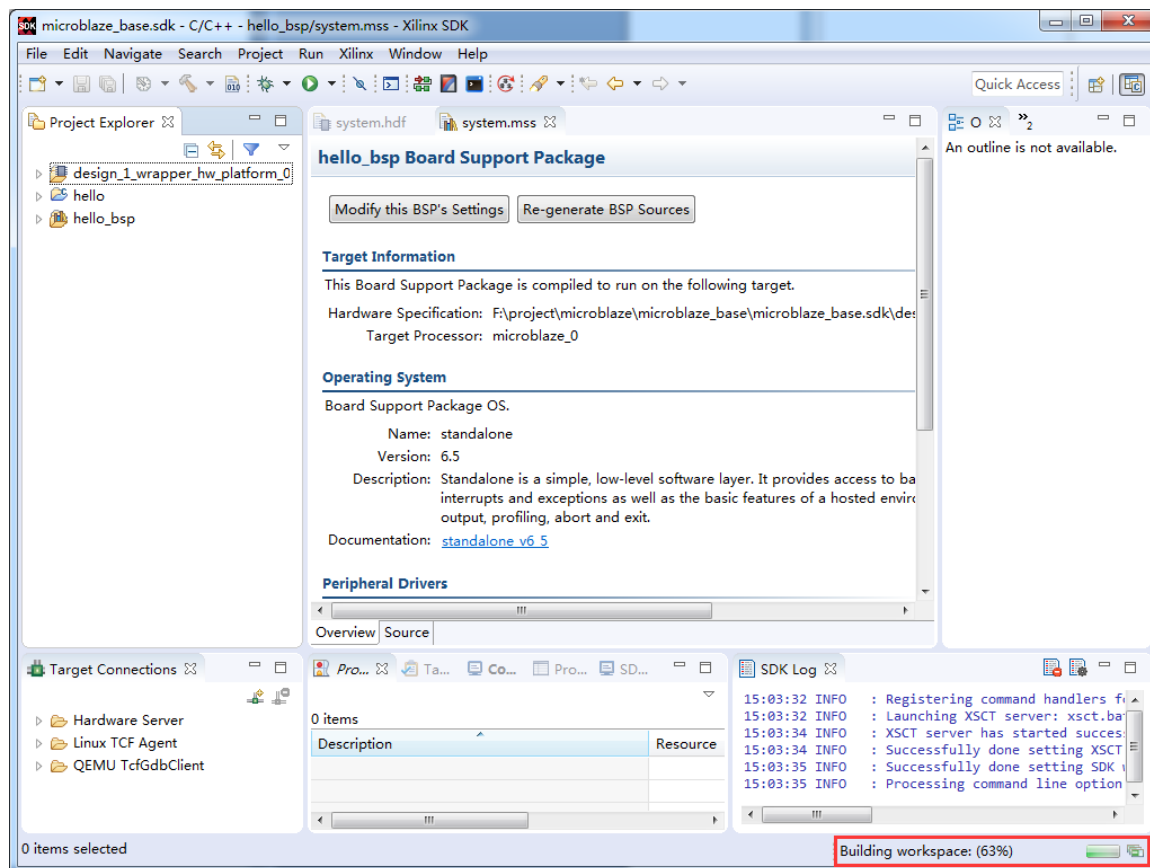
3. 设置工程名为 “hello”，点击 “Next”



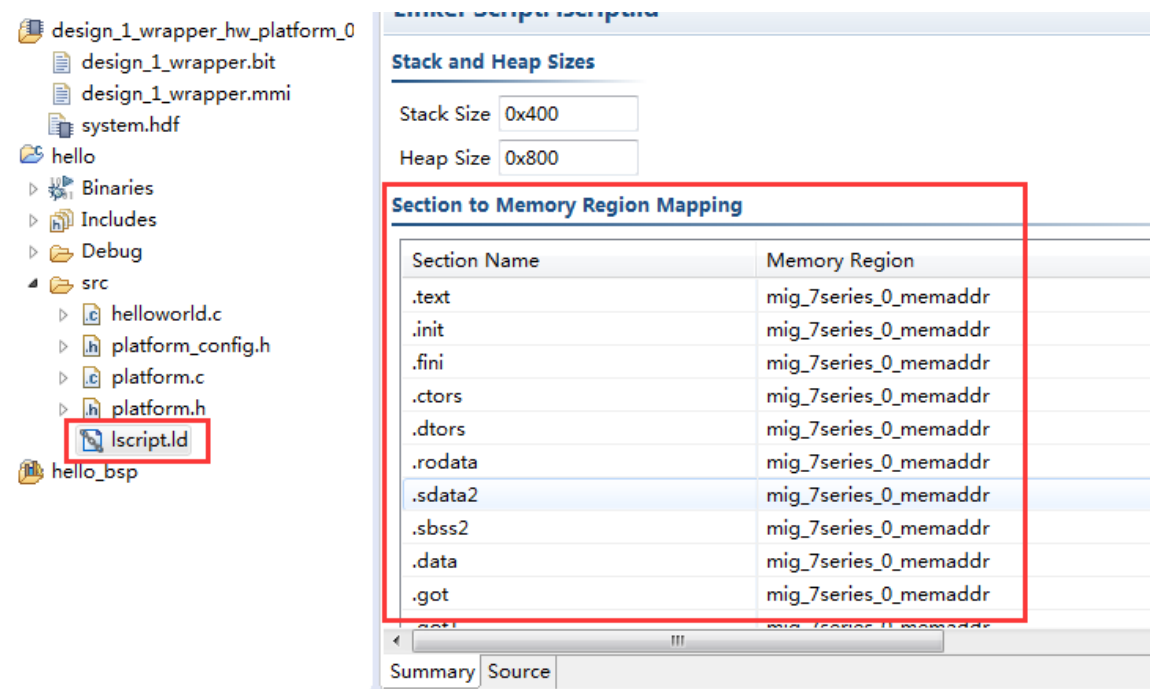
4. 模板选择 “Hello World”，点击 “Finish”



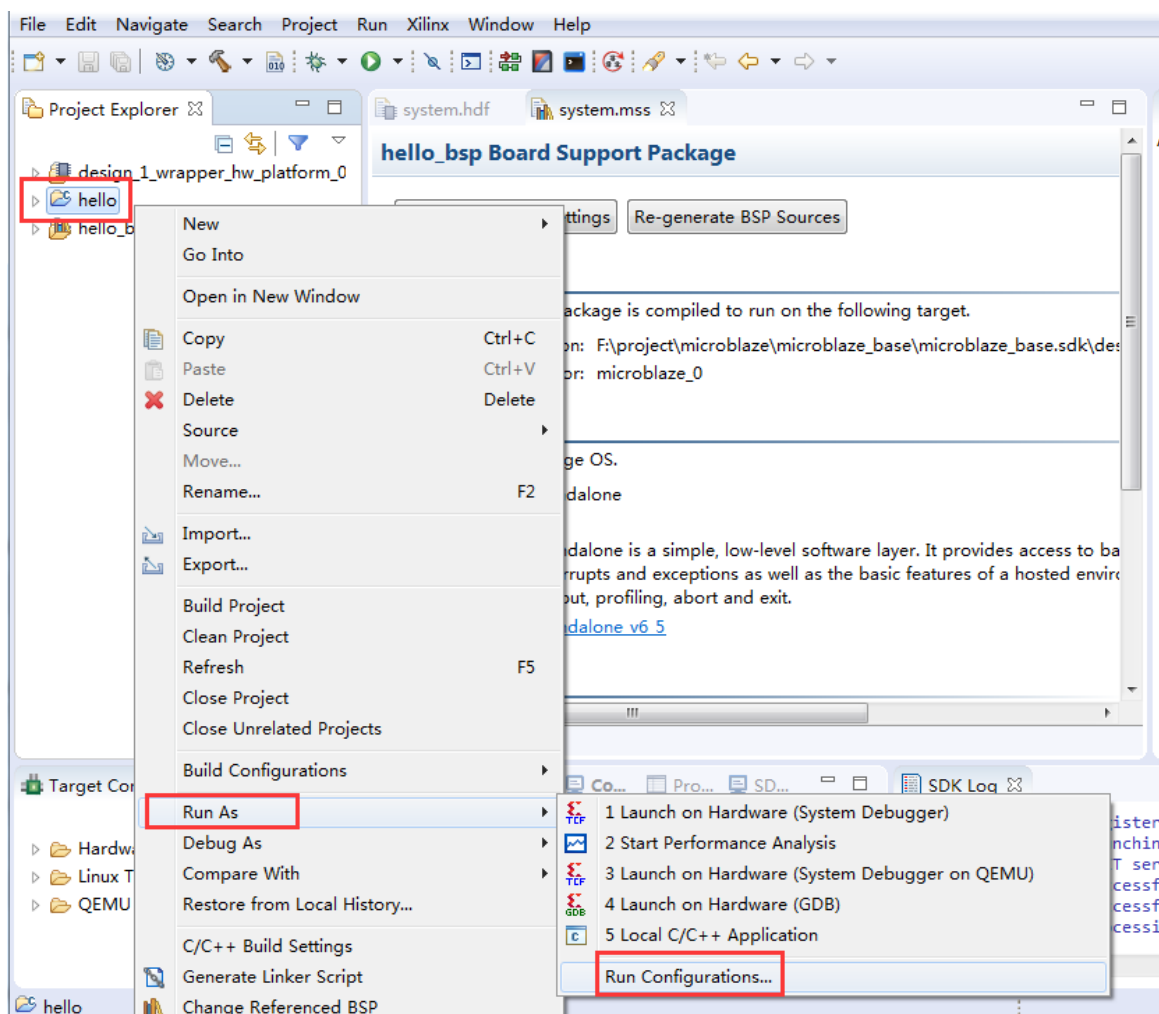
5. 工程建立完成以后就会自动编译，等待编译完成就可以调试运行



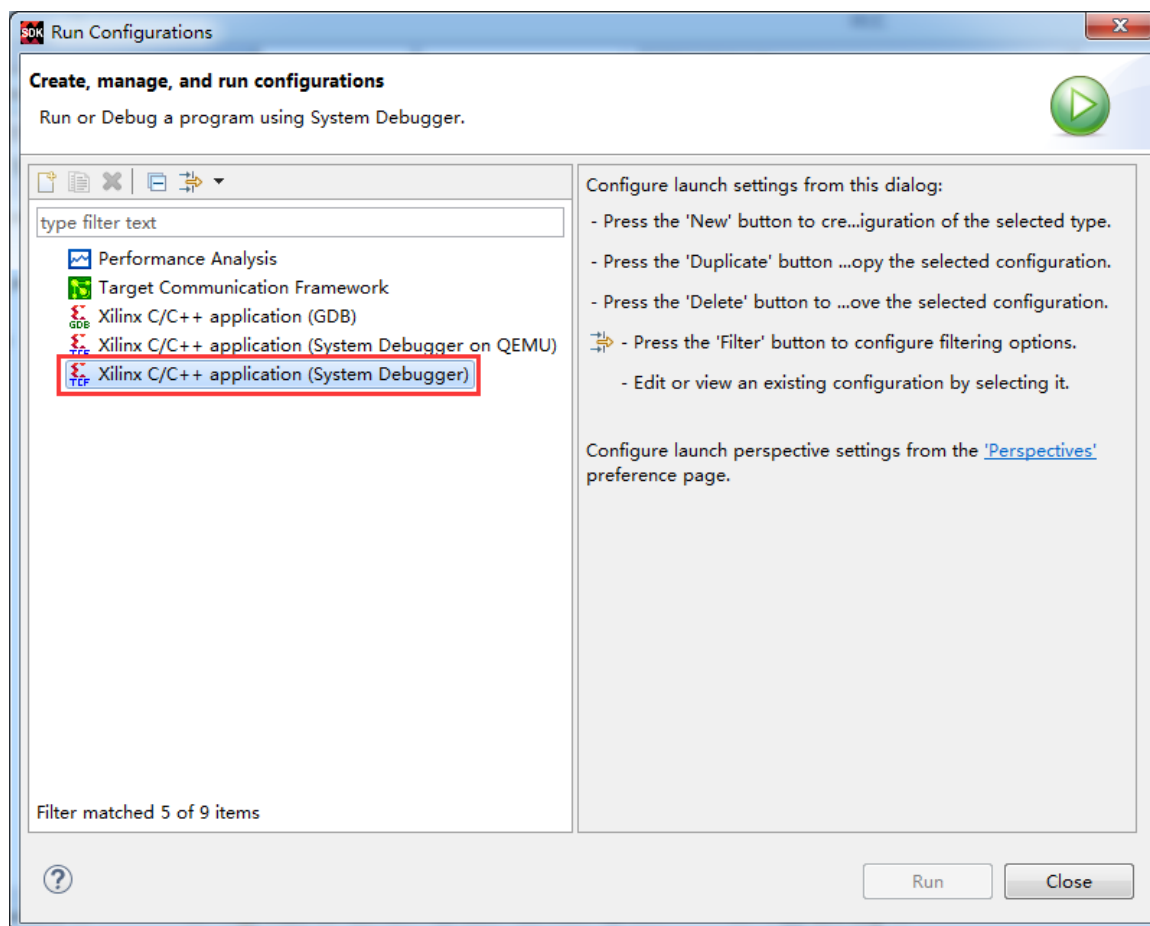
6. 双击 “lscript.ld” 文件可以查看堆栈设置，Hello 工程默认都运行在 ddr 上



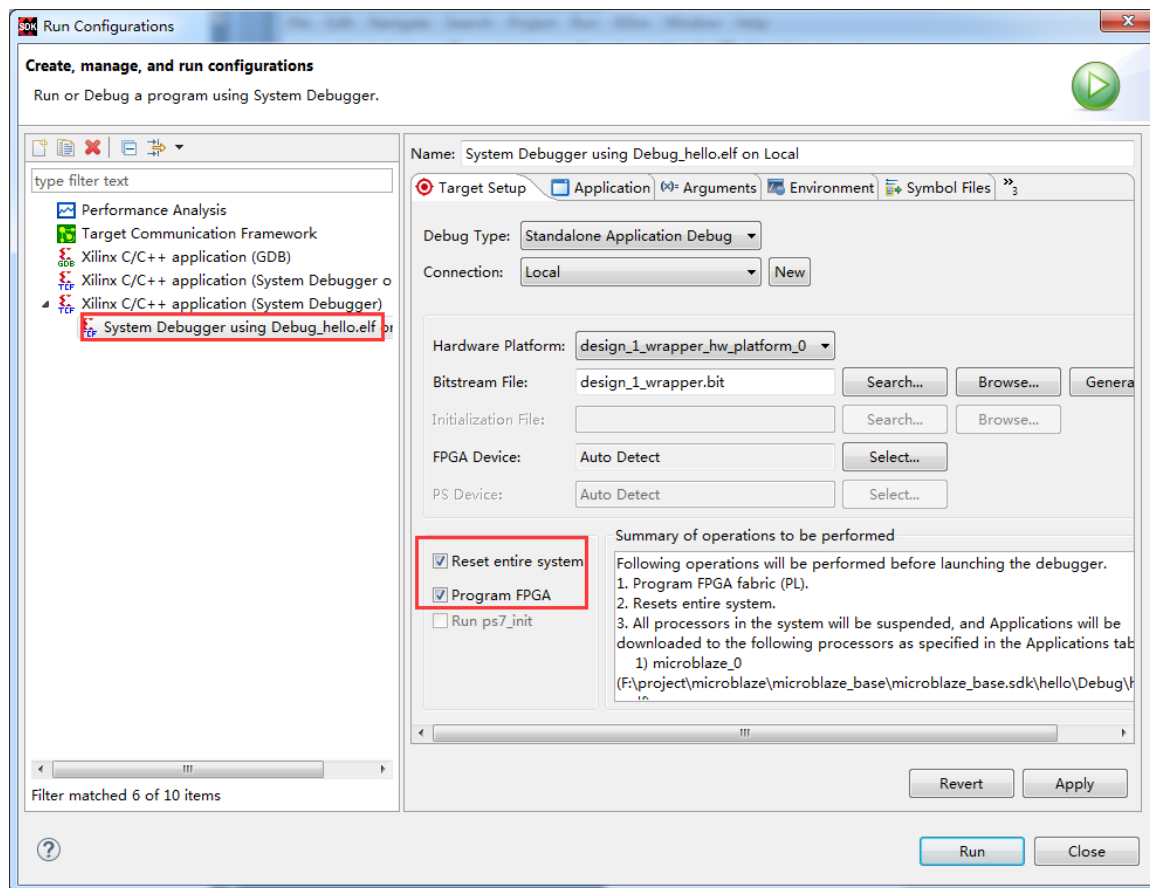
选择 “Hello” 工程，右键选择 “Run As -> Run Configuration...”



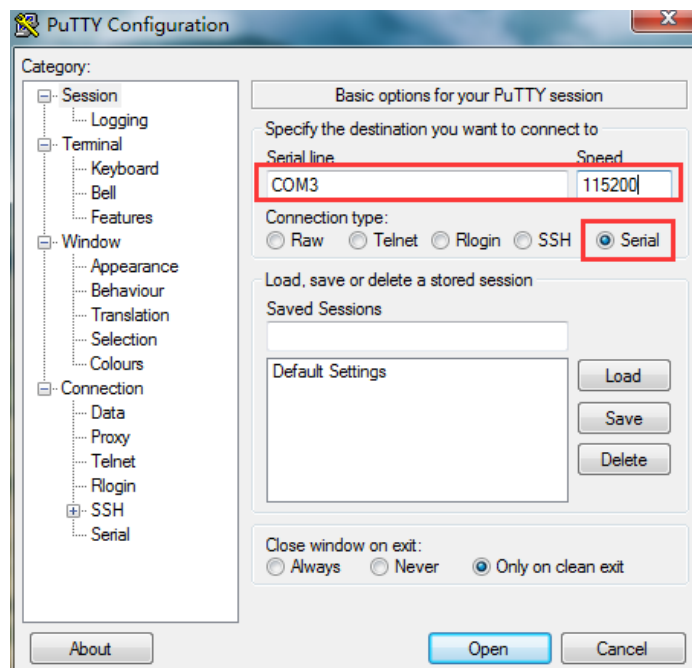
7. 双击 “Xilinx C/C++ application(System Debugger)”



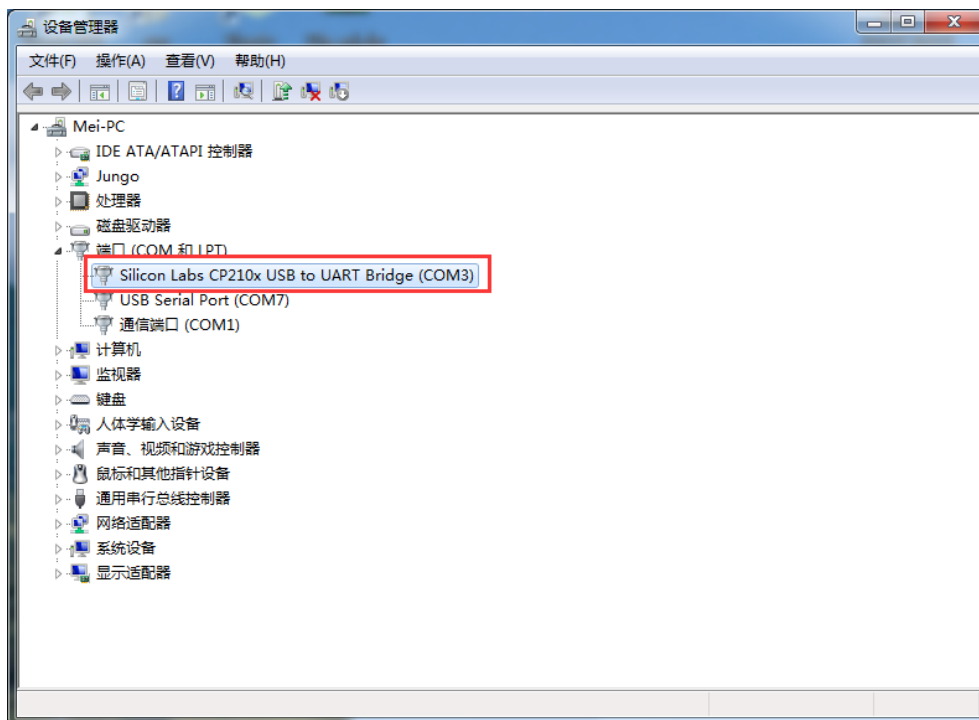
8. 勾选 “Reset entire system” 和 “Program FPGA” 复位整个系统并下载 FPGA



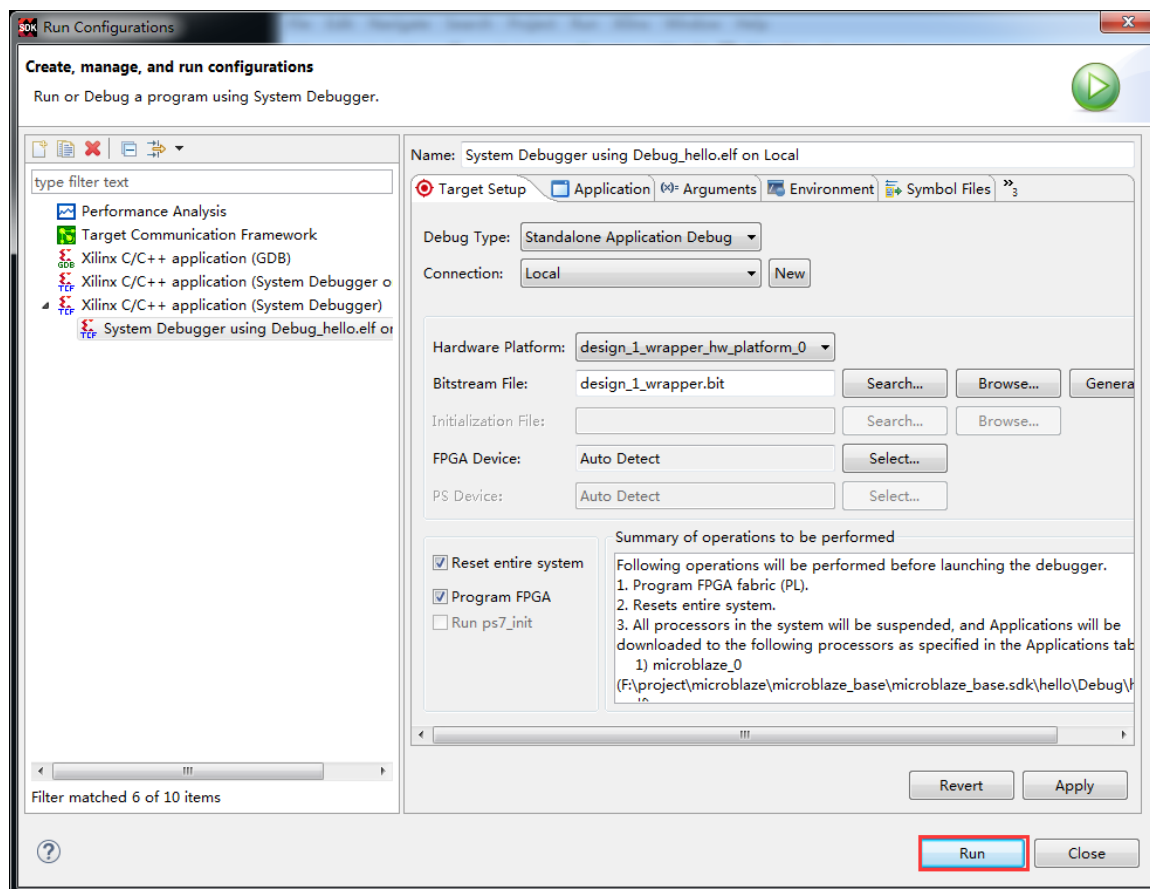
9. 连接 JTAG 线到开发板、UART 的 USB 线到 PC
10. 使用 PuTTY 软件做为串口终端调试工具，PuTTY 是一个免安装的小软件



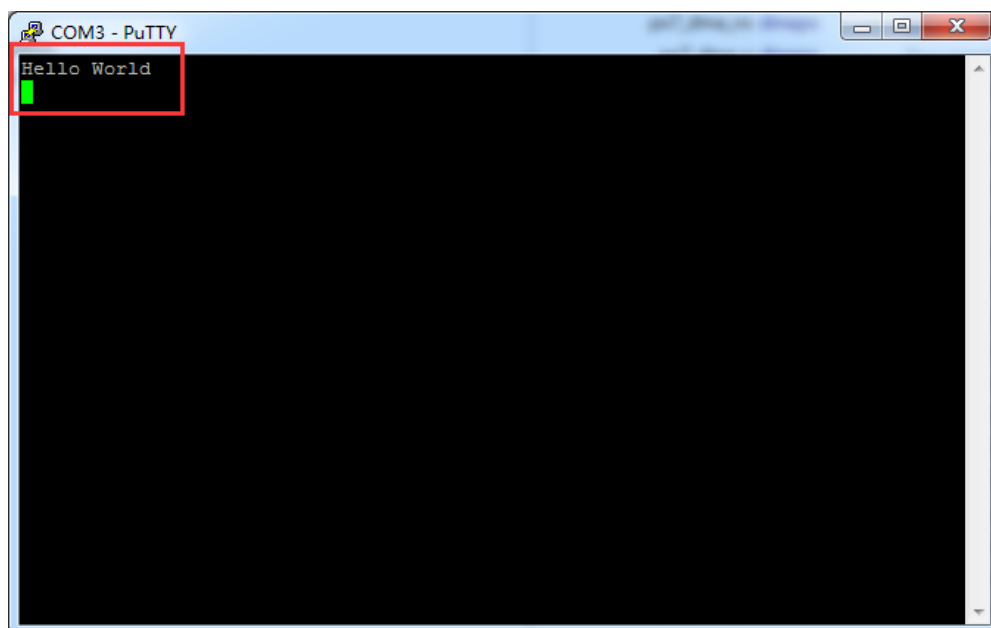
11. 选择 Serial，Serial line 填写 COM3，Speed 填写 115200，COM3 串口号根据设备管理器里显示的填写，点击“Open”



12. 点击 “Run”



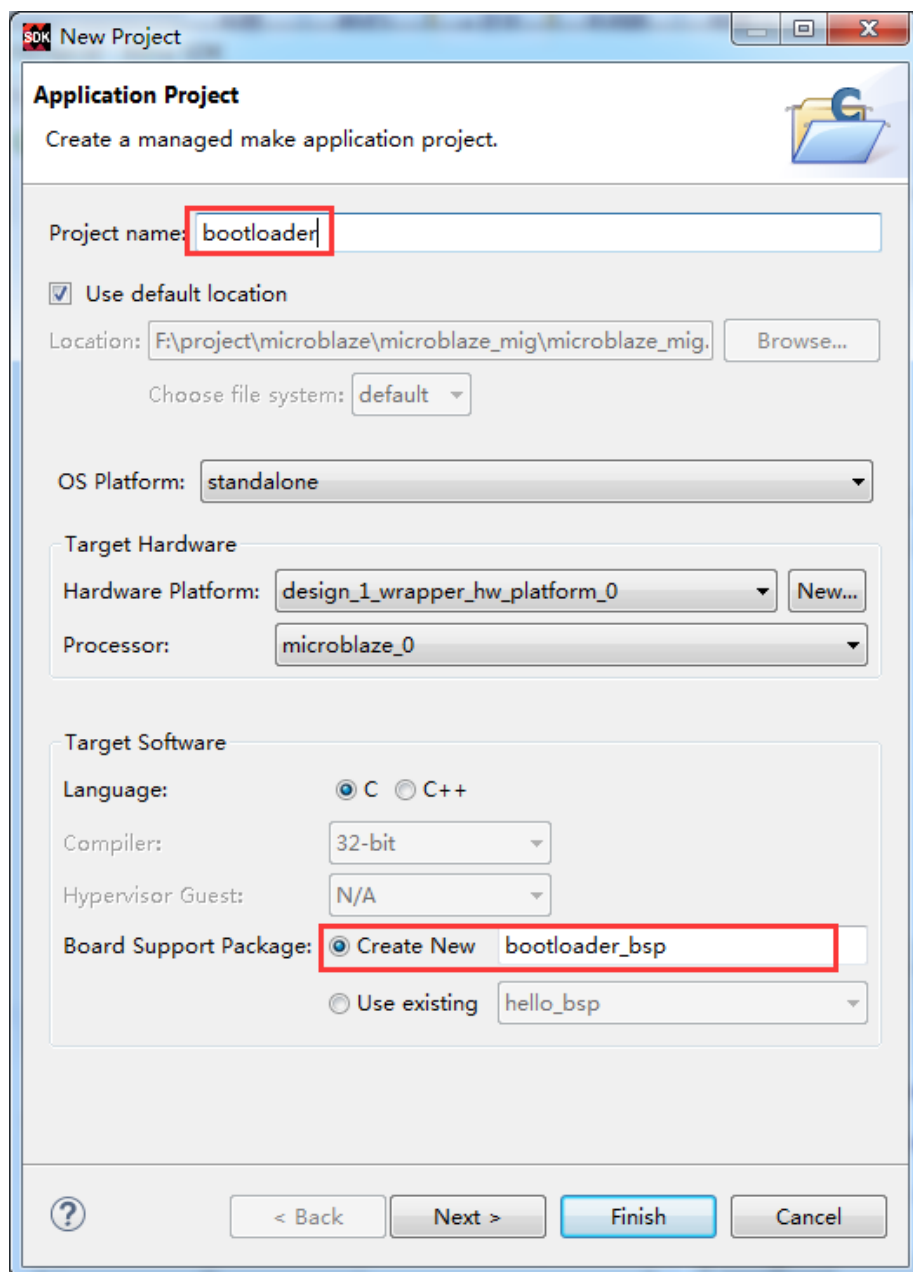
13. “Hello World” 显示出来了



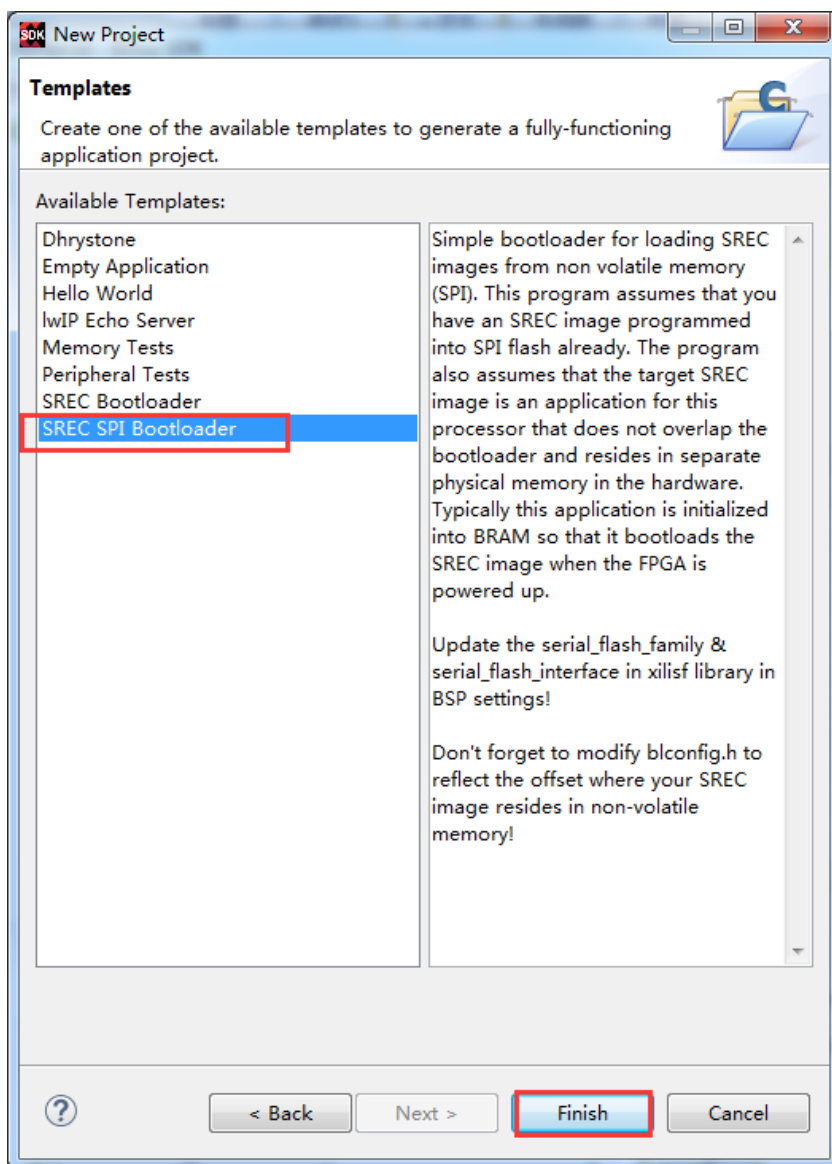
3.3 烧写程序到 Flash

3.3.1 BootLoader 工程

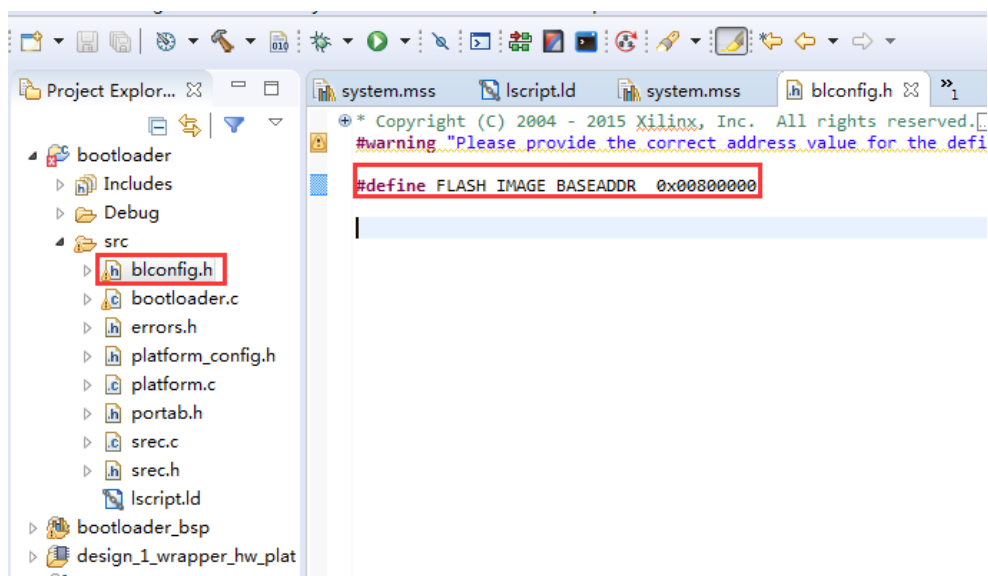
1. 建立 bootloader 工程



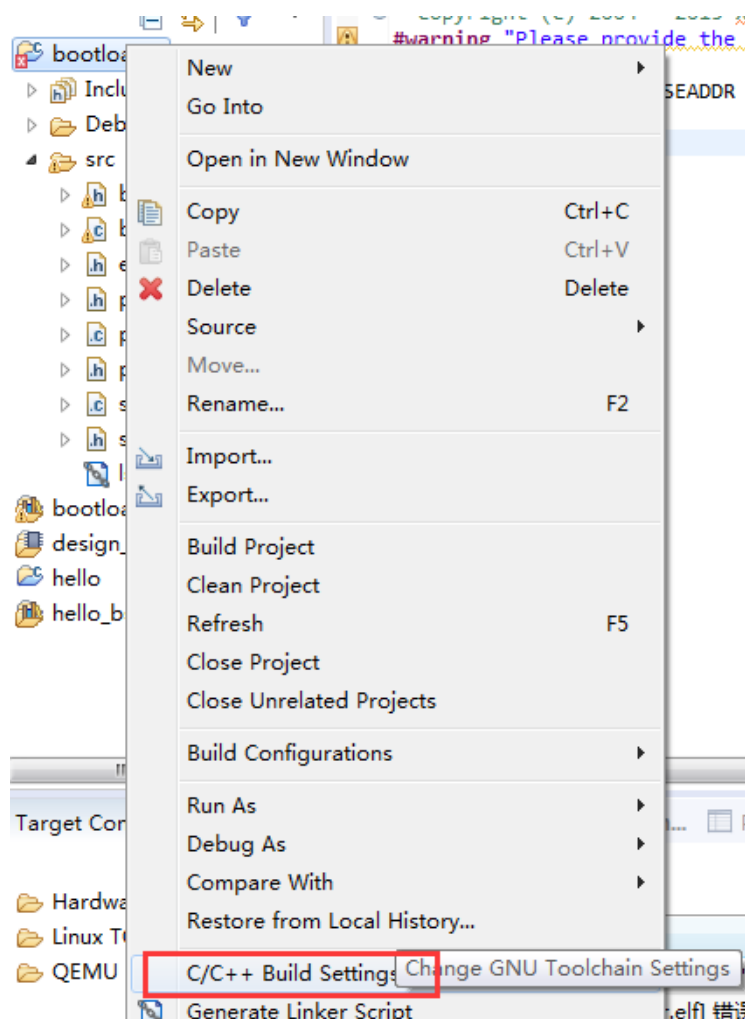
2. 选择 “SREC SPI Bootloader” 模板，点击 “Finish”



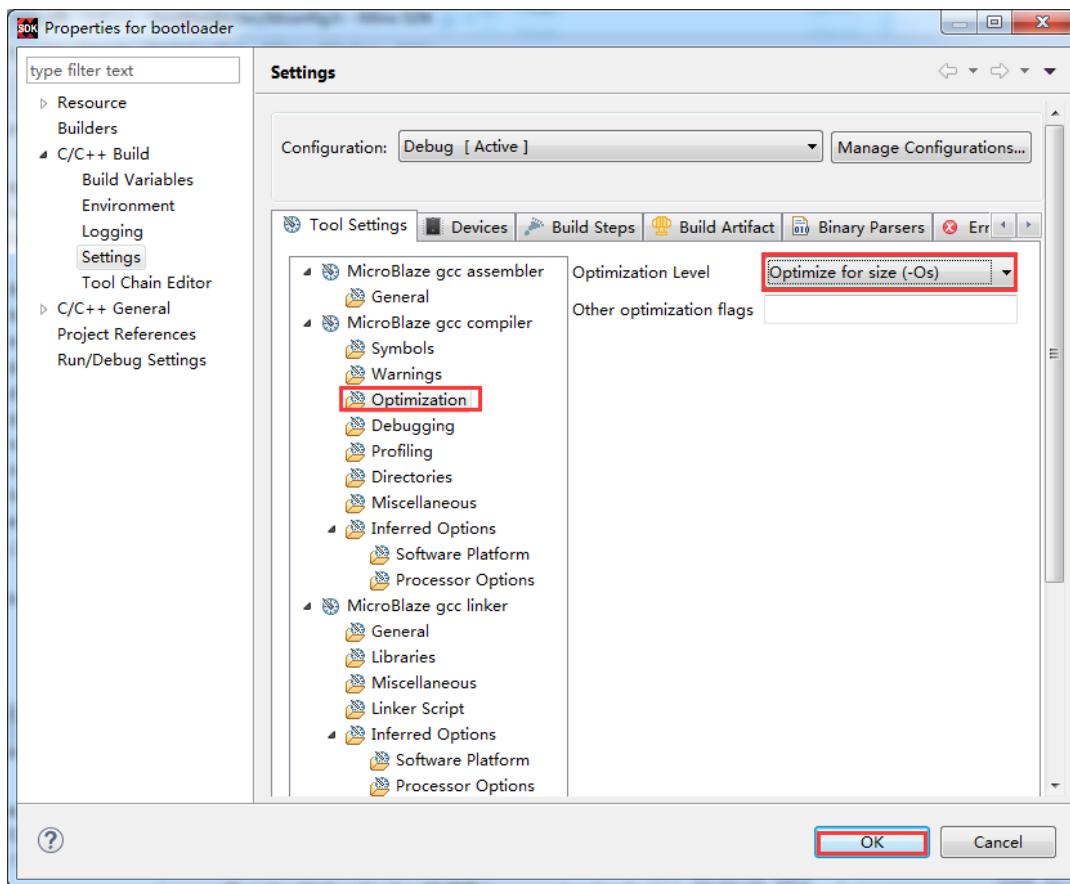
3. 修改 blconfig.h 文件，将 Microblaze 软件文件在 Flash 中的加载点修改为“0x00800000”，就是 Flash 前面的 8MB 保存 Bitstream 文件，8MB 开始保存软件。



11. Bootloader 自动编译出错，因为片上 RAM 分配 16K，要配置优化选项，减少代码体积，选择工程右键，点击 “C/C++ Build Settings”

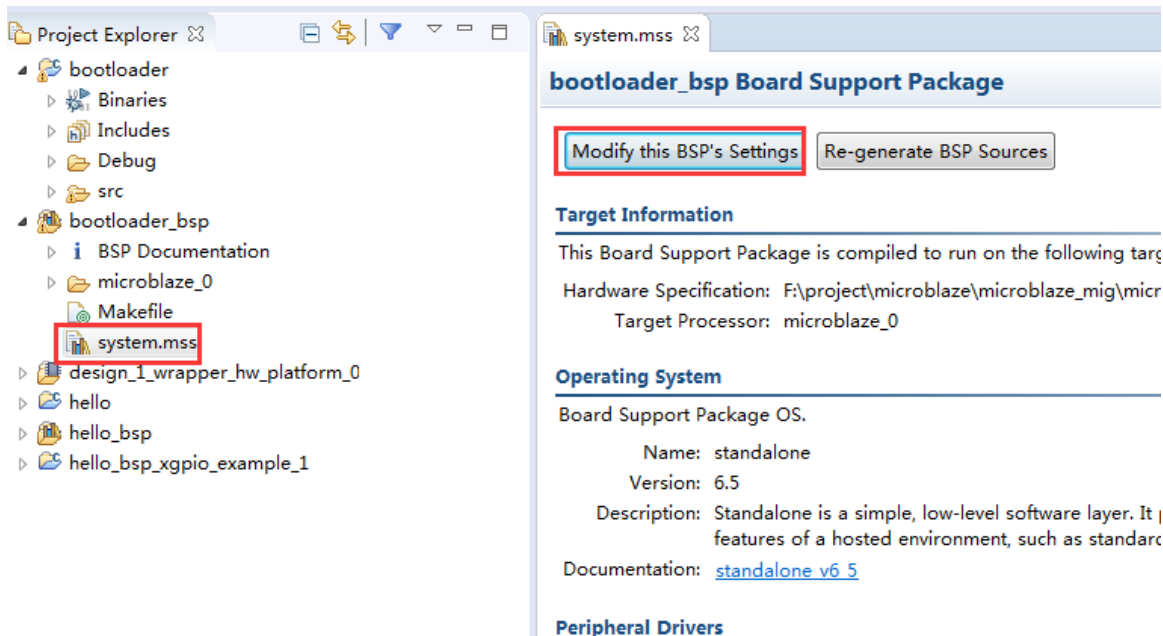


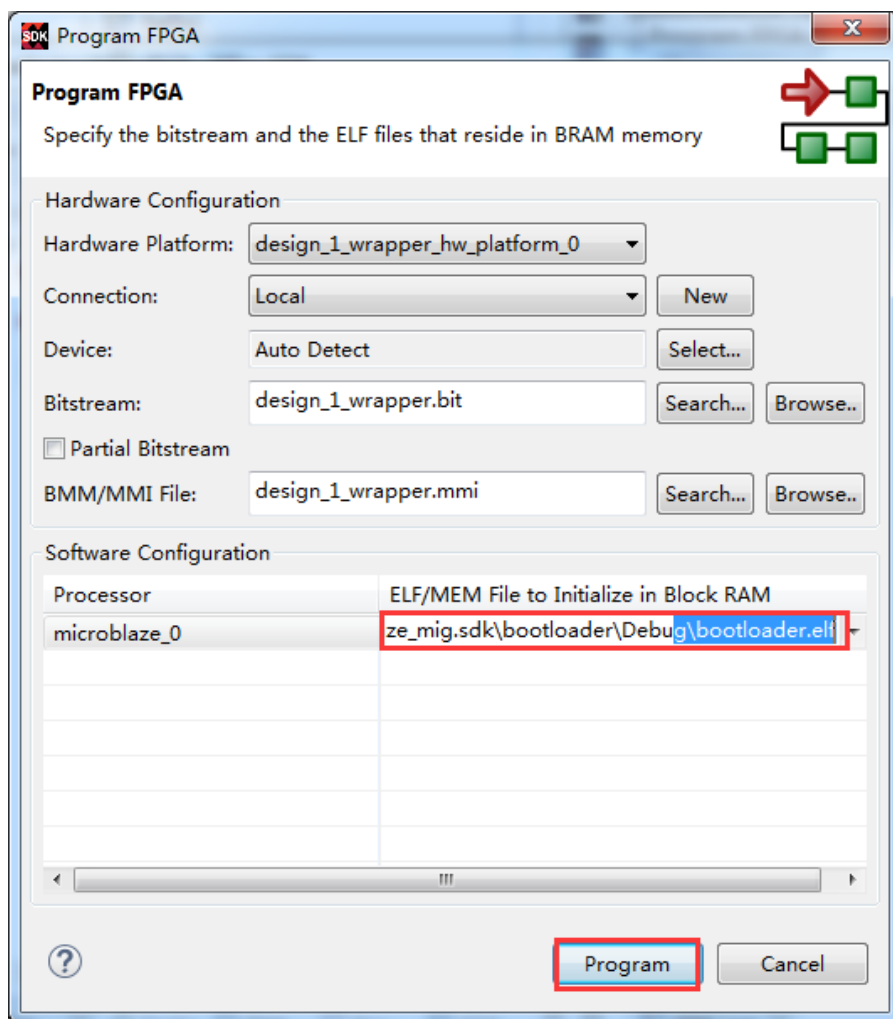
12. 优化等级设置为-Os



3.3.2 修改 bootloader bsp

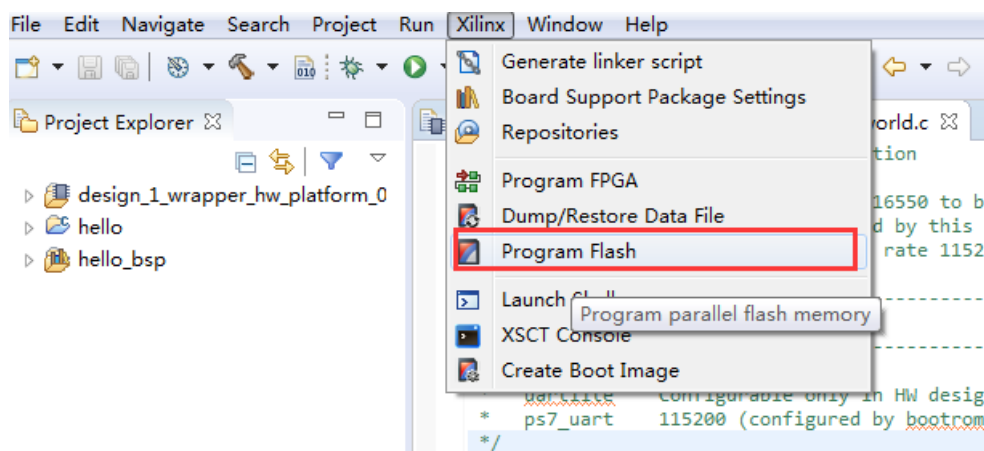
13. 双击 “bootloader_bsp” 目录的 “system.mss” ,点击 “Modify this BSP's Settings”





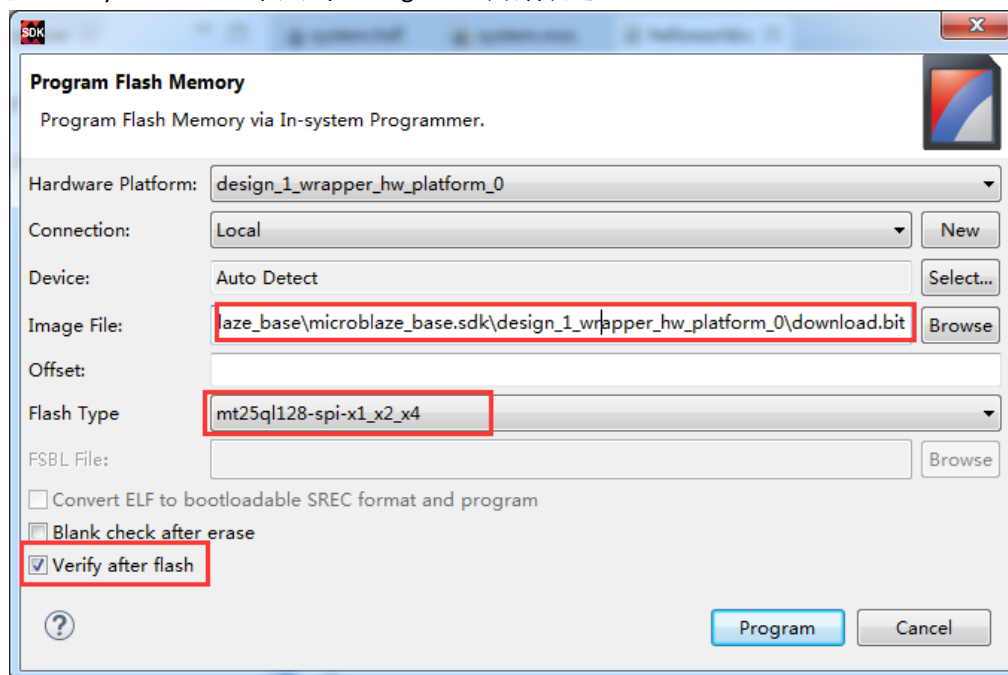
3.3.3 烧写 Bitstream 和 BootLoader

17. 点击 “Xilinx -> Program Flash”

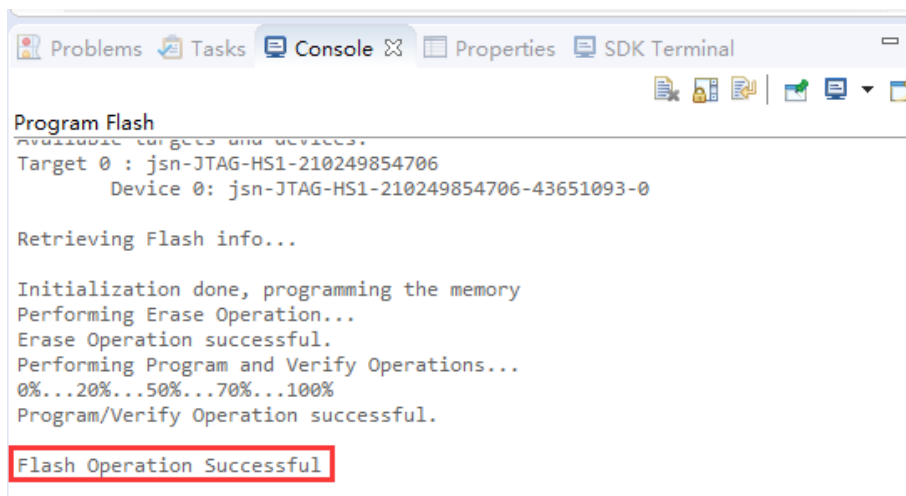


18. “Image File” 选择上一步生成的 “download.bit” 文件，位于目录
“*.sdk/design_1_wrapper_hw_platform_0” 中，“Flash Type” 选择 “mt25ql128-spi-x1_x2_x4”，

勾选 “Verify after flash”，点击 “Program” 开始烧写 Flash

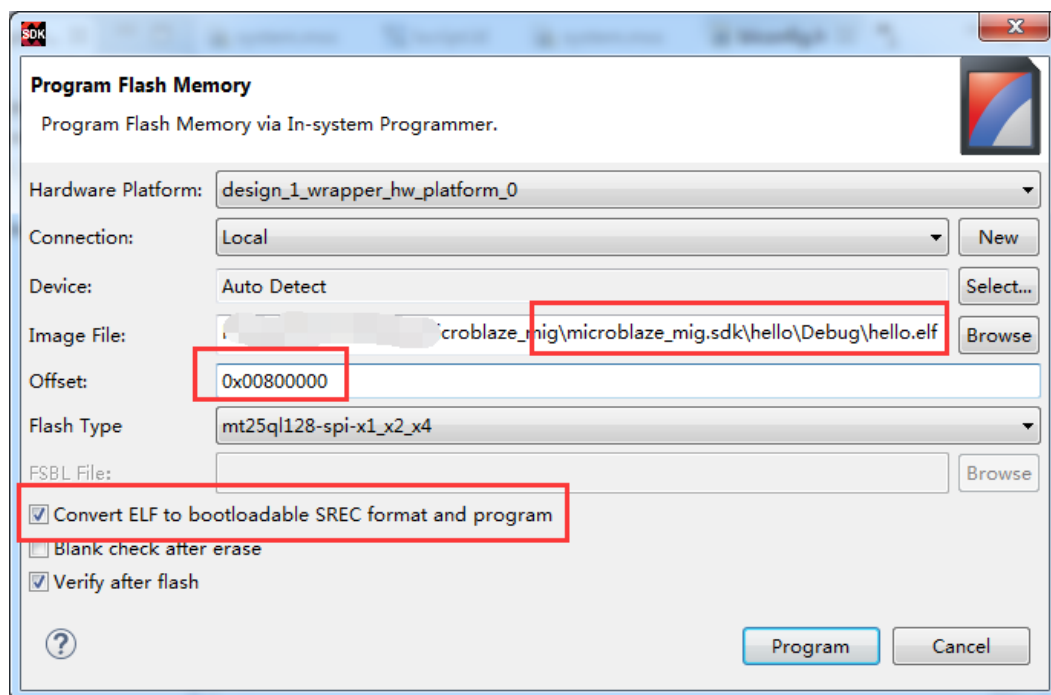


19. 烧写完成



3.3.4 烧写应用程序

将 “hello.elf” 烧写到 0x00800000 处，这个地址在前面的 BootLoader 工程里设置的，勾选 “Convert ELF to bootloadable SREC format and program”，烧写完成以后最好拔掉 JTAG 连接线，再重启验证是否烧写成功



20. 运行结果

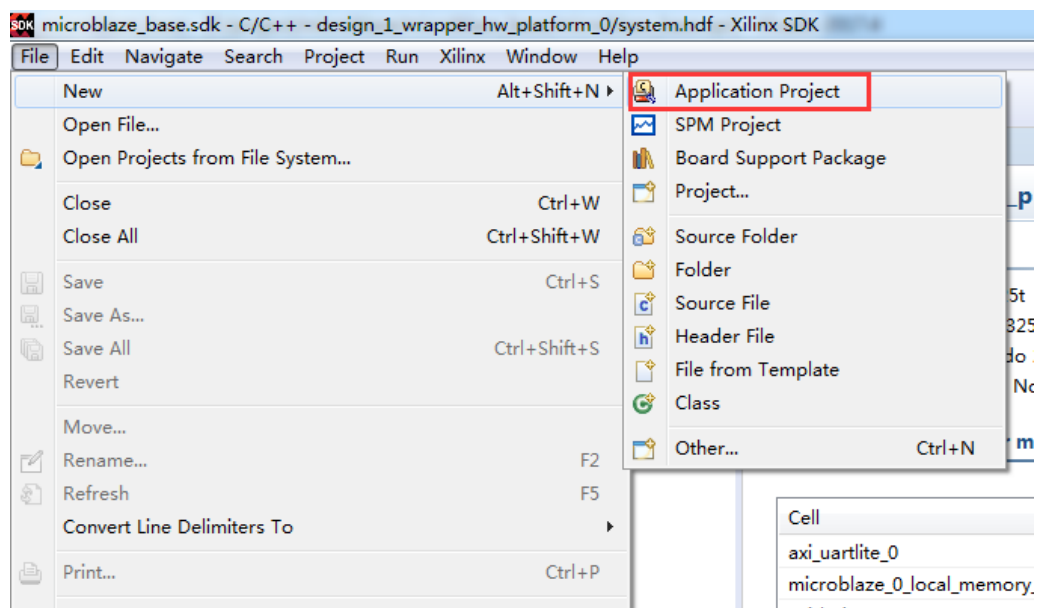
```
SREC SPI Bootloader
Loading SREC image from flash @ address: 00800000
Bootloader: Processed (0x)000000e3 S-records
Executing program starting at address: 00000000
Hello World
```

第四章 按键控制 LED 例程

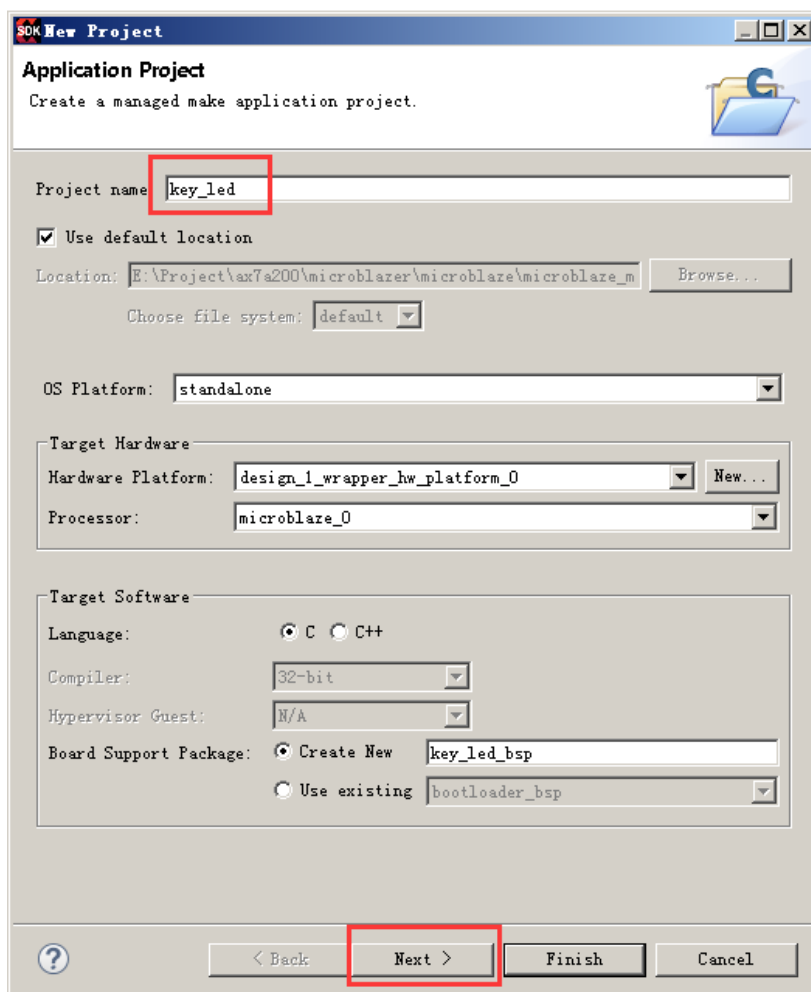
本章使用第三章的 Vivado 工程配置,因为在第三章里的 VIVADO 工程里已经添加了 2 个 GPIO 控制器 `axi_led` 和 `axi_key`。我们这里只要 SDK 软件编程实现按键控制 LED 灯,按键按下,LED 灯亮,按键松开,LED 灯灭。另外程序中会加入按键去抖的功能,使得按键功能更可靠。

4.1 SDK 软件编程

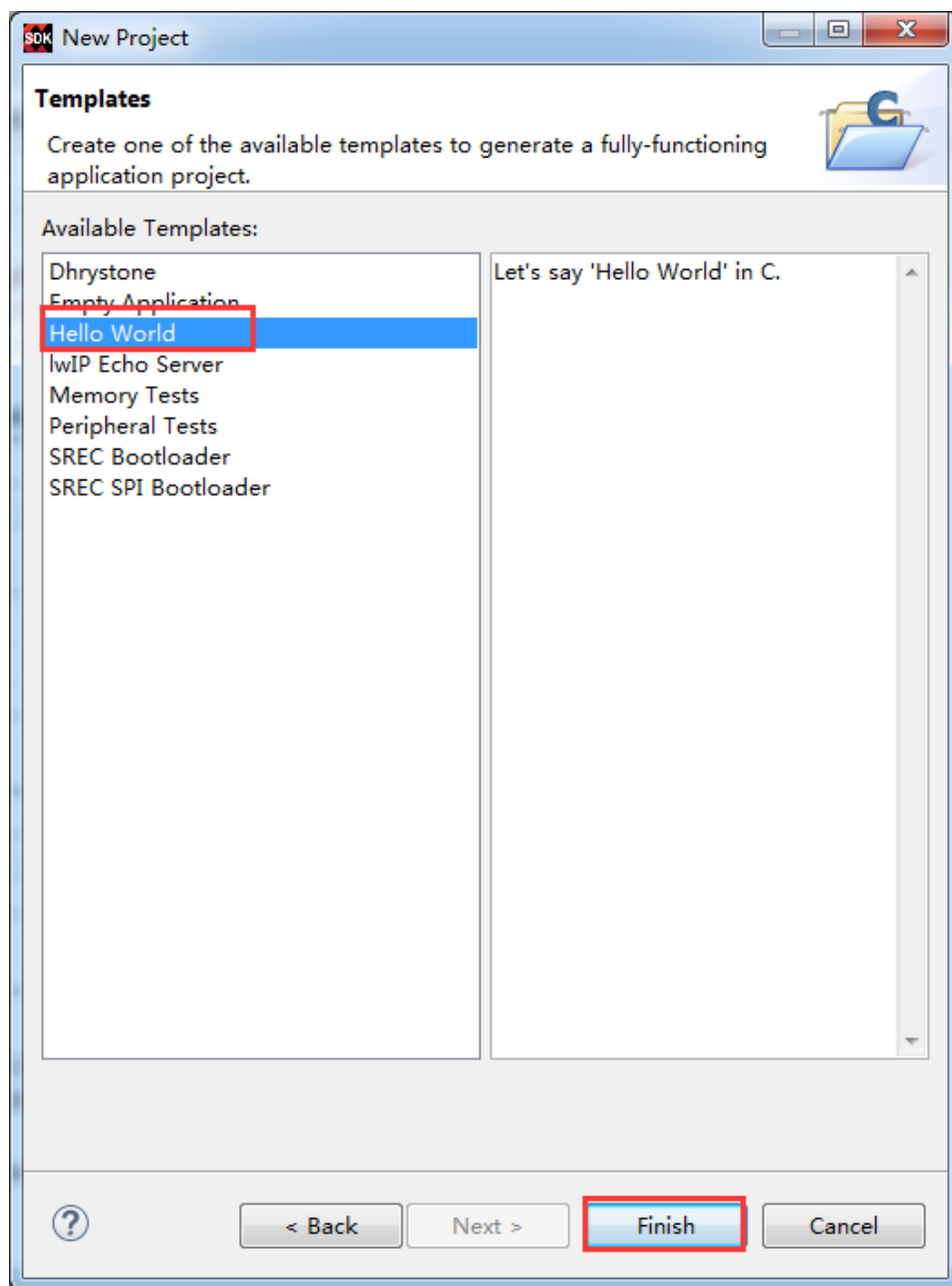
1. 新建工程



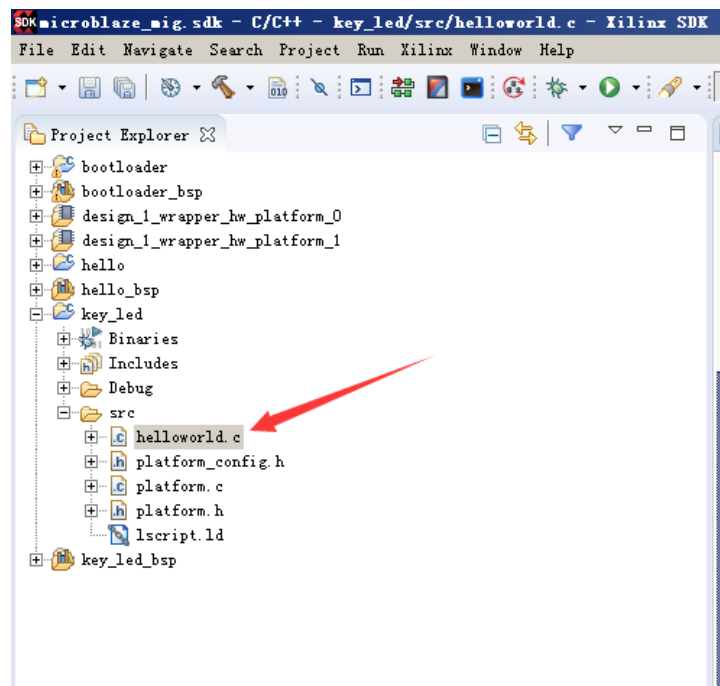
2. 设置工程名为 “key_led”, 点击 “Next”



3. 模板选择 “Hello World” , 点击 “Finish”



4. 工程建立后修改 helloworld.c 文件。



helloworld.c 修改为以下的代码。

```

/***** Include Files *****/
#include <stdio.h>
#include "xparameters.h"
#include "xgpio.h"
#include "platform.h"

/***** Variable Defintions *****/
XGpio Gpio_leds;
XGpio Gpio_keys;

int main()
{
    int Status;
    u32 Delay;
    u32 DataRead;

    init_platform();

    Status = XGpio_Initialize(&Gpio_keys, XPAR_AXI_KEY_DEVICE_ID);
    if (Status != XST_SUCCESS) {
        return XST_FAILURE;
    }

    XGpio_SetDataDirection(&Gpio_keys, 1, 0xFFFFFFFF);    //set Gpio_keys is
input

    Status = XGpio_Initialize(&Gpio_leds, XPAR_AXI_LED_DEVICE_ID);
    if (Status != XST_SUCCESS) {
        return XST_FAILURE;
    }

    XGpio_SetDataDirection(&Gpio_leds, 1, 0x0);           //set Gpio_leds is output

    while(1)
    {
        DataRead = XGpio_DiscreteRead(&Gpio_keys, 1);
    }
}

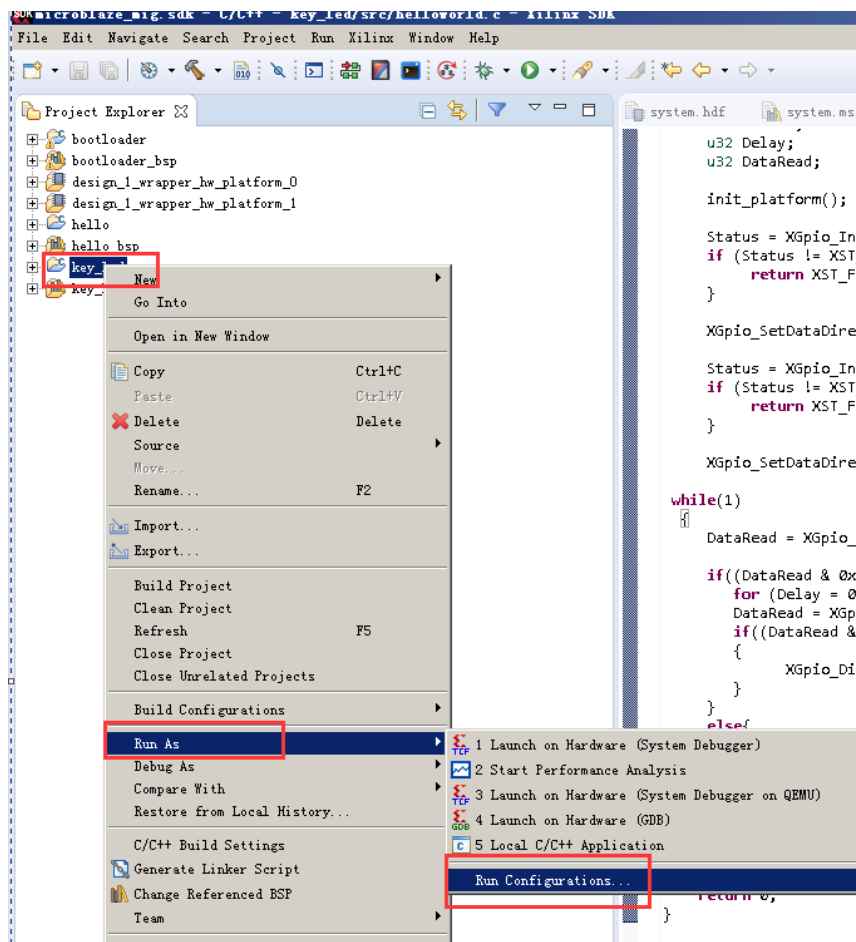
```

```
if((DataRead & 0x0f)!=0x0f){ //if key is pushed
    for (Delay = 0; Delay < 2000; Delay++);
    DataRead = XGpio_DiscreteRead(&Gpio_keys, 1); //read again
    if((DataRead & 0x0f)!=0x0f)
    {
        XGpio_DiscreteWrite(&Gpio_leds, 1, DataRead);
    }
}
else{ //if key is not pushed
    for (Delay = 0; Delay < 2000; Delay++);
    DataRead = XGpio_DiscreteRead(&Gpio_keys, 1); //read again
    if((DataRead & 0x0f)==0x0f)
    {
        XGpio_DiscreteWrite(&Gpio_leds, 1, DataRead);
    }
}
return 0;
}
```

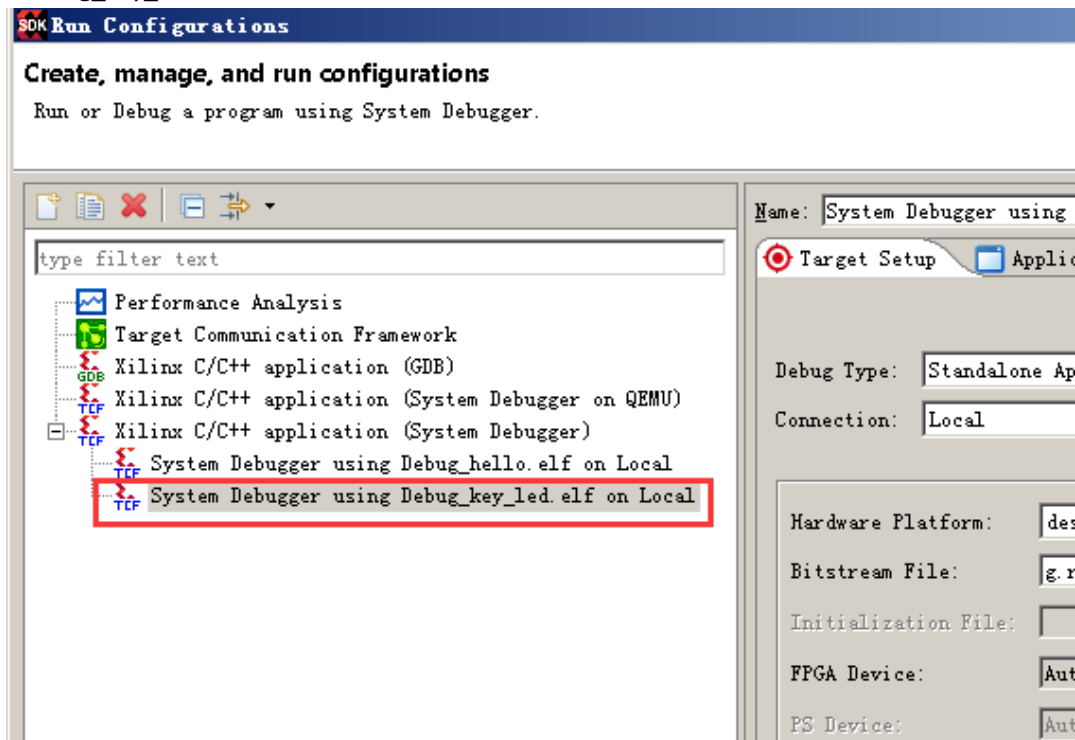
程序在 while 循环里检测按键 Gpio_keys 的输入值，如果发现按键按下，会等待一段时间后再判断按键的数值，如果还是有按键按下，写 Gpio_leds 的值，使得开发板上的 LED 灯亮来指示那个按键按下。同样程序也判断了按键松开的情况，按键松开，开发板上的 LED 灯也会熄灭。

4.2 程序下载测试

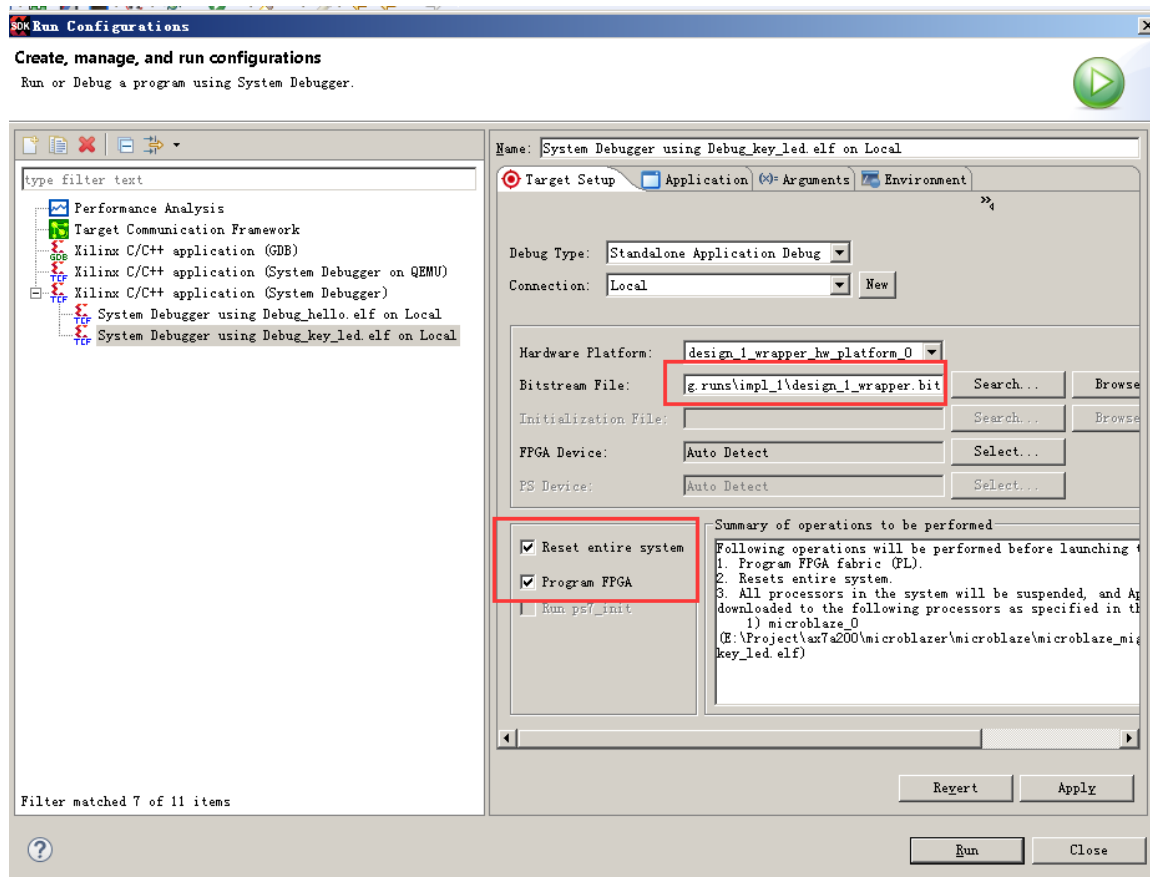
1. 选择 “key_led” 工程，右键选择 “Run As -> Run Configuration...”



2. 双击 “Xilinx C/C++ application(System Debugger)” ,选择 “System Debugger using Debug_key_led.elf on Local ”



3. 选择 design_1_wrapper.bit , 勾选 “Reset entire system” 和 “Program FPGA” 复位整个系统并下载 FPGA



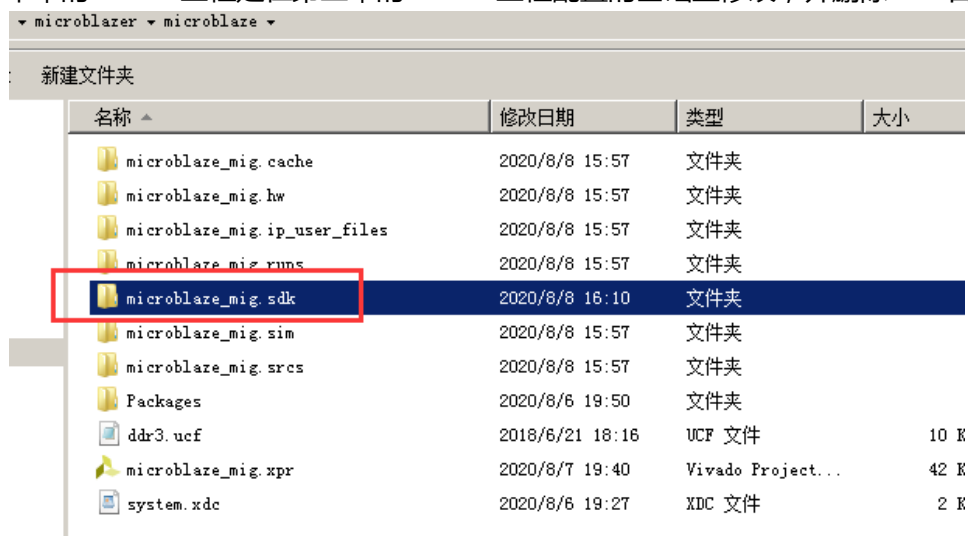
4. 连接 JTAG 线到开发板、Run 下载。下载后，我们按一下开发板上的 KEY1，这时 LED1 灯会亮起，按键松开，LED 灯会熄灭。

第五章 按键中断例程

前面一章我们有介绍按键的检测，需要程序实时查询的方式查看 GPIO 状态。本章介绍按键中断的实现，通过 GPIO 产生中断，用户程序只要在中断产生的时候产生一些相应操作。

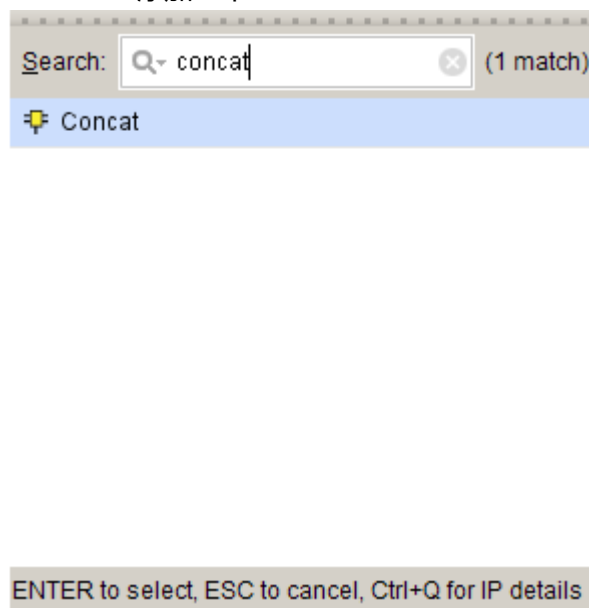
5.1 VIVADO 工程修改

本章的 Vivado 工程是在第三章的 Vivado 工程配置的基础上修改，并删除 SDK 目录。



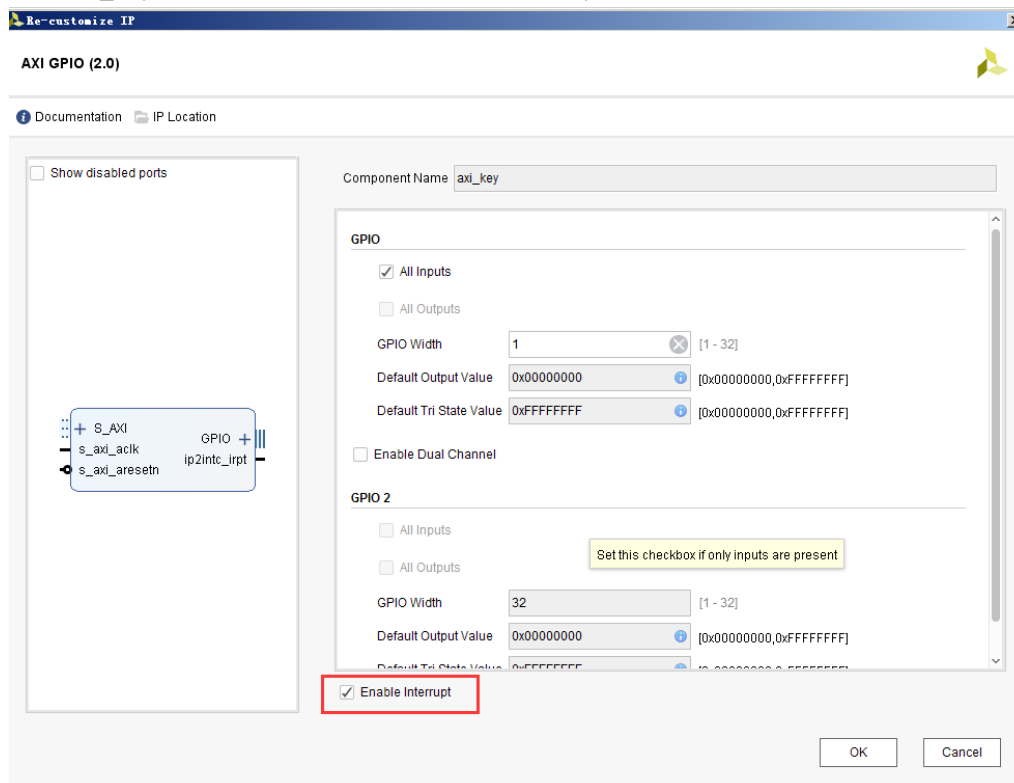
5.1.1 添加 Constant

1. 搜索“Concat”，双击“Constant” 添加一个 Concat IP



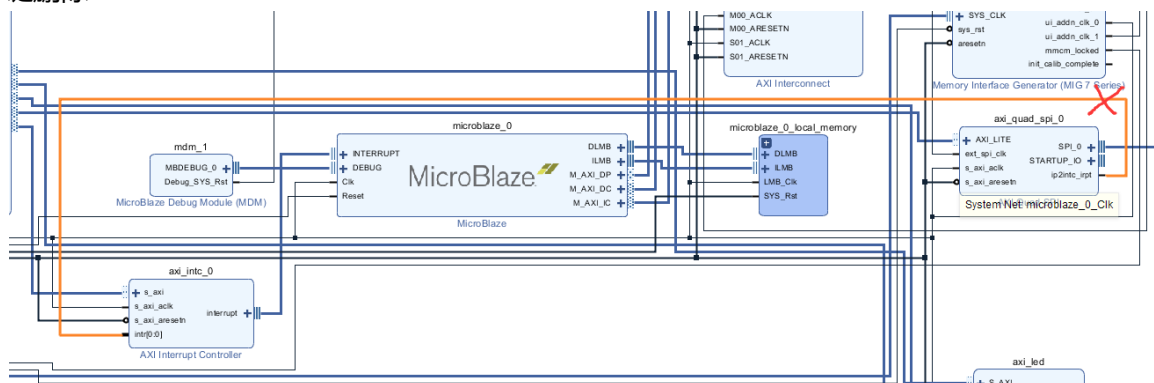
5.1.2 修改 axi_key

2. 双击 axi_key IP, 配置界面里选择 “Enable interrupt”

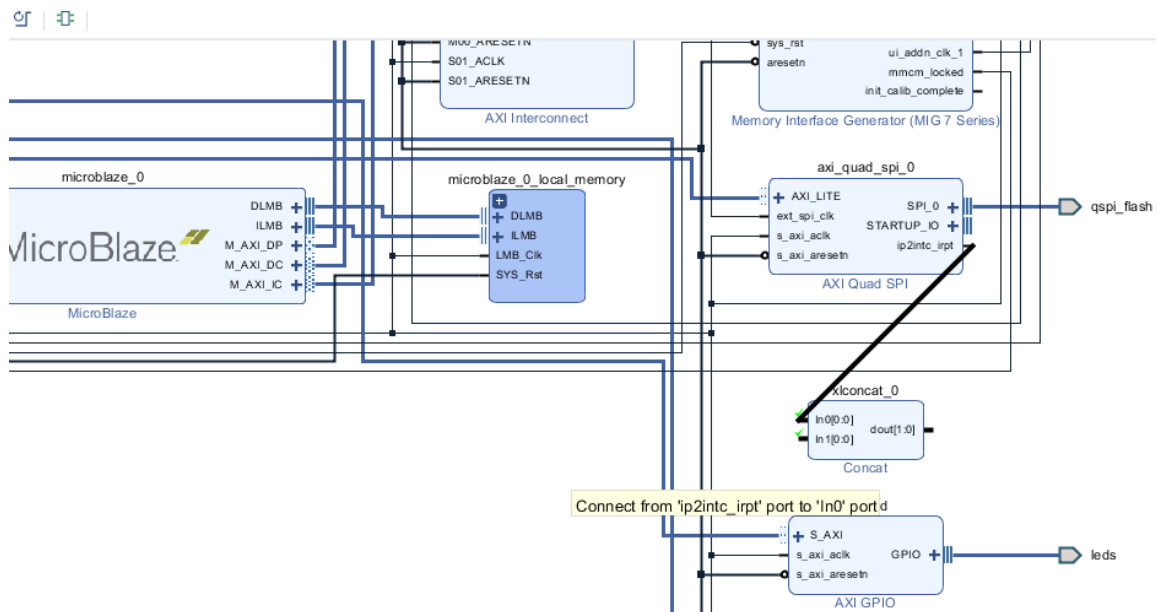


5.1.3 中断信号连线

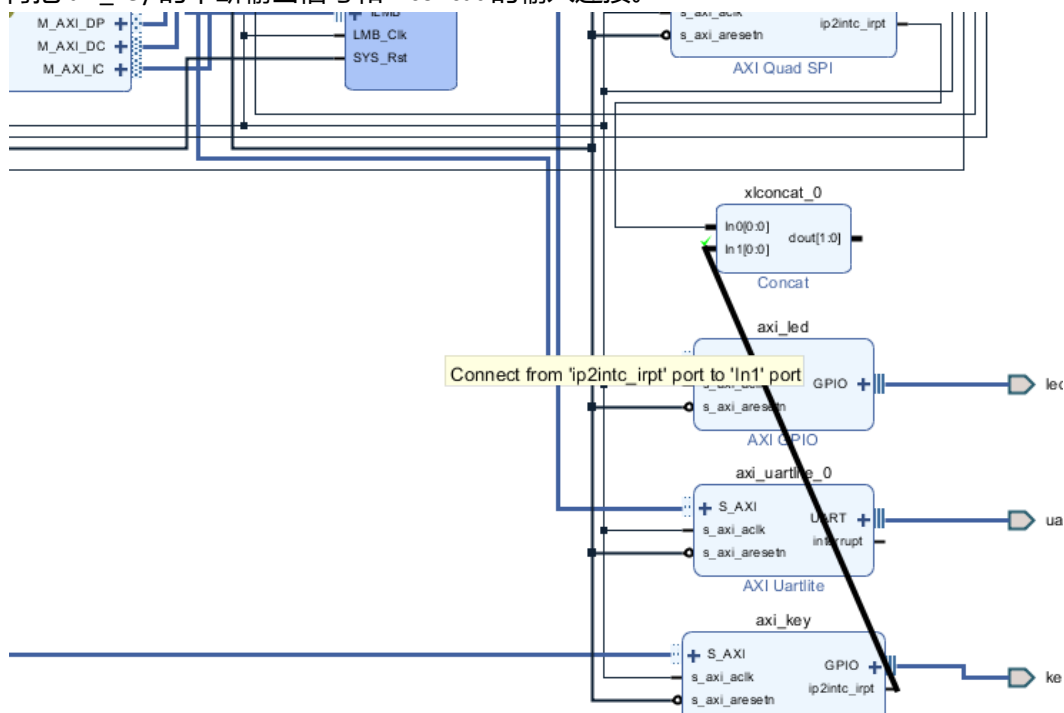
3. 先删除 axi_quad_spi_0 的中断输出信号和 axi_intc_0 之间的连线。选择这根线 然后按 Delete 键删除



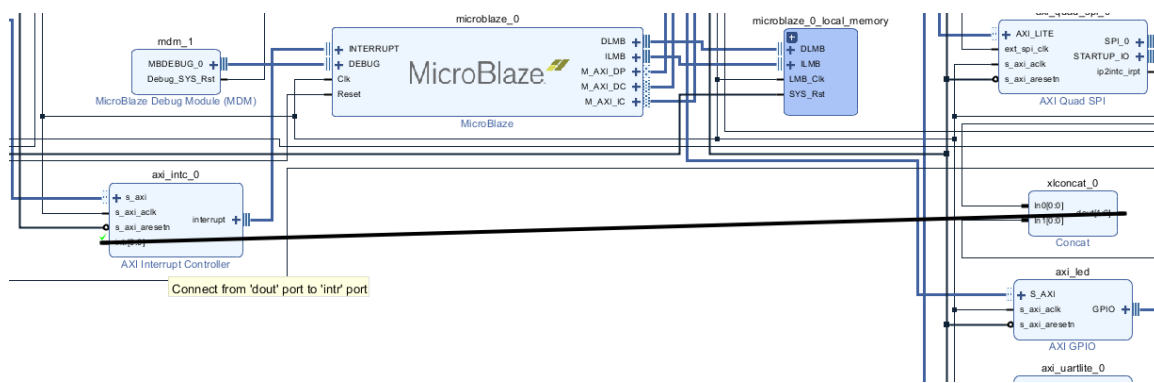
4. 再把 axi_quad_spi_0 的中断输出信号和新加的 xlconcat 的输入连接。



5. 再把 axi_key 的中断输出信号和 xlconcat 的输入连接。

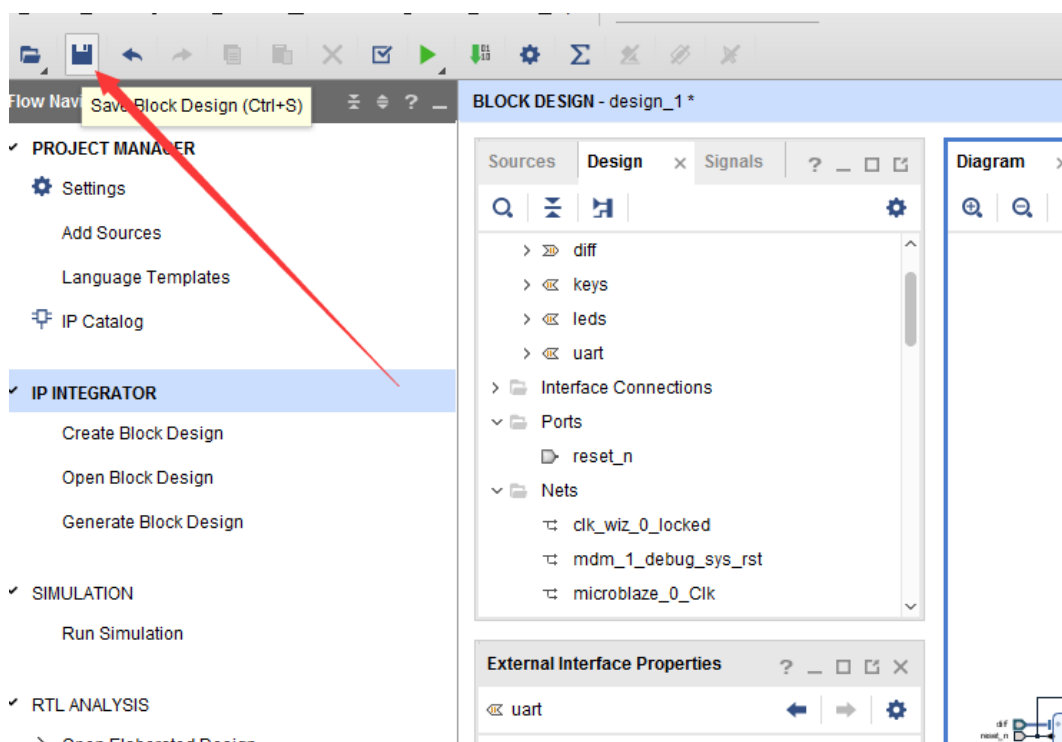


6. 把 xlconcat 输出信号和 axi_intc_0 的中断输入连接，这样相当于 2 个中断（FLASH 中断和按键中断）都连接到了中断控制器上。

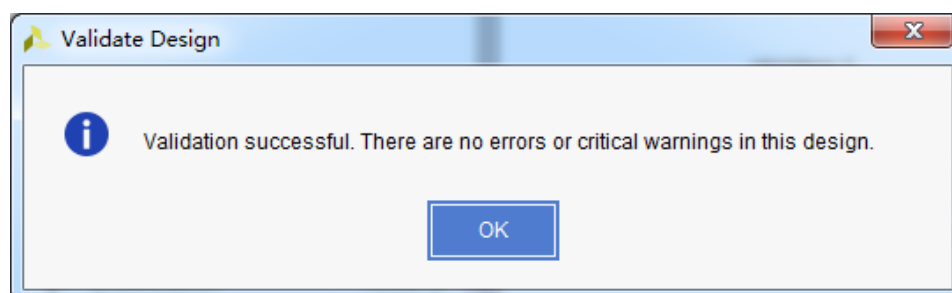


5.1.4 保存并检查错误

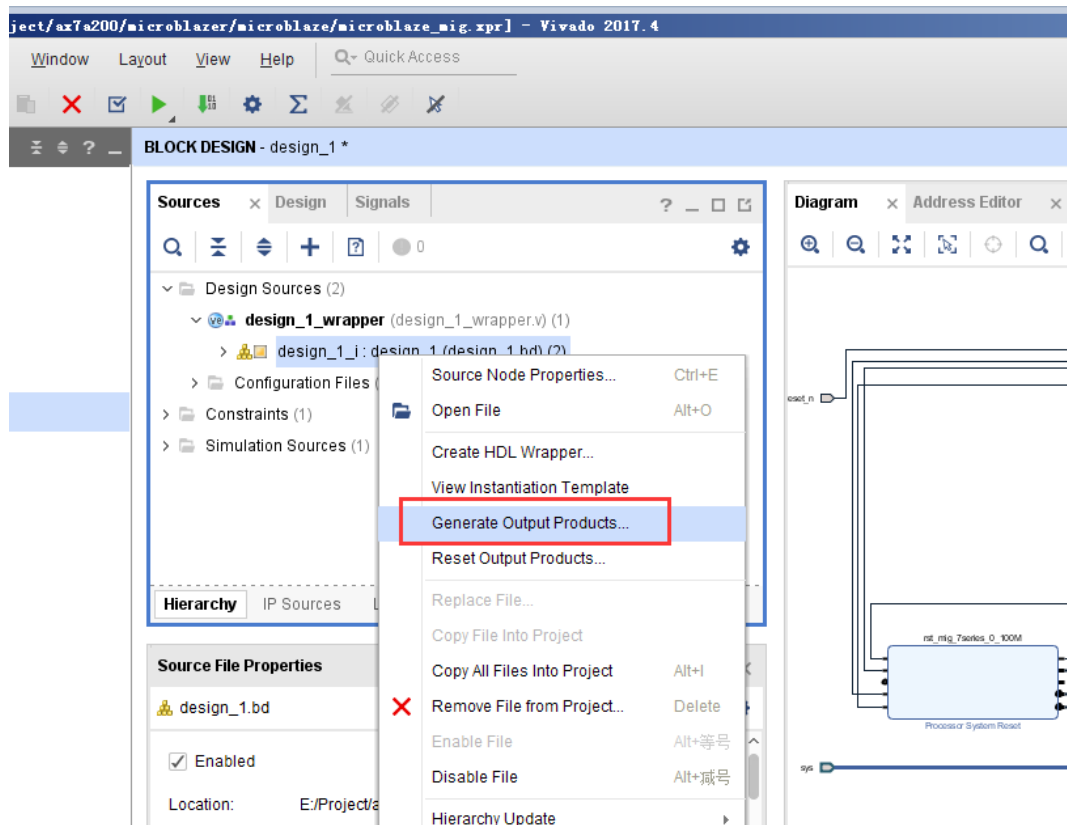
7. 保存设计，然后按“F6”检查设计



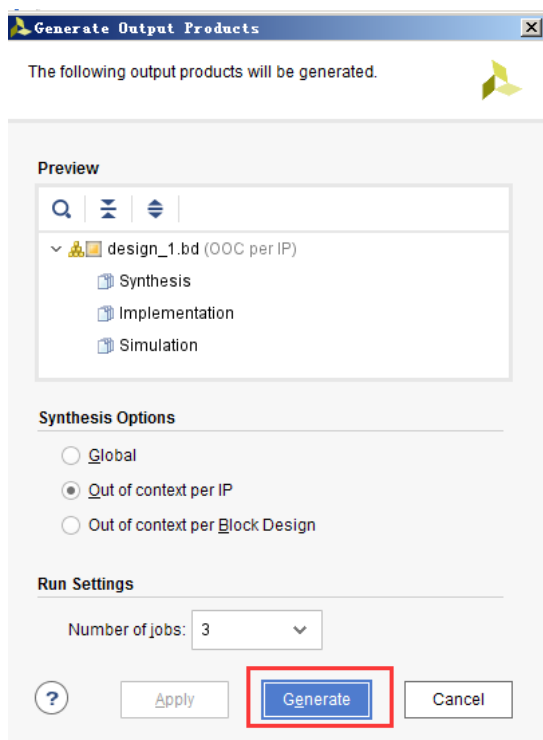
8. 设计中没有错误和警告



9. 选择 “design_1.bd” 文件，右键选择 “Generate Output Products” ，把 IP 参数和连接信息 update 到 project 中

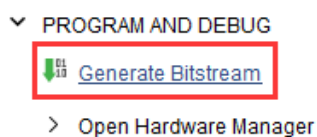


10. 选择 Generate 按钮

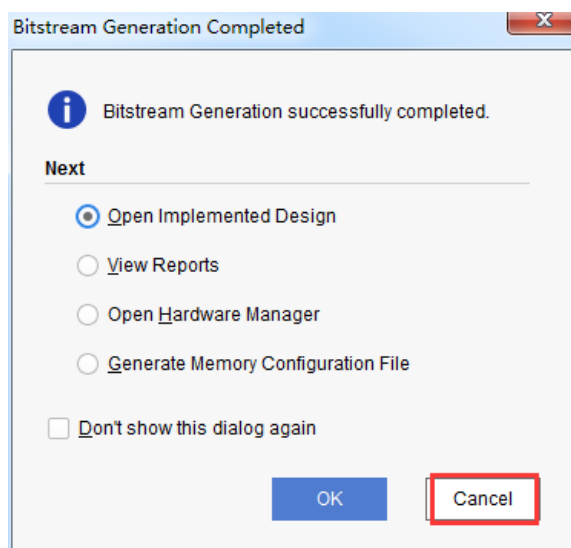


5.1.5 生成 Bitstream 文件

11. 点击 “Generate Bitstream” 生成 bit 流文件

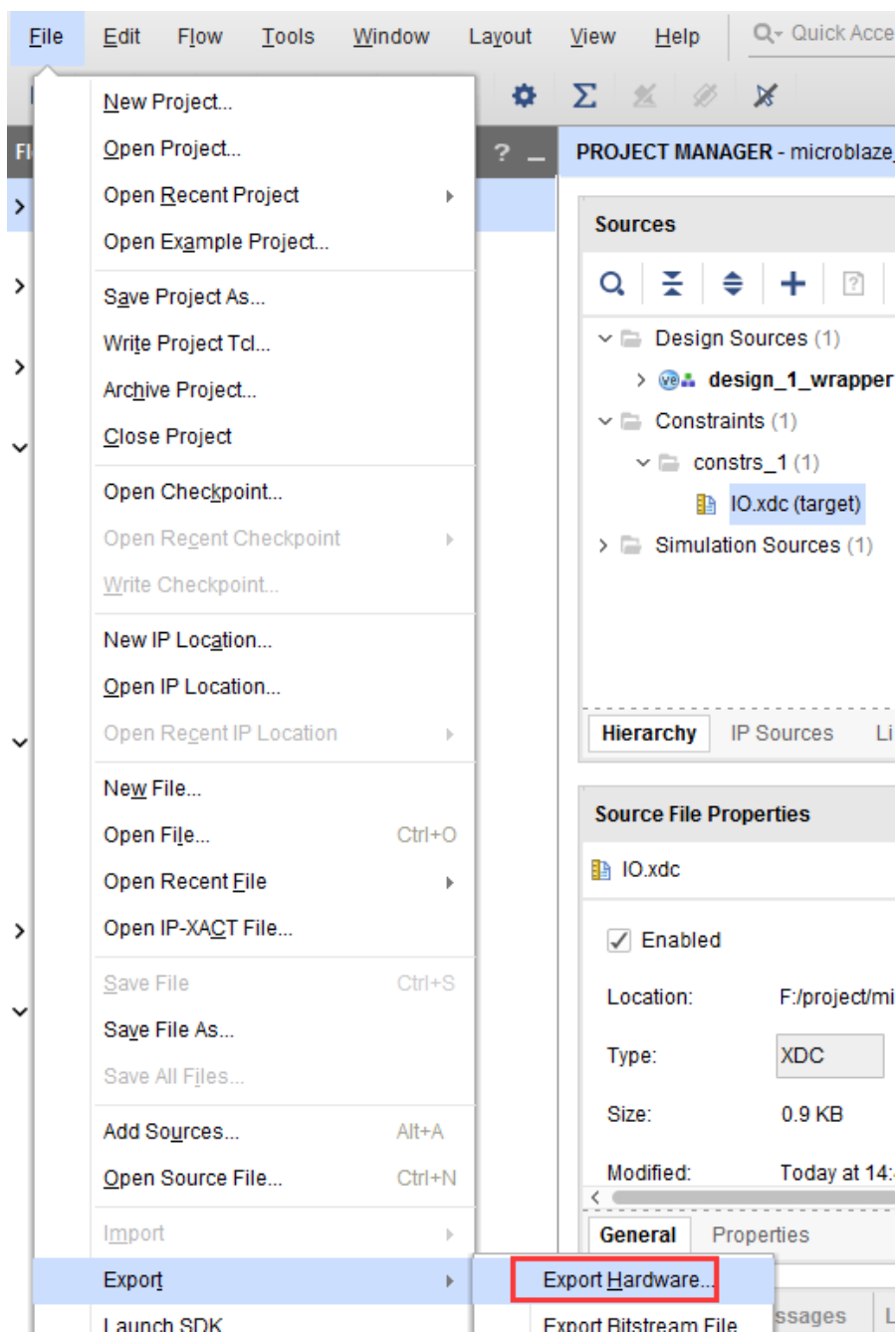


12. 生成完成以后，点击 “Cancel”，不做任何操作

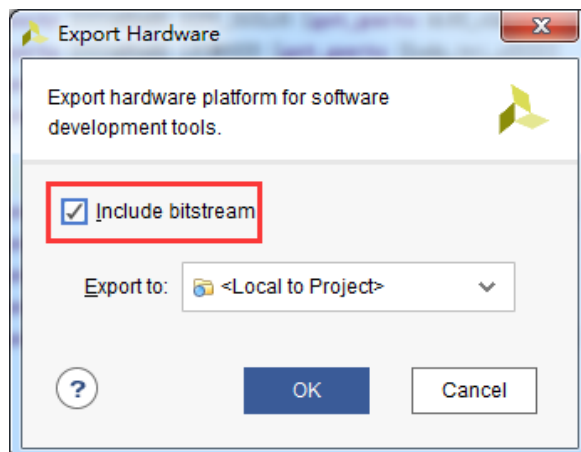


5.1.6 导出硬件

13. 在 “File -> Export -> Export Hardware...” 导出硬件

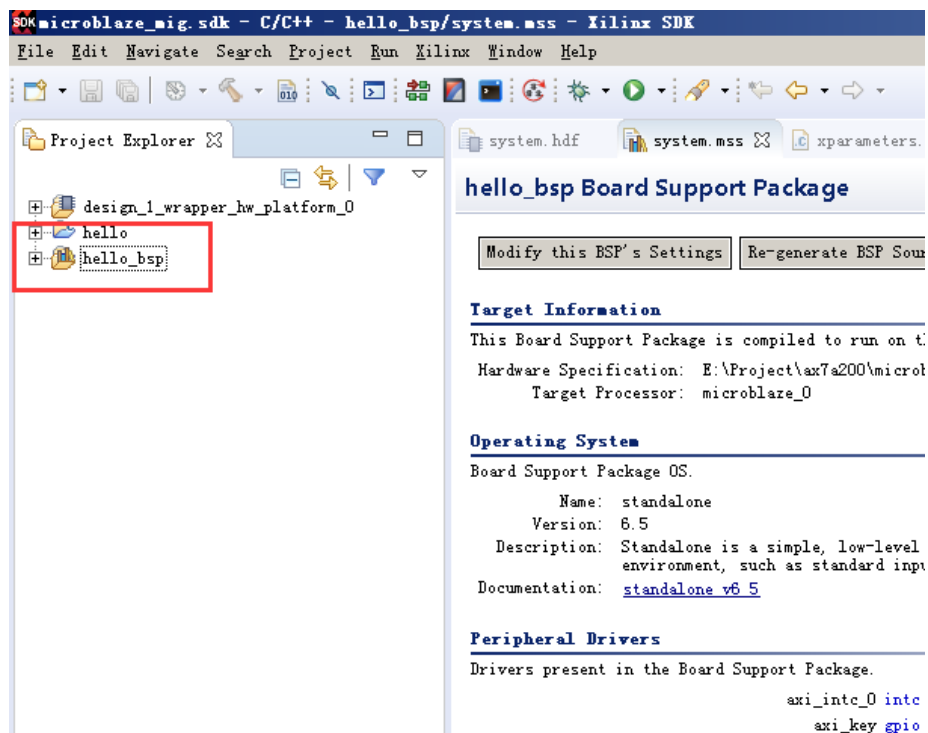


14. 在弹出窗口中选择 “Include bitstream” 包含 Bitstream 文件

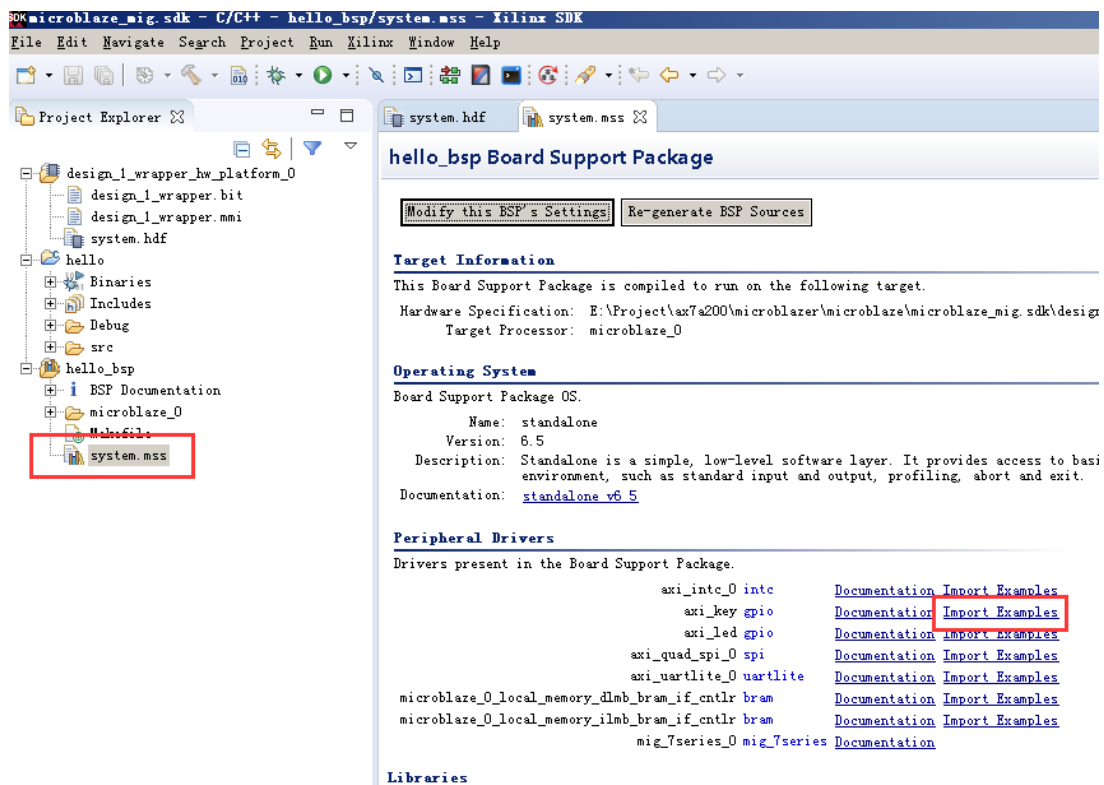


5.2 SDK 开发按键中断程序

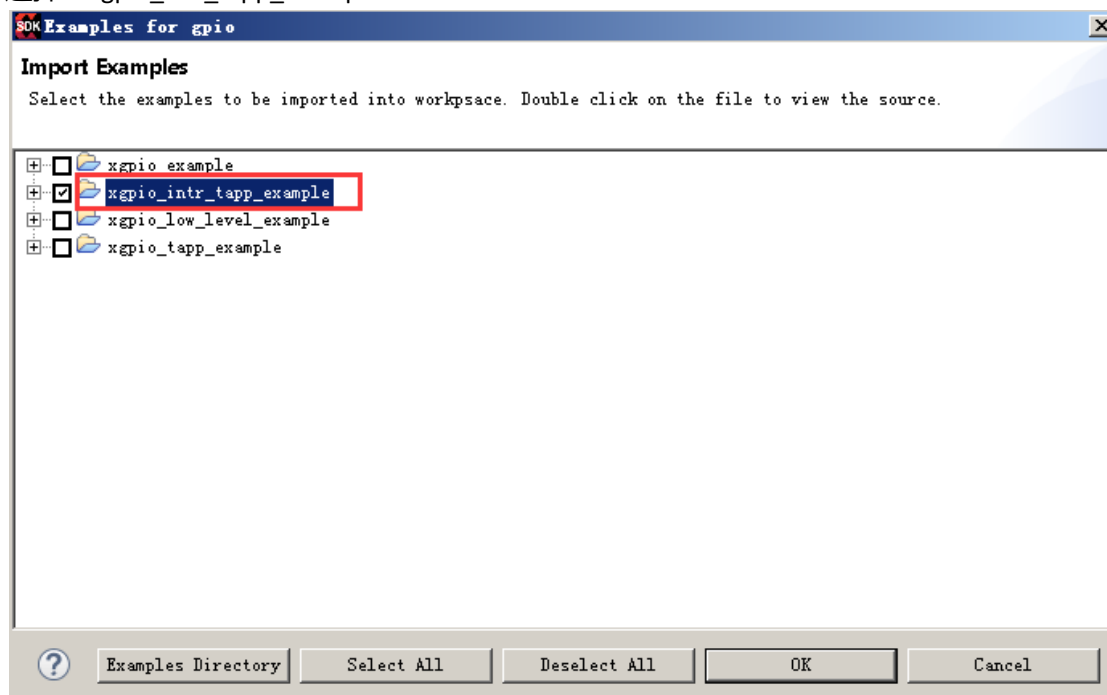
15. SDK 下新建一个 helloworld 工程。



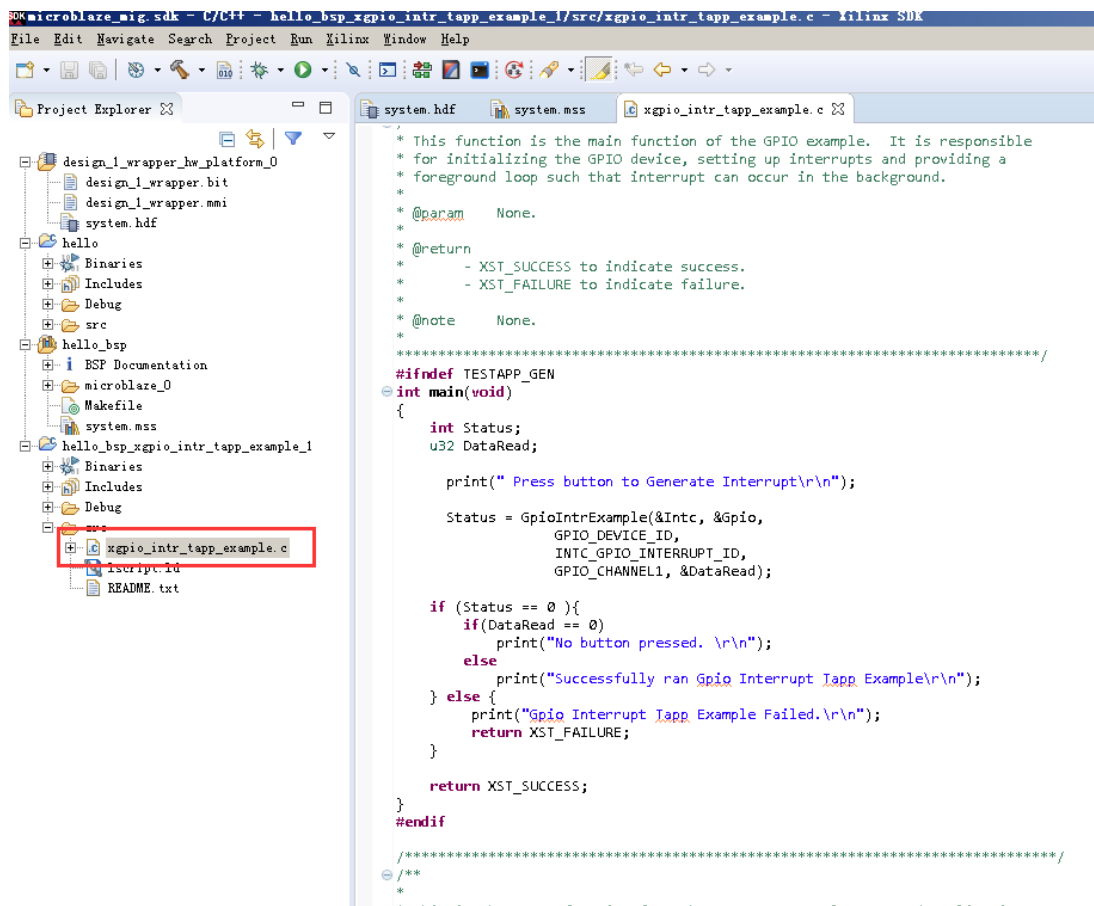
16. 双击 BSP 目录里的“system.mss”文件，点击 axi_key 旁的 “Import Examples”



17. 选择 “xgpio_intr_tapp_example”



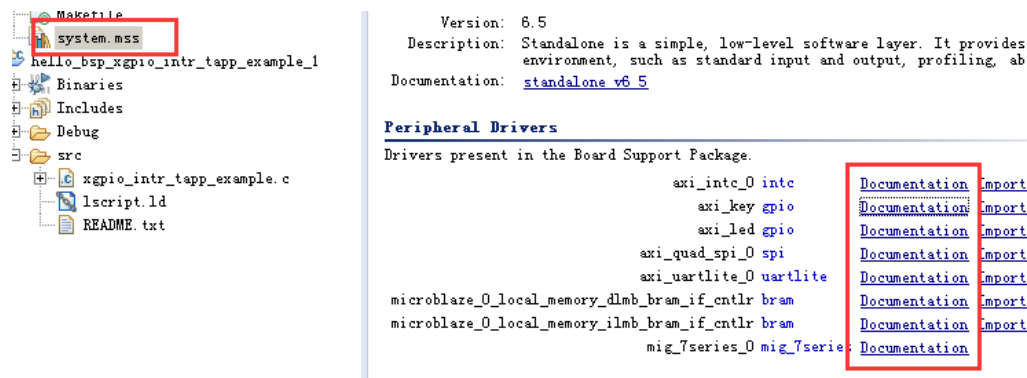
18. 打开生成的 xpio_intr_tapp_example.c , 先看看里面的程序。



程序里通过 GpioSetupIntrSystem 程序实现 GPIO 的中断初始化,里面主要用到的中断函数有:

XIntc_Initialize	中断初始化
XIntc_Connect	设置中断服务程序入口地址
XIntc_Enable	中断允许
XIntc_Start	中断开始
XGpio_InterruptEnable	相应 GPIO 中断允许
XGpio_InterruptGlobalEnable	GPIO 全局中断允许
Xil_ExceptionInit	异常处理函数

关于这些函数的定义和说明可以参考 document,寄存器定义可以在 vivado 软件里的查看 IP 文档。



19. 为了程序简洁一些 我们这里修改了 main 函数 在 main 函数里合并了个 gpioIntrExample。

```

/***** Include Files *****/
#include "xparameters.h"
#include "xgpio.h"
#include "xil_exception.h"
#include "xintc.h"
#include <stdio.h>

#define GPIO_DEVICE_ID      XPAR_AXI_KEY_DEVICE_ID
#define GPIO_CHANNEL1      1

#define INTC_GPIO_INTERRUPT_ID  XPAR_INTC_0_GPIO_0_VEC_ID
#define INTC_DEVICE_ID  XPAR_INTC_0_DEVICE_ID

#define INTC_DEVICE_ID  XPAR_INTC_0_DEVICE_ID
#define INTC      XIntc
#define INTC_HANDLER  XIntc_InterruptHandler

/***** Function Prototypes *****/
void GpioHandler(void *CallBackRef);

int GpioIntrExample(INTC *IntcInstancePtr, XGpio *InstancePtr,
    u16 DeviceId, u16 IntrId,
    u16 IntrMask, u32 *DataRead);

int GpioSetupIntrSystem(INTC *IntcInstancePtr, XGpio *InstancePtr,
    u16 DeviceId, u16 IntrId, u16 IntrMask);

void GpioDisableIntr(INTC *IntcInstancePtr, XGpio *InstancePtr,
    u16 IntrId, u16 IntrMask);

/***** Variable Definitions *****/
XGpio Gpio; /* The Instance of the GPIO Driver */
INTC Intc; /* The Instance of the Interrupt Controller Driver */

static u16 GlobalIntrMask; /* GPIO channel mask that is needed by
    * the Interrupt Handler */

static volatile u32 IntrFlag; /* Interrupt Handler Flag */

int main(void)
{
    int Status;

    print(" Press button to Generate Interrupt\r\n");

    Status = XGpio_Initialize(&Gpio, GPIO_DEVICE_ID);
    if (Status != XST_SUCCESS) {
        return XST_FAILURE;
    }

    Status = GpioSetupIntrSystem(&Intc, &Gpio, GPIO_DEVICE_ID, INTC_GPIO_INTERRUPT_ID, GPIO_CHANNEL1);
    if (Status != XST_SUCCESS) {
        return XST_FAILURE;
    }
}

```

```

    }

    IntrFlag = 0;

    while (1) {
        if (IntrFlag) {
            IntrFlag = 0;
            print("KEY1 is pressed, interrupt is happen \r\n");
        }
    }

    GpioDisableIntr(&Intc, &Gpio, INTC_GPIO_INTERRUPT_ID, GPIO_CHANNEL1);

return XST_SUCCESS;
}

/***** GPIO interrupt setup *****/
int GpioSetupIntrSystem(INTC *IntcInstancePtr, XGpio *InstancePtr,
                        u16 DeviceId, u16 IntrId, u16 IntrMask)
{
    int Result;

    GlobalIntrMask = IntrMask;

    /*
     * Initialize the interrupt controller driver so that it's ready to use.
     * specify the device ID that was generated in xparameters.h
     */
    Result = XIntc_Initialize(IntcInstancePtr, INTC_DEVICE_ID);
    if (Result != XST_SUCCESS) {
        return Result;
    }

    /* Hook up interrupt service routine */
    XIntc_Connect(IntcInstancePtr, IntrId,
                  (Xil_ExceptionHandler)GpioHandler, InstancePtr);

    /* Enable the interrupt vector at the interrupt controller */
    XIntc_Enable(IntcInstancePtr, IntrId);

    /*
     * Start the interrupt controller such that interrupts are recognized
     * and handled by the processor
     */
    Result = XIntc_Start(IntcInstancePtr, XIN_REAL_MODE);
    if (Result != XST_SUCCESS) {
        return Result;
    }

    /*
     * Enable the GPIO channel interrupts so that push button can be
     * detected and enable interrupts for the GPIO device
     */
    XGpio_InterruptEnable(InstancePtr, IntrMask);
    XGpio_InterruptGlobalEnable(InstancePtr);

    /*
     * Initialize the exception table and register the interrupt
     * controller handler with the exception table
     */
    Xil_ExceptionInit();

    Xil_ExceptionRegisterHandler(XIL_EXCEPTION_ID_INT,
                                (Xil_ExceptionHandler)INTC_HANDLER, IntcInstancePtr);

    /* Enable non-critical exceptions */
    Xil_ExceptionEnable();

    return XST_SUCCESS;
}

/***** This is the interrupt handler routine for the GPIO for this example. *****/

```

```

*****/
void GpioHandler(void *CallbackRef)
{
    XGpio *GpioPtr = (XGpio *)CallbackRef;

    IntrFlag = 1;

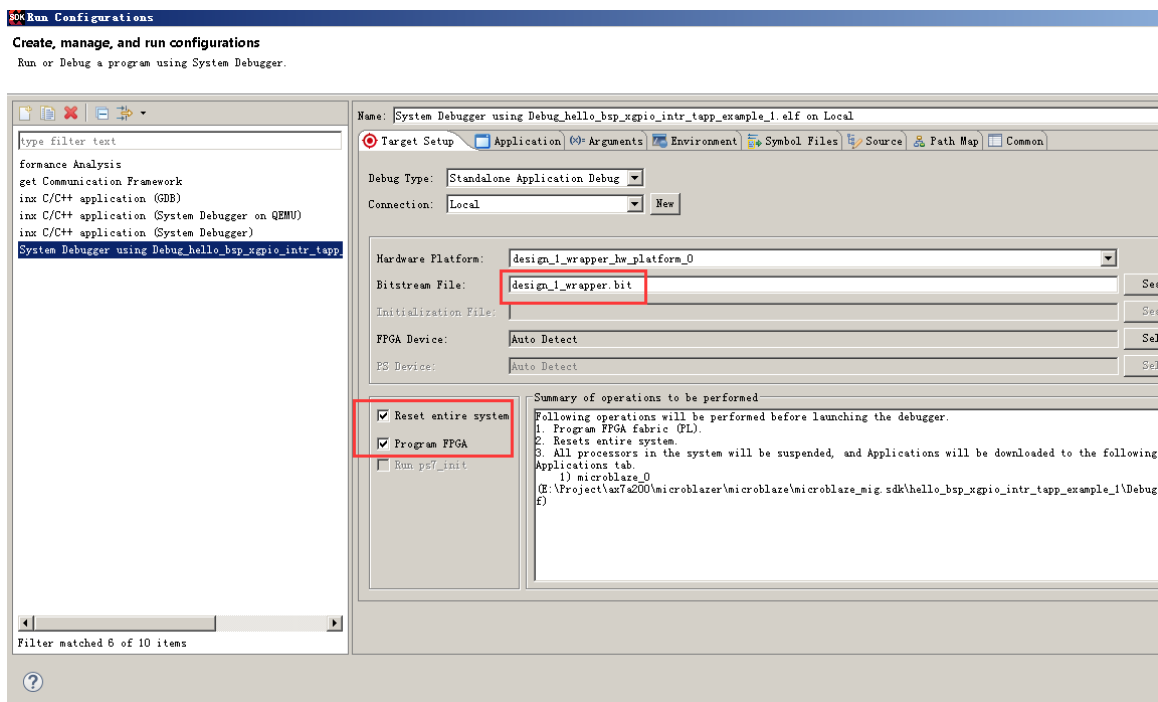
    /* Clear the Interrupt */
    XGpio_InterruptClear(GpioPtr, GlobalIntrMask);
}

/*****
**This function disables the interrupts for the GPIO
*****/
void GpioDisableIntr(INTC *IntcInstancePtr, XGpio *InstancePtr,
                    u16 IntrId, u16 IntrMask)
{
    XGpio_InterruptDisable(InstancePtr, IntrMask);
#ifdef XPAR_INTC_0_DEVICE_ID
    XIntc_Disable(IntcInstancePtr, IntrId);
#else
    /* Disconnect the interrupt */
    XScuGic_Disable(IntcInstancePtr, IntrId);
    XScuGic_Disconnect(IntcInstancePtr, IntrId);
#endif
    return;
}

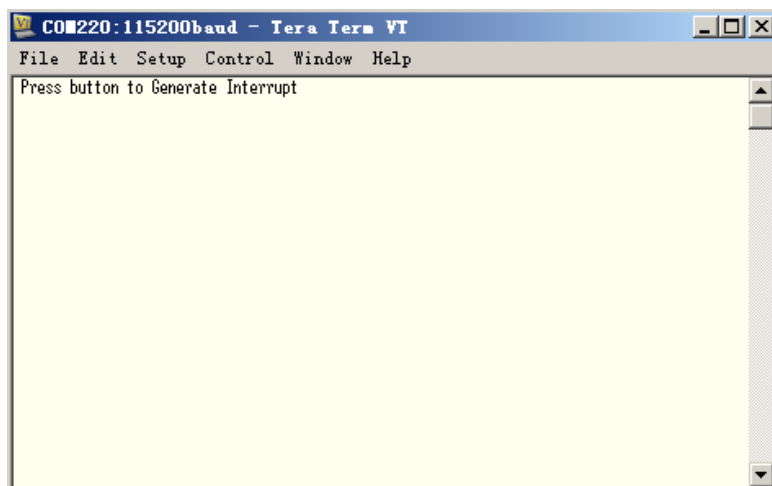
```

5.3 程序下载测试

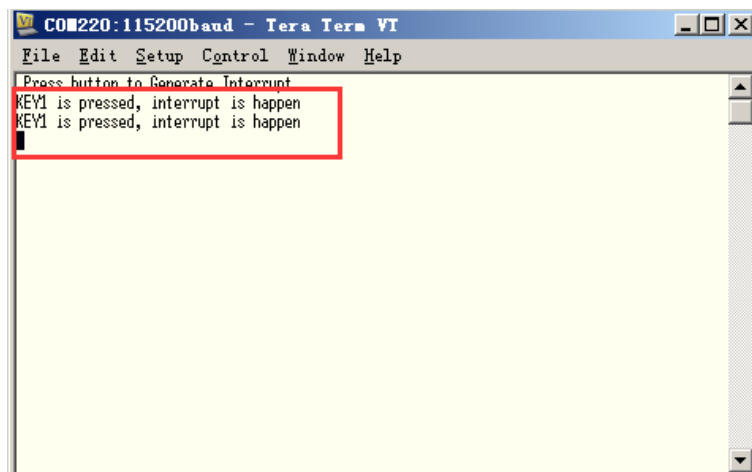
20. 选择“hello_bsp_xgpio_intr_tapp_example”工程,右键选择“Run As -> Run Configuration...”,在弹出的界面里配置如下。



21. 连接 JTAG 线到开发板、Run 下载。下载后，串口显示打印信息。



22. 按一下 KEY1，串口会打印 2 条消息，说明中断触发了 2 次。因为 GPIO 的输入产生变化的时候都会产生中断。所以当按键按下时，会触发一次中断，当按键松开时会又触发一次中断。



23. 如果用户想要判断这次中断是按键按下产生的呢，还是松开产生的呢？那就需要在中断产生的时候读取 GPIO 的值了。关于如何读取按键的值，看第四章内容，这里不重复了。

第六章 串口通信例程

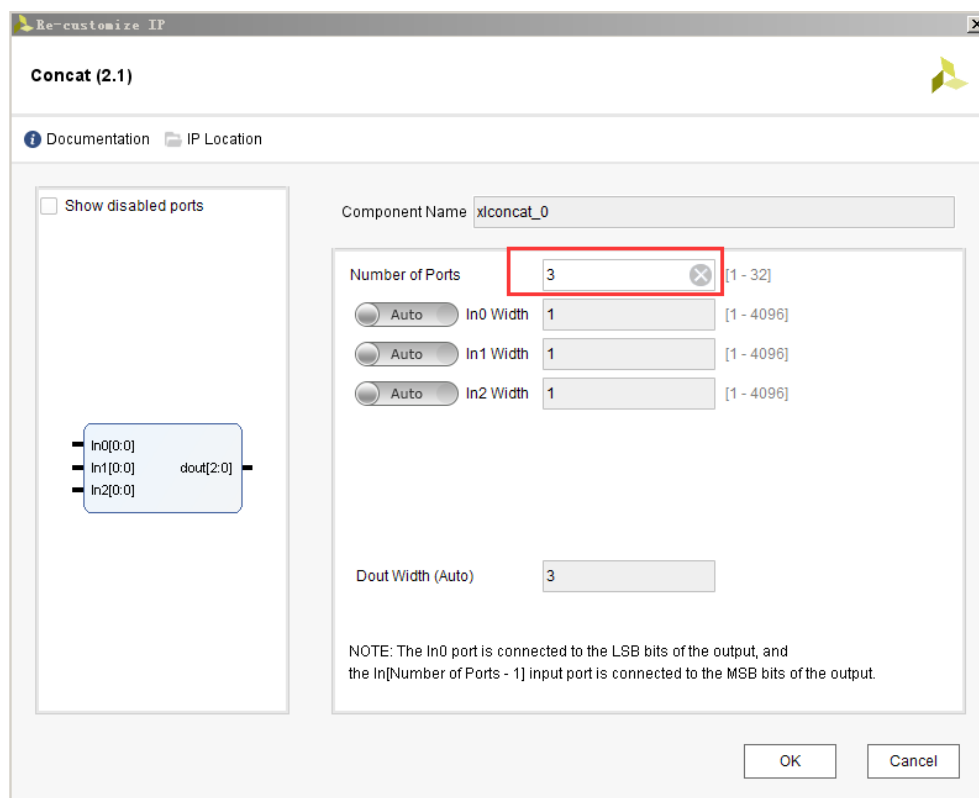
前面我们已经使用 print 函数打印信息到电脑的串口上，本章介绍在 Microblaze 里用查询方式或者用中断的方式实现串口通信。这里的 Vivado 工程是在第五章的 Vivado 工程配置的基础上修改，并删除 SDK 目录。



6.1 VIVADO 工程修改

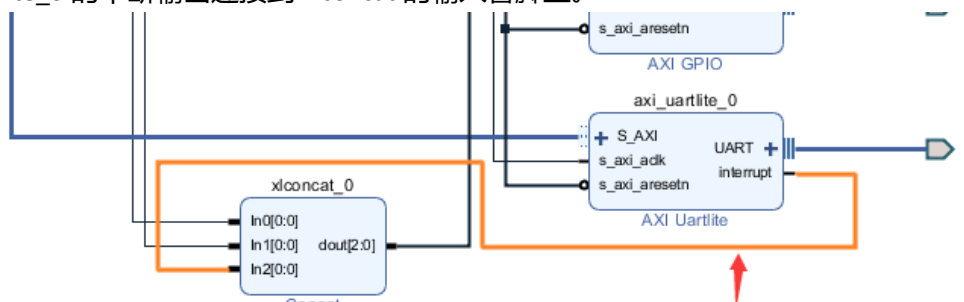
6.1.1 修改 Constant

1. 双击“Constant” IP，输入的通道数量改成 3



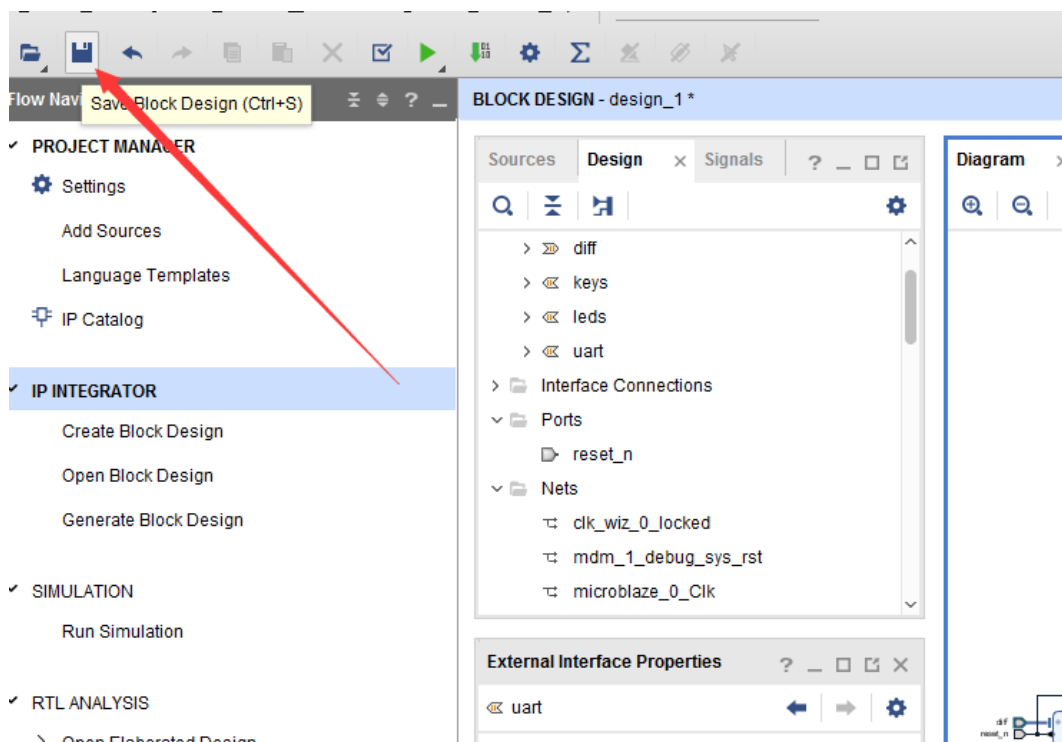
6.1.2 中断信号连线

2. axi_uartlite_0 的中断输出连接到 xlconcat 的输入管脚上。

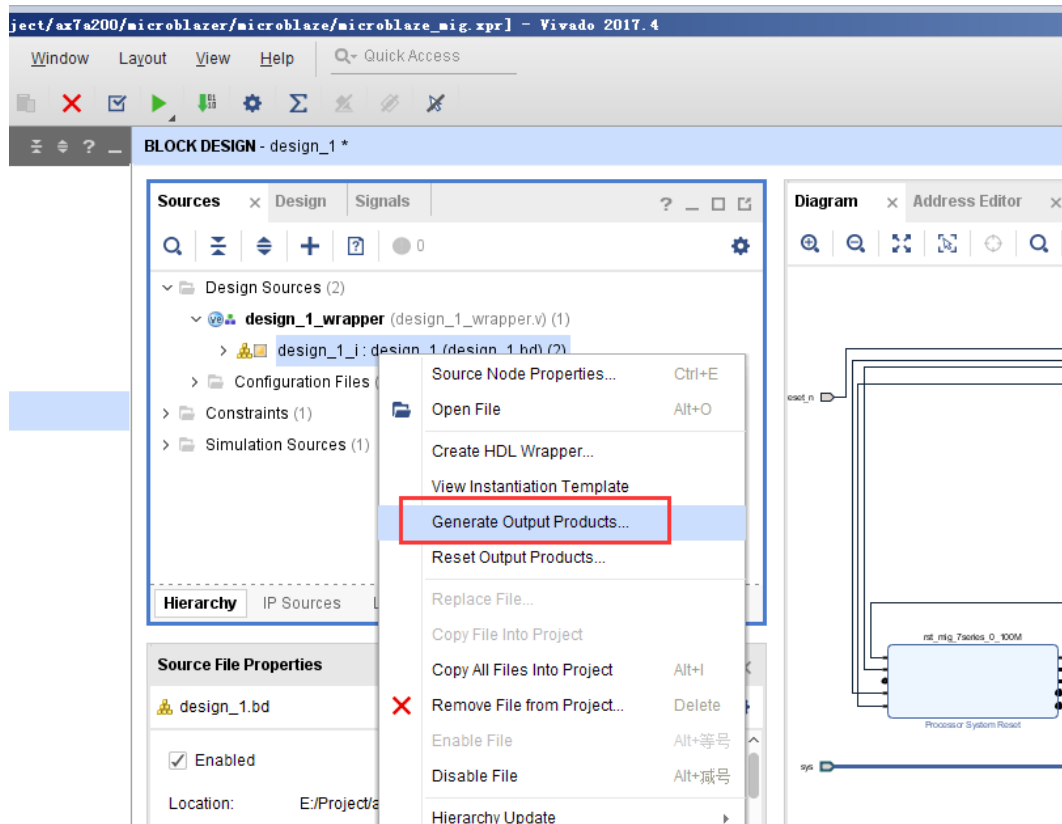


6.1.3 保存并检查错误

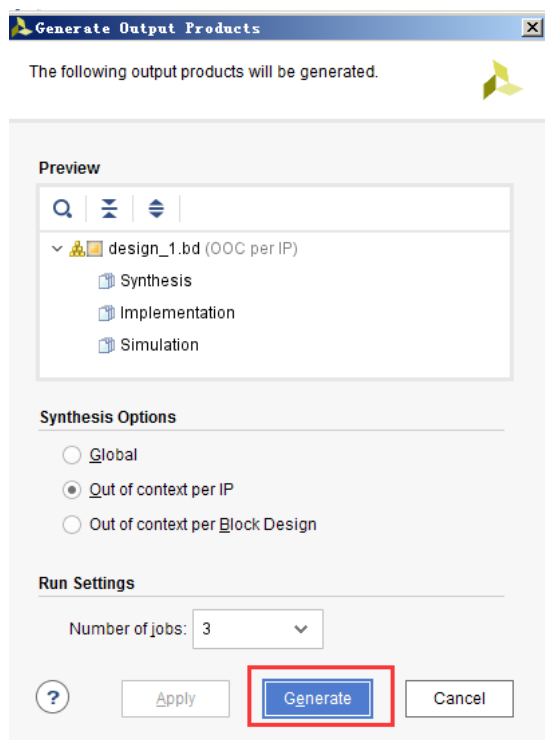
3. 保存设计，然后按 “F6” 检查设计



4. 选择 “design_1.bd” 文件，右键选择 “Generate Output Products” ，把 IP 参数和连接信息 update 到 project 中



5. 选择 Generate 按钮



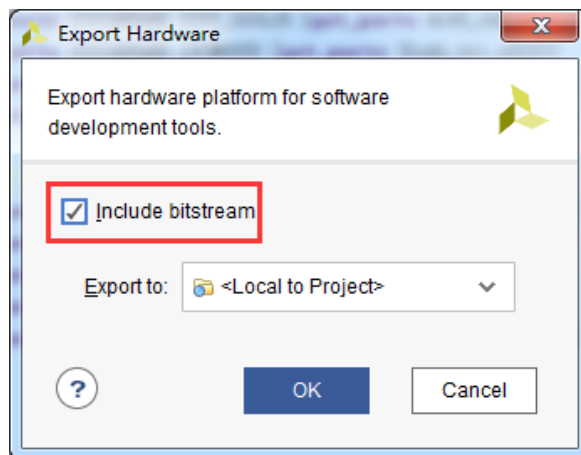
6.1.4 生成 Bitstream 文件

6. 点击 “Generate Bitstream” 生成 bit 流文件



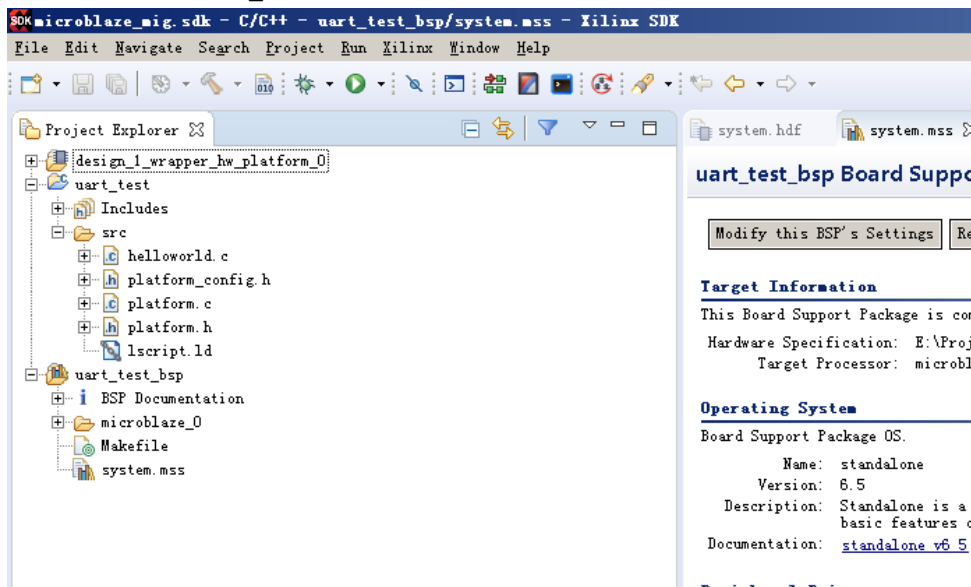
6.1.5 导出硬件

7. 在 “File -> Export -> Export Hardware...” 导出硬件，在弹出窗口中选择 “Include bitstream” 包含 Bitstream 文件



6.2 SDK 开发串口通信程序（查询方式）

8. SDK 下新建一个 uart_test 工程。



9. 修改 helloworld.c 文件如下

```
/* ***** Include Files ***** */
#include <stdio.h>
#include "xparameters.h"
#include "xuartlite_1.h"
#include "xuartlite.h"
#include "platform.h"

/* ----- main function ----- */
#define uart_baseaddr XPAR_AXI_UARTLITE_0_BASEADDR

int main()
{
    unsigned char rxdb;
    unsigned char i;
```

```

init_platform();

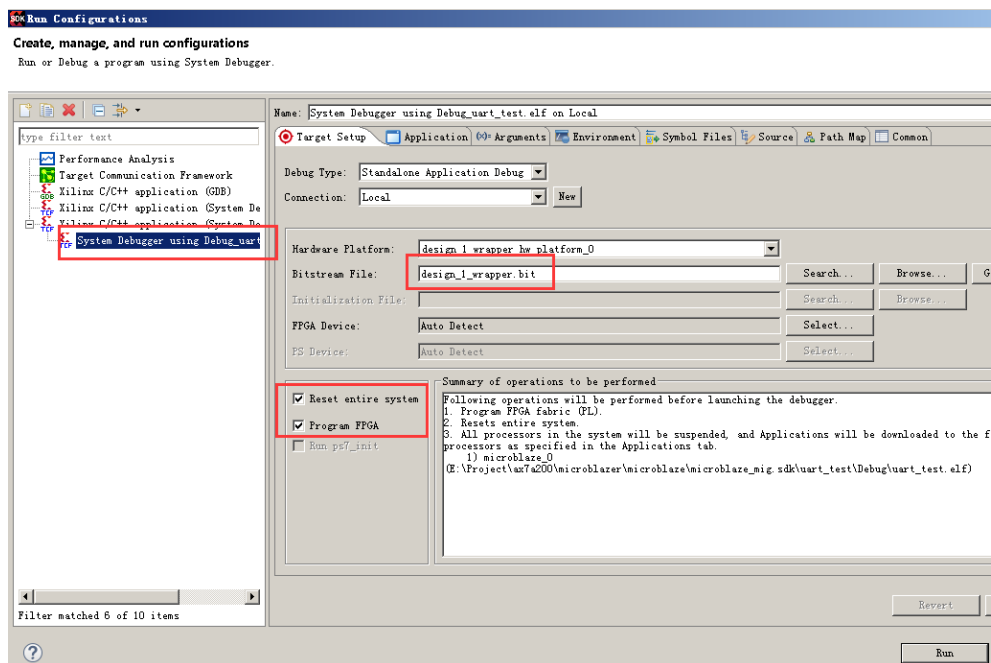
while(1)
{
    if(!XUartLite_IsReceiveEmpty(uart_baseaddr))    //check receiver buffer is empty or
not
    {
        for (i=100;i>0;i--);
        rxdb = XUartLite_RecvByte(uart_baseaddr);    //receive one byte
        for (i=100;i>0;i--);
        XUartLite_SendByte(uart_baseaddr,rxdb);    //send one byte
        for (i=100;i>0;i--);
    }
}
return 0;
}

```

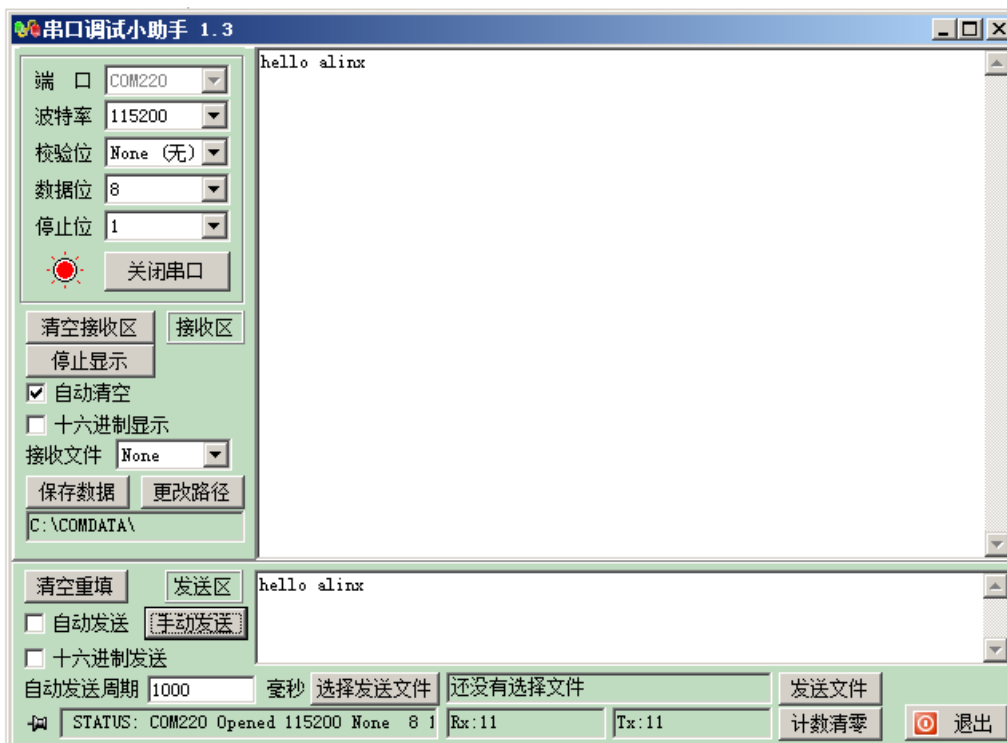
程序通过控制 uartlite 控制器的 3 个寄存器实现串口通信，寄存器的定义参考
"pg142-axi-uartlite.pdf"

6.3 程序下载测试

10. 选择 "uart_test" 工程，右键选择 "Run As -> Run Configuration..."，在弹出的界面里配置如下。

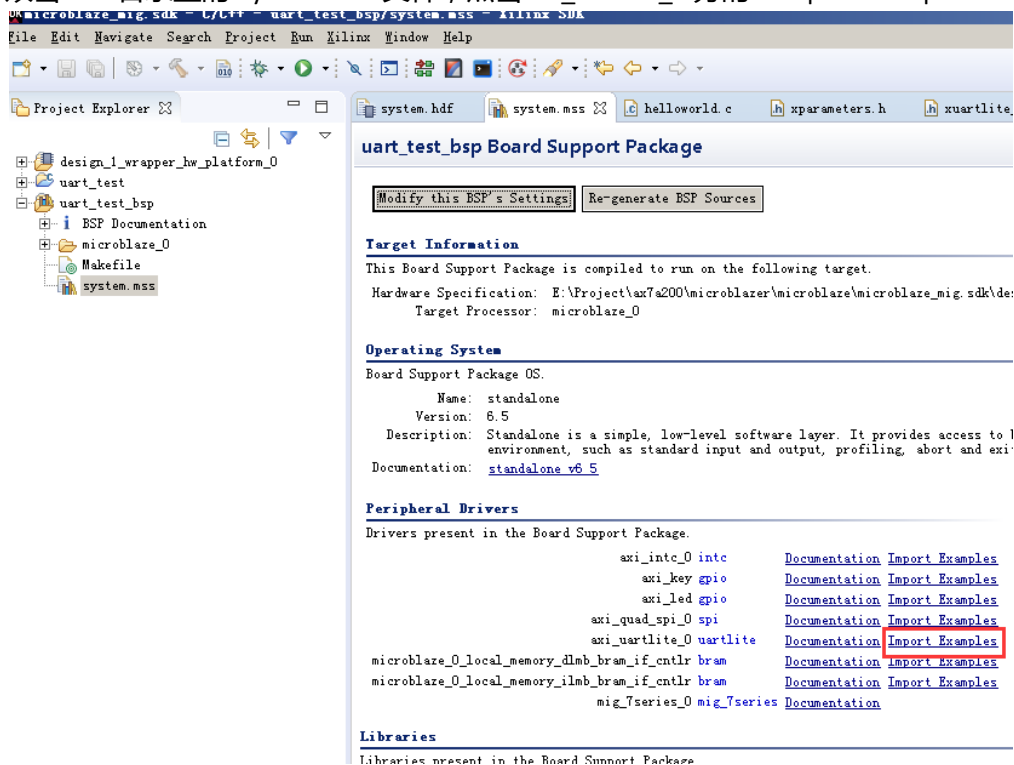


连接 JTAG 线到开发板、Run 下载。下载后在串口调试工具里输入 "hello alinx" 字符，然后发送，在接收的窗口会返回同样的字符。

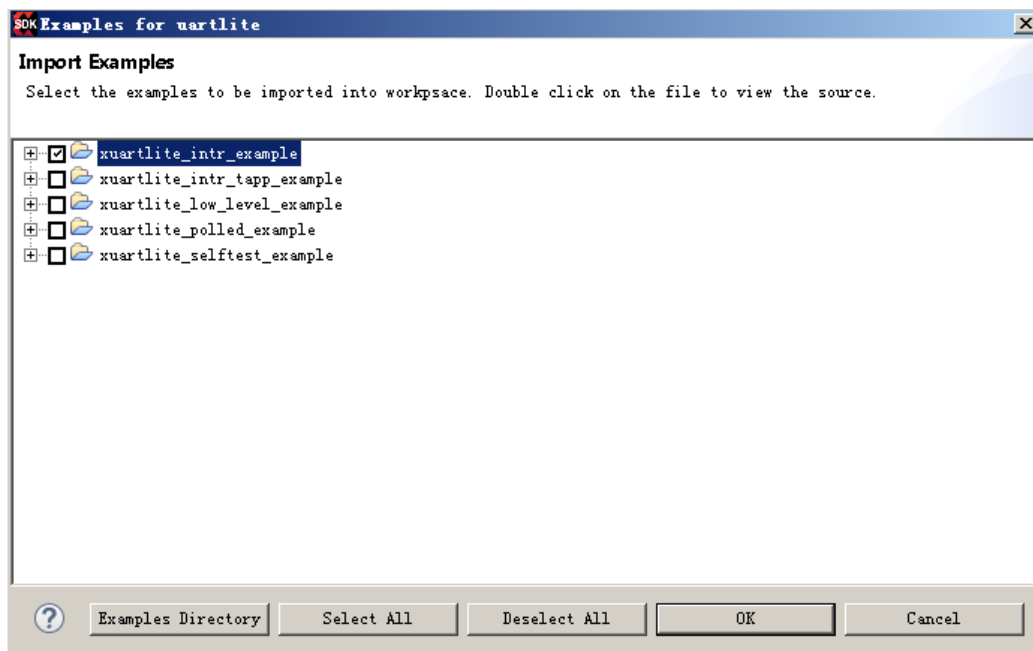


6.4 SDK 开发串口通信程序（中断方式）

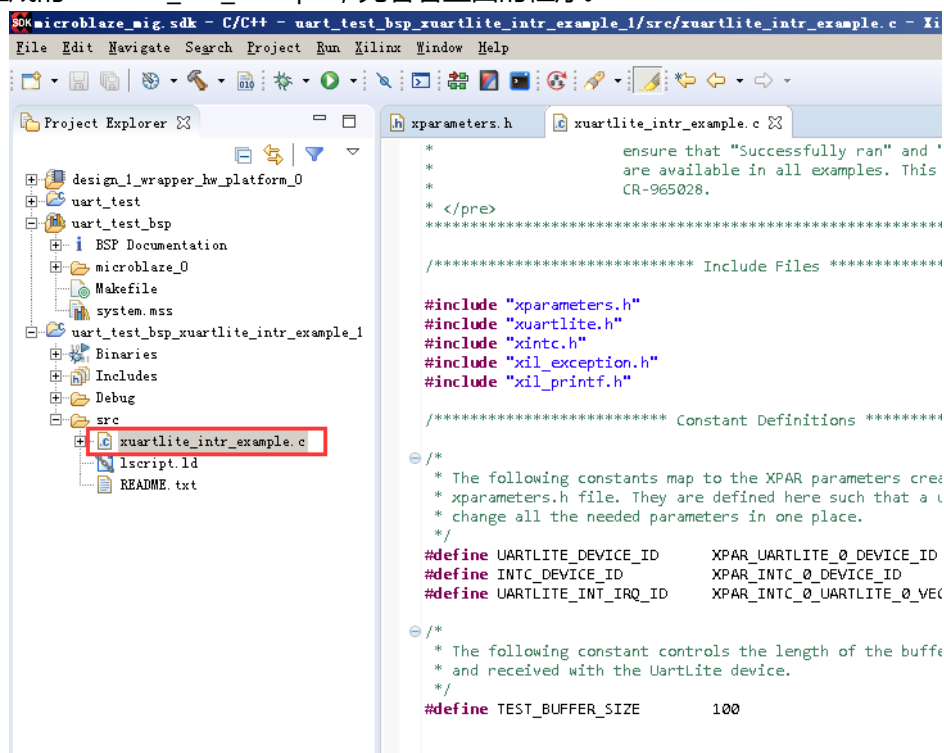
11. 双击 BSP 目录里的“system.mss”文件，点击 axi_uartlite_0 旁的 “Import Examples”



12. 选择 “uart_test_bsp_xuartlite_intr_example_1”



13. 打开生成的 xuartlite_intr_example，先看看里面的程序。



程序里通过 UartLiteIntrExample 程序实现 uart 中断测试，里面主要用到的函数有：

XUartLite_Initialize (&UartLite, DeviceId)	串口初始化
XUartLite_SelfTest (&UartLite)	串口自我测试
SetupInterruptSystem (&UartLite)	建立中断，这个在按键中断也有使用

XUartLite_SetSendHandler(&UartLite, SendHandler, &UartLite)	发送中断调用函数
XUartLite_SetRecvHandler(&UartLite, RecvHandler, &UartLite)	接收中断调用函数
XUartLite_EnableInterrupt(&UartLite)	串口中断使能
XUartLite_Recv(&UartLite, ReceiveBuffer, TEST_BUFFER_SIZE)	接收一定数量的数据到 buffer
XUartLite_Send(&UartLite, SendBuffer, TEST_BUFFER_SIZE)	发送 buffer 中一定数量的数据

14. 为了方便测试，我们这里修改一下测试的数据数量为 10。

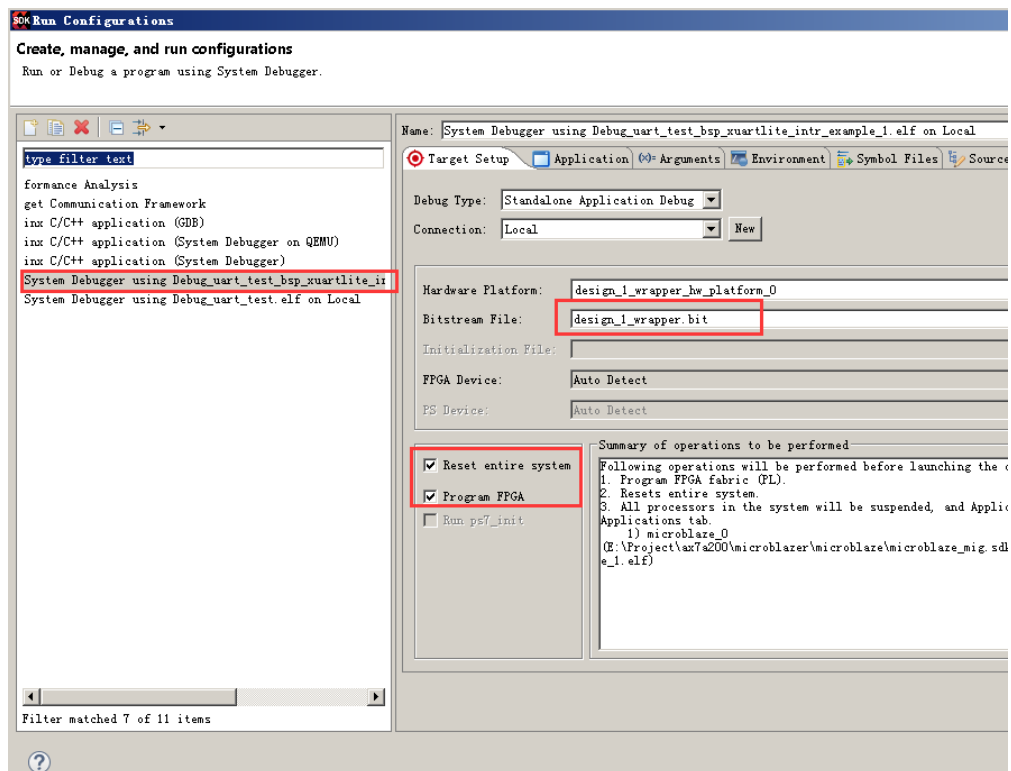
```

/*
 *
 */
#define TEST_BUFFER_SIZE 10

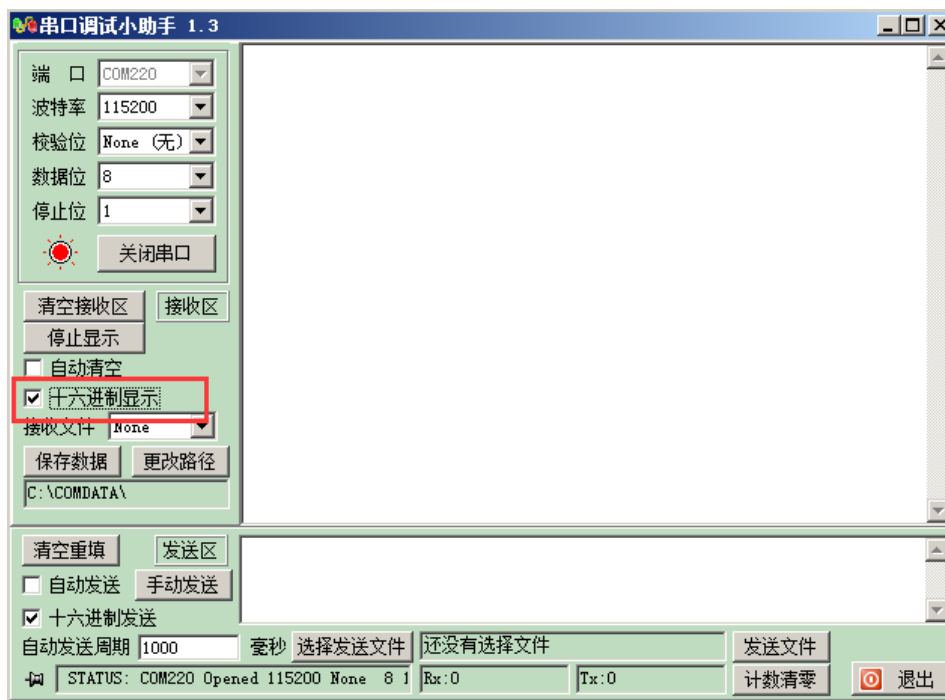
```

6.5 程序下载测试

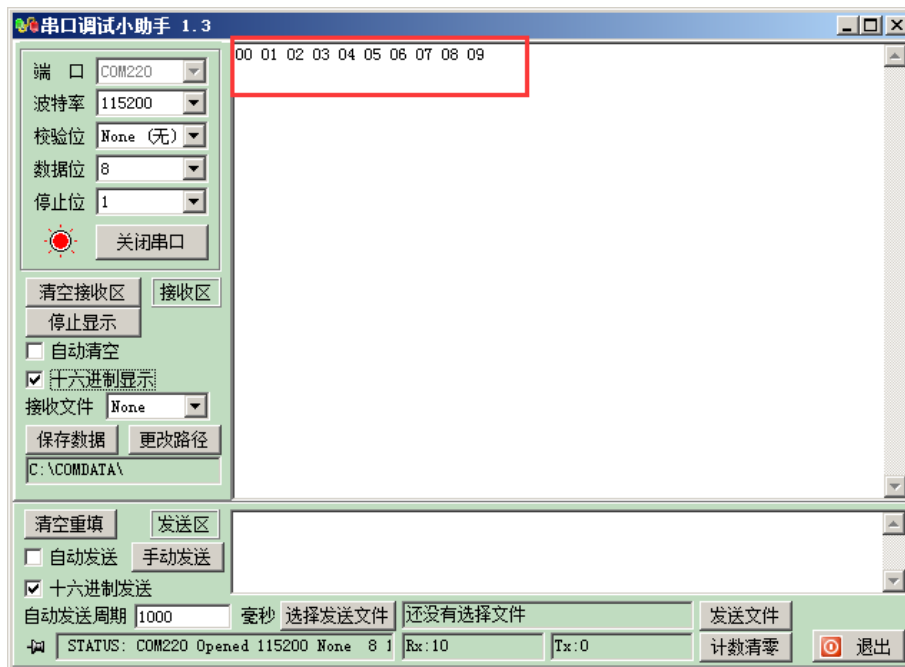
15. 选择 “uart_test_bsp_xuartlite_intr_example_1” 工程，右键选择 “Run As -> Run Configuration...” ,在弹出的界面里配置如下。



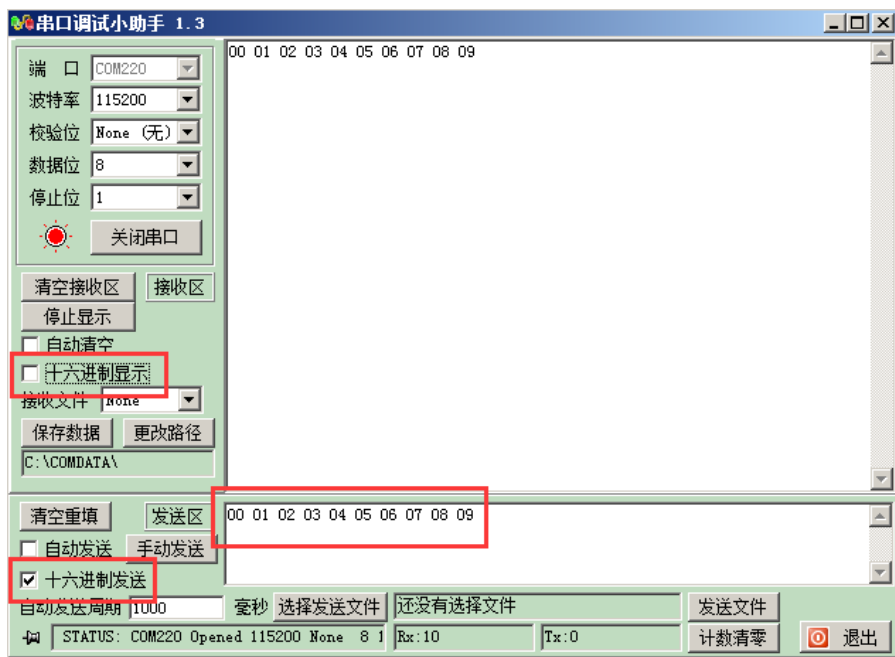
16. 串口工具设置成 16 进制显示。



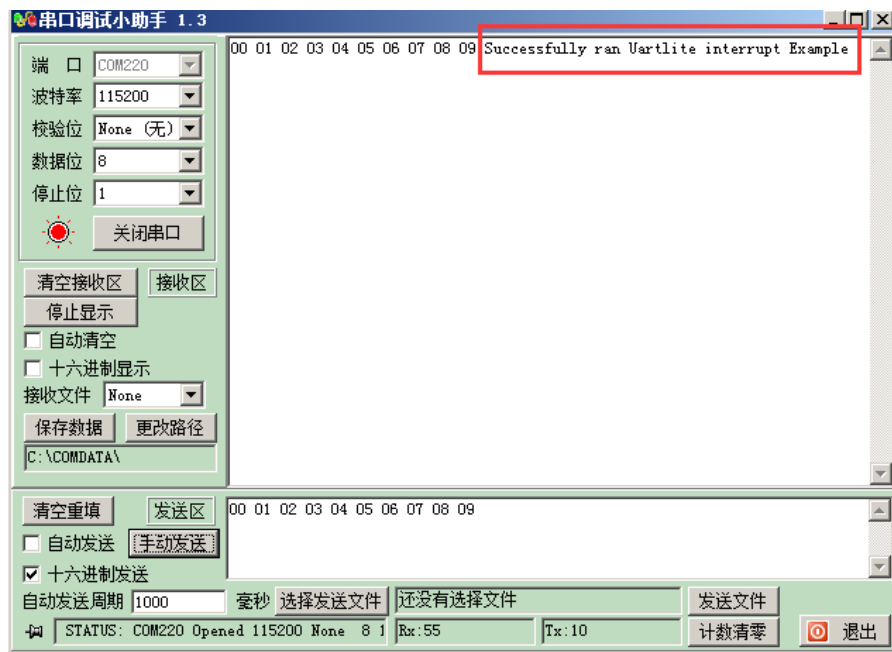
17. 连接 JTAG 和串口线到开发板、Run 下载。下载后，串口显示接收到 10 个测试数据。



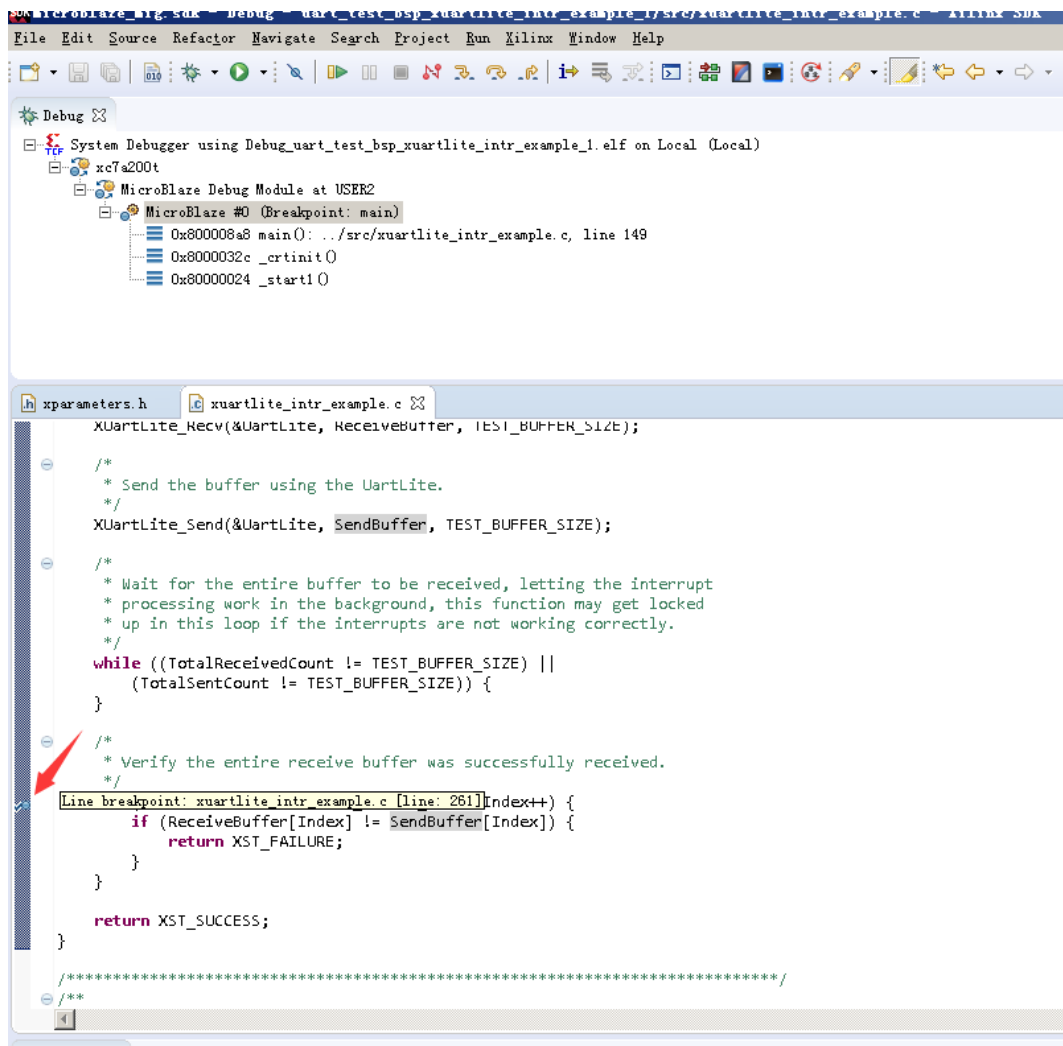
18. 然后我们这里需要拷贝这 10 个数据，选择 16 进制发送。另外要把 16 进制显示的勾给去掉，这样可以继续接收后面程序发送的字符串。点击手动发送



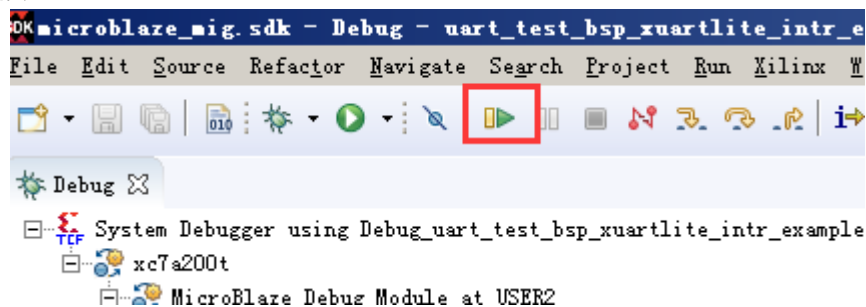
19. 程序接收到 10 个数据后会跟发送的数据进行比较，正确的话会发成功的消息。



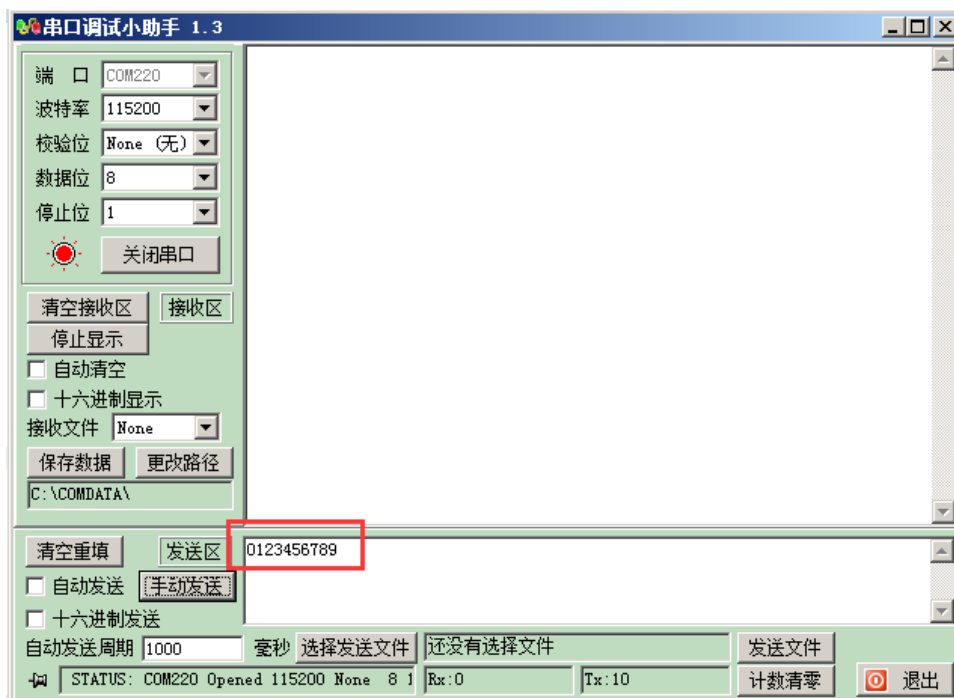
20. 我们也可以通过 Debug 的方式，在程序里查看接收到的数据。进入 Debug as hardware 后在以下的语句这边加一个断点，让程序运行到这里停止。



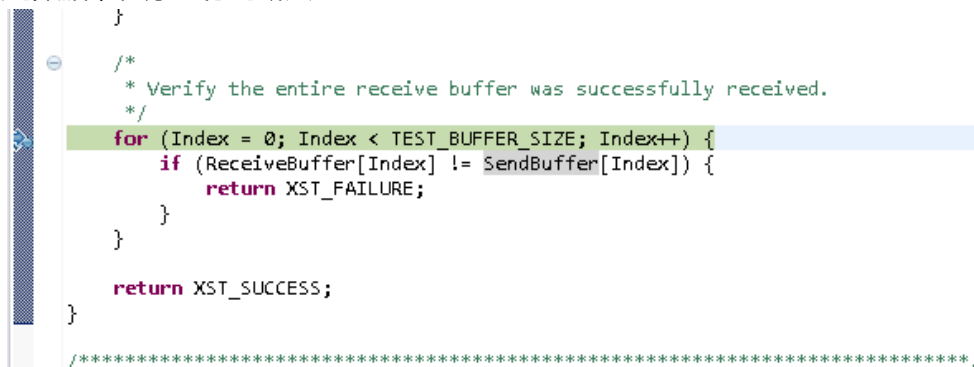
21. 点击运行按钮。



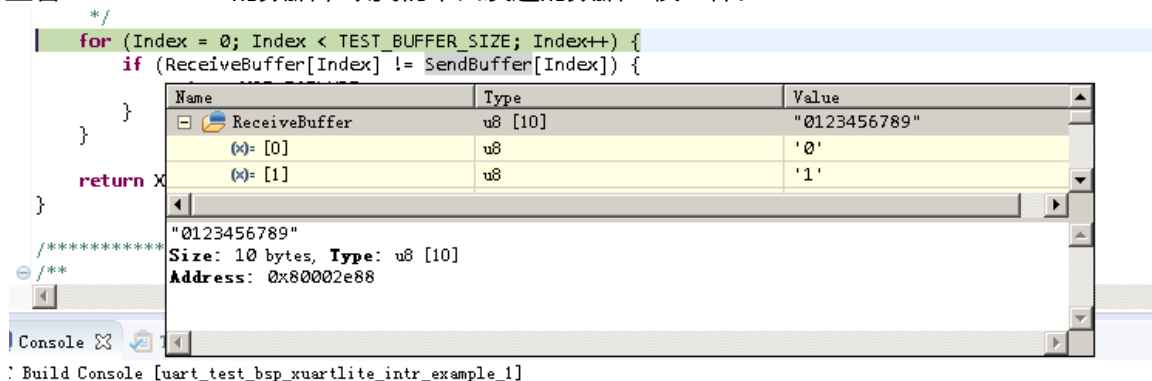
22. 通过串口发送 10 个字符数据，比如 “0123456789”，不要选择 16 进制。



23. 接收到数据，程序运行到断点。



24. 查看 ReceiveBuffer 的数据，跟我们串口发送的数据一模一样。



第七章 以太网实验 (LWIP)

实验 Vivado 工程为 “net_test”。

这里以 AX7A200 开发板为例，板上有 1 路千兆以太网，通过 RGMII 接口连接，可以由 FPGA 控制，本实验演示如何使用 SDK 自带的 LWIP 模板进行千兆以太网 TCP 通信。

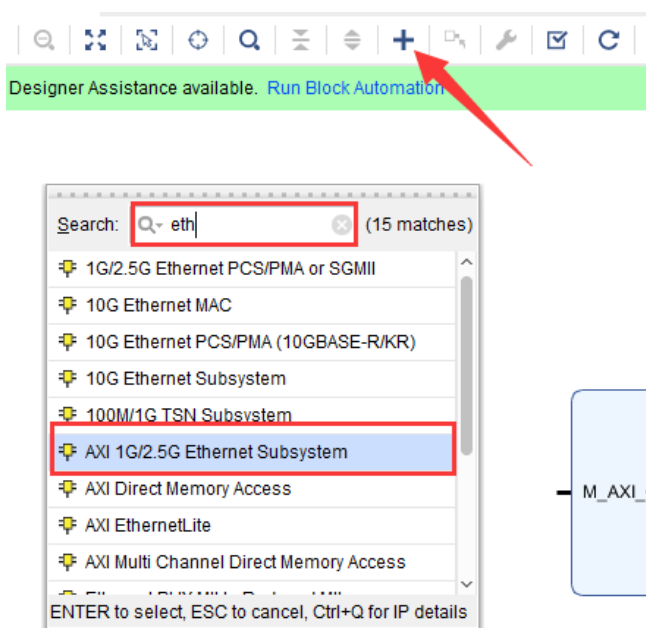
LWIP 虽然是轻量级协议栈，但如果从来没有使用过，使用起来会有一定的困难，建议先熟悉 LWIP 的相关知识。

7.1 Vivado 工程修改

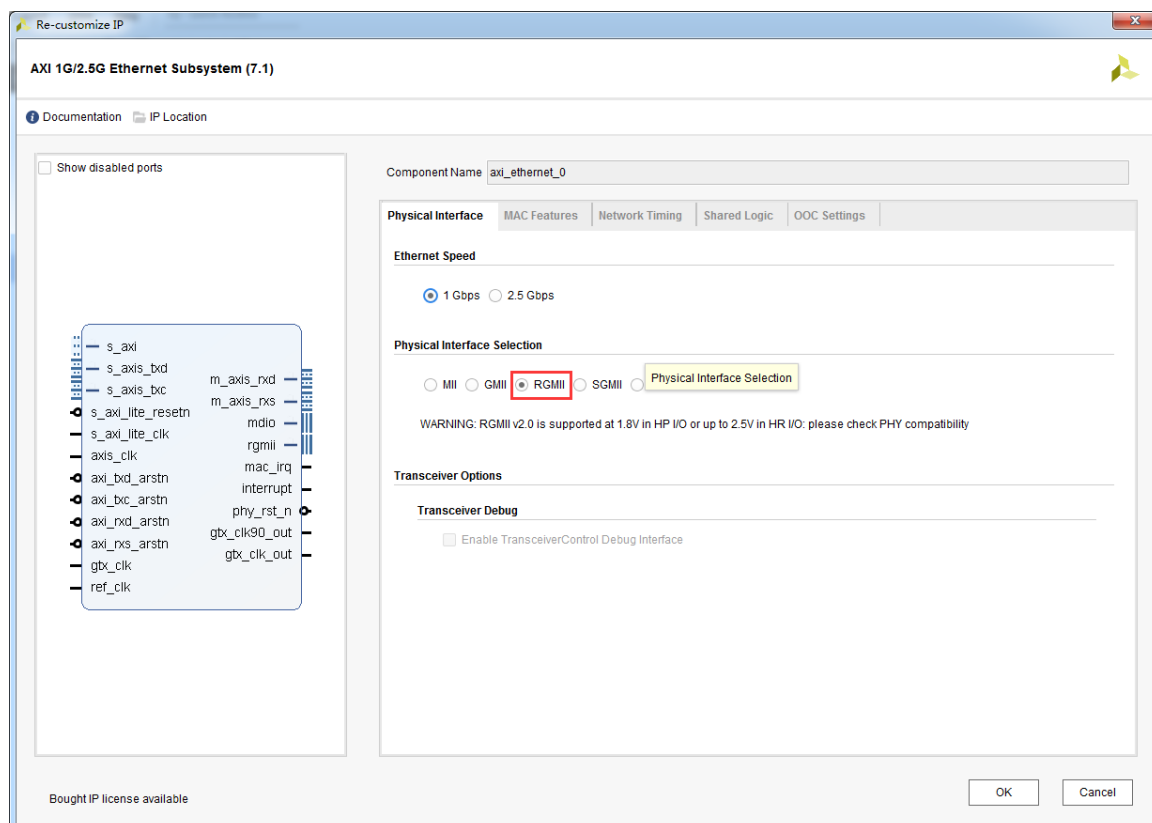
Vivado 工程是在第六章的 Vivado 工程配置的基础上修改，并删除 SDK 目录。

7.1.1 添加以太网 IP

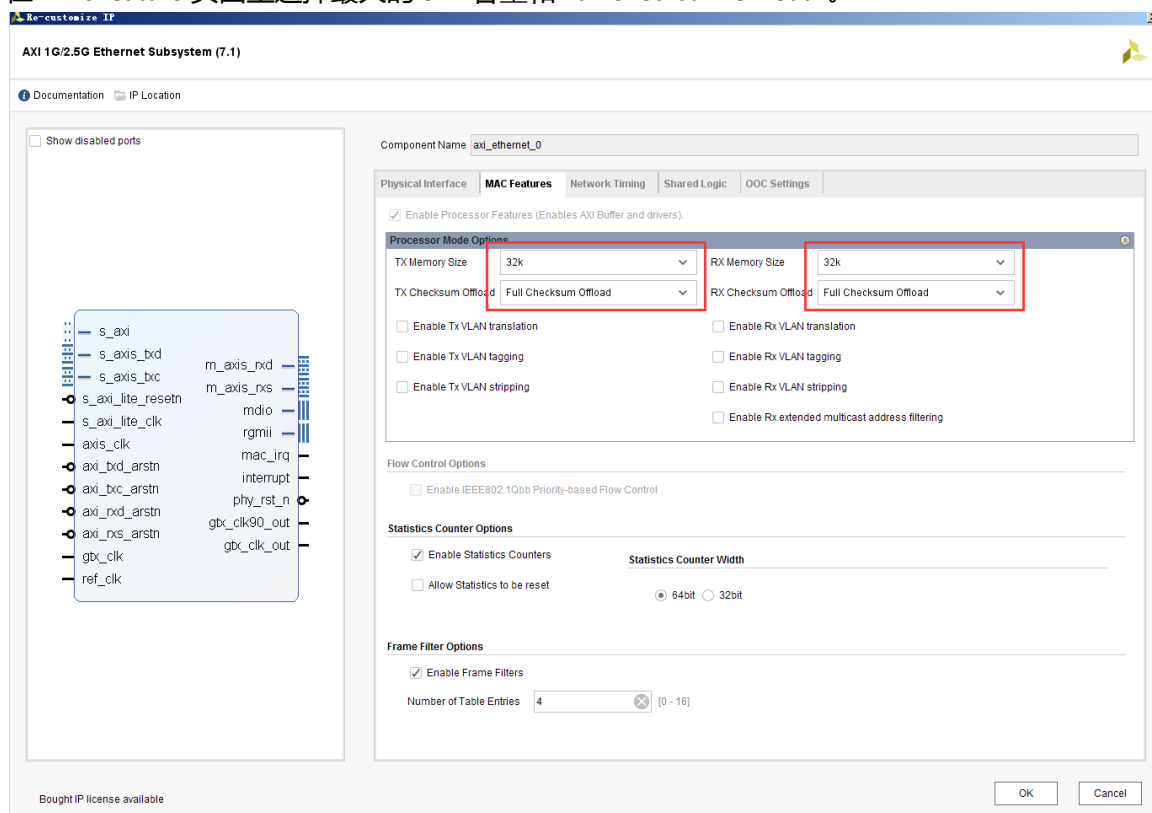
- 1) 搜索 “eth”，添加一个 “AXI 1G/2.5G Ethernet Subsystem”



- 2) 双击刚才添加的模块，修改参数，物理接口选择 “RGMII” (如果是其它 GMII 接口的开发板，就保存默认的 GMII 接口)

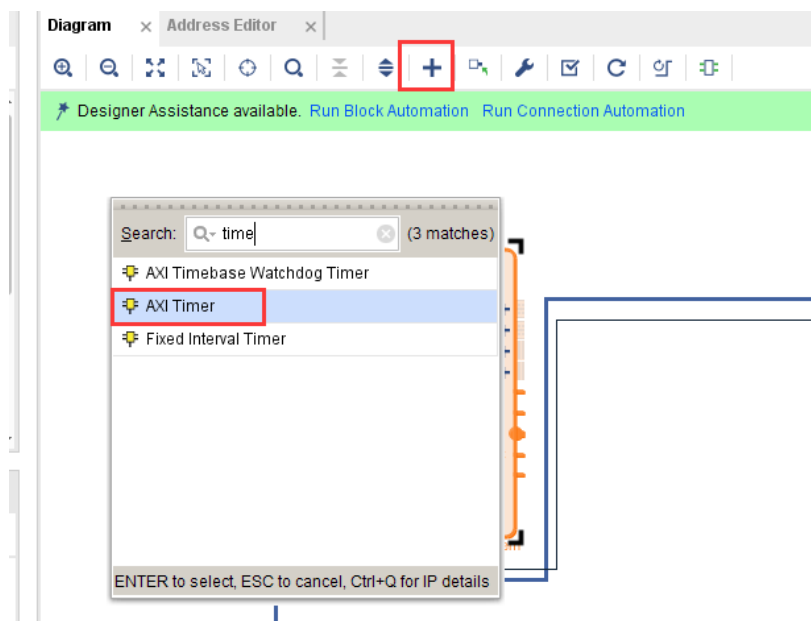


- 3) 在 MAC feature 页面里选择最大的 32K 容量和"full Checksum Offload".

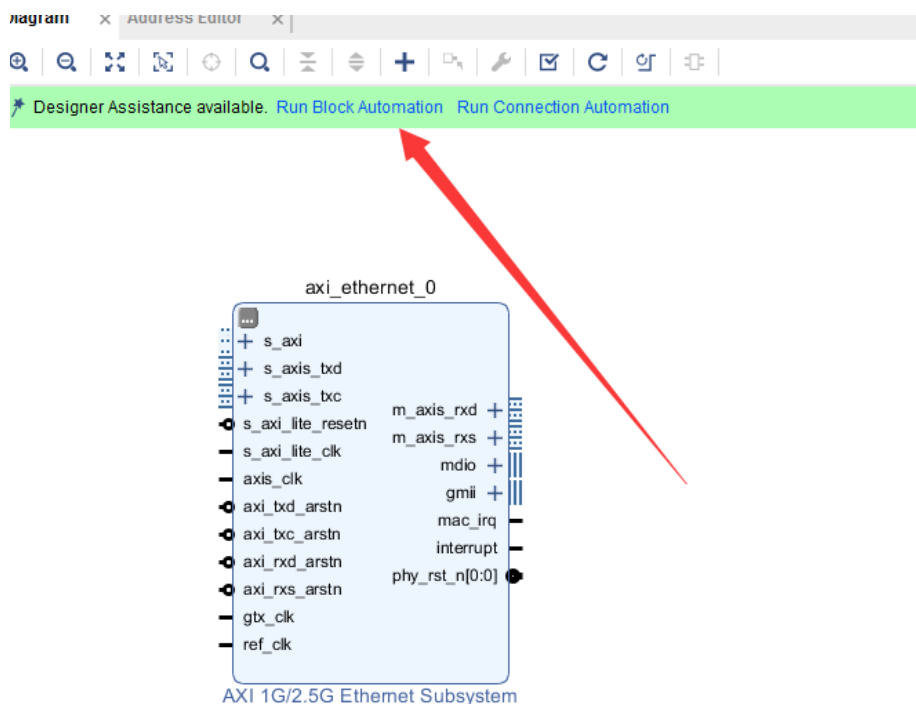


7.1.2 添加定时器

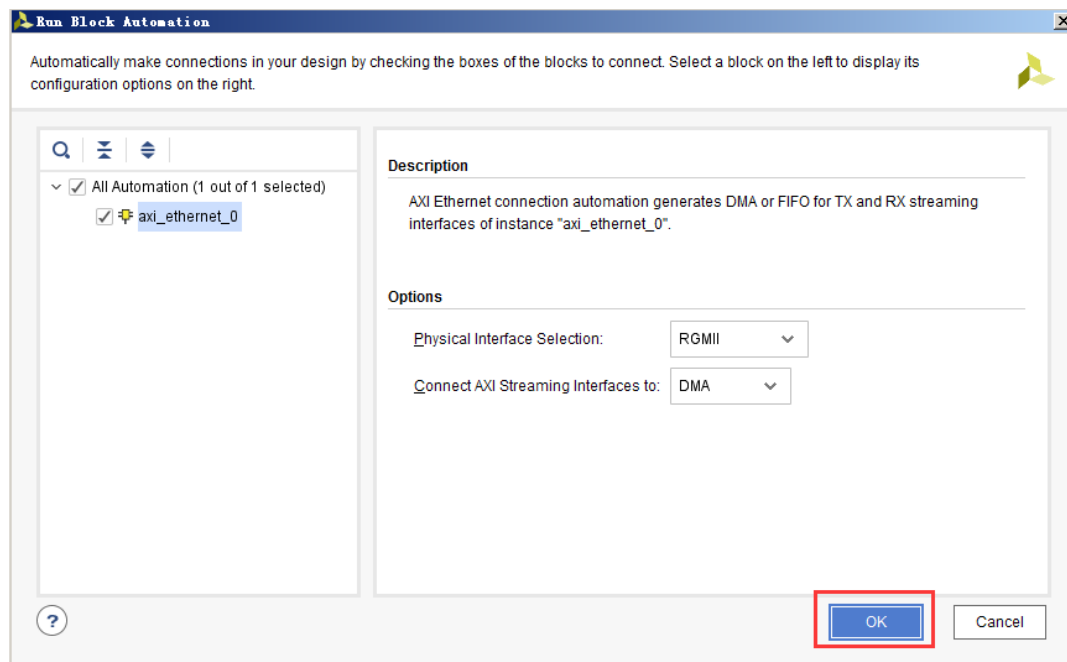
- 4) 以太网通信需要一个 axi timer 的 IP。搜索 "timer"，选择 AXI Timer。



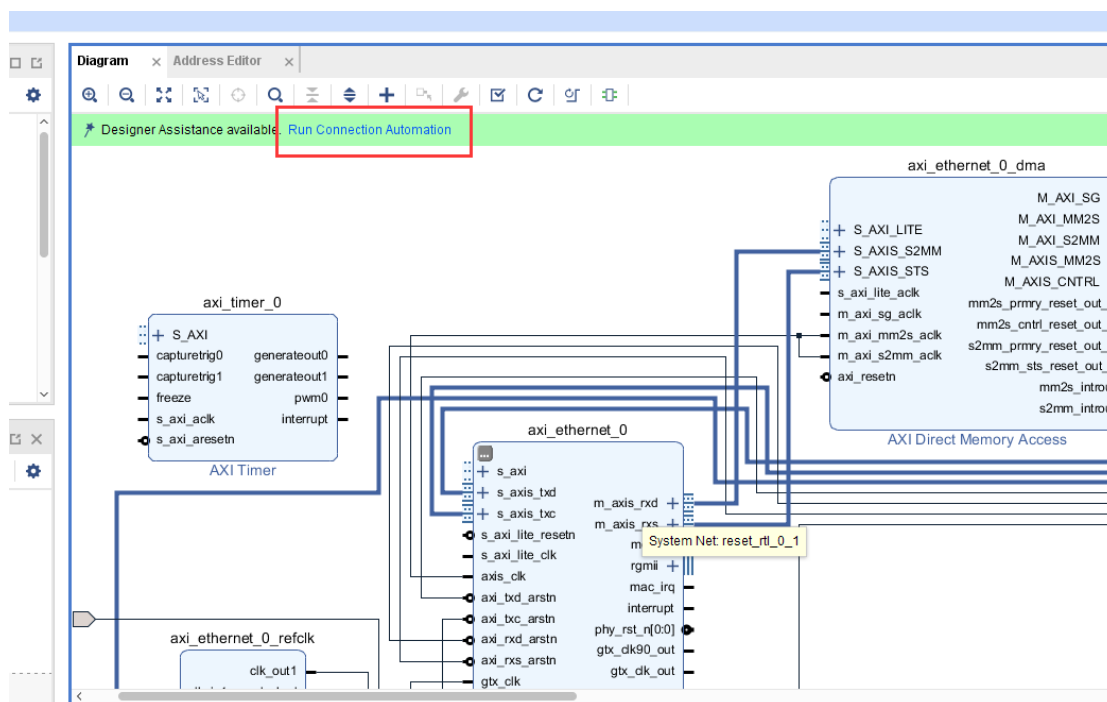
5) 点击 “Run Block Automation”



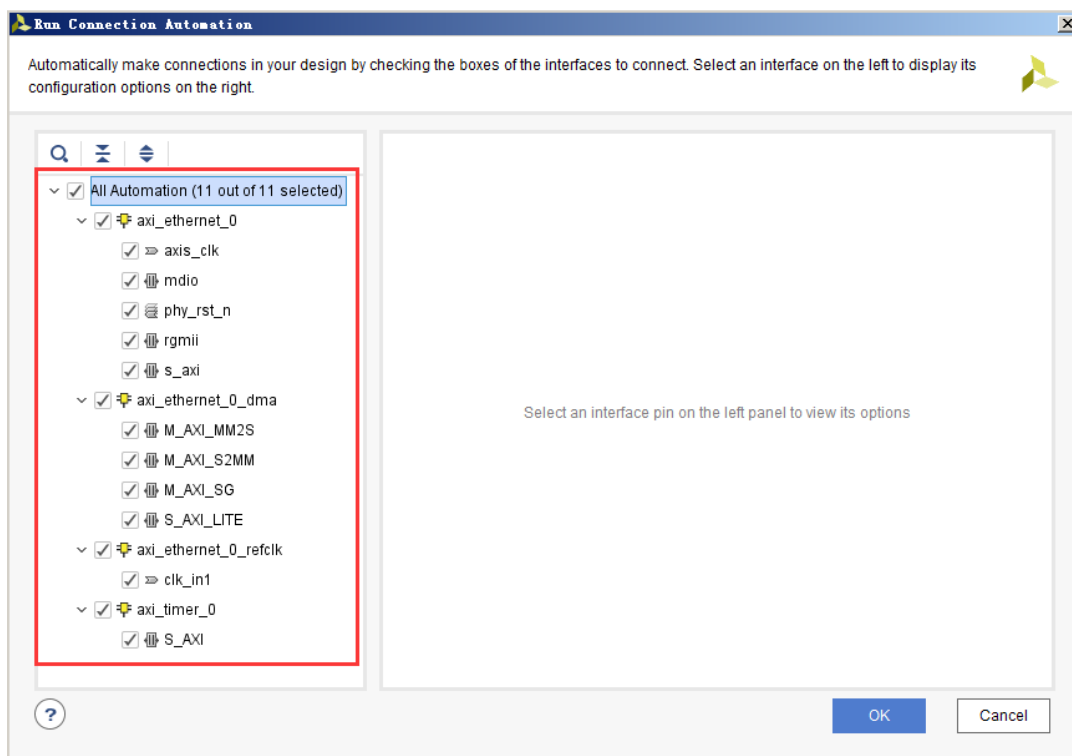
6) 选择 ethernet IP 的自动连接，软件会自动添加 DMA IP。



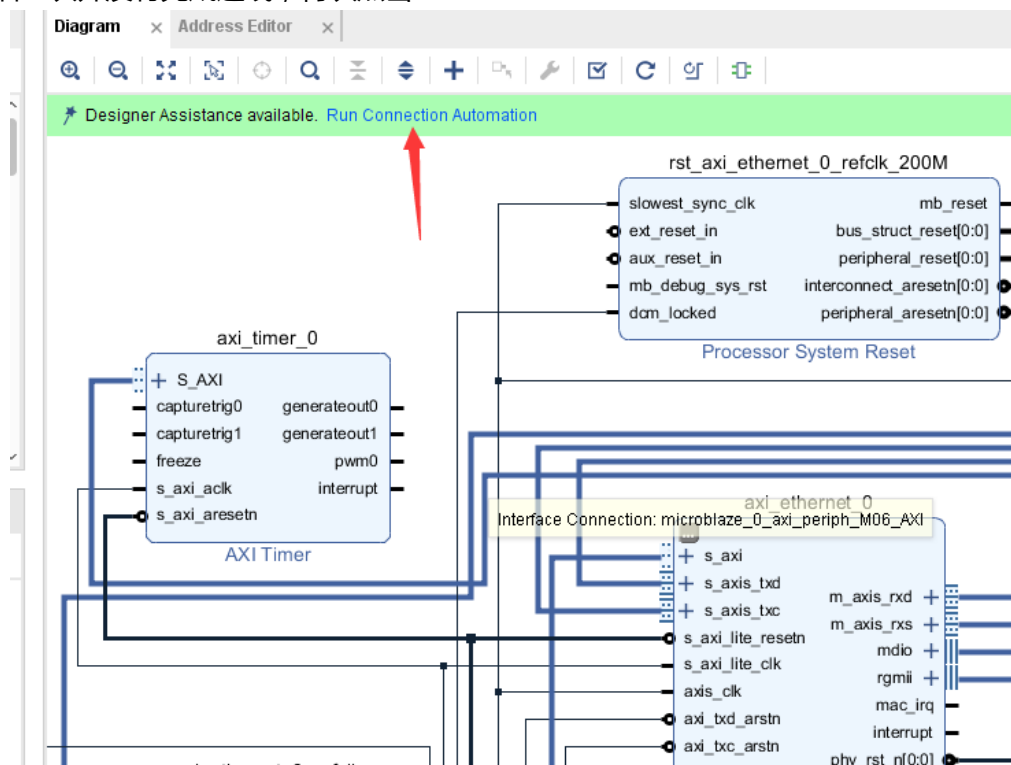
- 7) 这个时候可以 Vivado 自动插入了很多模块，比如 DMA IP 和 clock IP，点击“Run Connection Automation”自动连线 and 端口



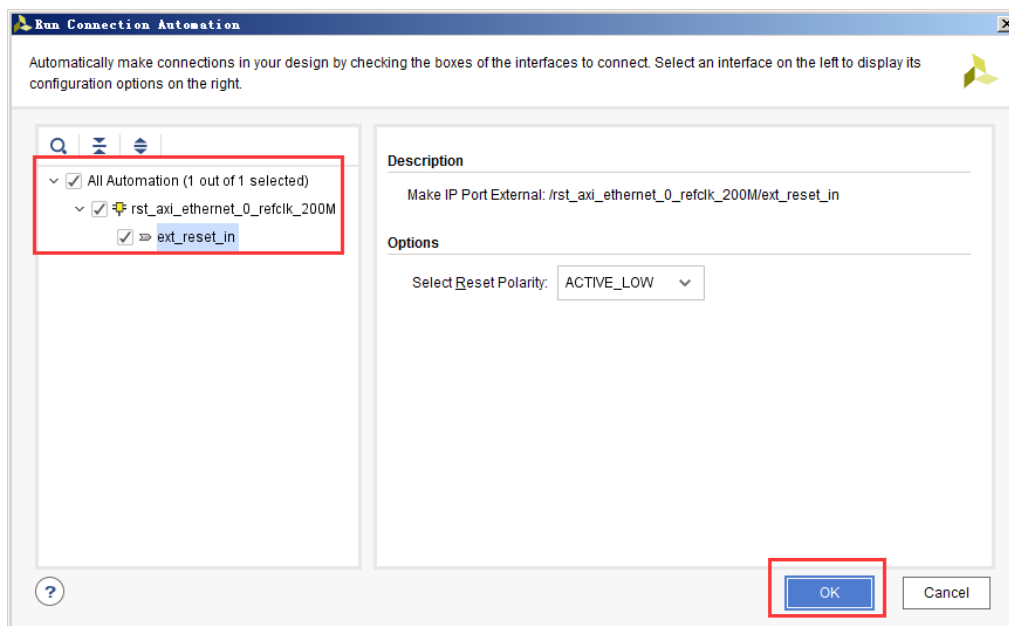
- 8) 选择所有自动连接



9) 然后一次并没有完成连线，再次点击“Run Connection Automation”

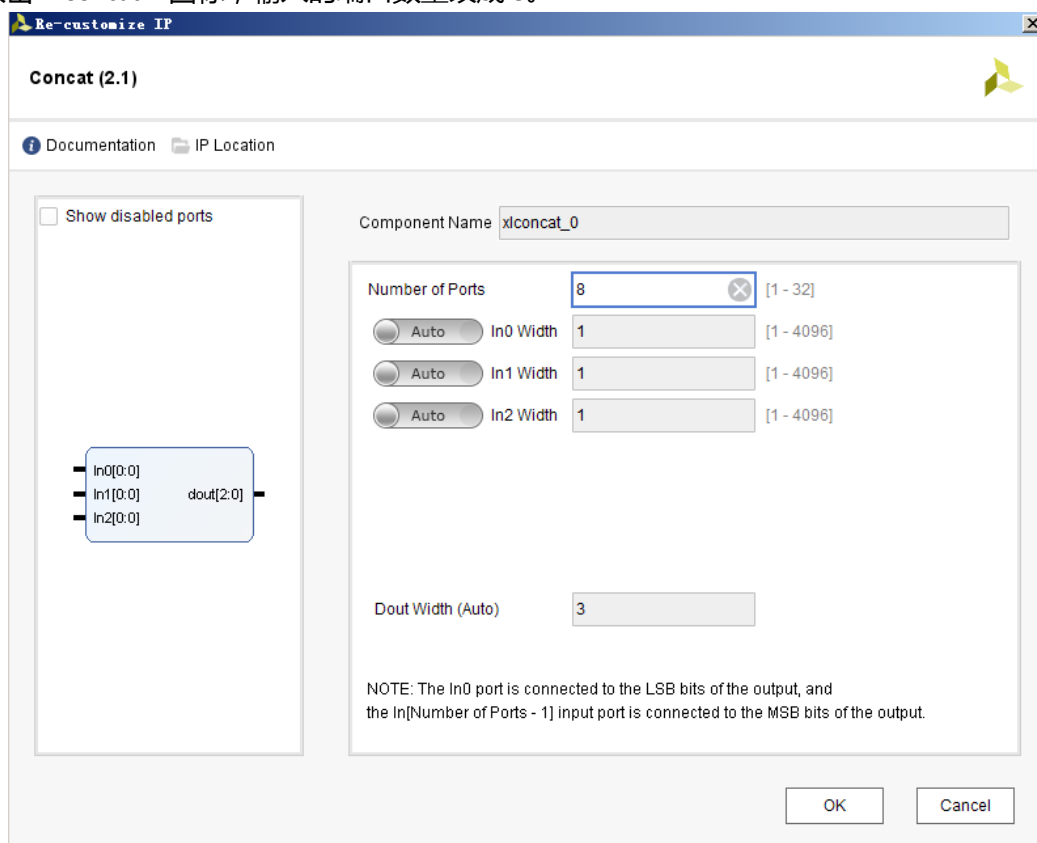


10) 选择所有



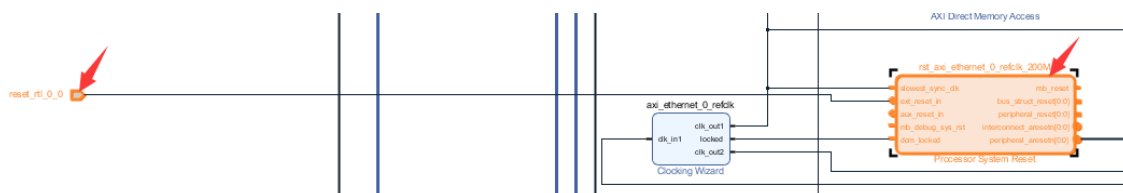
7.1.3 修改 Constant

11) 双击“Concat”图标，输入的端口数量改成 8。



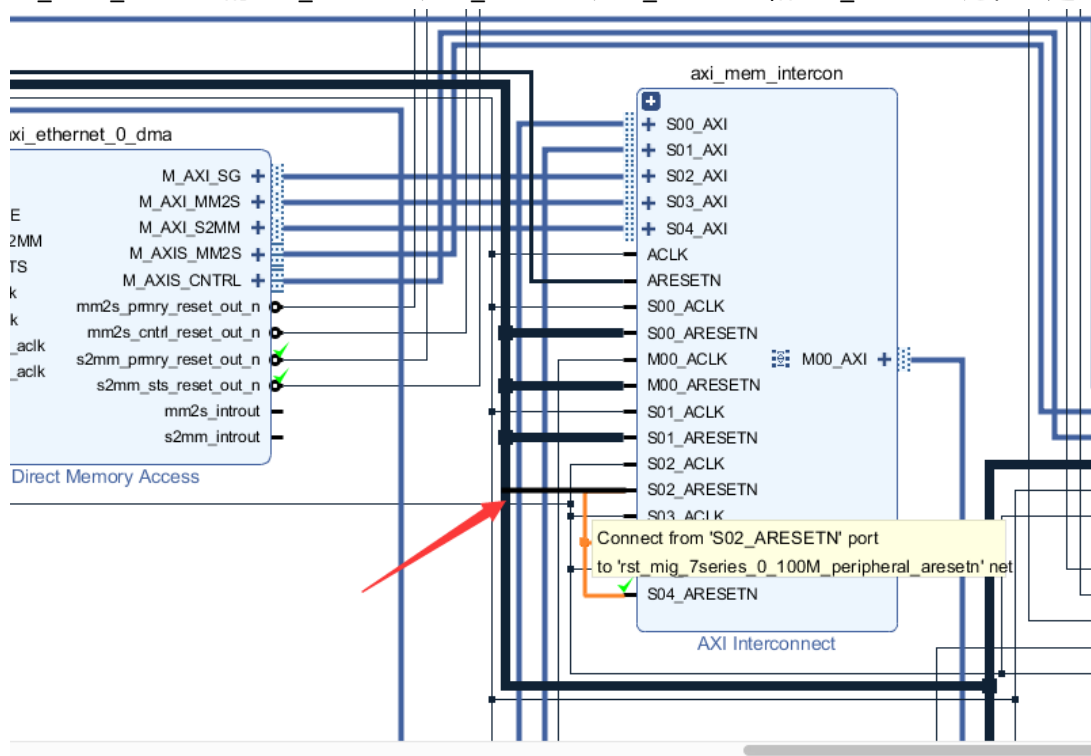
7.1.4 删除 rst_axi_ethernet_0_refclk_200M

12) 删除 rst_axi_ethernet_0_refclk_200M 和输入的端口 reset_rtl_0_0

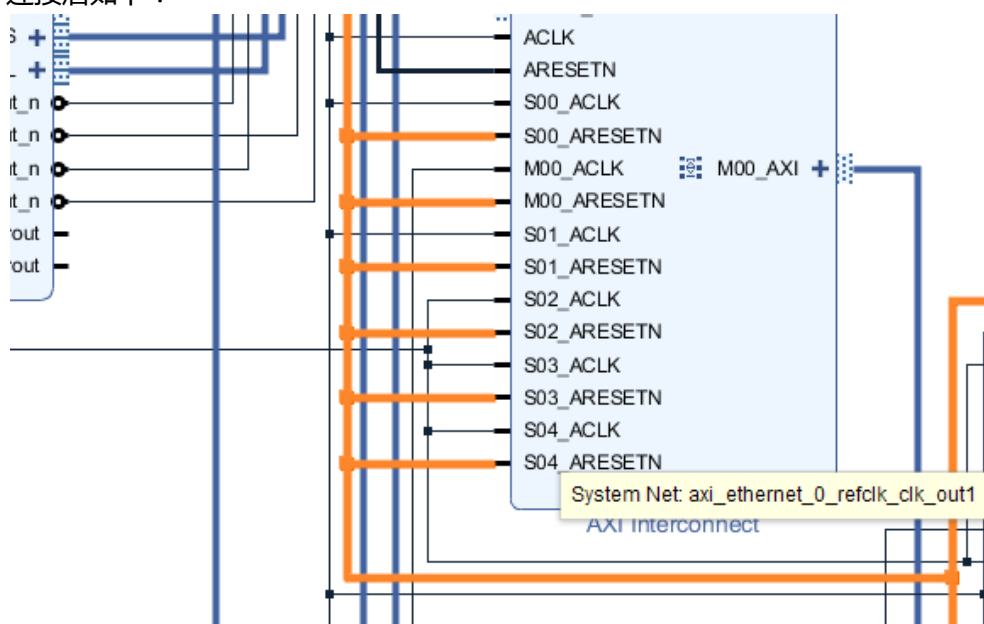


7.1.5 信号连线

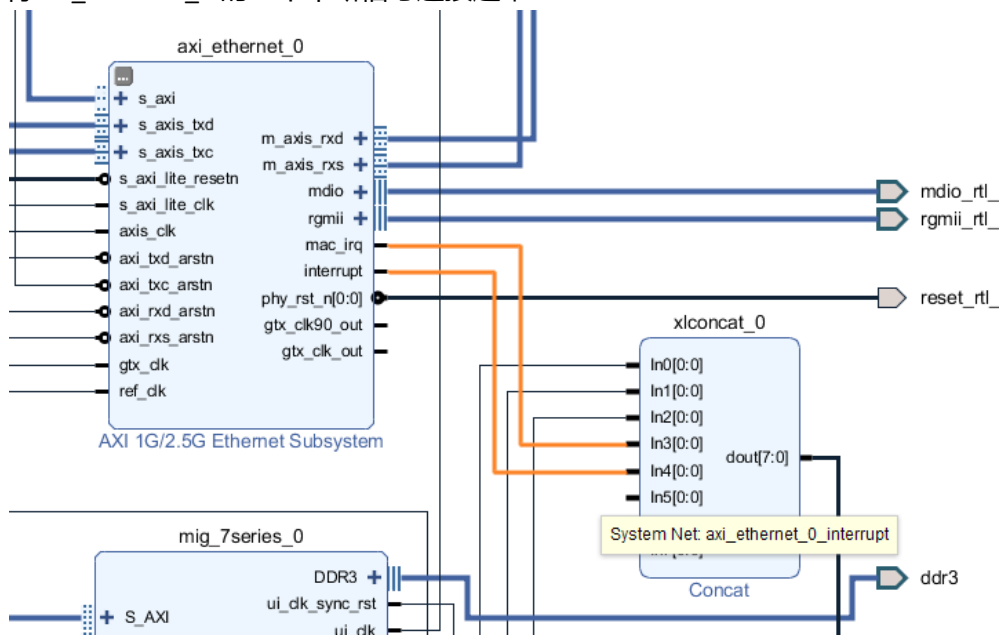
13) axi_mem_intercon 的 S02_ARESETN、S03_ARESETN、S04_ARESETN 和 S01_ARESETN 连在一起。



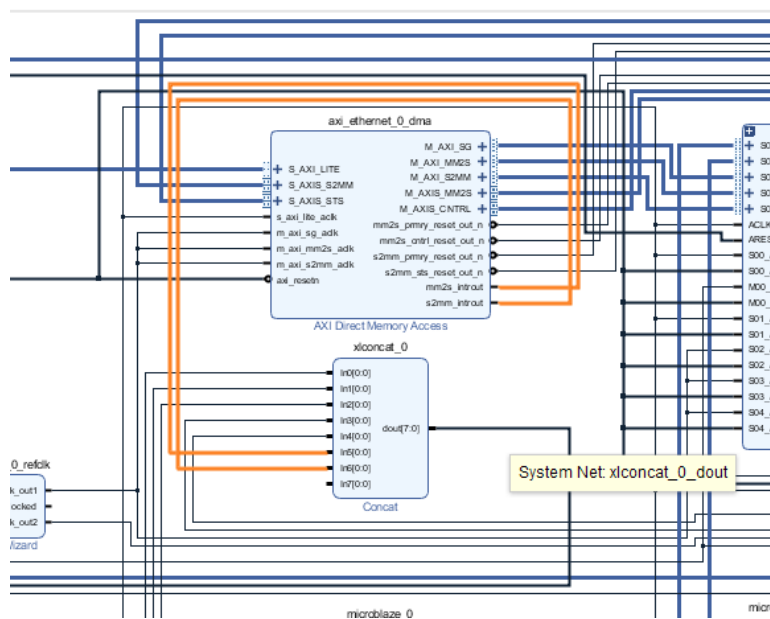
连接后如下：



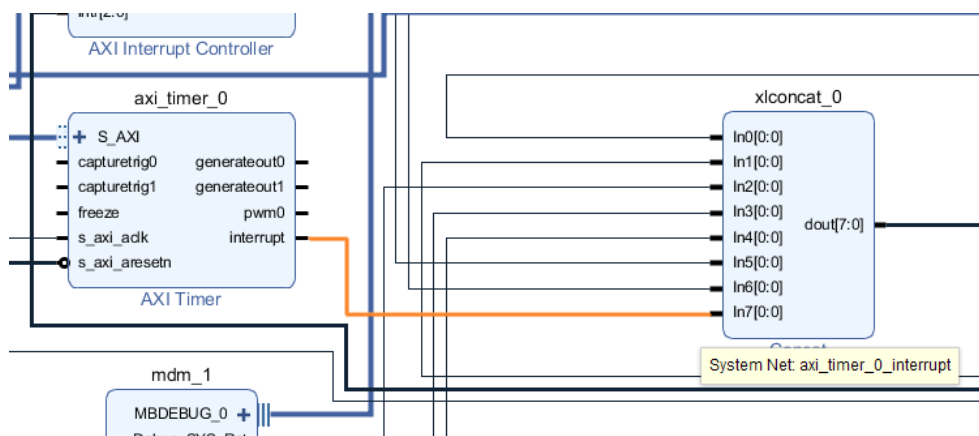
14) 将 axi_ethernet_0 的 2 个中断信号连接起来



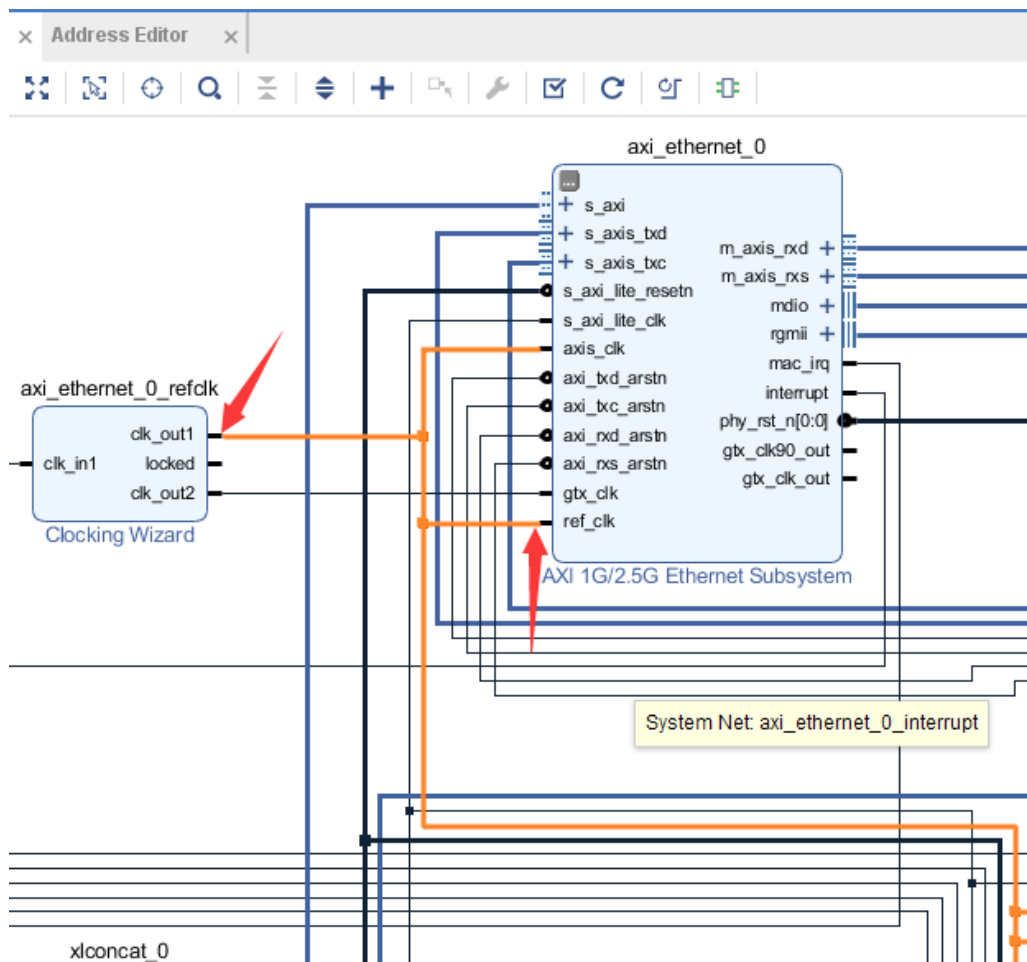
15) 将 axi_ethernet_0_dma 的 2 个中断信号连接起来



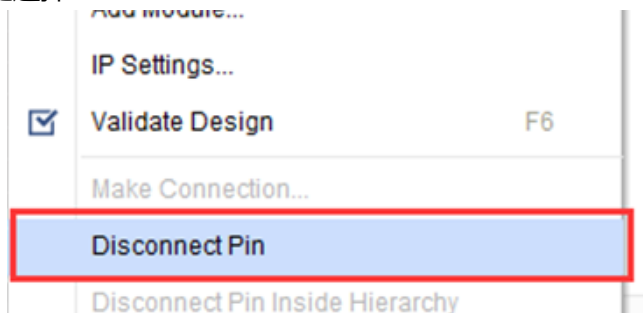
16) 将 axi_timer_0 的中断信号连接



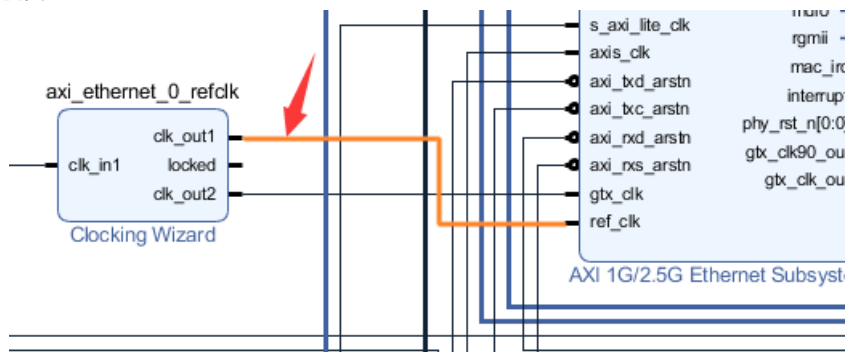
17) 对以下 2 个管脚连线断开



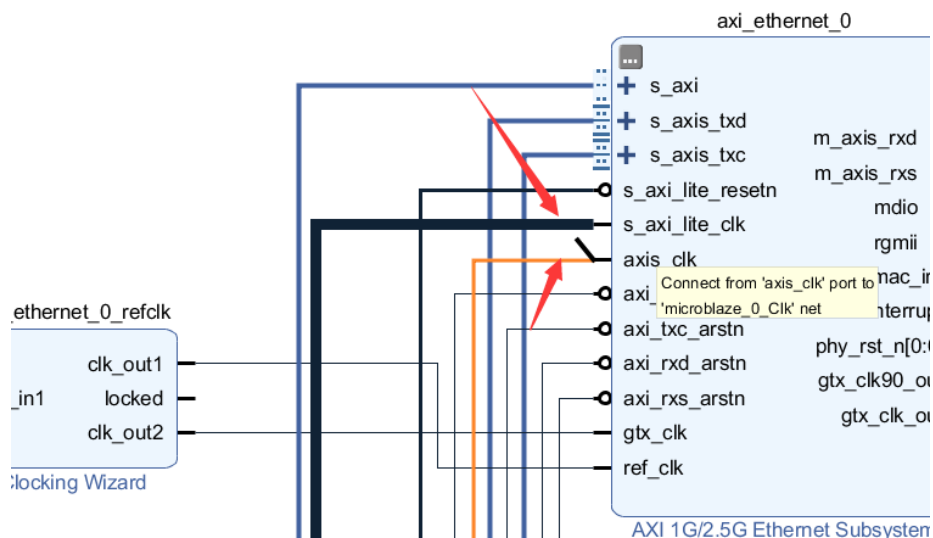
分别选择 Pin, 右键选择 "Disconnect Pin"



18) 然后单端连接。



19) 把 axis_clk 的时钟跟 s_axi_lite_clk 的时钟连接器



20) 确认 axi_ethernet_0_refclk 的输出时钟一个是 200M，一个是 125M。

Re-customize IP

Cloning Wizard (5.4)

Documentation IP Location

IP Symbol Resource

Show disabled ports

Component Name: axi_ethernet_0_refclk

Cloning Options Output Clocks MMCM Settings Summary

The phase is calculated relative to the active input clock.

Output Clock	Port Name	Output Freq (MHz)	Requested	Actual	Phase (degrees)	Requested	Actual
☒ clk_out1	clk_out1	200	⊗	200.000	0.000	⊗	0.000
☒ clk_out2	clk_out2	125	⊗	125.000	0.000	⊗	0.000
☐ clk_out3	clk_out3	100.000		N/A	0.000		N/A
☐ clk_out4	clk_out4	100.000		N/A	0.000		N/A
☐ clk_out5	clk_out5	100.000		N/A	0.000		N/A
☐ clk_out6	clk_out6	100.000		N/A	0.000		N/A
☐ clk_out7	clk_out7	100.000		N/A	0.000		N/A

☐ USE CLOCK SEQUENCING

Cloning Feedback

Output Clock	Sequence Number
clk_out1	1
clk_out2	1

Source Signaling

☒ Automatic Control On: If the path is complete off-chip feedback. sele

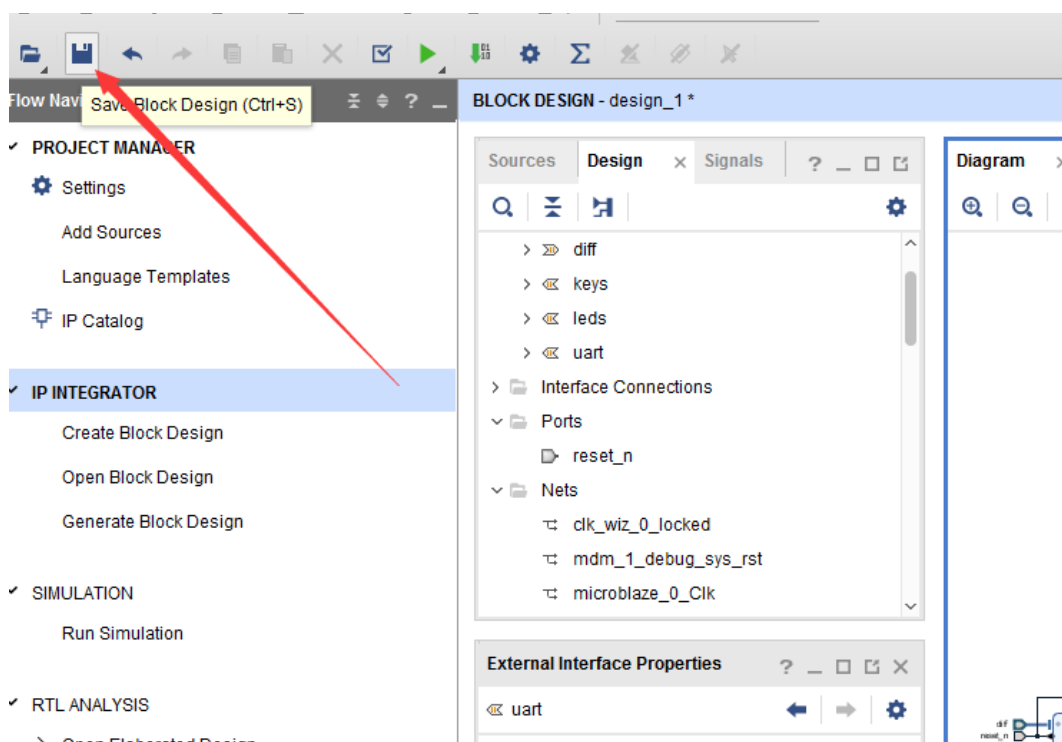
☐ Automatic Control Off: clkfb_in port for the use

21) 修改其他端口的名称

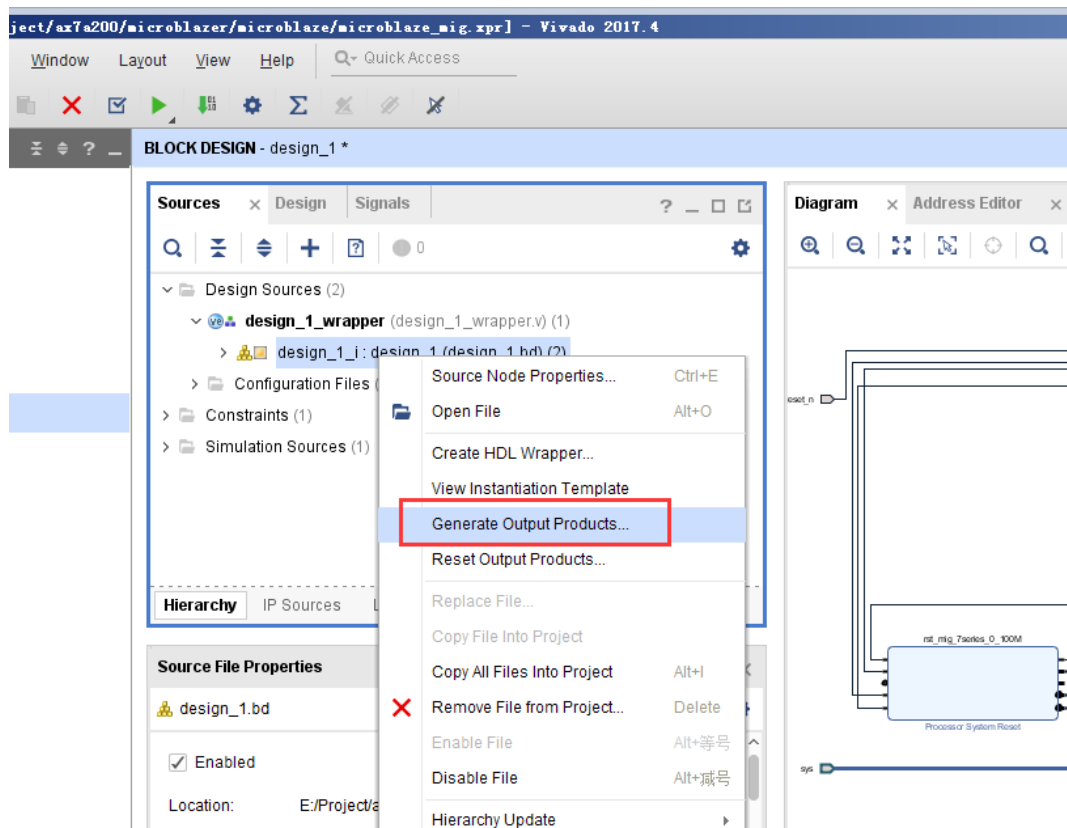


7.1.6 保存并检查错误

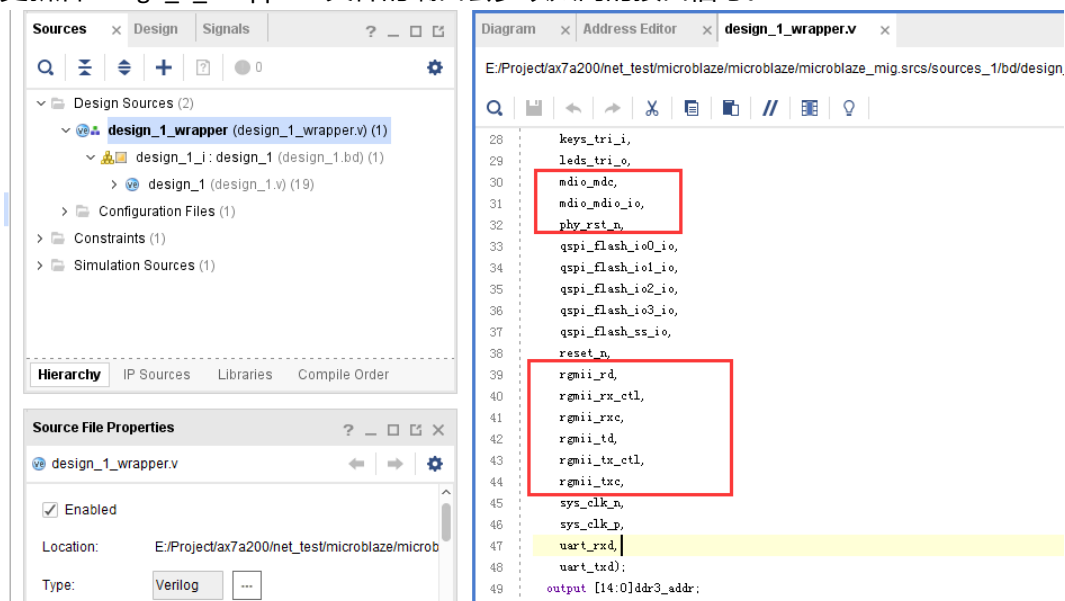
22) 保存设计，然后按“F6”检查设计



23) 选择“design_1.bd”文件，右键选择“Generate Output Products”，把IP参数和连接信息update到project中



更新后 design_1_wrapper.v 文件的端口会多以太网接口信号。



7.1.7 增加以太网约束文件（以 ax7a200 为例）

```
set_property PACKAGE_PIN N13 [get_ports {mdio_mdc      }}
set_property PACKAGE_PIN P14 [get_ports {mdio_mdio_io   }}
set_property PACKAGE_PIN R14 [get_ports {phy_rst_n      }}
set_property PACKAGE_PIN V18 [get_ports {rgmii_rxc      }}
set_property PACKAGE_PIN R19 [get_ports {rgmii_rx_ctl   }}
set_property PACKAGE_PIN P19 [get_ports {rgmii_rd[0]    }}
```

```

set_property PACKAGE_PIN U18 [get_ports {rgmii_rd[1]}    ]]
set_property PACKAGE_PIN U17 [get_ports {rgmii_rd[2]}    ]]
set_property PACKAGE_PIN P17 [get_ports {rgmii_rd[3]}    ]]
set_property PACKAGE_PIN P15 [get_ports {rgmii_txc}      ]]
set_property PACKAGE_PIN N17 [get_ports {rgmii_tx_ctl}   ]]
set_property PACKAGE_PIN N14 [get_ports {rgmii_td[0]}    ]]
set_property PACKAGE_PIN P16 [get_ports {rgmii_td[1]}    ]]
set_property PACKAGE_PIN R17 [get_ports {rgmii_td[2]}    ]]
set_property PACKAGE_PIN R16 [get_ports {rgmii_td[3]}    ]]

set_property IOSTANDARD LVCMOS33 [get_ports {mdio_mdc}    ]]
set_property IOSTANDARD LVCMOS33 [get_ports {mdio_mdio_io}]
set_property IOSTANDARD LVCMOS33 [get_ports {phy_rst_n}  ]]
set_property IOSTANDARD LVCMOS33 [get_ports {rgmii_rxc}  ]]
set_property IOSTANDARD LVCMOS33 [get_ports {rgmii_rx_ctl}]
set_property IOSTANDARD LVCMOS33 [get_ports {rgmii_rd[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {rgmii_rd[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {rgmii_rd[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {rgmii_rd[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {rgmii_txc}  ]]
set_property IOSTANDARD LVCMOS33 [get_ports {rgmii_tx_ctl}]
set_property IOSTANDARD LVCMOS33 [get_ports {rgmii_td[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {rgmii_td[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {rgmii_td[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {rgmii_td[3]}]

```

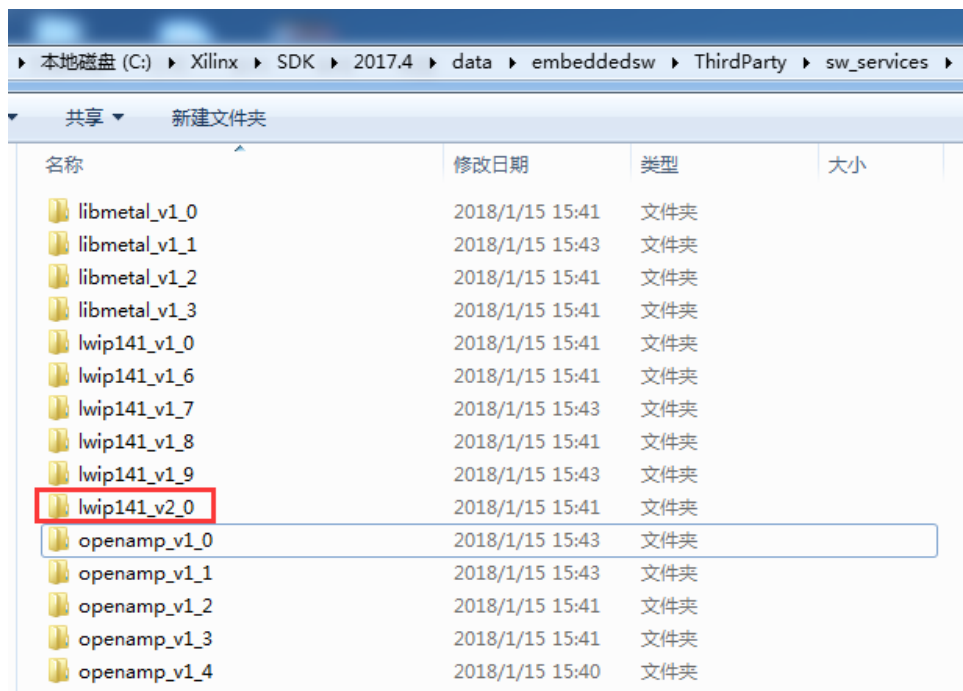
24) 编译生成 bit 文件，然后导出硬件信息，启动 SDK

7.2 SDK 程序

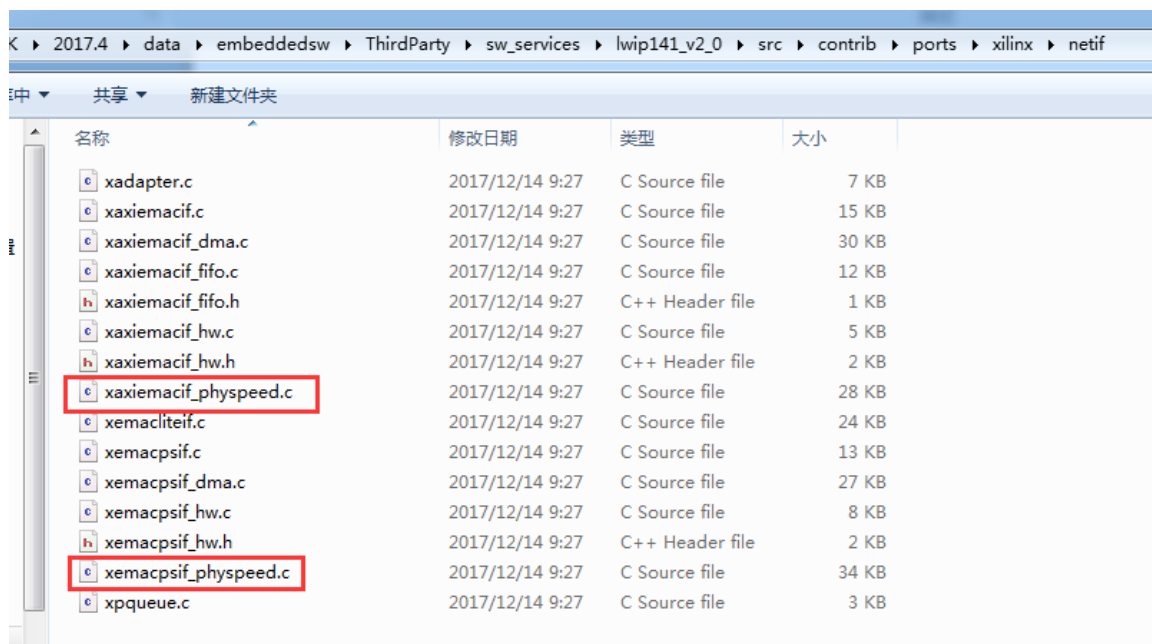
7.2.1 LWIP 库修改

由于自带的 LWIP 库只能识别部分 phy 芯片，如果开发板所用的 phy 芯片不在默认支持范围内，要修改库文件。也可以使用修改过的库替换原有的库。

1) 找到库文件目录 “X:\Xilinx\SDK\2017.4\data\embeddedsd\ThirdParty\sw_services”



- 2) 找到要修改的文件目录 “lwip141_v2_0\src\contrib\ports\xilinx\netif” 中文件 “xaxiemacif_physpeed.c” 和 “xemacpsif_physpeed.c” 要修改。



- 3) 修改 “xaxiemacif_physpeed.c” 文件，添加相关宏定义

```

#define IEEE_MMD_ACCESS_CTRL_DEVID_MASK      0x1F
#define IEEE_MMD_ACCESS_CTRL_PIDEVID_MASK    0x801F
#define IEEE_MMD_ACCESS_CTRL_NOPIDEVID_MASK  0x401F

#define PHY_R0_ISOLATE                        0x0400
#define PHY_DETECT_REG                       1
#define PHY_IDENTIFIER_1_REG                 2
#define PHY_IDENTIFIER_2_REG                 3
#define PHY_DETECT_MASK                      0x1808
#define PHY_MARVELL_IDENTIFIER              0x0141
#define PHY_TI_IDENTIFIER                    0x2000

/* Marvel PHY flags */
#define MARVEL_PHY_IDENTIFIER                0x141
#define MARVEL_PHY_MODEL_NUM_MASK           0x3F0
#define MARVEL_PHY_88E1111_MODEL             0xC0
#define MARVEL_PHY_88E1116R_MODEL           0x240
#define PHY_88E1111_RGMII_RX_CLOCK_DELAYED_MASK 0x0080

/* TI PHY Flags */
#define TI_PHY_DETECT_MASK                   0x796D
#define TI_PHY_IDENTIFIER                    0x2000
#define TI_PHY_DP83867_MODEL                 0xA231
#define DP83867_RGMII_CLOCK_DELAY_CTRL_MASK 0x0003
#define DP83867_RGMII_TX_CLOCK_DELAY_MASK   0x0030
#define DP83867_RGMII_RX_CLOCK_DELAY_MASK   0x0003

/* TI DP83867 PHY Registers */
#define DP83867_R32_RGMIICTL1                0x32
#define DP83867_R86_RGMIIIDCTL               0x86

#define MICREL_PHY_IDENTIFIER                0x22
#define MICREL_PHY_KSZ9031_MODEL             0x220

#define TI_PHY_REGCR                          0xD
#define TI_PHY_ADDDR                          0xE
#define TI_PHY_PHYCTR                          0x10

```

4) 添加 phy 速度获取函数

```

unsigned int get_phy_speed_ksz9031(XAxiEthernet *xaxiemacp, u32 phy_addr)
{
    u16 control;
    u16 status;
    u16 partner_capabilities;
    xil_printf("Start PHY autonegotiation \r\n");

    XAxiEthernet_PhyWrite(xaxiemacp, phy_addr, IEEE_PAGE_ADDRESS_REGISTER, 2);
    XAxiEthernet_PhyRead(xaxiemacp, phy_addr, IEEE_CONTROL_REG_MAC, &control);
    //control |= IEEE_RGMII_TXRX_CLOCK_DELAYED_MASK;
    control &= ~(0x10);
    XAxiEthernet_PhyWrite(xaxiemacp, phy_addr, IEEE_CONTROL_REG_MAC, control);

    XAxiEthernet_PhyWrite(xaxiemacp, phy_addr, IEEE_PAGE_ADDRESS_REGISTER, 0);

    XAxiEthernet_PhyRead(xaxiemacp, phy_addr, IEEE_AUTONEGO_ADVERTISE_REG, &control);
    control |= IEEE_ASYMMETRIC_PAUSE_MASK;
    control |= IEEE_PAUSE_MASK;
    control |= ADVERTISE_100;
    control |= ADVERTISE_10;
    XAxiEthernet_PhyWrite(xaxiemacp, phy_addr, IEEE_AUTONEGO_ADVERTISE_REG, control);

    XAxiEthernet_PhyRead(xaxiemacp, phy_addr, IEEE_1000_ADVERTISE_REG_OFFSET, &control);

    control |= ADVERTISE_1000;
    XAxiEthernet_PhyWrite(xaxiemacp, phy_addr, IEEE_1000_ADVERTISE_REG_OFFSET, control);

    XAxiEthernet_PhyWrite(xaxiemacp, phy_addr, IEEE_PAGE_ADDRESS_REGISTER, 0);
    XAxiEthernet_PhyRead(xaxiemacp, phy_addr, IEEE_COPPER_SPECIFIC_CONTROL_REG, &control);

    control |= (7 << 12); /* max number of gigabit attempts */
    control |= (1 << 11); /* enable downshift */
    XAxiEthernet_PhyWrite(xaxiemacp, phy_addr, IEEE_COPPER_SPECIFIC_CONTROL_REG, control);

    XAxiEthernet_PhyRead(xaxiemacp, phy_addr, IEEE_CONTROL_REG_OFFSET, &control);
    control |= IEEE_CTRL_AUTONEGOTIATE_ENABLE;
}

```

```

control |= IEEE_STAT_AUTONEGOTIATE_RESTART;

XAXiEthernet_PhyWrite(xaxiemacp, phy_addr, IEEE_CONTROL_REG_OFFSET, control);

XAXiEthernet_PhyRead(xaxiemacp, phy_addr, IEEE_CONTROL_REG_OFFSET, &control);
control |= IEEE_CTRL_RESET_MASK;
XAXiEthernet_PhyWrite(xaxiemacp, phy_addr, IEEE_CONTROL_REG_OFFSET, control);

while (1) {
    XAXiEthernet_PhyRead(xaxiemacp, phy_addr, IEEE_CONTROL_REG_OFFSET, &control);
    if (control & IEEE_CTRL_RESET_MASK)
        continue;
    else
        break;
}
xil_printf("Waiting for PHY to complete autonegotiation.\r\n");

XAXiEthernet_PhyRead(xaxiemacp, phy_addr, IEEE_STATUS_REG_OFFSET, &status);
while ( !(status & IEEE_STAT_AUTONEGOTIATE_COMPLETE) ) {
    sleep(1);
    XAXiEthernet_PhyRead(xaxiemacp, phy_addr, IEEE_STATUS_REG_OFFSET,
                          &status);
}

xil_printf("autonegotiation complete \r\n");

XAXiEthernet_PhyRead(xaxiemacp, phy_addr, 0x1f, &partner_capabilities);

if ( (partner_capabilities & 0x40) == 0x40) /* 1000Mbps */
    return 1000;
else if ( (partner_capabilities & 0x20) == 0x20) /* 100Mbps */
    return 100;
else if ( (partner_capabilities & 0x10) == 0x10) /* 10Mbps */
    return 10;
else
    return 0;
}

```

5) 修改函数 “get_IEEE_phy_speed”，添加对 KSZ9031 的支持。

```

unsigned get_IEEE_phy_speed(XAXiEthernet *xaxiemacp)
{
    u16 phy_identifier;
    u16 phy_model;
    u8 phytype;

#ifdef XPAR_AXIETHERNET_0_BASEADDR
    u32 phy_addr = detect_phy(xaxiemacp);

    /* Get the PHY Identifier and Model number */
    XAXiEthernet_PhyRead(xaxiemacp, phy_addr, PHY_IDENTIFIER_1_REG, &phy_identifier);
    XAXiEthernet_PhyRead(xaxiemacp, phy_addr, PHY_IDENTIFIER_2_REG, &phy_model);

    /* Depending upon what manufacturer PHY is connected, a different mask is
    * needed to determine the specific model number of the PHY. */
    if (phy_identifier == MARVEL_PHY_IDENTIFIER) {
        phy_model = phy_model & MARVEL_PHY_MODEL_NUM_MASK;

        if (phy_model == MARVEL_PHY_88E1116R_MODEL) {
            return get_phy_speed_88E1116R(xaxiemacp, phy_addr);
        } else if (phy_model == MARVEL_PHY_88E1111_MODEL) {
            return get_phy_speed_88E1111(xaxiemacp, phy_addr);
        }
    } else if (phy_identifier == TI_PHY_IDENTIFIER) {
        phy_model = phy_model & TI_PHY_DP83867_MODEL;
        phytype = XAXiEthernet_GetPhysicalInterface(xaxiemacp);

        if (phy_model == TI_PHY_DP83867_MODEL && phytype == XAE_PHY_TYPE_SGMII) {
            return get_phy_speed_TI_DP83867_SGMII(xaxiemacp, phy_addr);
        }

        if (phy_model == TI_PHY_DP83867_MODEL) {
            return get_phy_speed_TI_DP83867(xaxiemacp, phy_addr);
        }
    } else if (phy_identifier == MICREL_PHY_IDENTIFIER)
    {
        xil_printf("Phy %d is KSZ9031\n\r", phy_addr);
        get_phy_speed_ksz9031(xaxiemacp, phy_addr);
    }
    else {

```

```

        LWIP_DEBUGF(NETIF_DEBUG, ("XAXiEthernet get_IEEE_phy_speed: Detected PHY with unknown
        identifier/model.\r\n"));
    }
#endif
#ifdef PCM_PMA_CORE_PRESENT
    return get_phy_negotiated_speed(xaxiemacp, phy_addr);
#endif
}

```

6) 修改 "xemacpsif_physpeed.c" 文件添加宏定义

```

#define IEEE_PAGE_ADDRESS_REGISTER 22
#define IEEE_CTRL_1GBPS_LINKSPEED_MASK 0x2040
#define IEEE_CTRL_LINKSPEED_MASK 0x0040
#define IEEE_CTRL_LINKSPEED_1000M 0x0040
#define IEEE_CTRL_LINKSPEED_100M 0x2000
#define IEEE_CTRL_LINKSPEED_10M 0x0000
#define IEEE_CTRL_RESET_MASK 0x8000

#define IEEE_SPEED_MASK 0xC000
#define IEEE_SPEED_1000 0x8000
#define IEEE_SPEED_100 0x4000

#define IEEE_CTRL_RESET_MASK 0x8000
#define IEEE_CTRL_AUTONEGOTIATE_ENABLE 0x1000
#define IEEE_STAT_AUTONEGOTIATE_COMPLETE 0x0020
#define IEEE_STAT_AUTONEGOTIATE_RESTART 0x0200
#define IEEE_RGMII_TXRX_CLOCK_DELAYED_MASK 0x0030
#define IEEE_ASYMMETRIC_PAUSE_MASK 0x0800
#define IEEE_PAUSE_MASK 0x0400
#define IEEE_AUTONEG_ERROR_MASK 0x8000

#define PHY_DETECT_REG 1
#define PHY_IDENTIFIER_1_REG 2
#define PHY_IDENTIFIER_2_REG 3
#define PHY_DETECT_MASK 0x1808
#define PHY_MARVELL_IDENTIFIER 0x0141
#define PHY_TI_IDENTIFIER 0x2000
#define PHY_XILINX_PCS_PMA_ID1 0x0174
#define PHY_XILINX_PCS_PMA_ID2 0x0C00

#define MICREL_PHY_IDENTIFIER 0x22
#define MICREL_PHY_KSZ9031_MODEL 0x220

#define XEMACPS_GMII2RGMII_SPEED1000_FD 0x140
#define XEMACPS_GMII2RGMII_SPEED100_FD 0x2100
#define XEMACPS_GMII2RGMII_SPEED10_FD 0x100
#define XEMACPS_GMII2RGMII_REG_NUM 0x10

```

7) 添加 phy 速度获取函数

```

static u32_t get_phy_speed_ksz9031(XEmacPs *xemacsp, u32_t phy_addr)
{
    u16_t temp;
    u16_t control;
    u16_t status;
    u16_t status_speed;
    u32_t timeout_counter = 0;
    u32_t temp_speed;
    u32_t phyregtemp;

    xil_printf("Start PHY autonegotiation \r\n");

    XEmacPs_PhyWrite(xemacsp, phy_addr, IEEE_PAGE_ADDRESS_REGISTER, 2);
    XEmacPs_PhyRead(xemacsp, phy_addr, IEEE_CONTROL_REG_MAC, &control);
    control |= IEEE_RGMII_TXRX_CLOCK_DELAYED_MASK;
    XEmacPs_PhyWrite(xemacsp, phy_addr, IEEE_CONTROL_REG_MAC, control);

    XEmacPs_PhyWrite(xemacsp, phy_addr, IEEE_PAGE_ADDRESS_REGISTER, 0);

    XEmacPs_PhyRead(xemacsp, phy_addr, IEEE_AUTONEGO_ADVERTISE_REG, &control);
    control |= IEEE_ASYMMETRIC_PAUSE_MASK;
    control |= IEEE_PAUSE_MASK;
    control |= ADVERTISE_100;
    control |= ADVERTISE_10;
    XEmacPs_PhyWrite(xemacsp, phy_addr, IEEE_AUTONEGO_ADVERTISE_REG, control);
}

```

```

XEmacPs_PhyRead(xemacpsp, phy_addr, IEEE_1000_ADVERTISE_REG_OFFSET,
                &control);
control |= ADVERTISE_1000;
XEmacPs_PhyWrite(xemacpsp, phy_addr, IEEE_1000_ADVERTISE_REG_OFFSET,
                 control);

XEmacPs_PhyWrite(xemacpsp, phy_addr, IEEE_PAGE_ADDRESS_REGISTER, 0);
XEmacPs_PhyRead(xemacpsp, phy_addr, IEEE_COPPER_SPECIFIC_CONTROL_REG,
                &control);

control |= (7 << 12); /* max number of gigabit attempts */
control |= (1 << 11); /* enable downshift */
XEmacPs_PhyWrite(xemacpsp, phy_addr, IEEE_COPPER_SPECIFIC_CONTROL_REG,
                 control);

XEmacPs_PhyRead(xemacpsp, phy_addr, IEEE_CONTROL_REG_OFFSET, &control);
control |= IEEE_CTRL_AUTONEGOTIATE_ENABLE;
control |= IEEE_STAT_AUTONEGOTIATE_RESTART;
XEmacPs_PhyWrite(xemacpsp, phy_addr, IEEE_CONTROL_REG_OFFSET, control);

XEmacPs_PhyRead(xemacpsp, phy_addr, IEEE_CONTROL_REG_OFFSET, &control);
control |= IEEE_CTRL_RESET_MASK;
XEmacPs_PhyWrite(xemacpsp, phy_addr, IEEE_CONTROL_REG_OFFSET, control);

while (1) {
    XEmacPs_PhyRead(xemacpsp, phy_addr, IEEE_CONTROL_REG_OFFSET, &control);
    if (control & IEEE_CTRL_RESET_MASK)
        continue;
    else
        break;
}

XEmacPs_PhyRead(xemacpsp, phy_addr, IEEE_STATUS_REG_OFFSET, &status);

xil_printf("Waiting for PHY to complete autonegotiation.\r\n");

while ( !(status & IEEE_STAT_AUTONEGOTIATE_COMPLETE) ) {
    sleep(1);
    XEmacPs_PhyRead(xemacpsp, phy_addr,
                    IEEE_COPPER_SPECIFIC_STATUS_REG_2, &temp);
    timeout_counter++;

    if (timeout_counter == 30) {
        xil_printf("Auto negotiation error \r\n");
        return;
    }
    XEmacPs_PhyRead(xemacpsp, phy_addr, IEEE_STATUS_REG_OFFSET, &status);
}
xil_printf("autonegotiation complete \r\n");

XEmacPs_PhyRead(xemacpsp, phy_addr, 0x1f,
                &status_speed);

if ( (status_speed & 0x40) == 0x40) /* 1000Mbps */
    return 1000;
else if ( (status_speed & 0x20) == 0x20) /* 100Mbps */
    return 100;
else if ( (status_speed & 0x10) == 0x10) /* 10Mbps */
    return 10;
else
    return 0;
return XST_SUCCESS;
}

```

8) 修改函数 “get_IEEE_phy_speed”，添加对 KSZ9031 的支持

```

static u32_t get_IEEE_phy_speed(XEmacPs *xemacpsp, u32_t phy_addr)
{
    u16_t phy_identity;
    u32_t RetStatus;

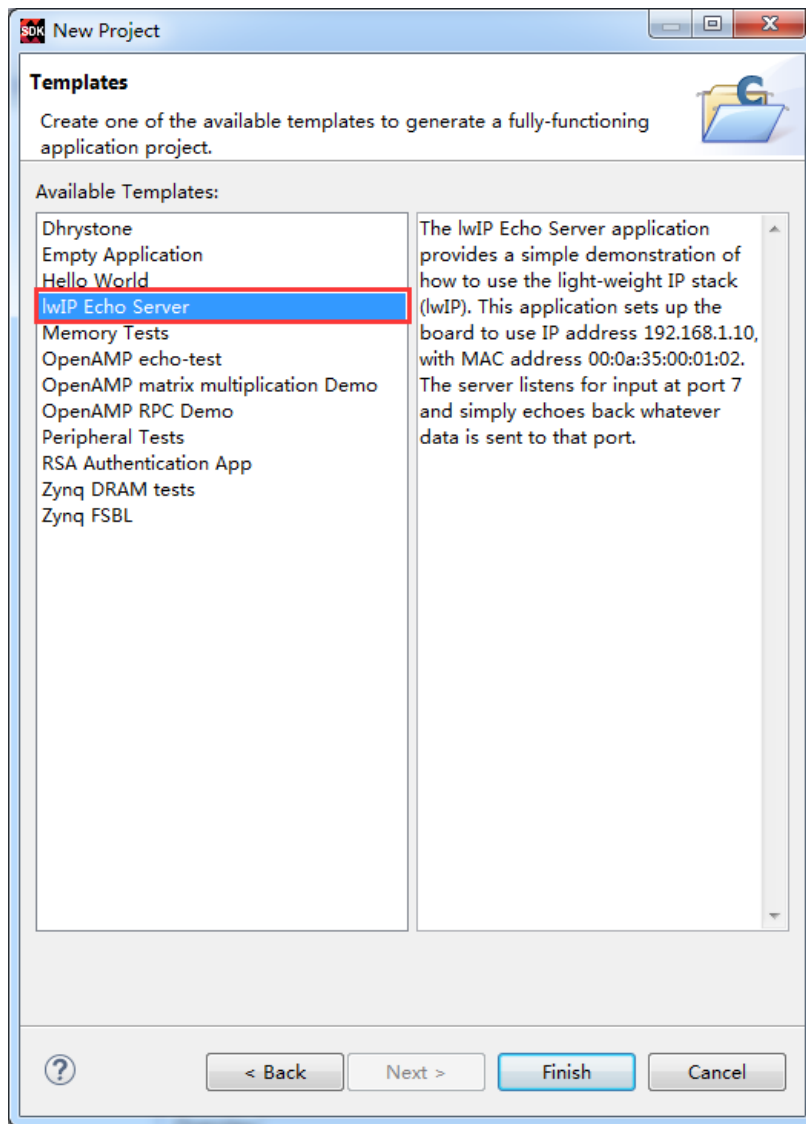
    XEmacPs_PhyRead(xemacpsp, phy_addr, PHY_IDENTIFIER_1_REG,
                    &phy_identity);

    if(phy_identity == MICREL_PHY_IDENTIFIER)
    {
        RetStatus = get_phy_speed_ksz9031(xemacpsp, phy_addr);
    }
    else if (phy_identity == PHY_TI_IDENTIFIER) {
        RetStatus = get_TI_phy_speed(xemacpsp, phy_addr);
    } else {
        RetStatus = get_Marvell_phy_speed(xemacpsp, phy_addr);
    }
}

```

```
    return RetStatus;  
}
```

7.2.2 SDK 下创建基于 LWIP 模板的 APP

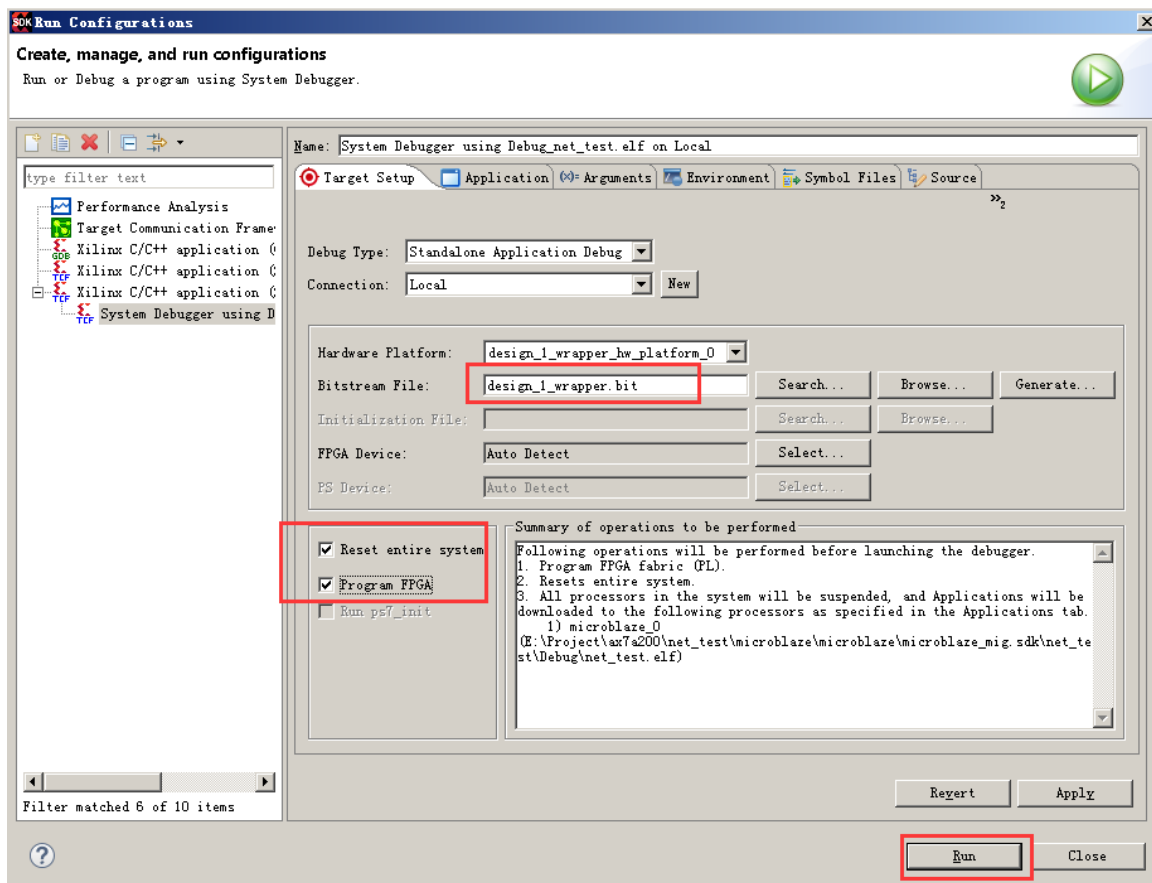


7.3 下载调试

测试环境要求有一台支持 dhcp 的路由器，开发板通过网线连接路由器可以自动获取 IP 地址，实验主机和开发板在一个网络，可以相互通信。

7.3.1 以太网测试

- 1) 连接串口打开串口调试终端，连接好以太网网线到路由器（ETH2）
- 2) 运行 SDK



- 3) 可以看到串口打印出一些信息，可以看到自动获取到地址为 “192.168.1.100”，连接速度 1000Mbps，tcp 端口为 7

```

-----luIP TCP echo server -----
TCP packets sent to port 6001 will be echoed back
WARNING: Not a Marvell or TI Ethernet PHY. Please verify the initialization sequence
Phy 1 is KSZ9031
Start PHY autonegotiation
Waiting for PHY to complete autonegotiation.
autonegotiation complete
auto-negotiated link speed: 1000
Board IP: 192.168.1.100
Netmask : 255.255.255.0
Gateway : 192.168.1.1
TCP echo server started @ port 7

```

- 4) 这时电脑也连接在同一个路由器的话，就可以 ping 这个开发板的 IP 地址

```
C:\管理员: C:\Windows\system32\cmd.exe
Microsoft Windows [版本 6.1.7601]
版权所有 (c) 2009 Microsoft Corporation。保留所有权利。

C:\Users\Administrator>ping 192.168.1.100

正在 Ping 192.168.1.100 具有 32 字节的数据:
来自 192.168.1.100 的回复: 字节=32 时间<1ms TTL=255
来自 192.168.1.100 的回复: 字节=32 时间<1ms TTL=255
来自 192.168.1.100 的回复: 字节=32 时间<1ms TTL=255
来自 192.168.1.100 的回复: 字节=32 时间<1ms TTL=255

192.168.1.100 的 Ping 统计信息:
    数据包: 已发送 = 4, 已接收 = 4, 丢失 = 0 (0% 丢失),
往返行程的估计时间(以毫秒为单位):
    最短 = 0ms, 最长 = 0ms, 平均 = 0ms

C:\Users\Administrator>
```

第八章 自定义 IP 实验

实验 Vivado 工程为 “custom_pwm_ip”。

Xilinx 官方为大家提供了很多 IP 核，在 Vivado 的 IP Catalog 中可以查看这些 IP 核，用户在构建自己的系统中，不可能只使用 Xilinx 官方的免费 IP 核，很多时候需要创建属于自己的用户 IP 核，创建自己的 IP 核有很多好处，例如系统设计定制化，设计复用，可以在 IP 核中加入 license，有偿提供给别人使用；简化系统设计和缩短设计时间。用 Microblaze 系统设计 IP 核，最常用的就是使用 AXI 总线将 Microblaze 同 PL 部分的 IP 核连接起来。本实验将为大家介绍如何在 Vivado 中构建 AXI 总线类型的 IP 核，此 IP 核用来产生一个 PWM，用这个控制开发板上的 LED，做一个呼吸灯的效果。

8.1 PWM 介绍

我们经常使用 PWM 来控制 LED，蜂鸣器等，通过调节脉冲的占空比来调节 LED 的亮度。在其他开发板中我们使用过的一个 pwm 模块如下：

```
//=====
//
//
// Author: meisq
// msq@qq.com
// ALINX(shanghai) Technology Co.,Ltd
// heijin
// WEB: http://www.alinx.cn/
// BBS: http://www.heijin.org/
//
//=====
//
// Copyright (c) 2017,ALINX(shanghai) Technology Co.,Ltd
// All rights reserved
//
// This source file may be used and distributed without restriction provided
// that this copyright statement is not removed from the file and that any
// derivative work contains the original copyright notice and the associated
// disclaimer.
//
//=====

//=====
// Description: pwm model
// pwm out period = frequency(pwm_out) * (2 ** N) / frequency(clk);
//
//=====
// Revision History:
// Date By Revision Change Description
//-----
// 2017/5/3 meisq 1.0 Original
//*****/
`timescale 1ns / 1ps
module ax_pwm
#(
    parameter N = 32 //pwm bit width
)
(
    input clk,
    input rst,
    input[N - 1:0]period,
    input[N - 1:0]duty,
    output pwm_out
);

reg[N - 1:0] period_r;
reg[N - 1:0] duty_r;
reg[N - 1:0] period_cnt;
```

```
reg pwm_r;  
assign pwm_out = pwm_r;  
always@(posedge clk or posedge rst)  
begin  
    if(rst==1)  
    begin  
        period_r <= { N {1'b0} };  
        duty_r <= { N {1'b0} };  
    end  
    else  
    begin  
        period_r <= period;  
        duty_r <= duty;  
    end  
end  
  
always@(posedge clk or posedge rst)  
begin  
    if(rst==1)  
        period_cnt <= { N {1'b0} };  
    else  
        period_cnt <= period_cnt + period_r;  
    end  
  
always@(posedge clk or posedge rst)  
begin  
    if(rst==1)  
    begin  
        pwm_r <= 1'b0;  
    end  
    else  
    begin  
        if(period_cnt >= duty_r)  
            pwm_r <= 1'b1;  
        else  
            pwm_r <= 1'b0;  
        end  
    end  
end  
endmodule
```

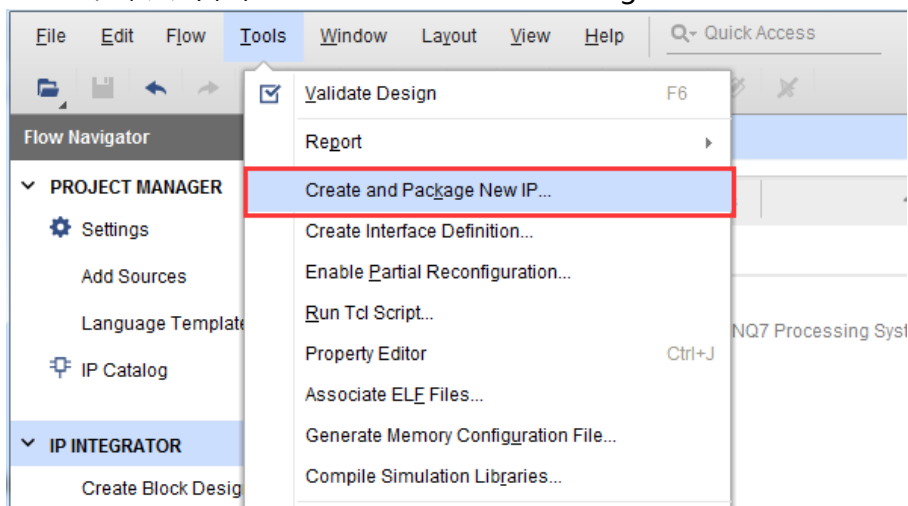
可以看到这个 PWM 模块需要 2 个参数 “period”、“duty” 来控制频率和占空比，我们需要设计一些寄存器来控制这些参数，这里需要使用 AXI 总线，PS 通过 AXI 总线来读写寄存器。

8.2 Vivado 工程建立

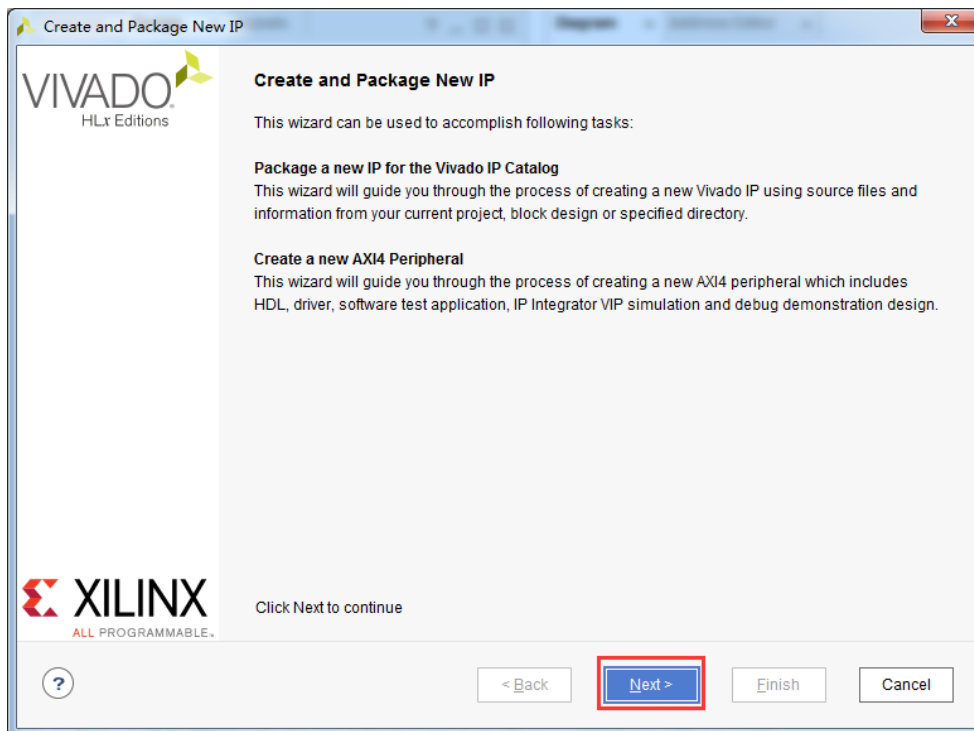
这里为了避免重复劳动，我们继续在第七章的以太网 Vivado 工程的基础上进行试验。只是删除 SDK 目录。

8.2.1 创建自定义 IP

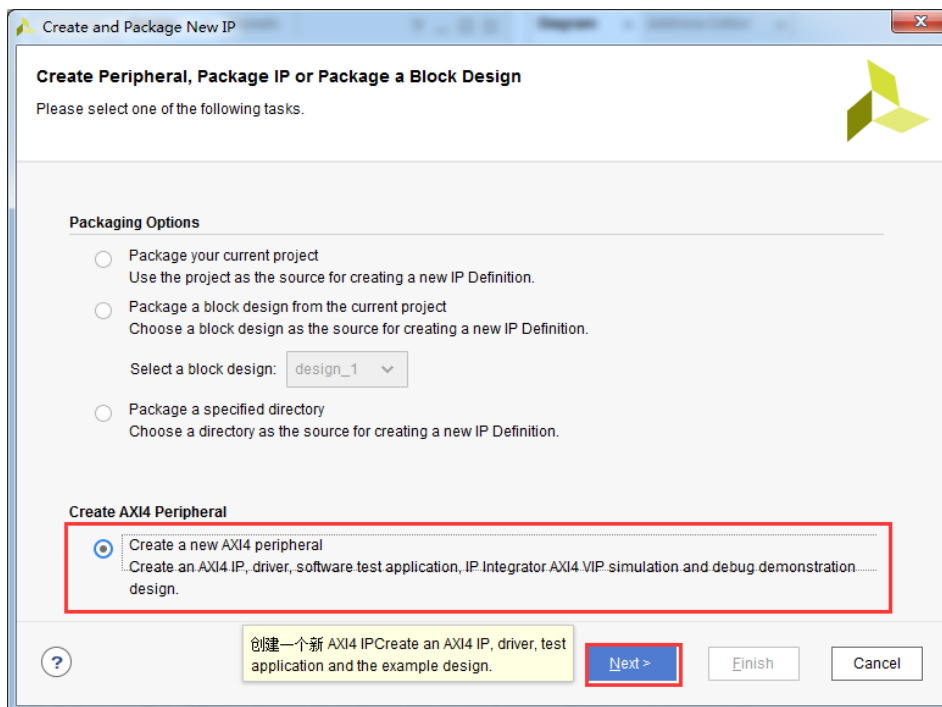
- 1) 打开 vivado 工程，点击菜单 “Tools->Create and Package IP...”



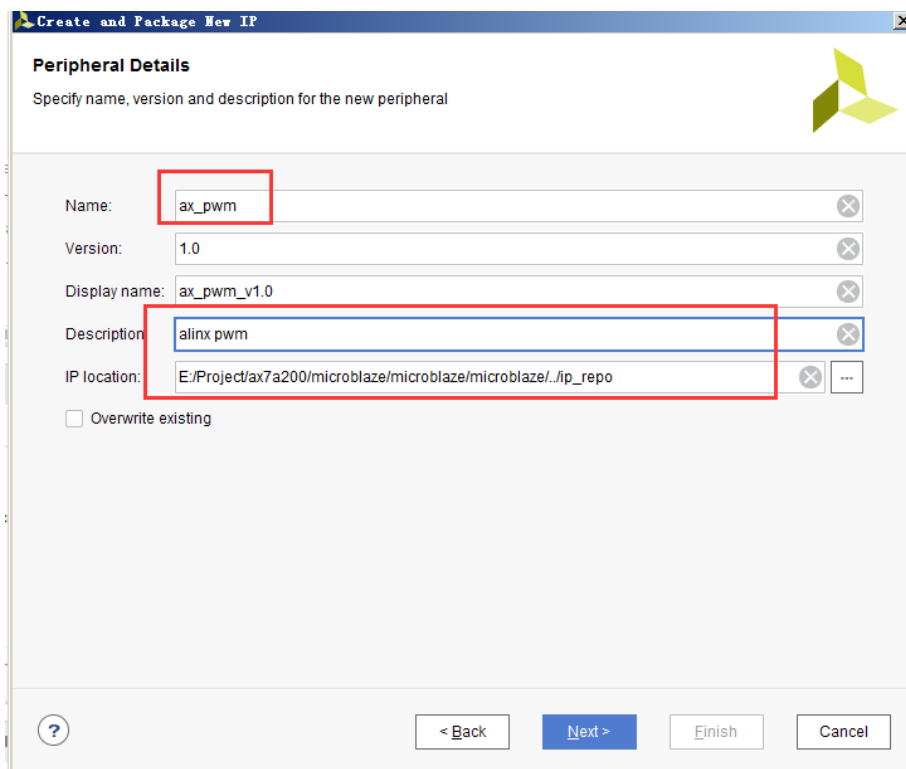
2) 选择 “Next”



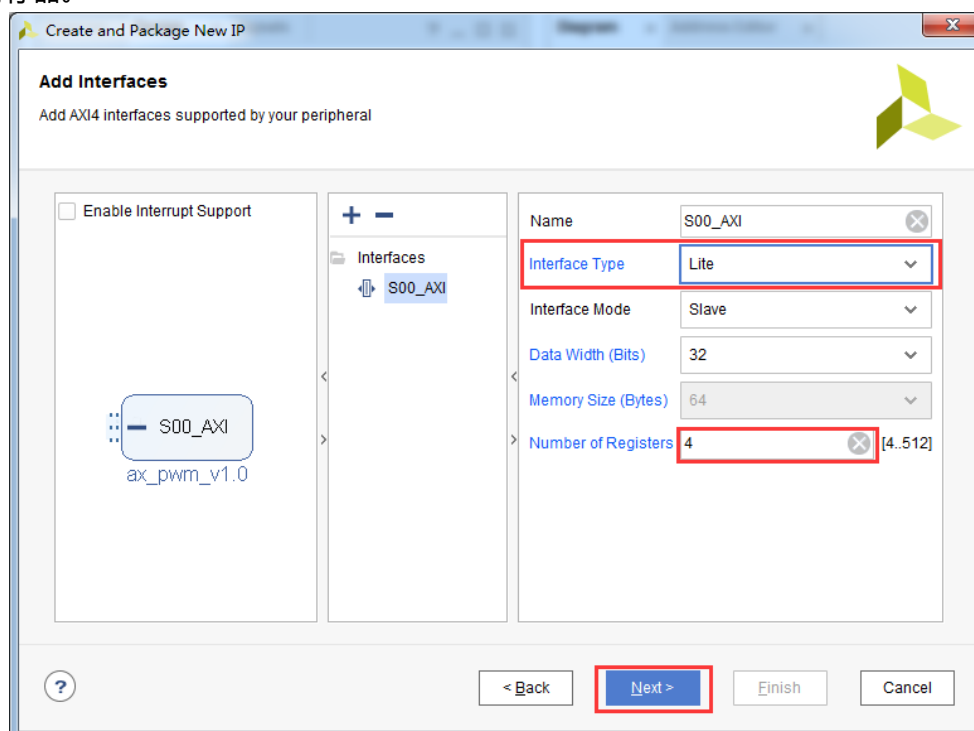
3) 选择创建一个新的 AXI4 设备



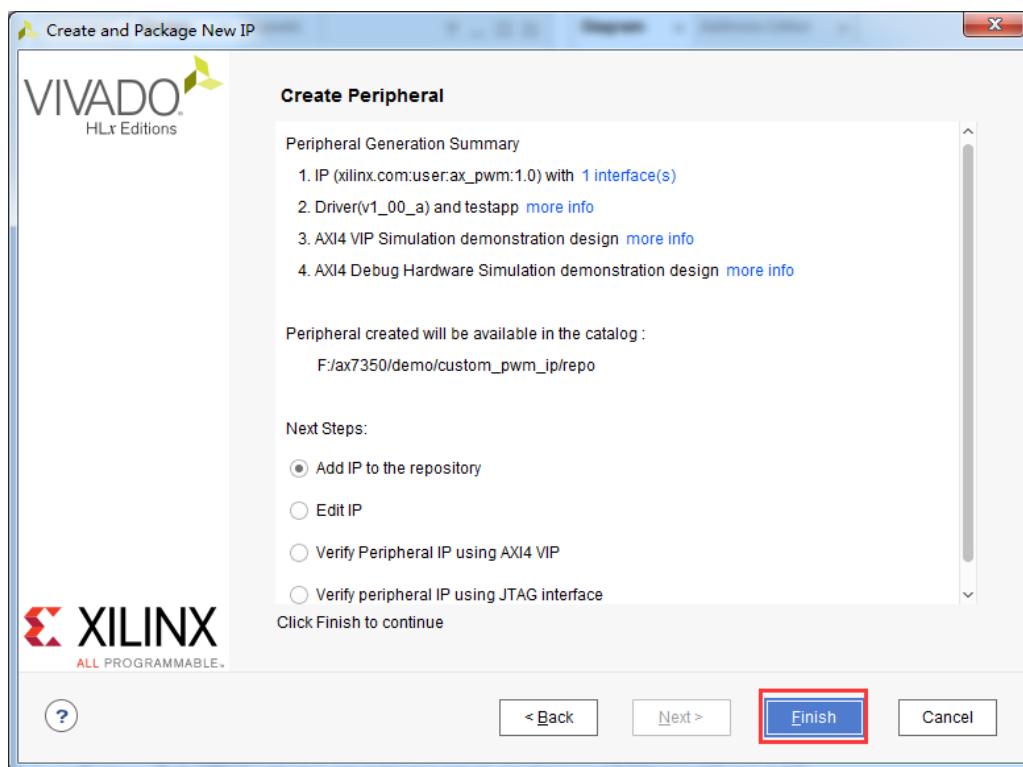
4) 名称填写 “ax_pwm”, 描述填写 “alinx pwm”, 然后选择一个合适的位置用来放 IP



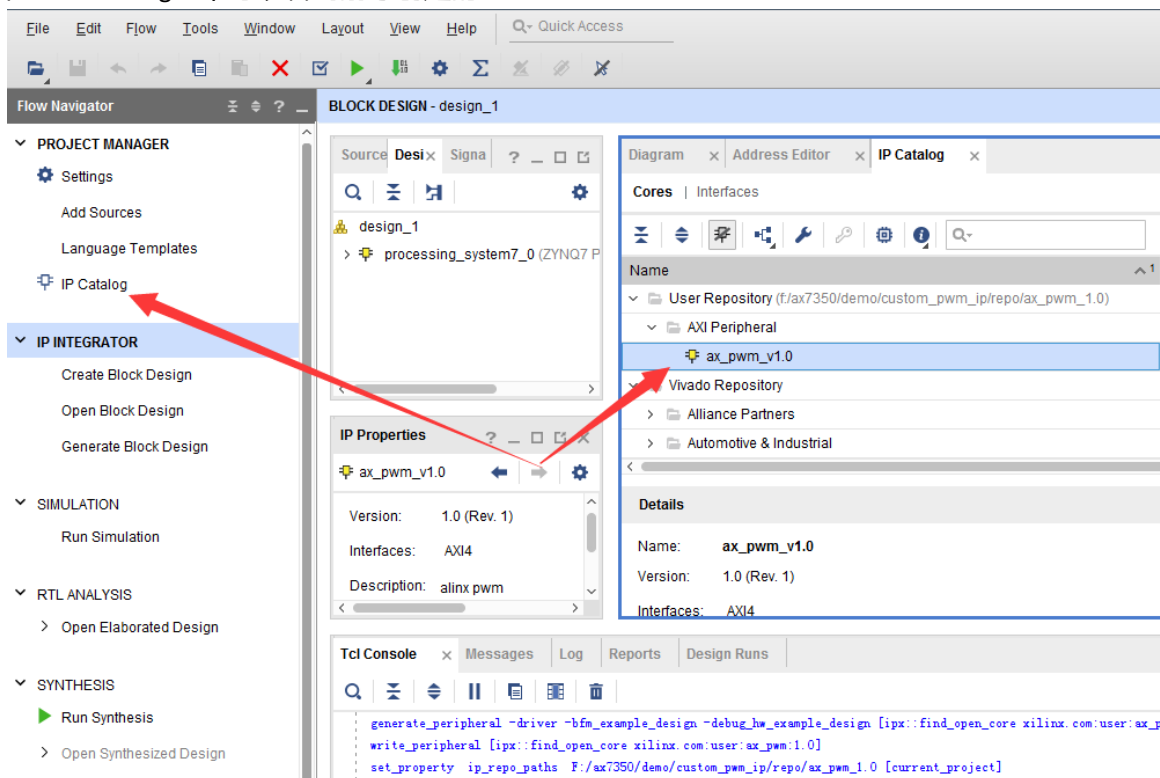
- 5) 下面参数可以指定接口类型、寄存器数量等，这里不需要修改，使用 AXI Lite Slave 接口，4 个寄存器。



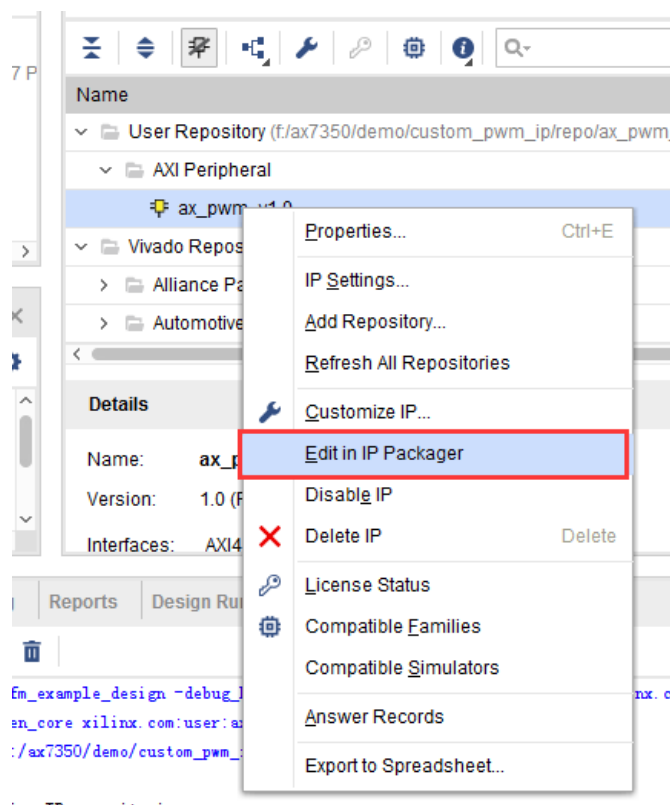
- 6) 点击 “Finish” 完成 IP 的创建



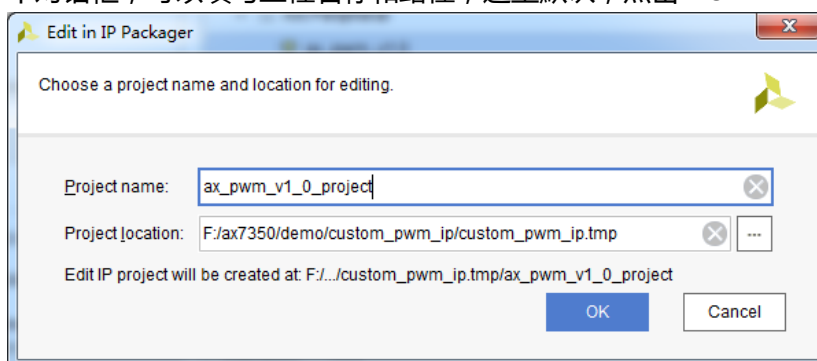
7) 在 “IP Catalog” 中可以看到刚才创建的 IP



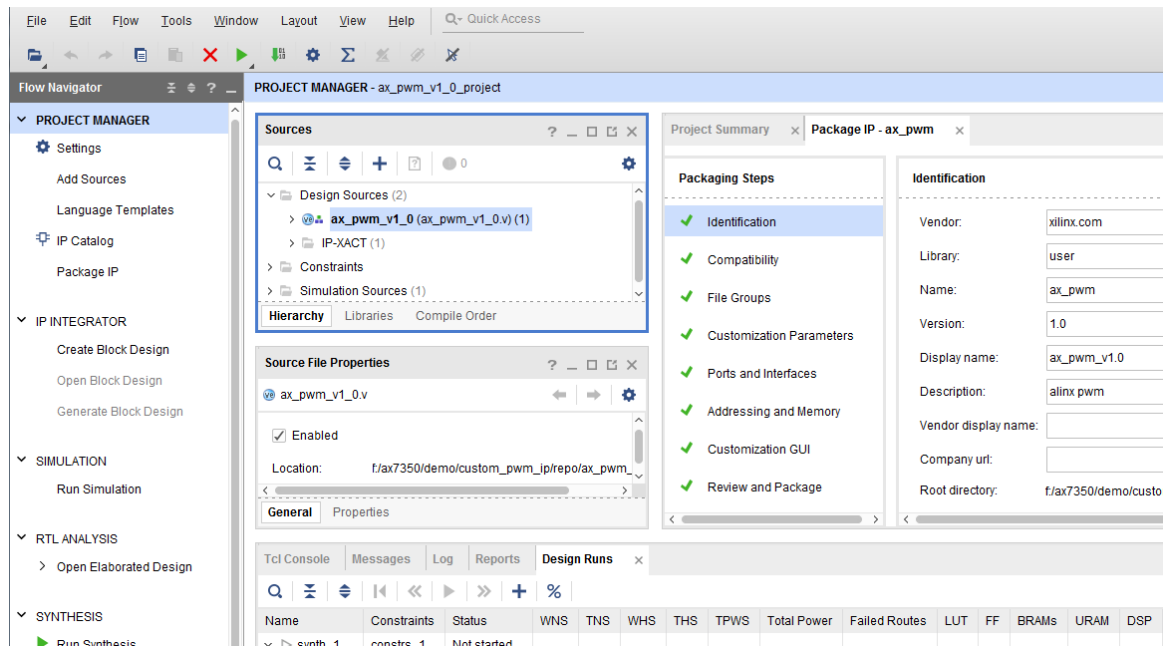
8) 这个时候的 IP 只有简单的寄存器读写功能，我们需要修改 IP，选择 IP，右键 “Edit in IP Packager”



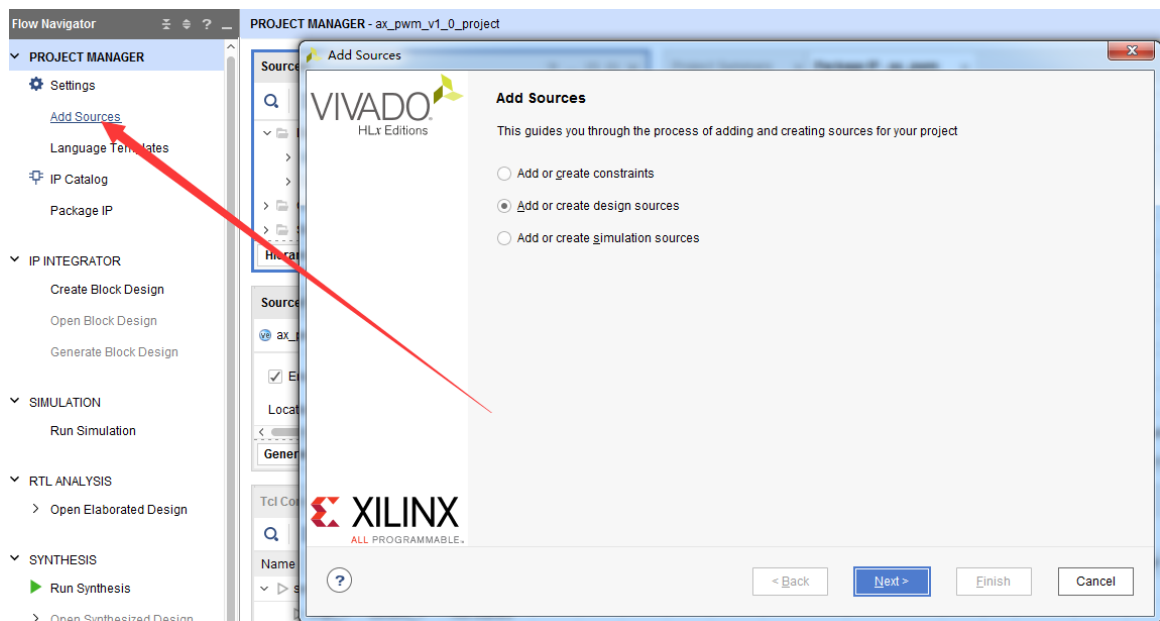
- 9) 这是弹出一个对话框，可以填写工程名称和路径，这里默认，点击“OK”



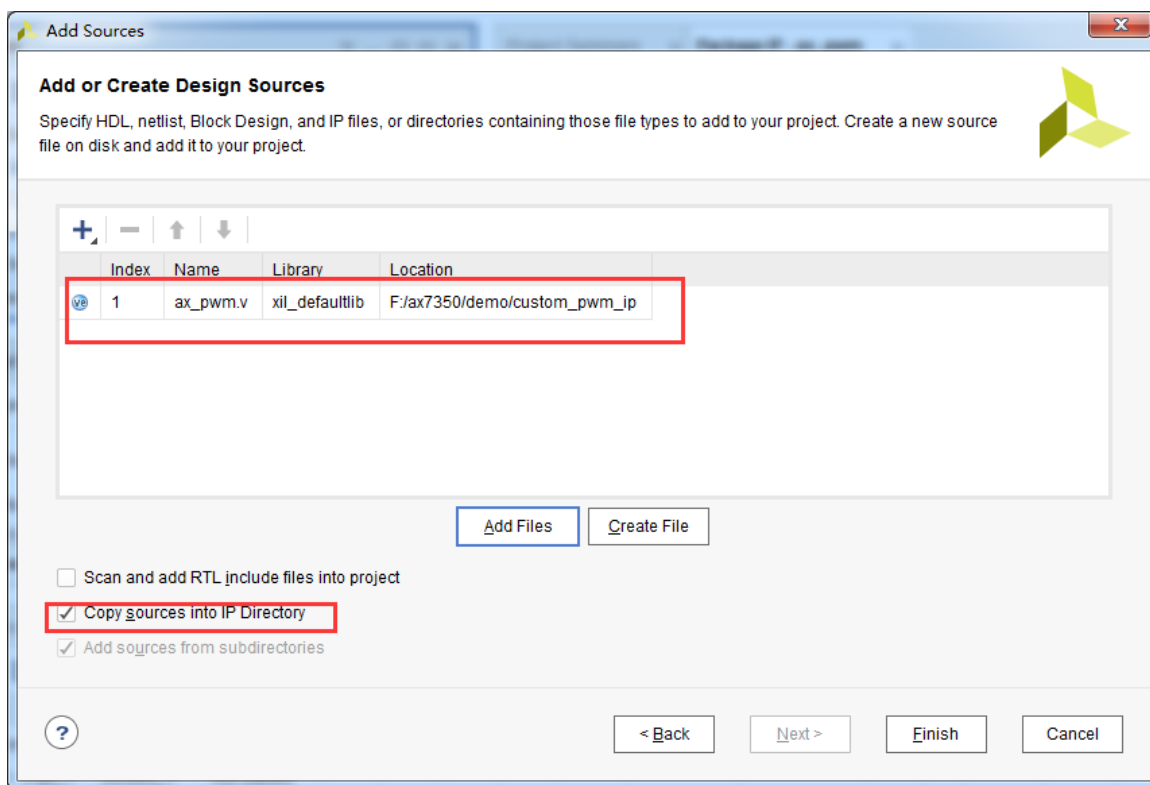
- 10) Vivado 打开了一个新的工程



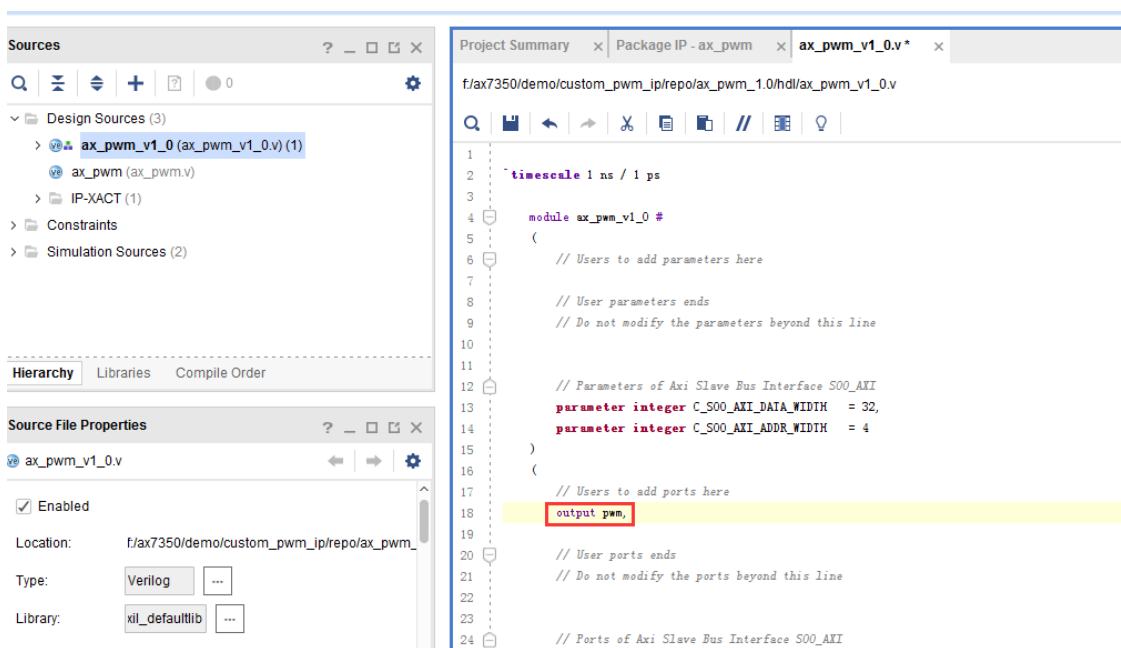
11) 添加 PWM 功能的核心代码



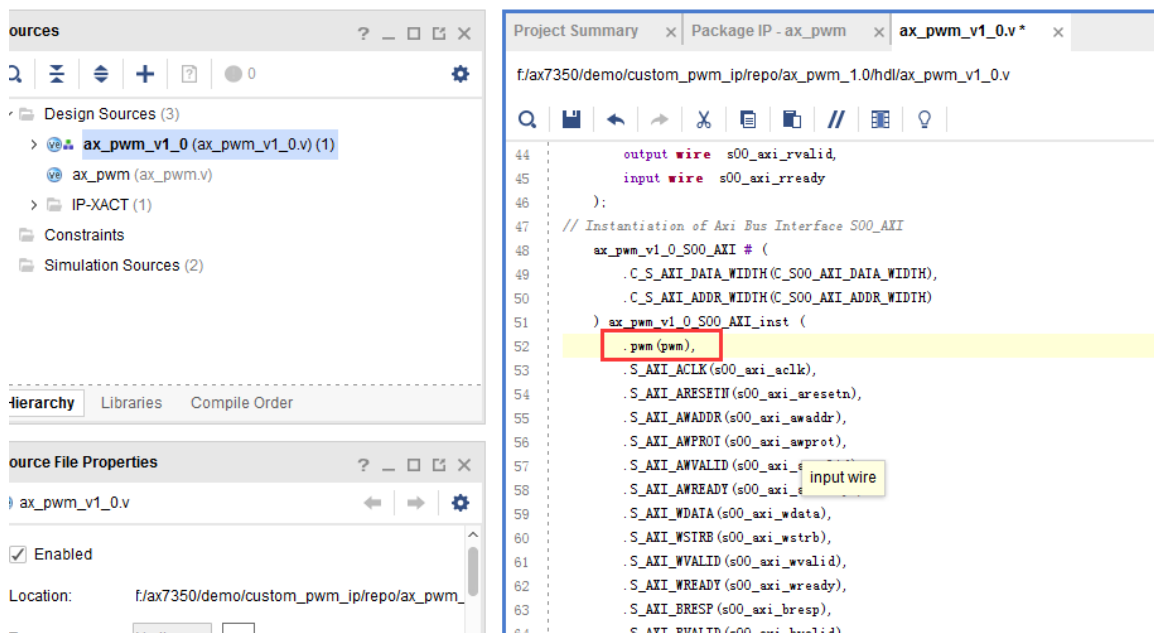
12) 添加代码时选择复制代码到 IP 目录



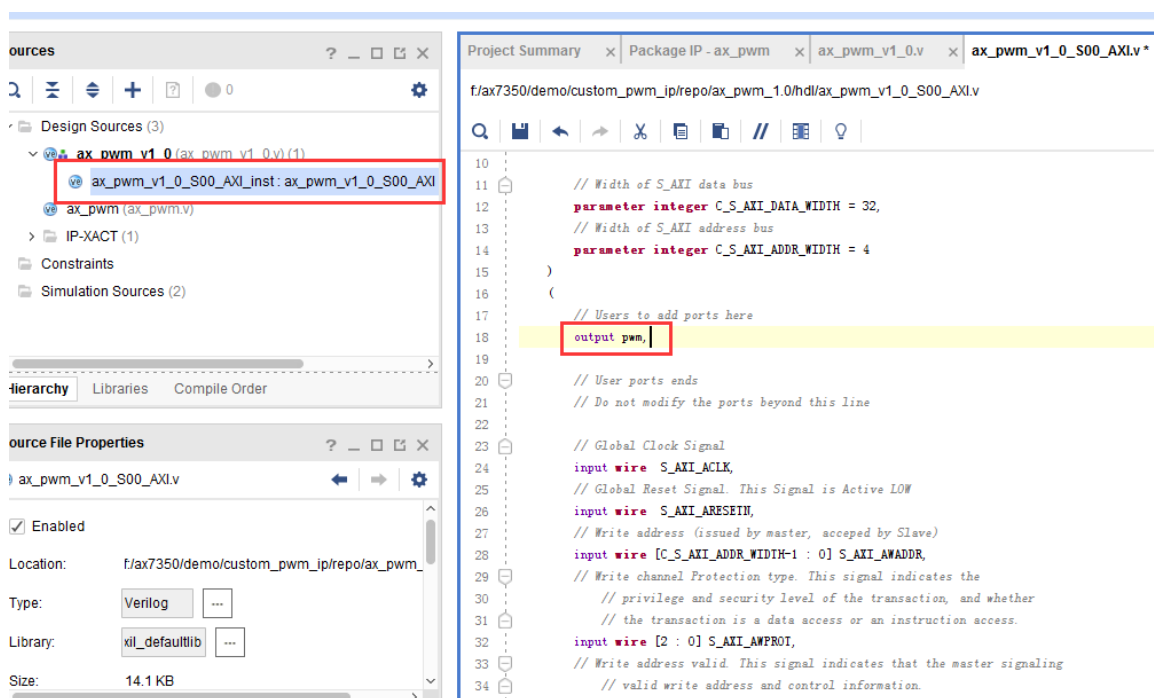
13) 修改 “ax_pwm_v1_0.v”, 添加一个 pwm 输出端口



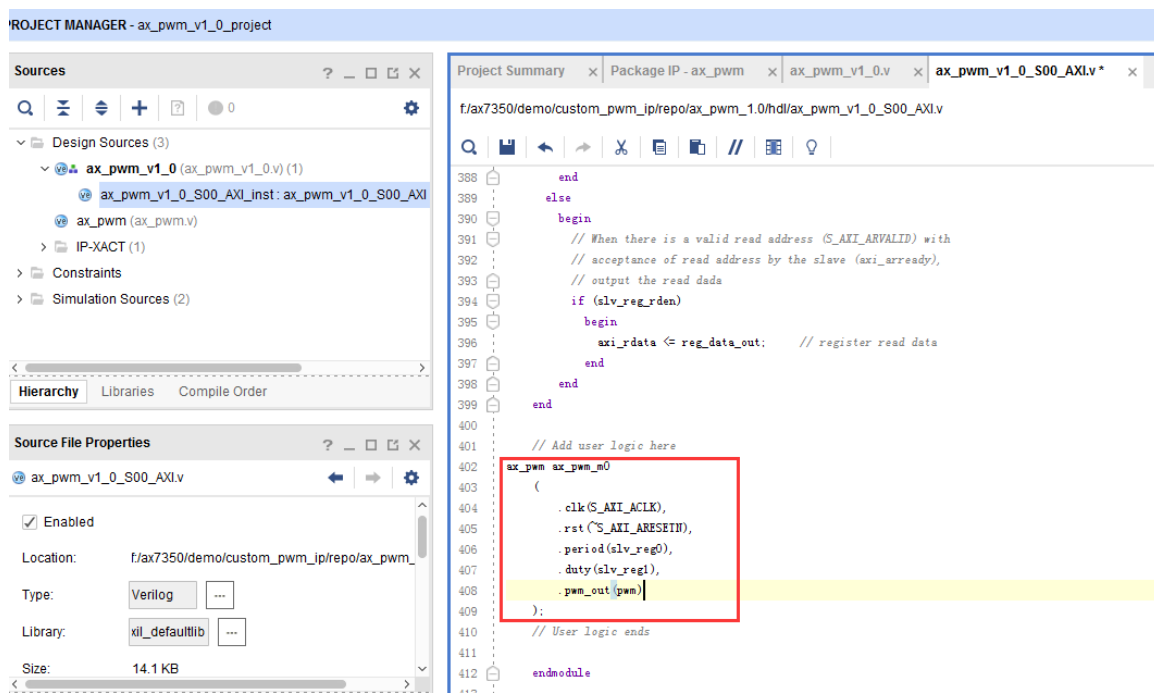
14) 修改 “ax_pwm_v1_0.v”, 在例化 “ax_pwm_v1_0_S00_AXI” ,中添加 pwm 端口的例化



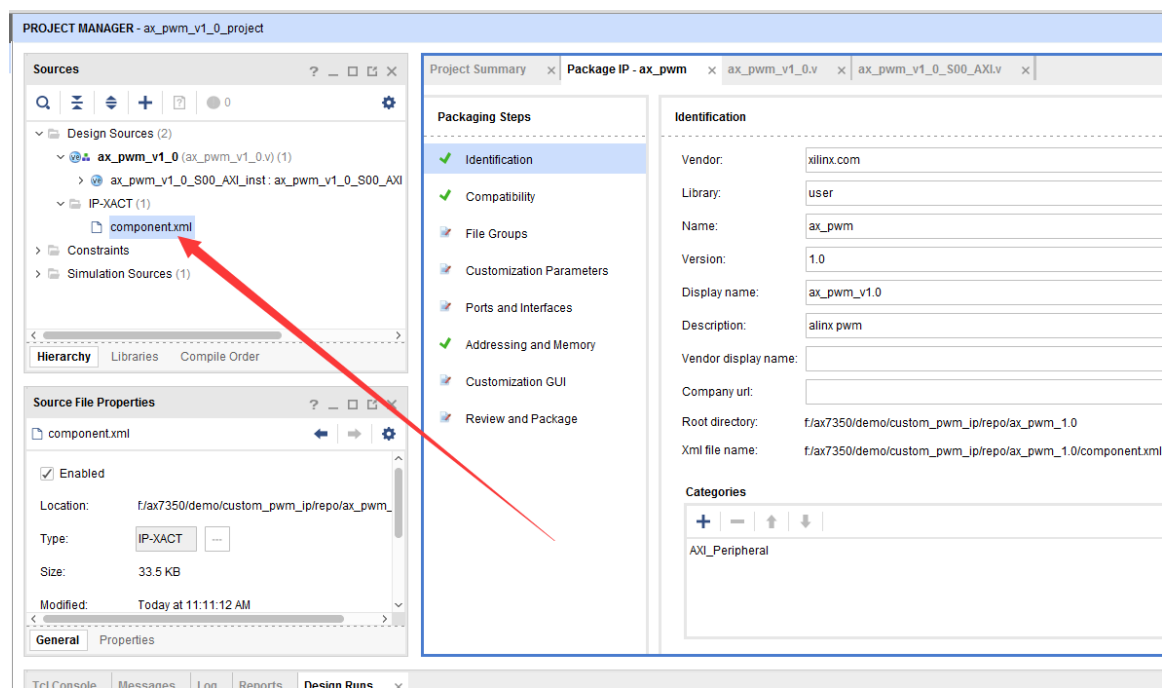
- 15) 修改 “ax_pwm_v1_0_s00_AXI.v” 文件，添加 pwm 端口，这个文件是实现 AXI4 Lite Slave 的核心代码



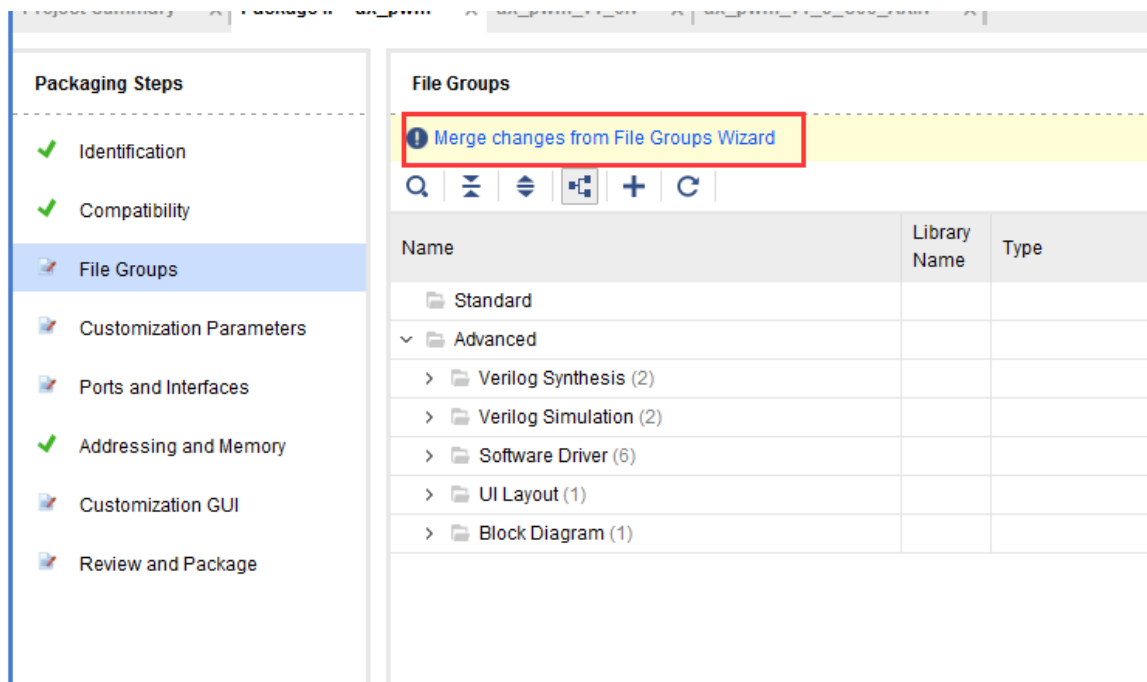
- 16) 修改“ax_pwm_v1_0_s00_AXI.v”文件，例化 pwm 核心功能代码，将寄存器 slv_reg0 和 slv_reg1 用于 pwm 模块的参数控制。



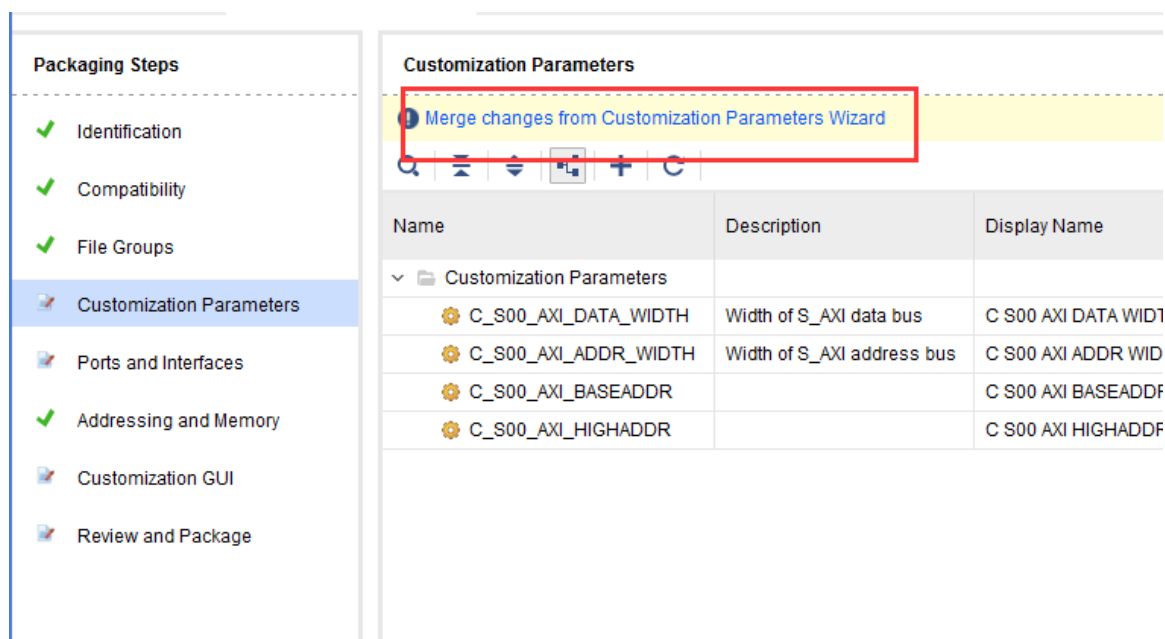
17) 双击 “component.xml” 文件



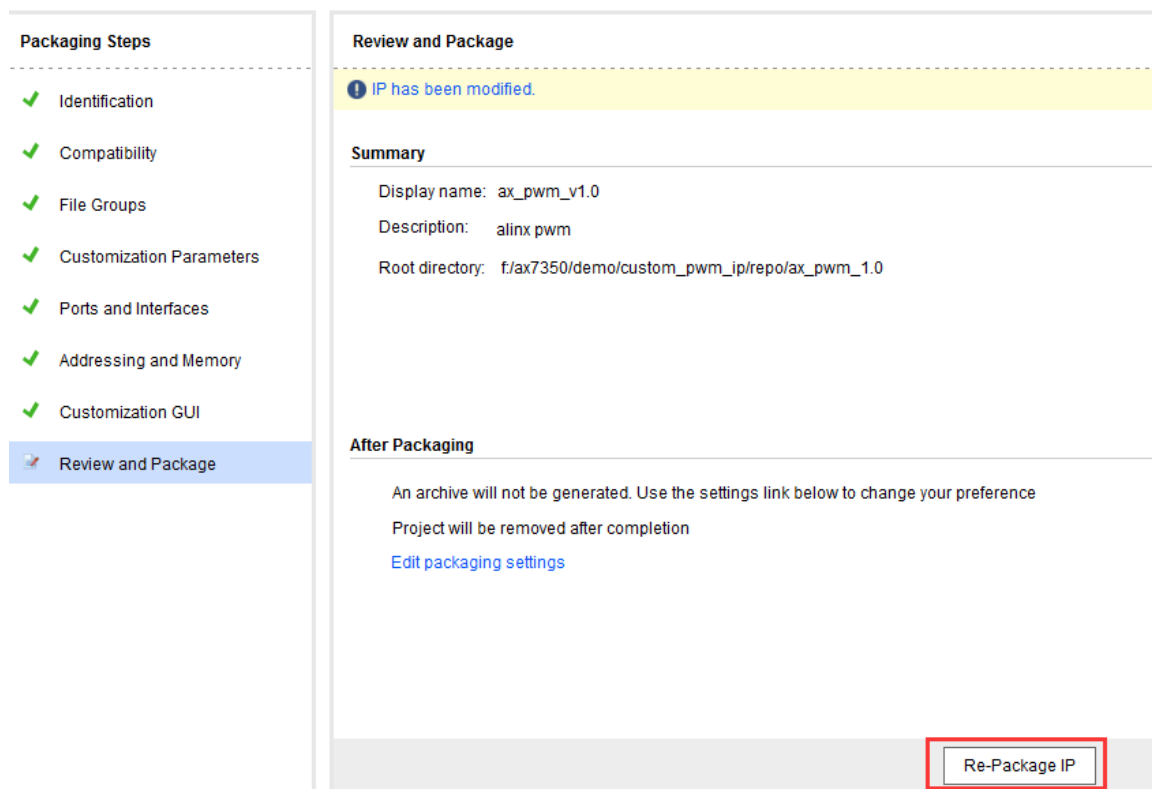
18) 在 “File Groups” 选项中点击 “Merge changes from File Groups Wizard”



19) 在 “Customization Parameters” 选项中点击 “Merge changes form Customization Parameters Wizard”

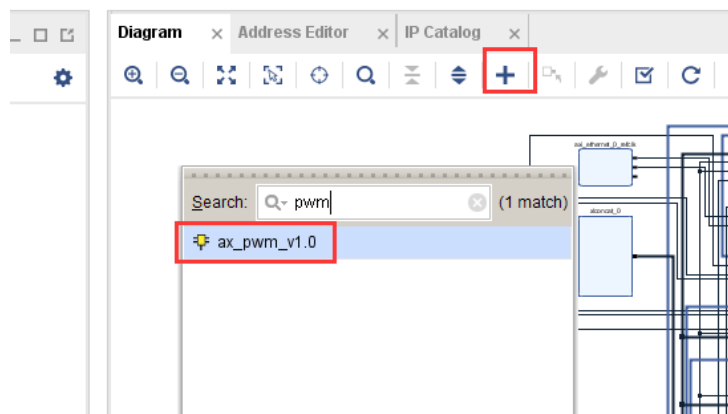


20) 点击 “Re-Package IP” 完成 IP 的修改



8.2.2 添加自定义 IP 到工程

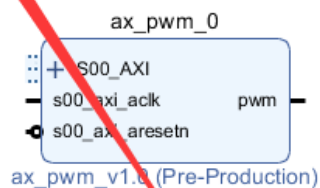
21) 搜索 “pwm”，添加 “ax_pwm_v1.0”



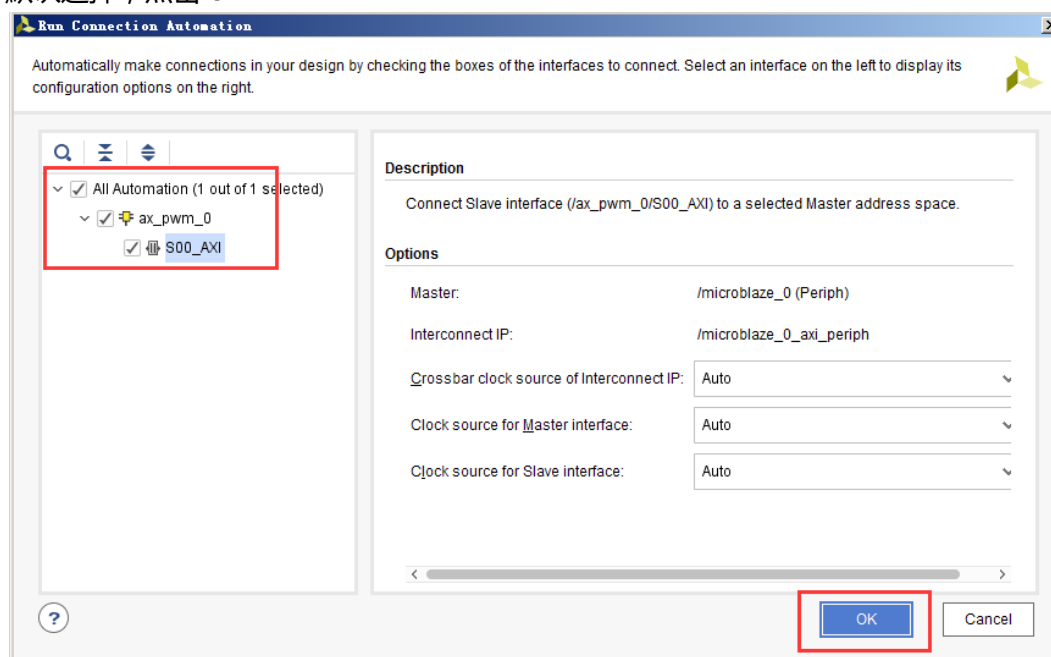
22) 点击 “Run Connection Automation”



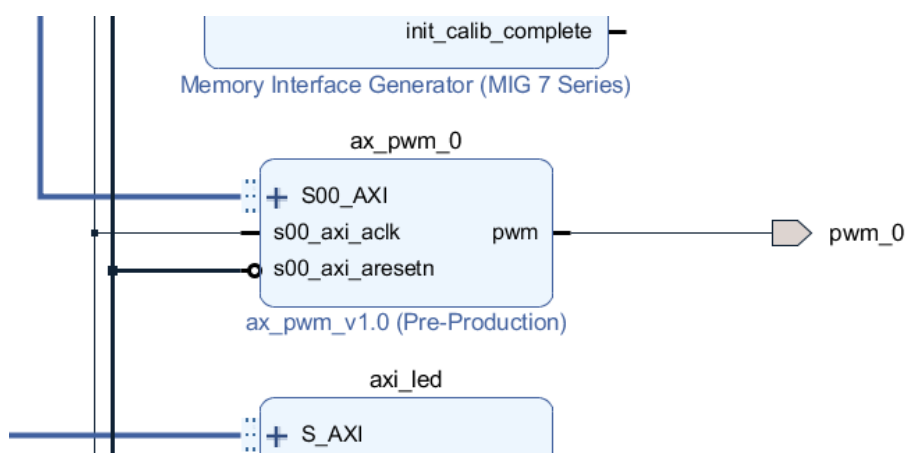
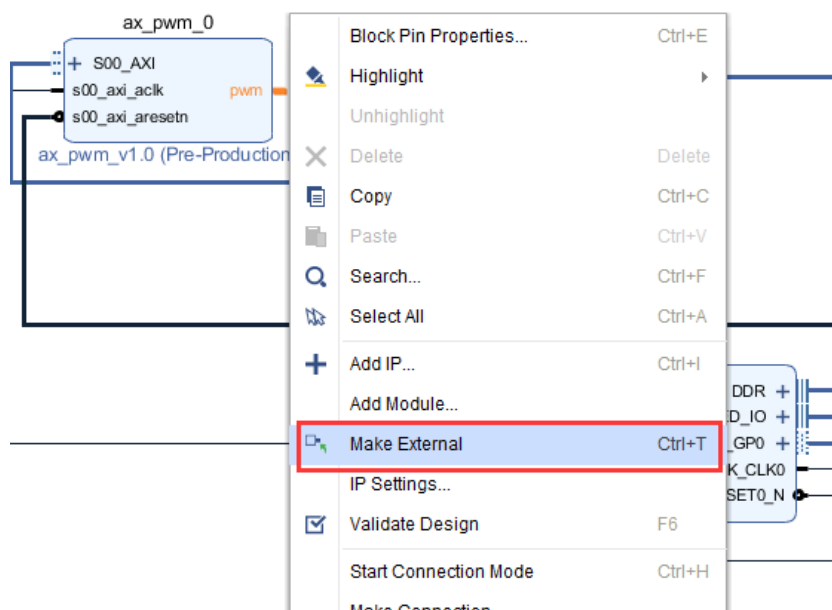
Designer Assistance available. [Run Connection Automation](#)



23) 默认选择，点击 OK

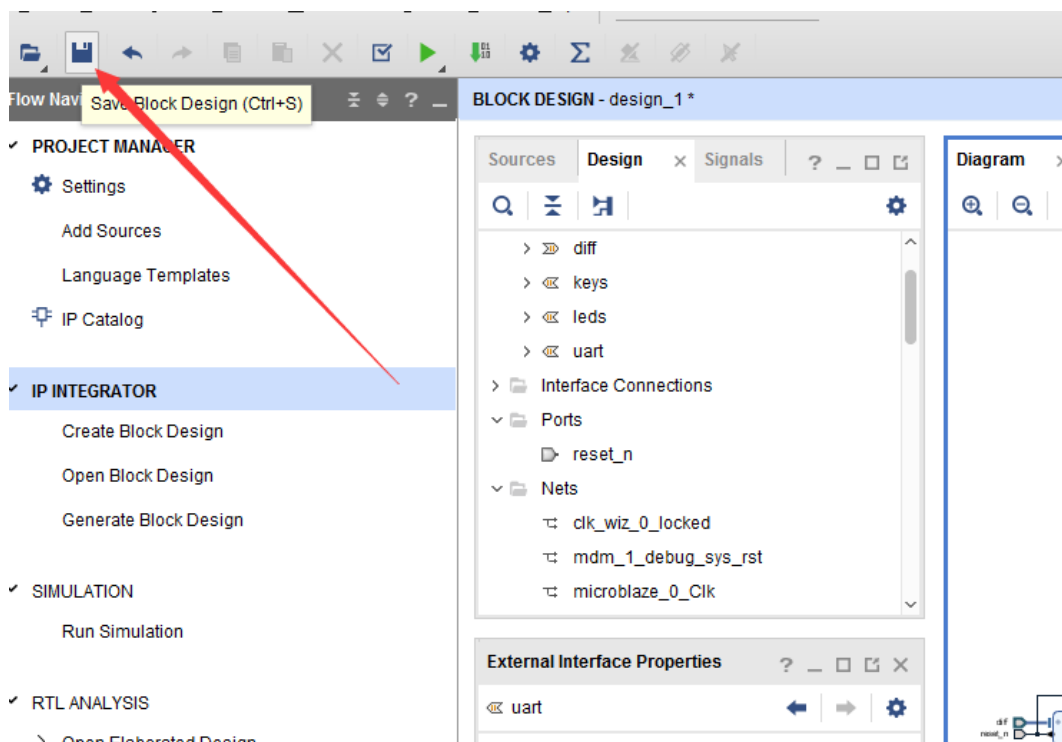


24) 导出 pwm 端口

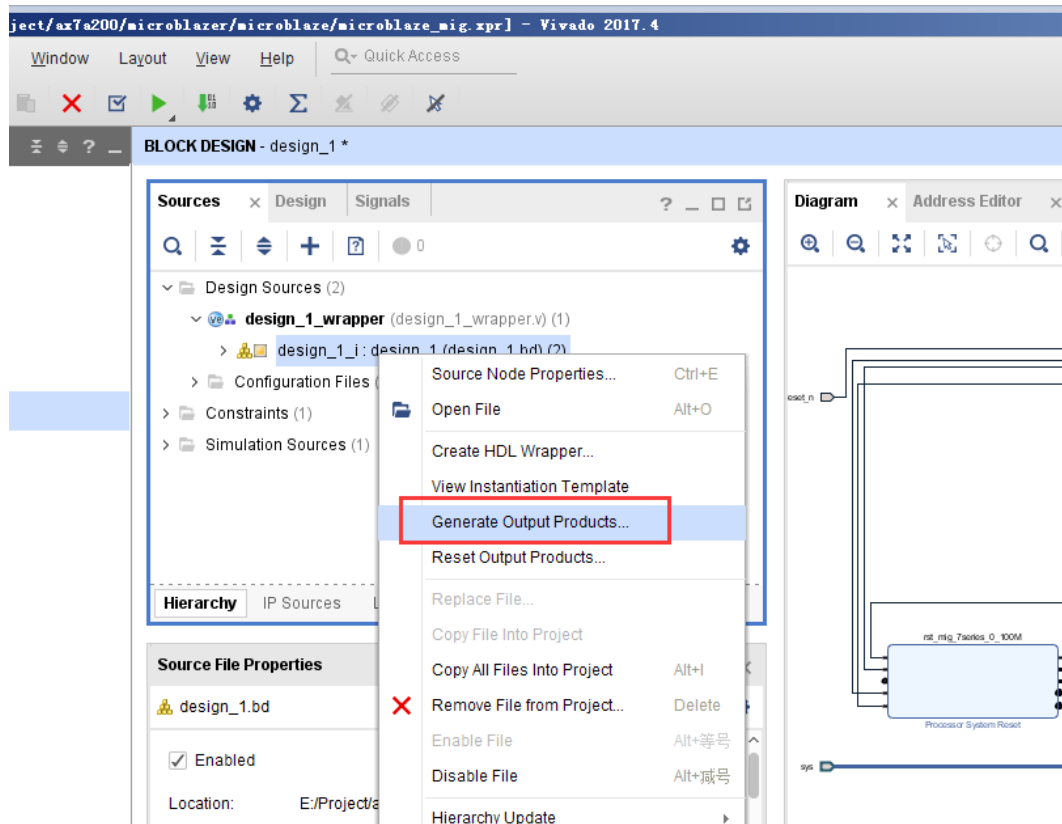


8.2.3 保存并检查错误

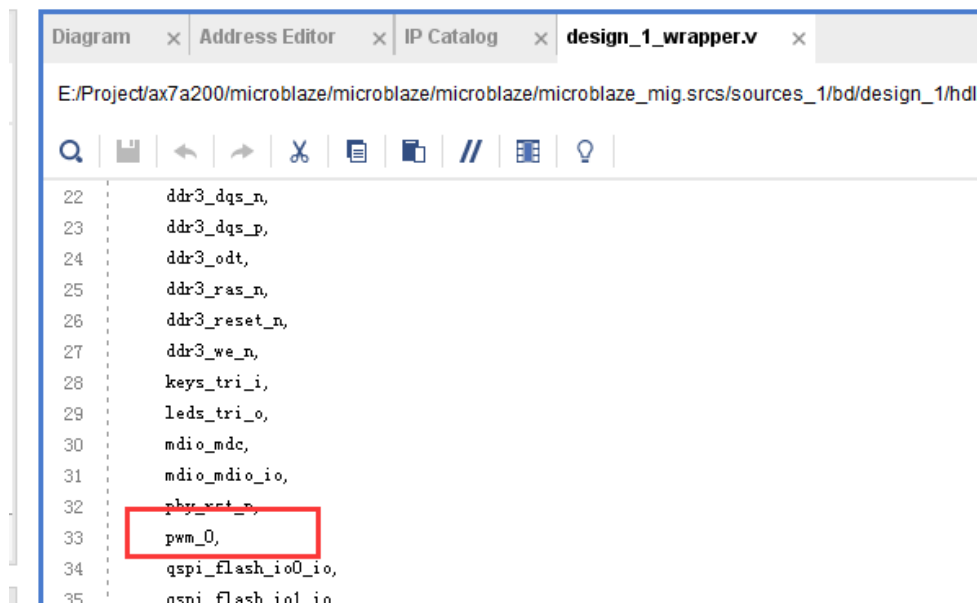
25) 保存设计，然后按“F6”检查设计



- 26) 选择 “design_1.bd” 文件，右键选择 “Generate Output Products” ，把 IP 参数和连接信息 update 到 project 中



更新后 design_1_wrapper.v 文件的端口会多一个 pwm 接口信号。



8.2.4 pwm 的管脚约束，

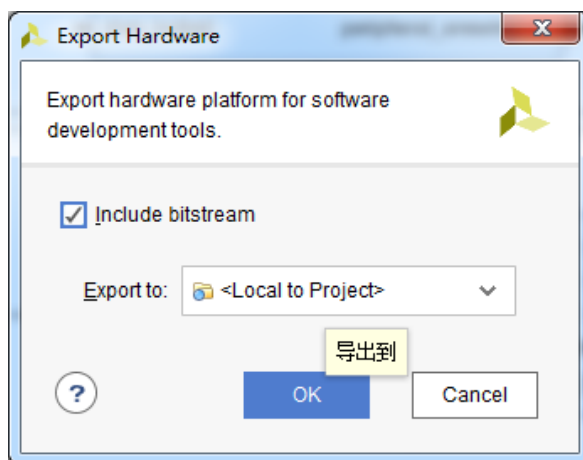
27) xdc 文件添加分配管脚，把 pwm_0 输出端口分配给 LED2 上，做一个呼吸灯（以 ax7a200 为例）

```
set_property PACKAGE_PIN M13 [get_ports {pwm_0      }]
set_property IOSTANDARD LVCMOS33 [get_ports {pwm_0      }]
```

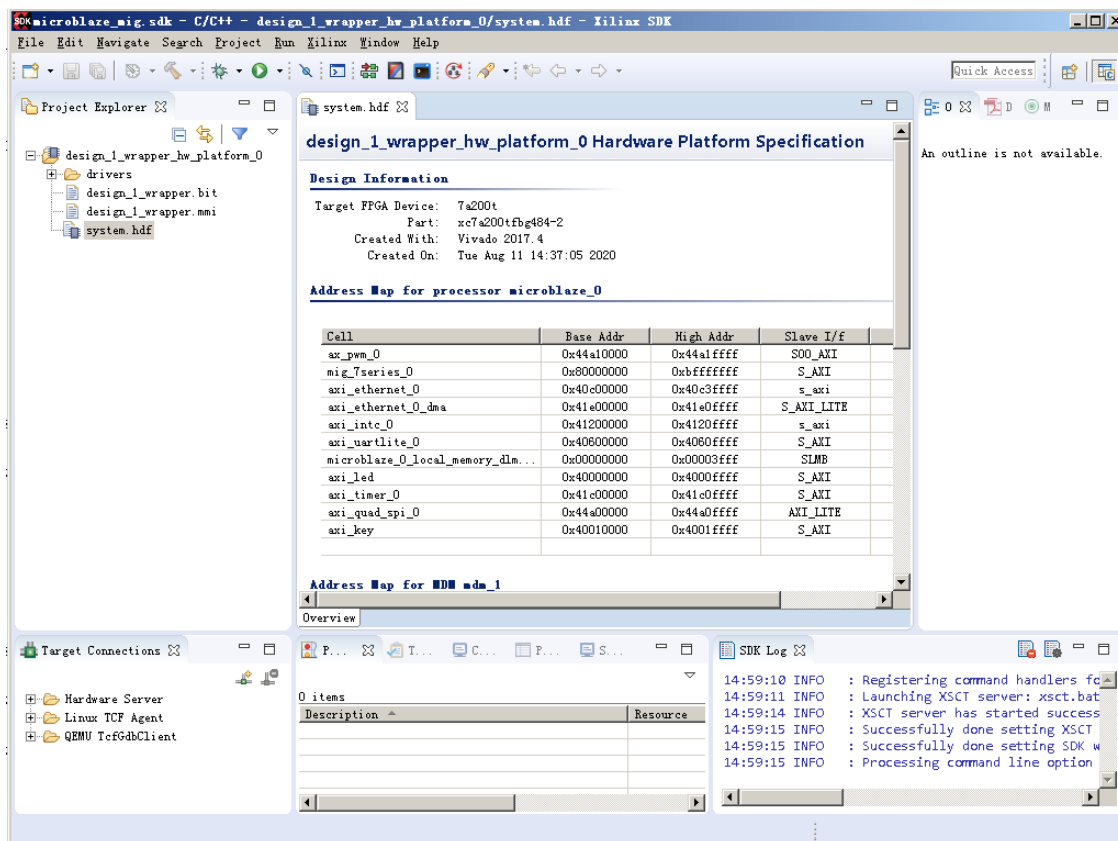
28) 编译生成 bit 文件，然后导出硬件信息，启动 SDK

8.3 SDK 软件编写调试

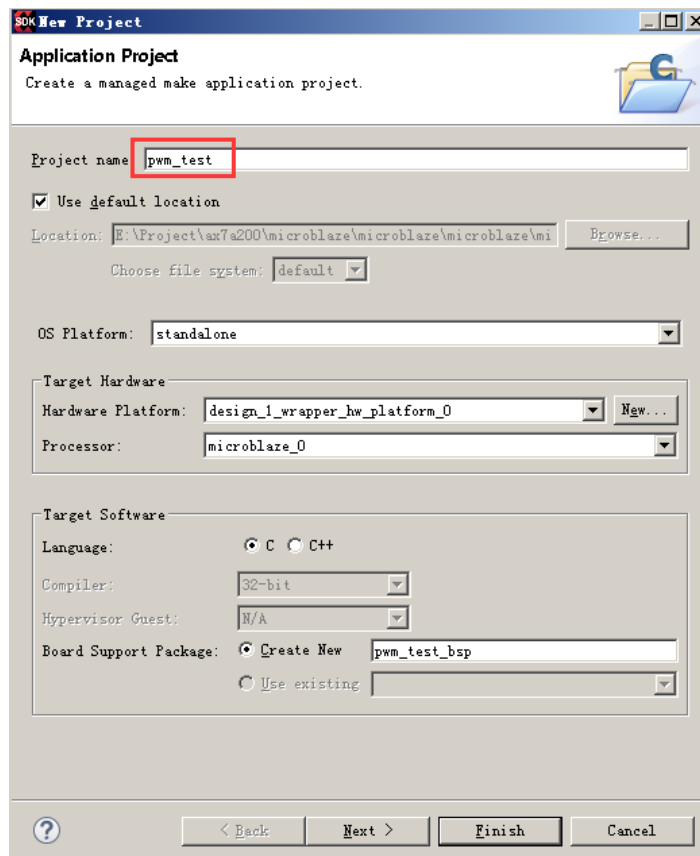
1) 导出硬件



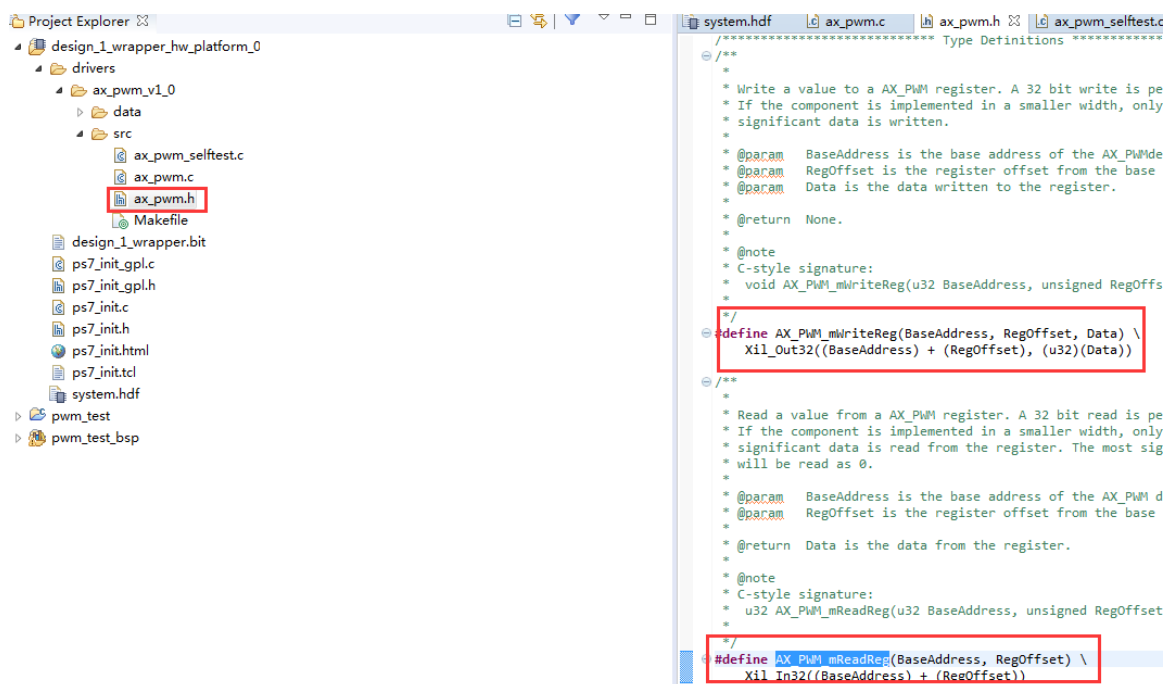
2) 启动 SDK



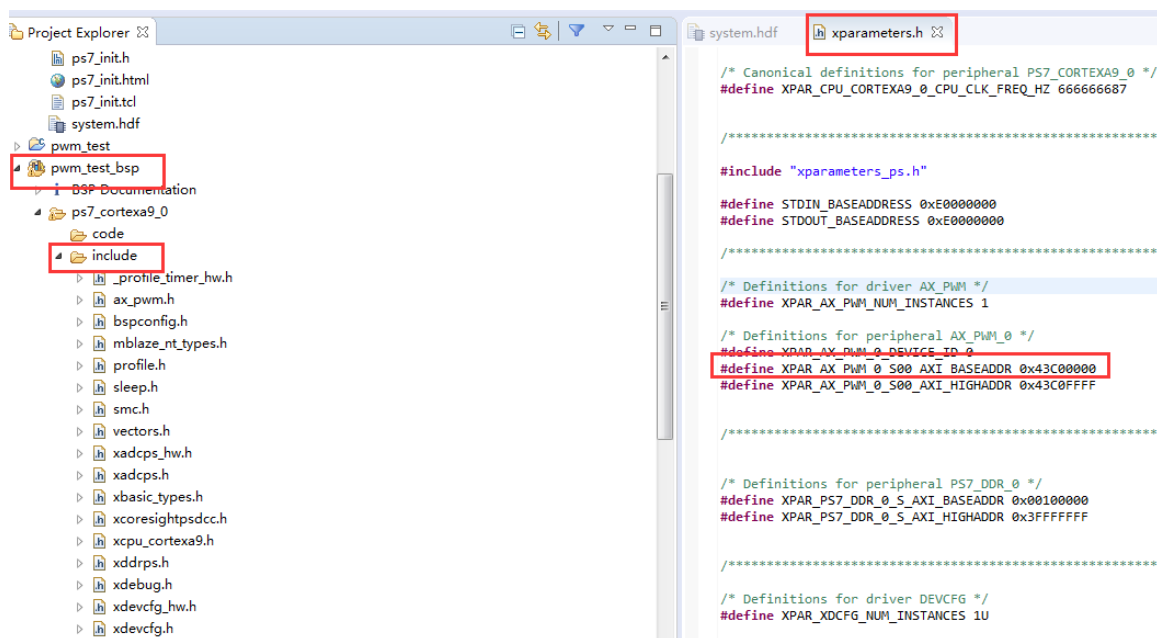
3) 新建 APP，模板选择 “Hello World”



- 4) 前面的例都是使用 xilinx 的 IP , xilinx 大多都提供一套 API , 对于这个自定义 IP , 我们需要自己开发 , 先看看 APP 的目录下的资源 , 可以找到一个 ax_pwm.h 的文件 , 这个文件里包含里对自定义 IP 寄存器的读写宏定义



- 5) 在 bsp 里找到 “xparameters.h” 文件 , 这个非常重要的文件 , 里面找到了自定义 IP 的寄存器基地址 , 可以找到自定义 IP 的基地址。



- 6) 有了寄存器读写宏和自定义 IP 的基地址 , 我们开始编写代码 , 测试自定义 IP , 我们先通过写寄存器 AX_PWM_S00_AXI_SLV_REG0_OFFSET , 控制 PWM 输出频率 , 然后通过写寄存器 AX_PWM_S00_AXI_SLV_REG1_OFFSET 控制 PWM 输出的占空比。

```
#include <stdio.h>
#include "platform.h"
#include "xil_printf.h"
#include "ax_pwm.h"
#include "xparameters.h"
#include "sleep.h"

unsigned int duty;

int main()
{
    u32 duty;

    init_platform();

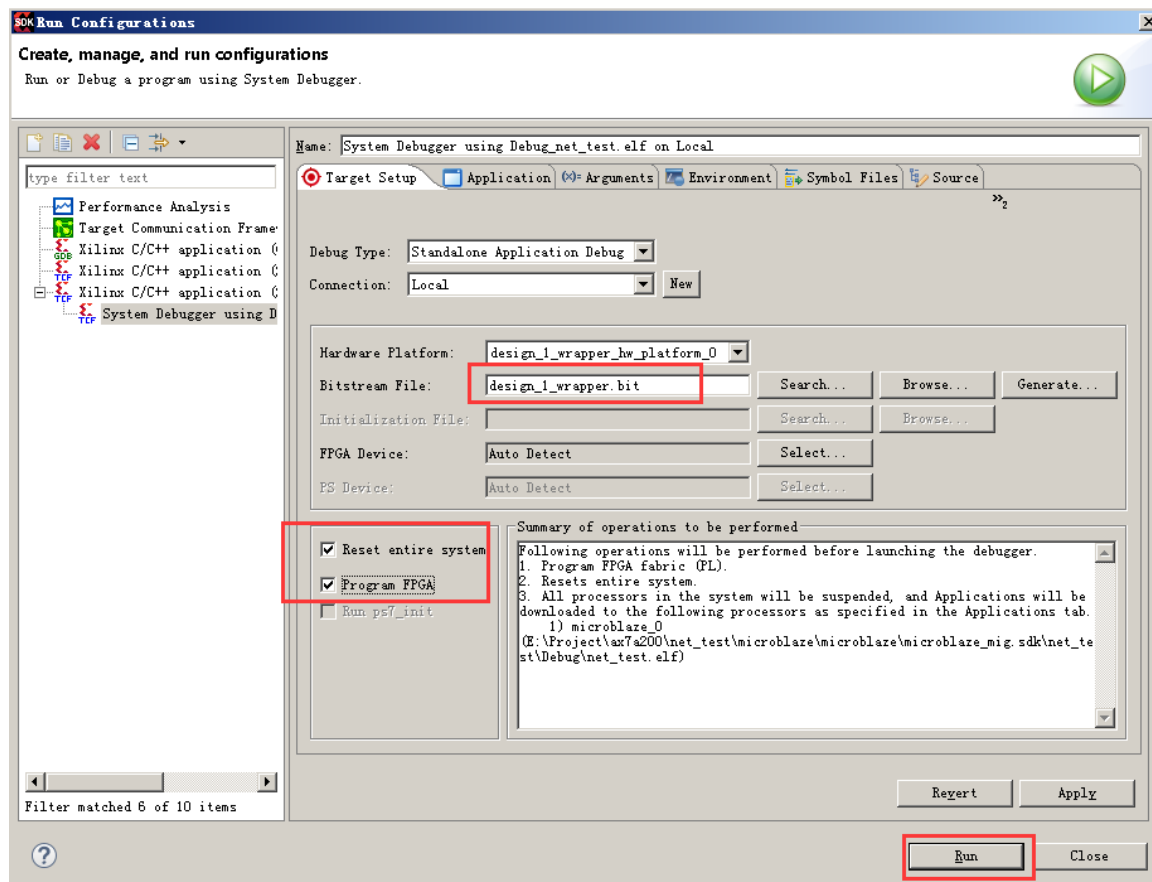
    print("Hello World\n\r");

    //pwm out period = frequency(pwm_out) * (2 ** N) / frequency(clk);
    AX_PWM_mWriteReg(XPAR_AX_PWM_0_S00_AXI_BASEADDR, AX_PWM_S00_AXI_SLV_REG0_OFFSET, 17179); //200hz

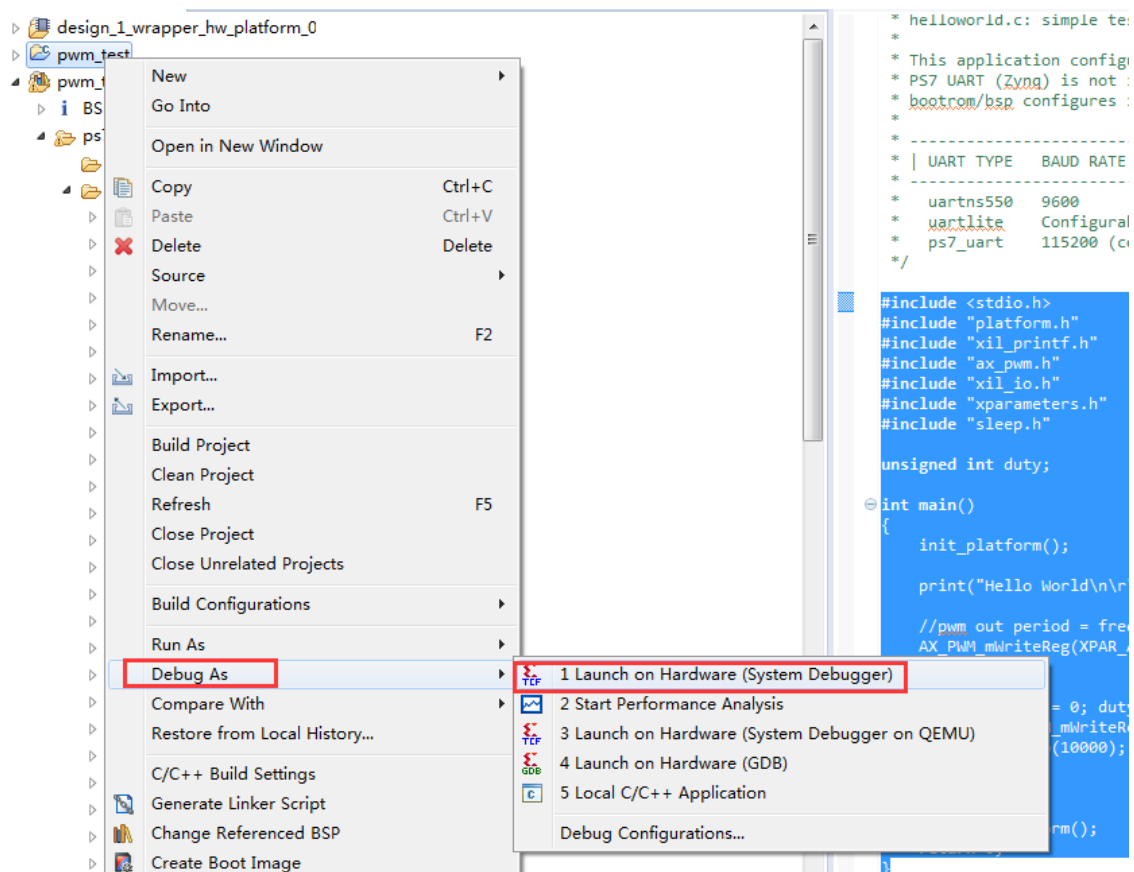
    while (1) {
        for (duty = 0; duty < 0xffffffff; duty = duty + 50000) {
            AX_PWM_mWriteReg(XPAR_AX_PWM_0_S00_AXI_BASEADDR, AX_PWM_S00_AXI_SLV_REG1_OFFSET, duty);
            usleep(20);
        }
    }

    cleanup_platform();
    return 0;
}
```

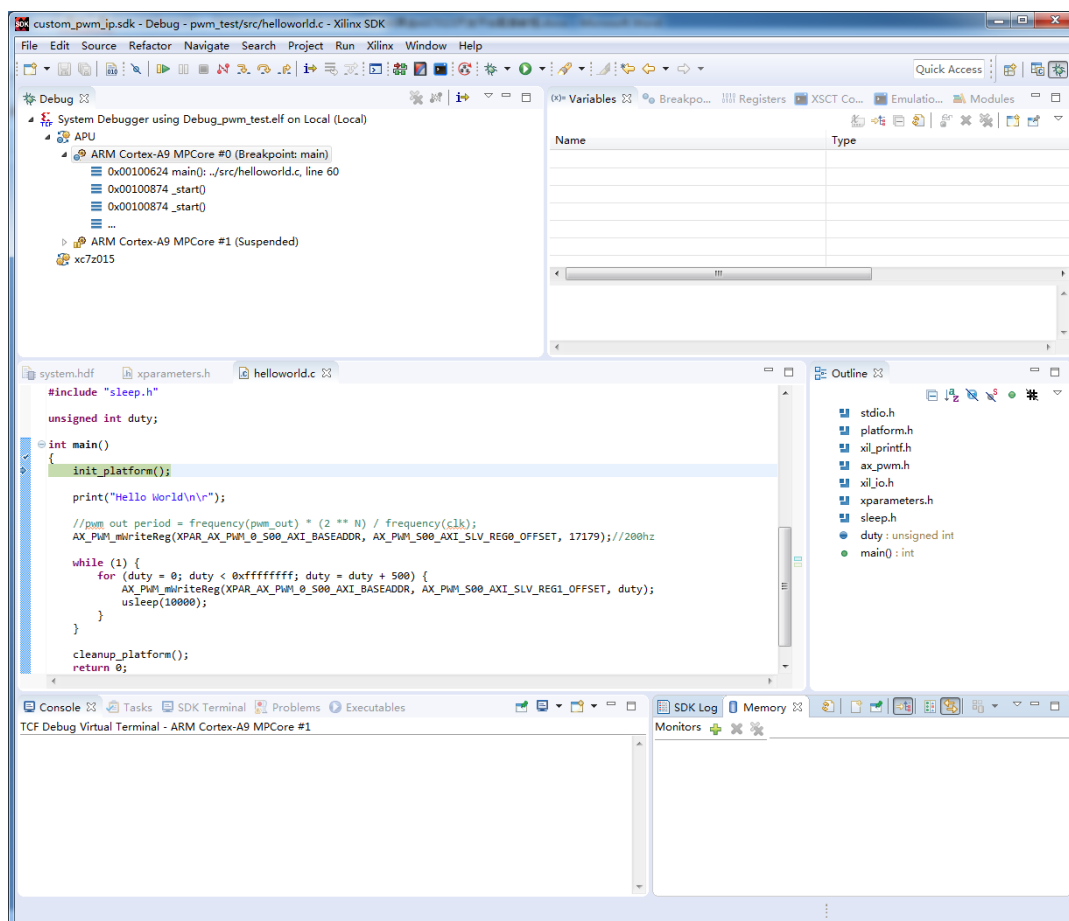
7) 通过运行代码，我们可以看到 LED2 呈现出一个呼吸灯的效果。



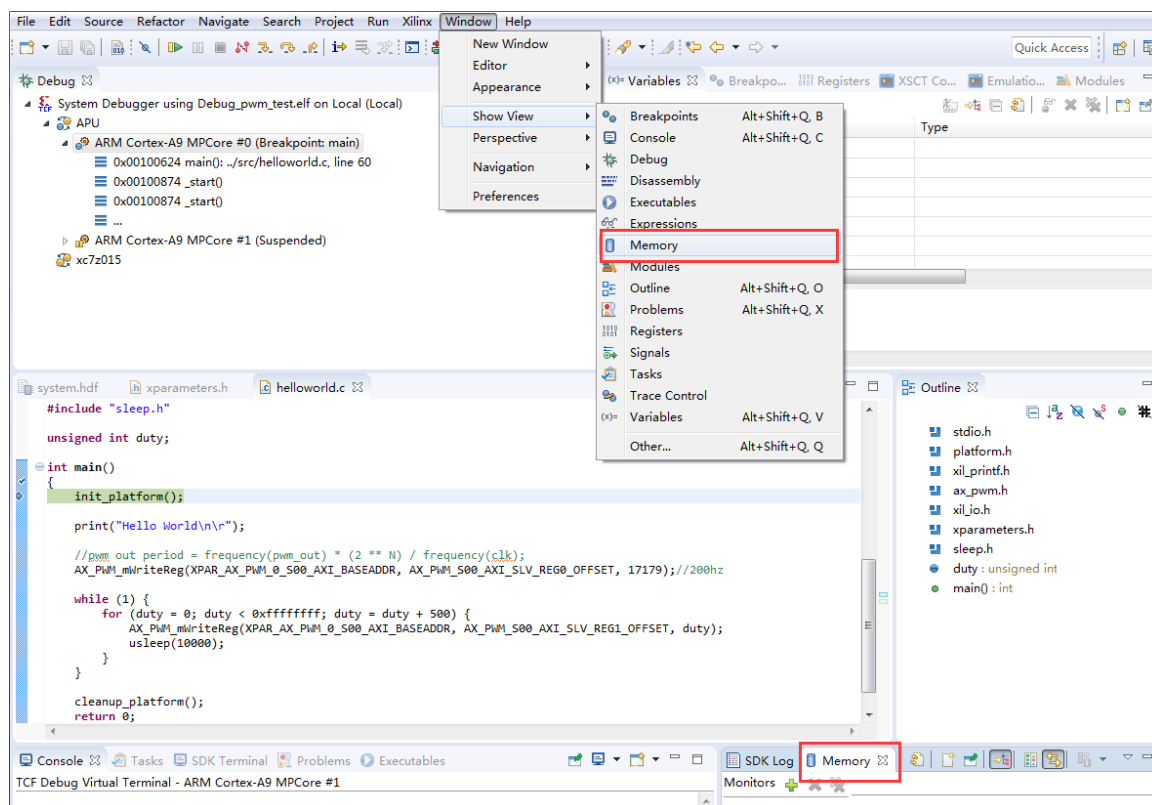
8) 通过 debug，我们来查看一下寄存器



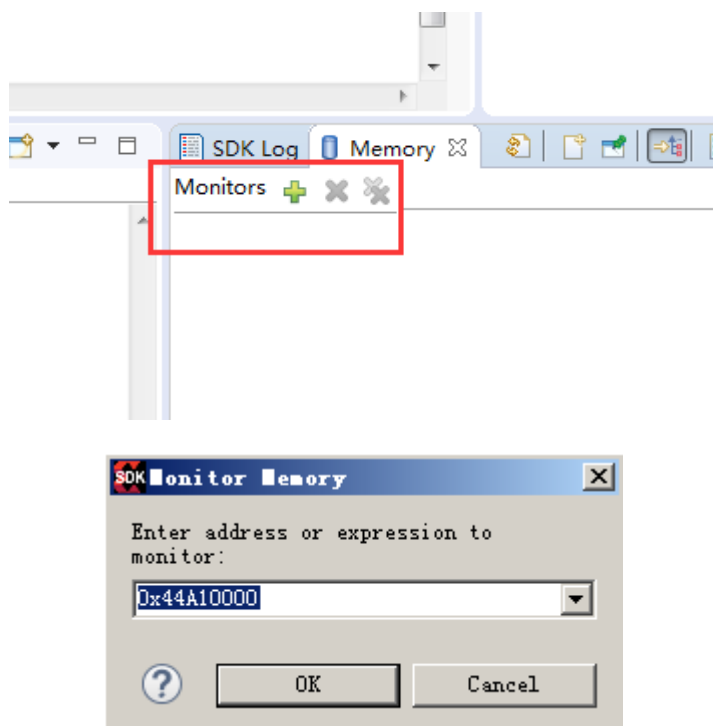
9) 进入 debug 状态，按“F6”可以单步运行。



10) 通过菜单可以查看 “Memory” 窗口



11) 添加一个监视地址 “0x43c00000”



12) 单步运行，观察变化

Ox44A10000 : Ox44A10000 <Hex Integer> + New Renderings...					
Address	0 - 3	4 - 7	8 - B	C - F	
44A0FF90	00000000	00000000	00000000	00000000	
44A0FFA0	00000400	00000000	00000000	00000000	
44A0FFB0	00000000	00000000	00000000	00000000	
44A0FFC0	00000000	00000000	00000000	00000000	
44A0FFD0	00000000	00000000	00000000	00000000	
44A0FFE0	00000180	000000A5	00000000	00000000	
44A0FFF0	00000001	00000000	00000000	00000000	
44A10000	0000431B	0000C350	00000000	00000000	
44A10010	0000431B	0000C350	00000000	00000000	
44A10020	0000431B	0000C350	00000000	00000000	
44A10030	0000431B	0000C350	00000000	00000000	

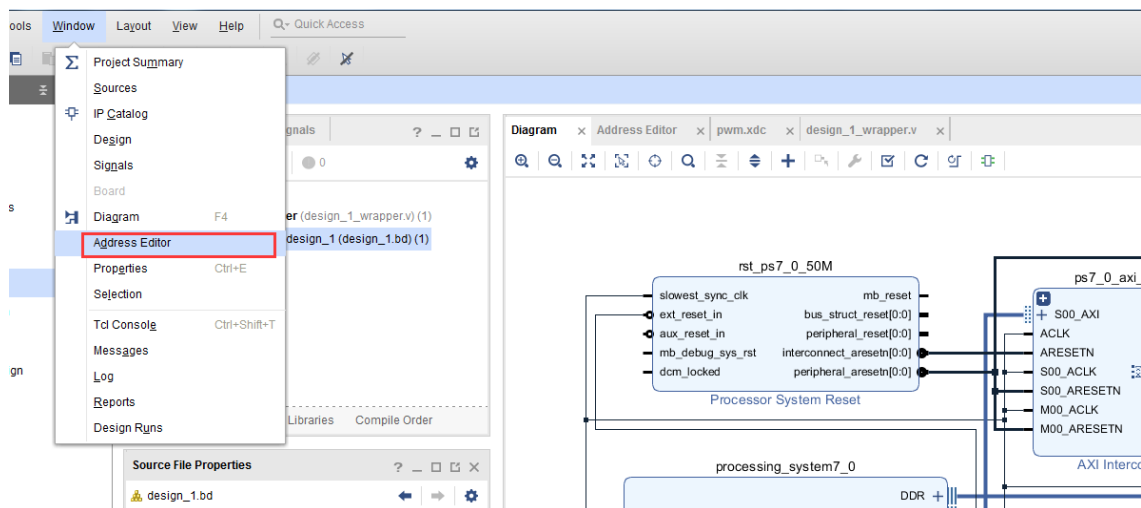
8.4 实验总结

通过本实验我们掌握了更多的 SDK 调试技巧，掌握了 ARM + FPGA 开发的核心内容，就是 ARM 和 FPGA 数据交互。

8.5 常见问题

8.5.1 如何知道 AXI IP 的基地址

- 1) 如下图所示，打开 “Address Editor”，可以看到地址分配情况



- 2) 地址一般是 Vivado 自动分配，我们也可以修改地址

Cell	Slave Interface	Base Name	Offset Address	Range	High Address
microblaze_0					
Data (32 address bits : 4G)					
axi_ethernet_0	s_axi	Reg0	0x40C0_0000	256K	0x40C3_FFFF
axi_ethernet_0_dma	S_AXI_LITE	Reg	0x41E0_0000	64K	0x41E0_FFFF
axi_intc_0	s_axi	Reg	0x4120_0000	64K	0x4120_FFFF
axi_key	S_AXI	Reg	0x4001_0000	64K	0x4001_FFFF
axi_led	S_AXI	Reg	0x4000_0000	64K	0x4000_FFFF
axi_quad_spi_0	AXI_LITE	Reg	0x44A0_0000	64K	0x44A0_FFFF
axi_timer_0	S_AXI	Reg	0x41C0_0000	64K	0x41C0_FFFF
axi_uartlite_0	S_AXI	Reg	0x4060_0000	64K	0x4060_FFFF
microblaze_0_local_memory/dlmb_bram_if_cntlr	SLMB	Mem	0x0000_0000	16K	0x0000_3FFF
mig_7series_0	S_AXI	memaddr	0x8000_0000	1G	0xBFFF_FFFF
ax_pwm_0	S00_AXI	S00_AXI_reg	0x44A1_0000	64K	0x44A1_FFFF
Instruction (32 address bits : 4G)					
microblaze_0_local_memory/ilmb_bram_if_cntlr	SLMB	Mem	0x0000_0000	16K	0x0000_3FFF
mig_7series_0	S_AXI	memaddr	0x8000_0000	1G	0xBFFF_FFFF

第九章 VDMA HDMI 显示

实验 Vivado 工程为 “vdma_hdmi_out”。

本实验里我们将结合构架 HDMI 显示系统，让用户可以在 SDK 环境里控制 HDMI 的视频和图像的显示。实现图像显示的方案有很多，但是都离不开 DMA 系统，DMA 系统可以完成显示数据从 ddr3 读出到显示器的显示，降低 CPU 的开销，VDMA 是 xilinx 开发的特殊 DMA，专门用于视频输入输出，是学习 xilinx FPGA 视频处理的重要内容。

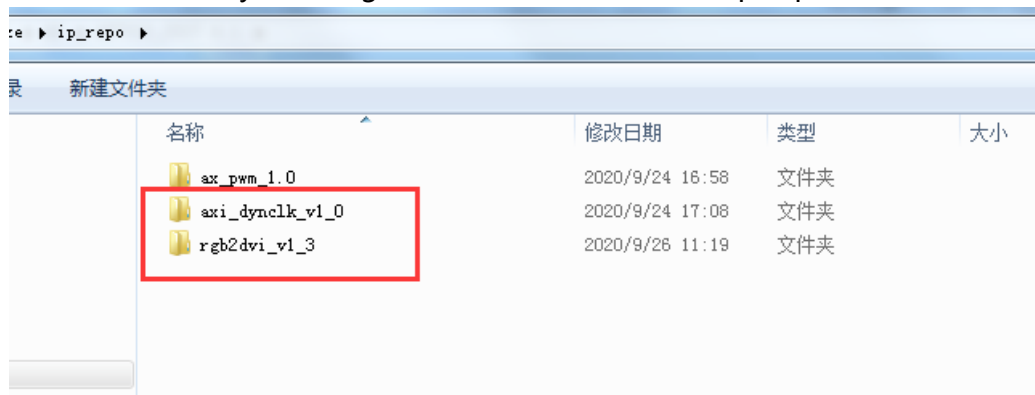
本实验中，我们会用到 Xilinx 提供的 IP，比如 VDMA, VID_OUT, V_TC 等等这些视频输出和控制的 IP。也会用到用户自定义的 IP，比如 DYNCLK，对应一些没有 HDMI 编解码芯片的开发板，还需要用到自定义的 RGB2DVI IP。

9.1 Vivado 工程修改

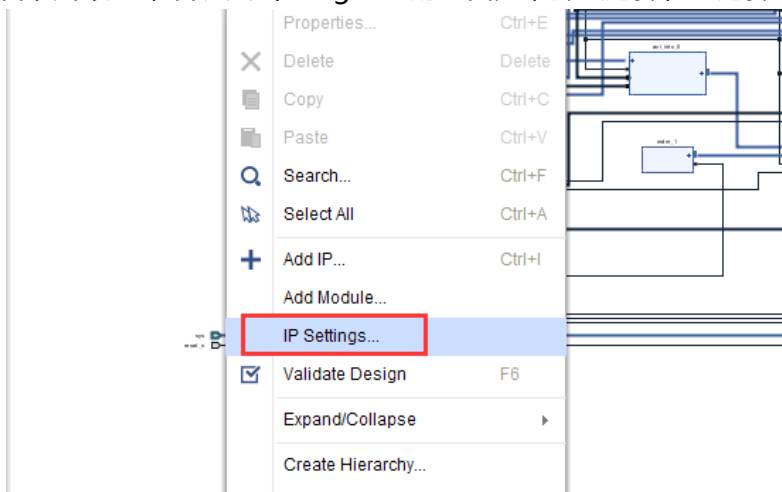
Vivado 工程是在第八章的 Vivado 工程配置的基础上修改，并删除 SDK 目录。

9.1.1 添加自定义 IP

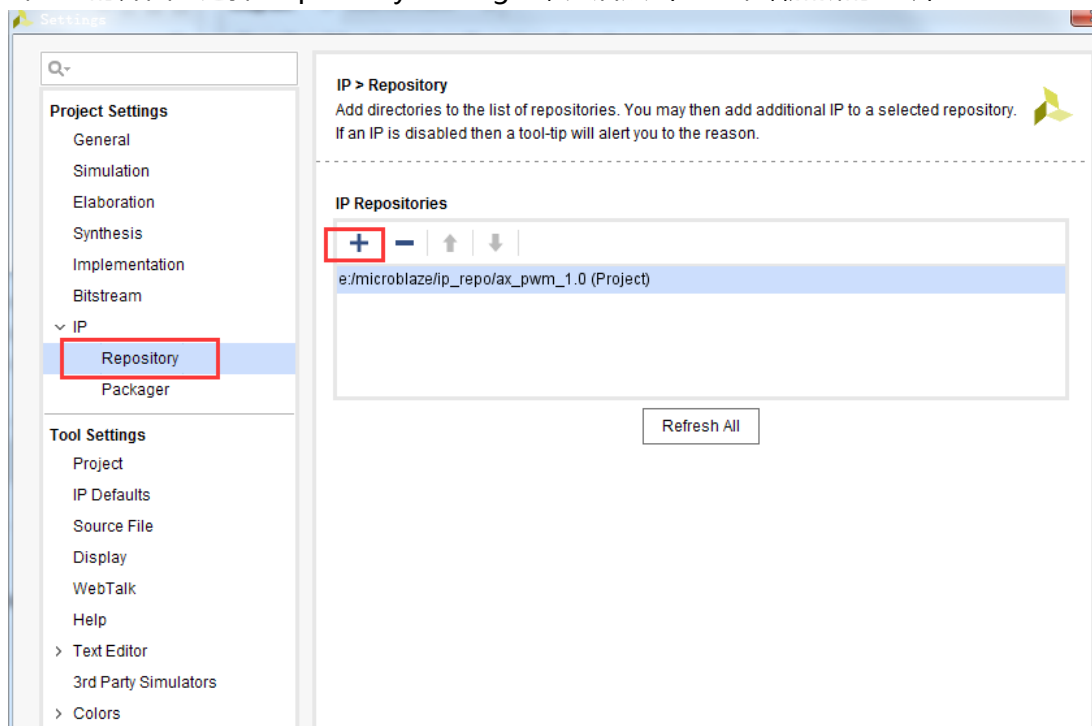
- 1) 把我们提供的 axi_dynclk 和 rgb2dvi 两个 IP 拷贝到工程的 ip_repo 目录下



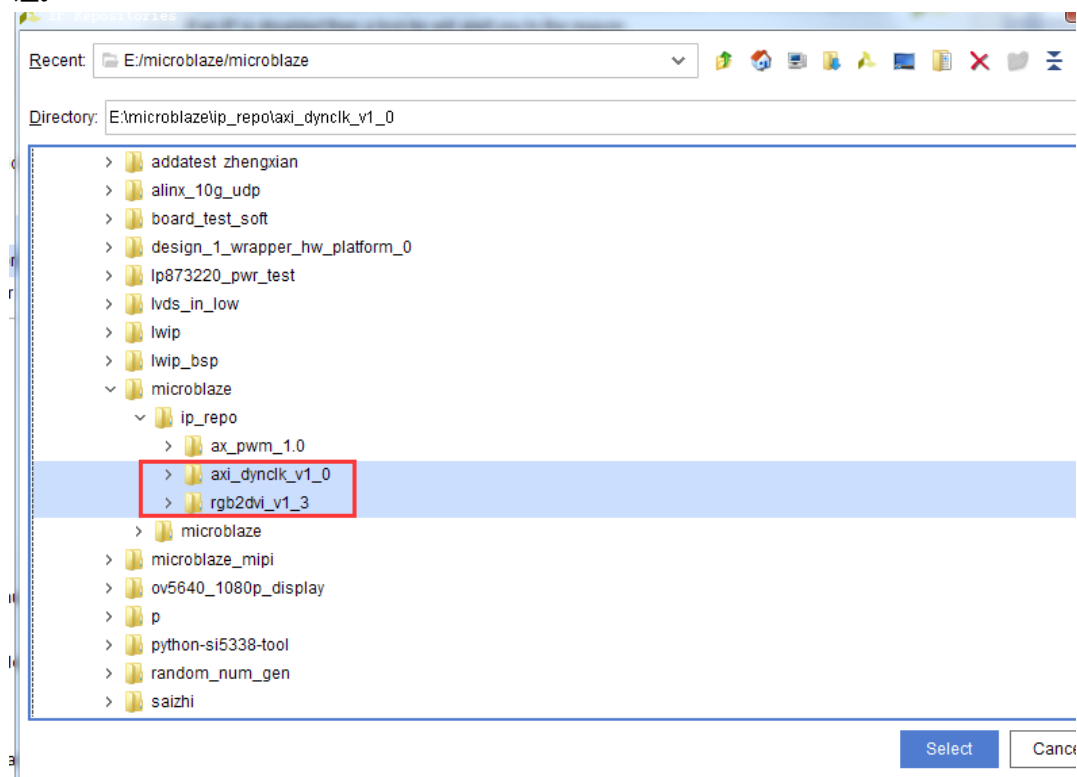
- 2) 在 Vivado 开发环境里，右键点击 Diagram 的空白处，弹出选择框里选择 IP Setting...



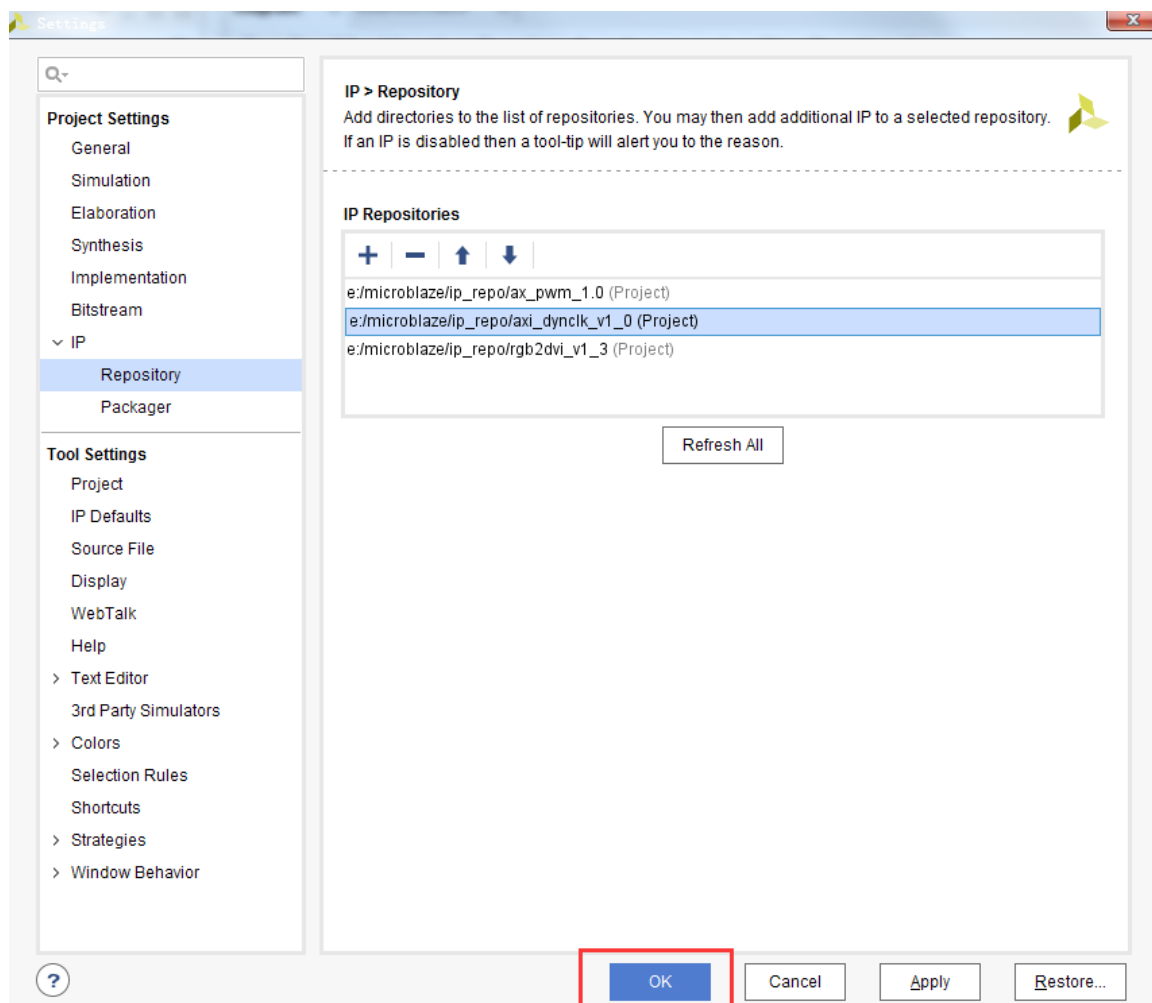
- 3) 在 IP 的界面里选择 Repository Manager，然后点击  来增加新的 IP 库



- 4) 选中 ip_repo 目录下的 axi_dynclock_v1_0 和 rgb2dvi_v1_3 文件目录夹，点击 Select 按钮。

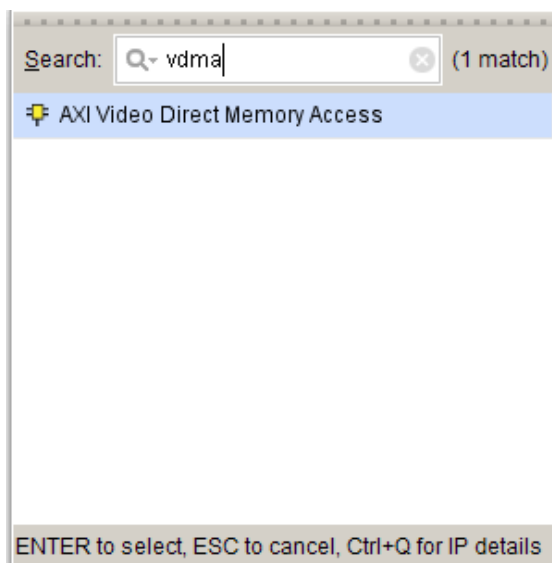


- 5) 自定义的 IP 已经添加项目中了，点击 OK 完成。这样我们在工程中就可以使用这两个用户自定义的 IP 了。

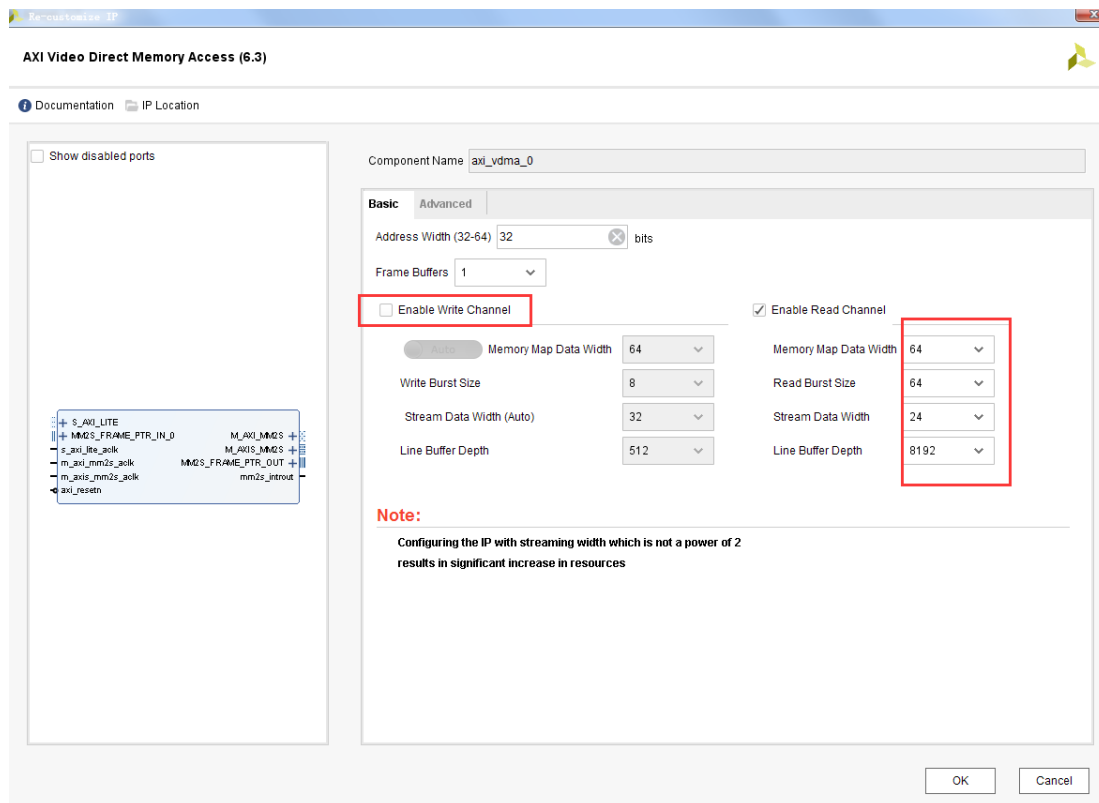


9.1.2 添加 VDMA

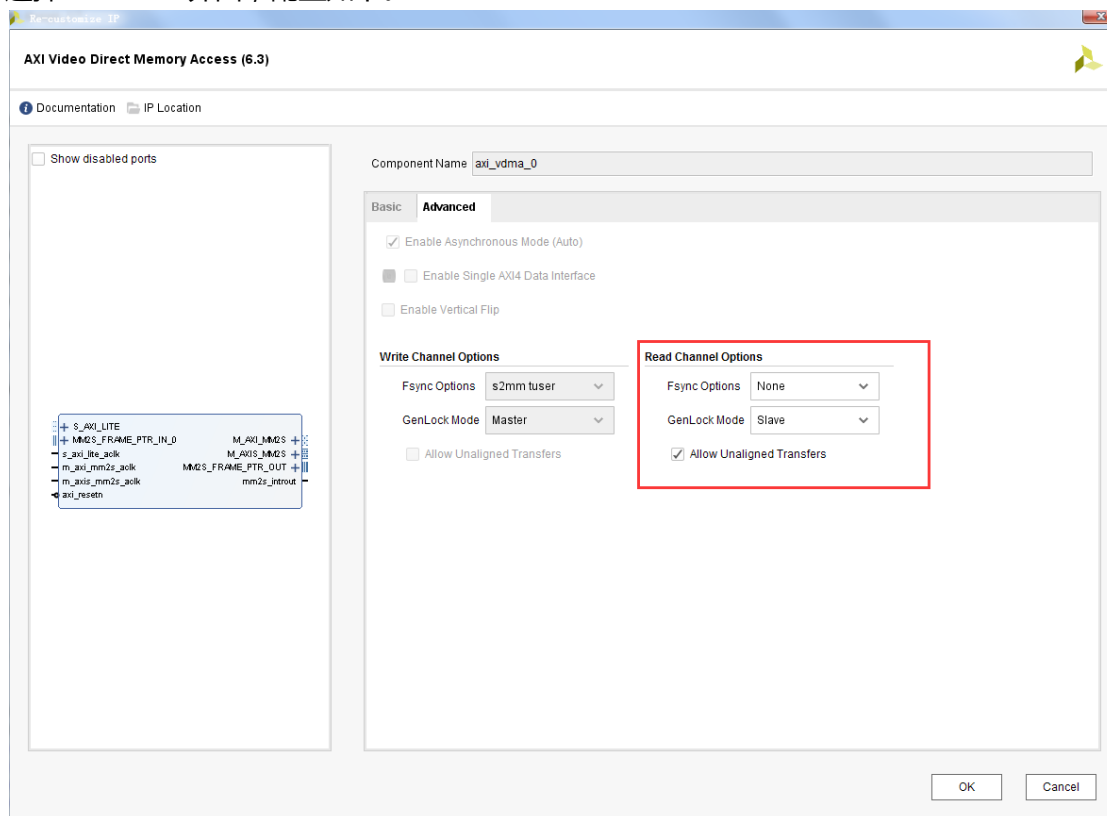
- 6) 添加一个 VDMA 的 IP 来读取 Microblaze 系统 DDR Memory 的数据。搜索 “vdma” ,双击添加。



- 7) 双击打开 VDMA 配置界面，在 Basic 界面取消 Enable Write Channel，因为本实验只是读取 Memory，所以只要 Enable Read Channel 就可以了。另外设置 Line Buffer Depth 为 8192，设置输出的 Stream Data Width 的数据宽度是 24 位。

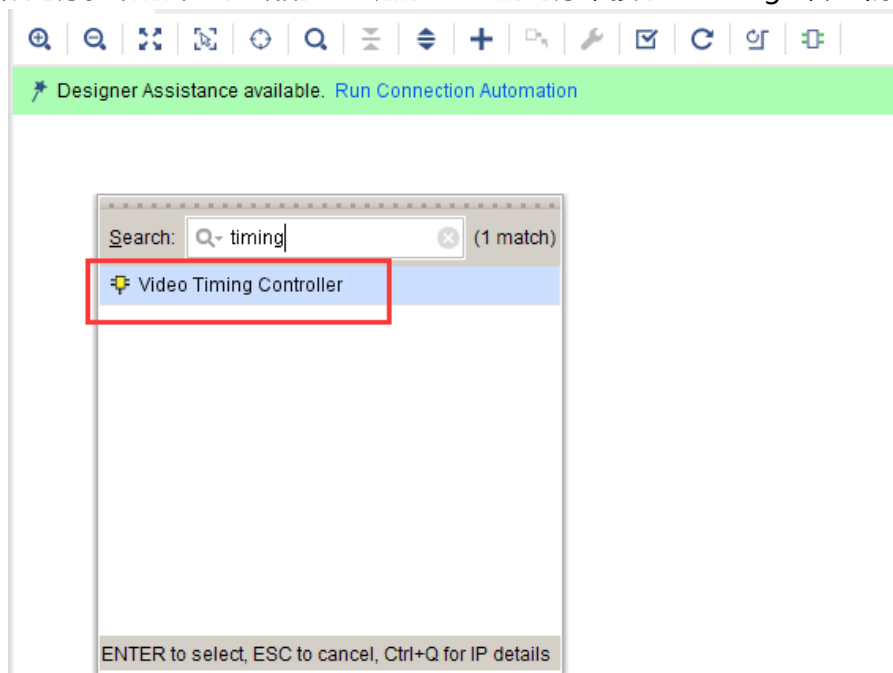


选择 Advanced 界面，配置如下。



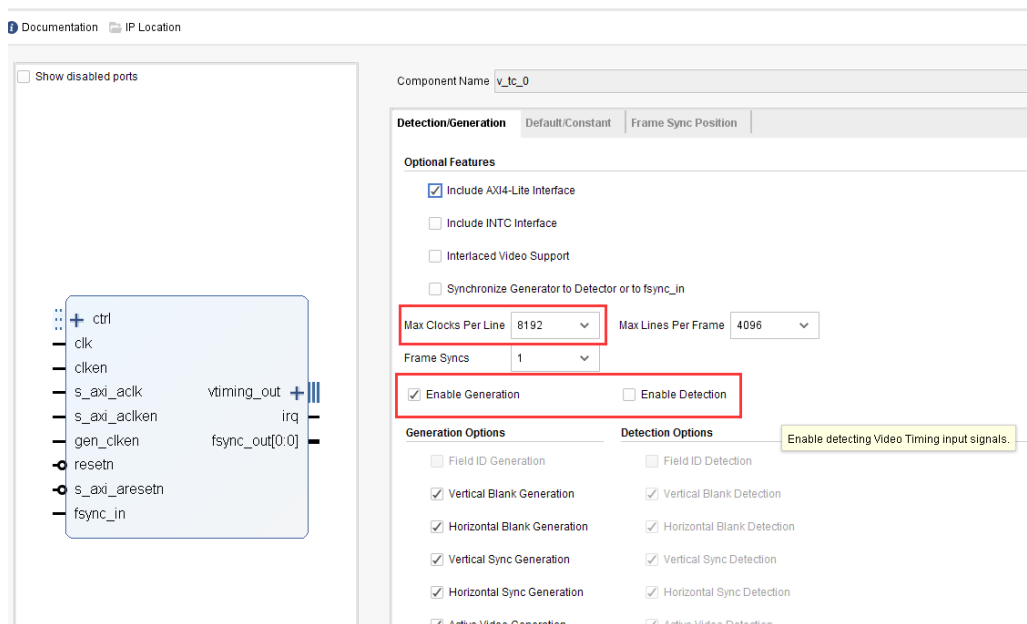
9.1.3 添加 Video Timing Controller

- 8) 添加视频时序控制器来生成或配置正确的 video 的时序，搜索“timing”，双击添加。



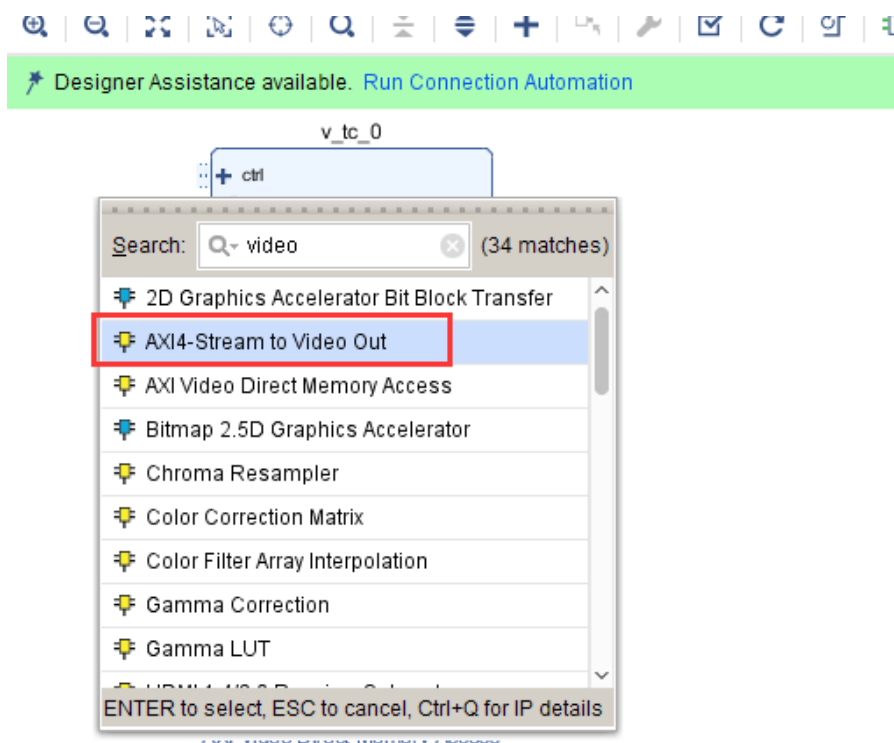
- 9) 配置视频时序控制器参数。

Video Timing Controller (6.1)

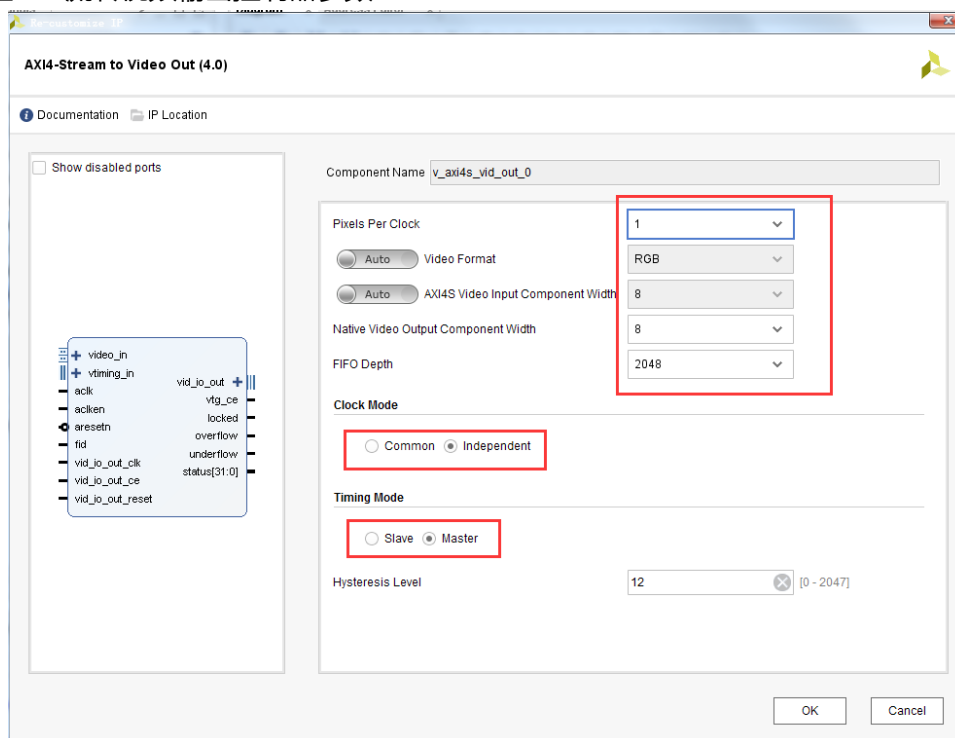


9.1.4 添加 AXI4-Stream to Video Out

- 10) AXI 流转视频输出控制器，搜索“video”，双击“AXI4-Stream to Video Out”添加。

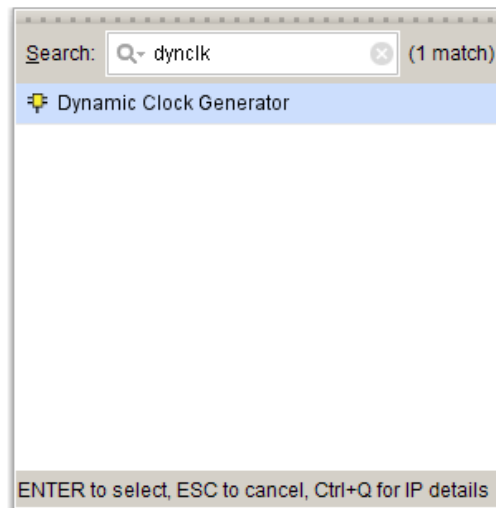


11) 配置 AXI 流转视频输出控制器参数



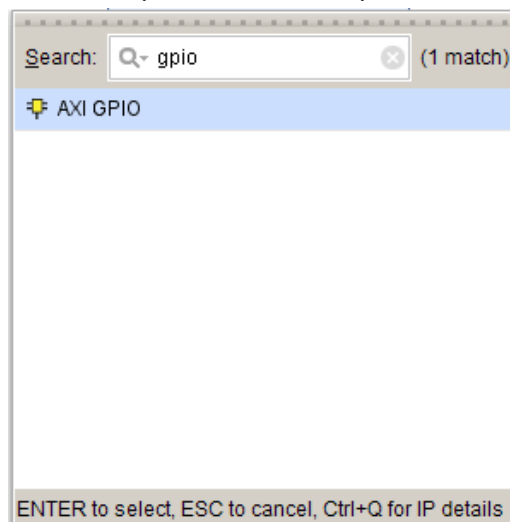
9.1.5 添加 axi_dyncclk

- 12) 由于视频有很多分辨率，各种分辨的时钟频率不相同，需要使用一个动态时钟控制器，这里就需要用到我们前面添加的自定义 IP。搜索“dyncclk”，双击“Dynamic Clock Genertor”添加。

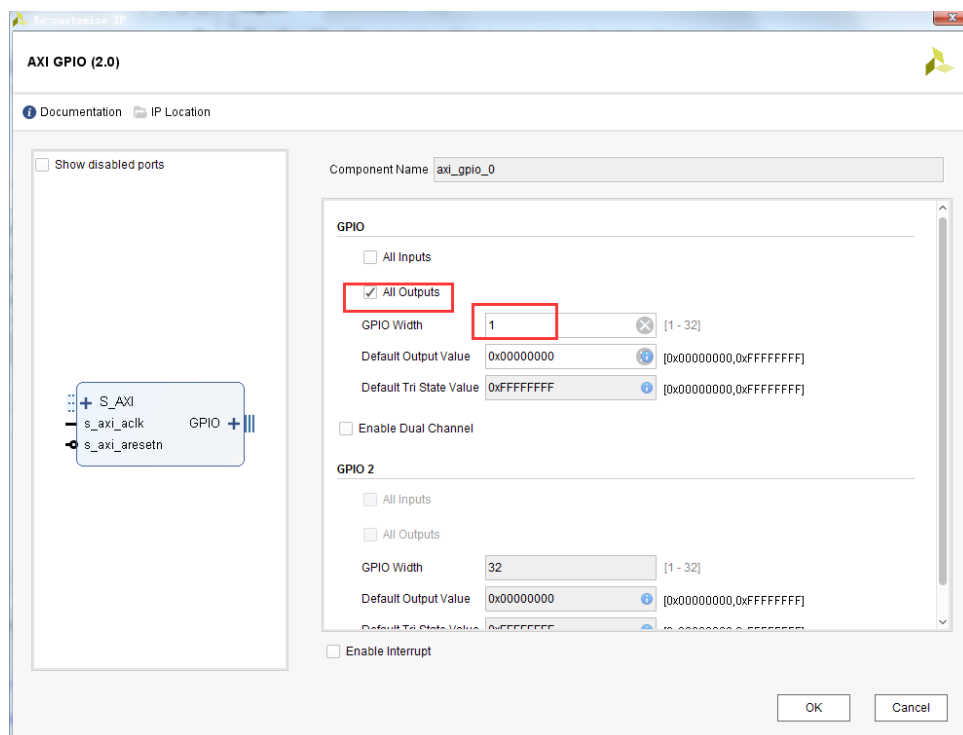


9.1.6 添加 I2C 和 GPIO

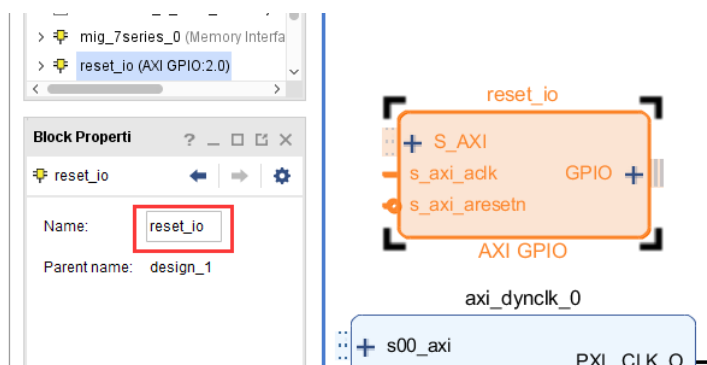
13) 添加一个 GPIO , 用于 HDMI 芯片 (SI9134 解码芯片) 复位。



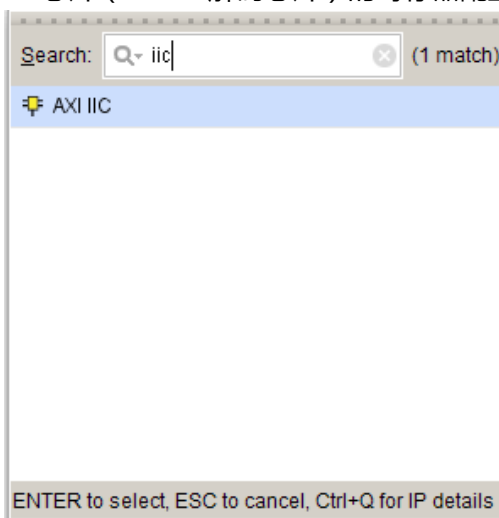
14) 配置 GPIO 参数



15) IP 名字取为 reset_io。

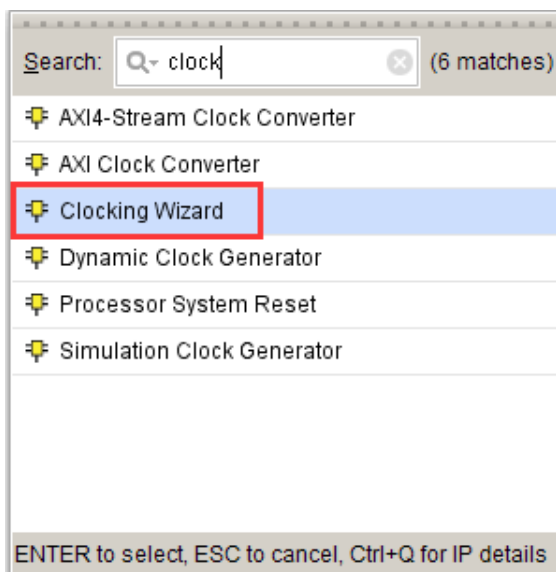


16) 添加一个 i2c , 用于 HDMI 芯片 (SI9134 解码芯片) 的寄存器配置。

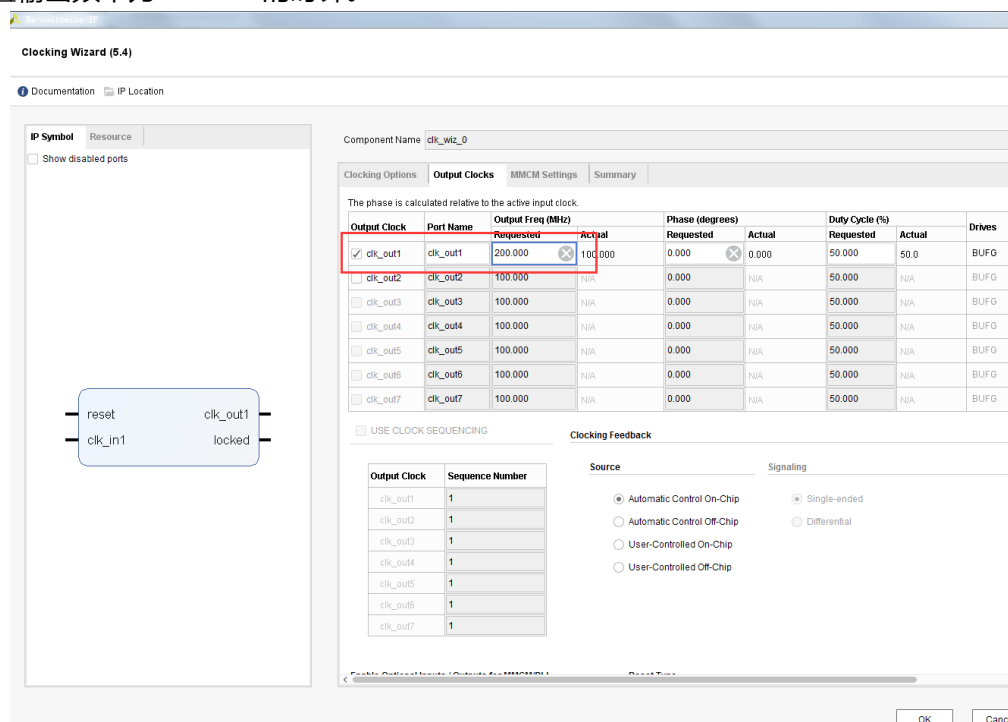


9.1.7 添加 PII

17) 添加一个 Clocking Wizard , 用于输出 200Mhz 的时钟。



18) 配置输出频率为 200Mhz 的时钟。

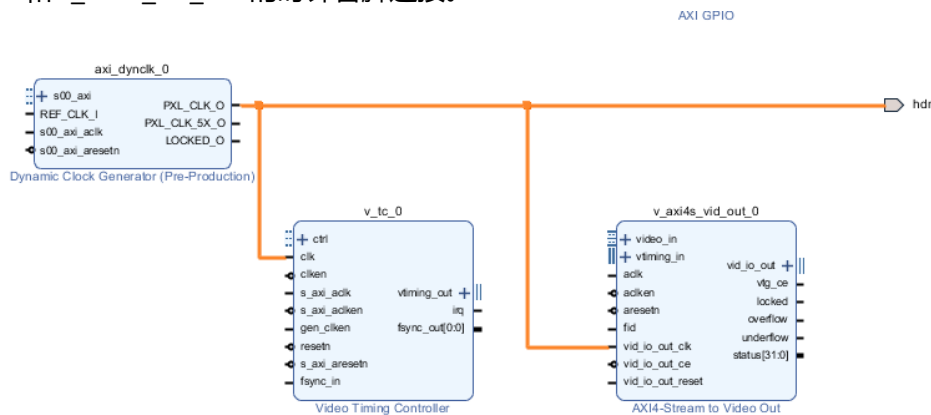


9.1.8 信号连线

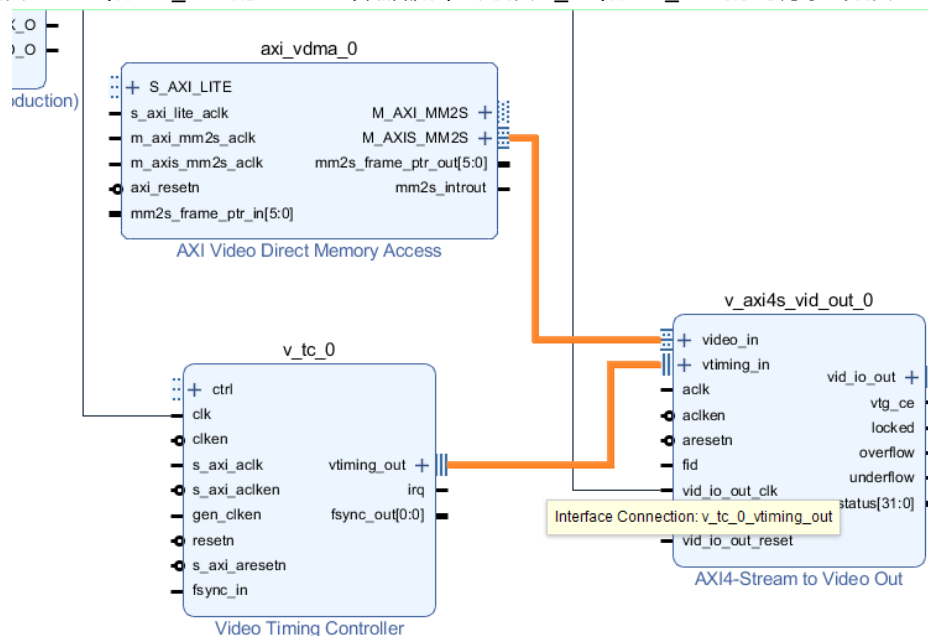
19) axi_dynclk 的 PXL_CLK_O 设置成 "Make External" ,然后改端口名为 "hdmi_out_clk"。



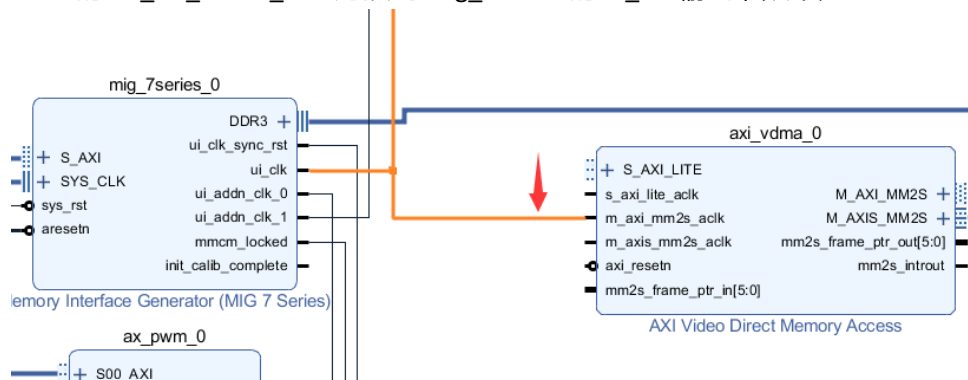
并跟 v_tc 和 v_axi4s_vid_out 的时钟管脚连接。



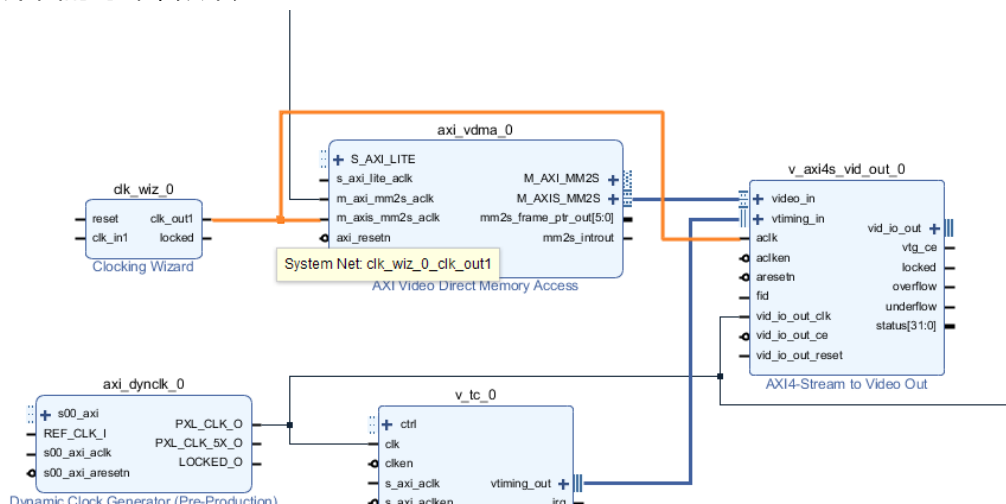
20) 连接 vdma 和 vid_out 的 stream 数据流，连接 v_tc 和 vid_out 的时序控制接口。



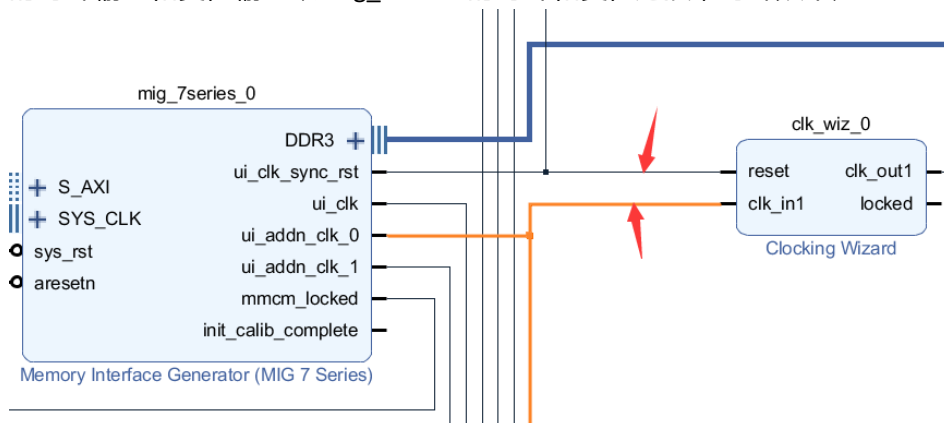
21) 连接 vdma 的 m_axi_mm2s_aclk 连接到 mig_7series 的 ui_clk 输出，频率为 100Mhz。



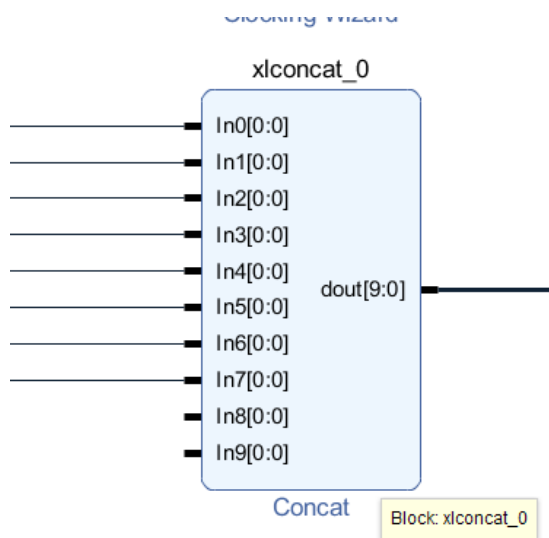
22) 连接下面的时钟，频率为 200MHz。

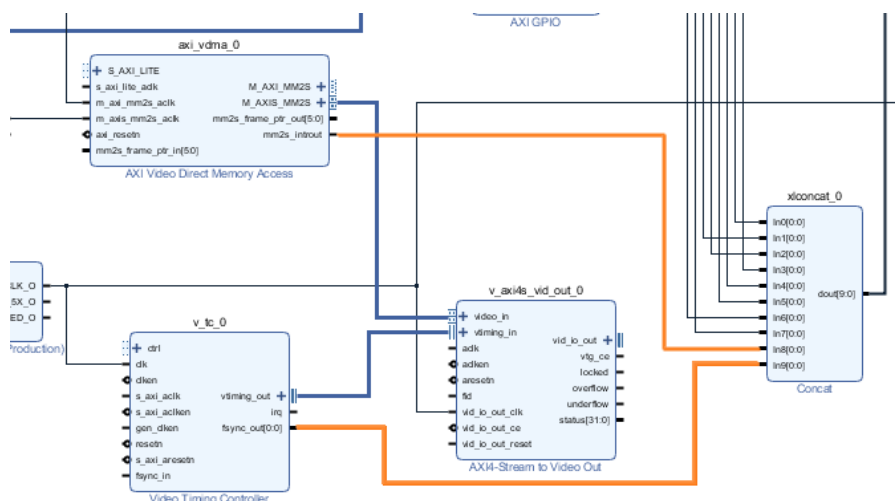


clk_wiz 的时钟输入和复位输入跟 mig_7series 的时钟和复位连接，时钟频率为 100MHz。

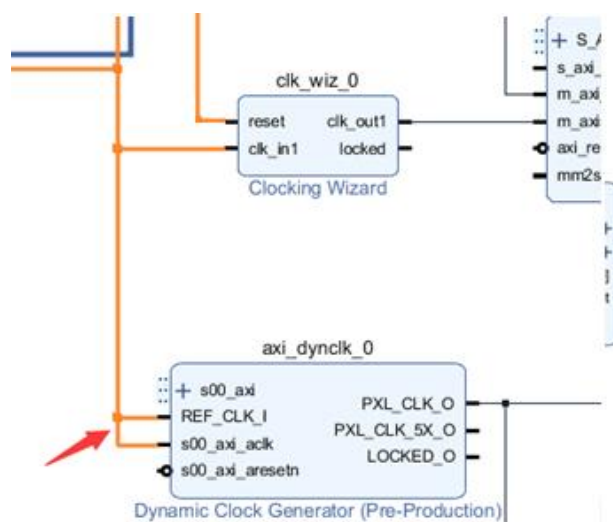


23) 修改 xlconcat 宽度，增加两个端口，连接 vdma 和 v_tc 的中断输入。

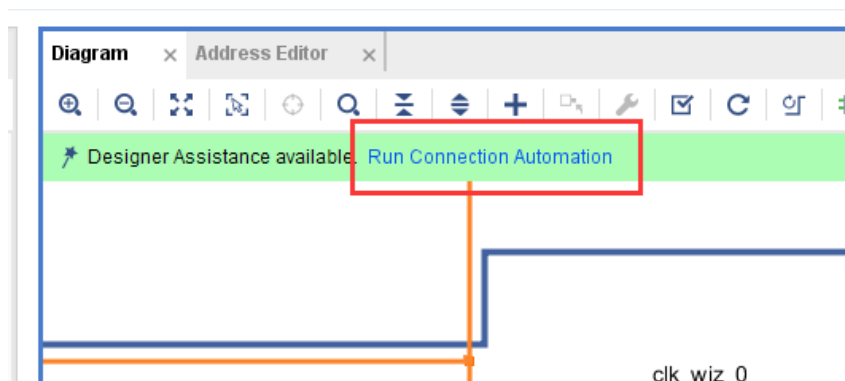




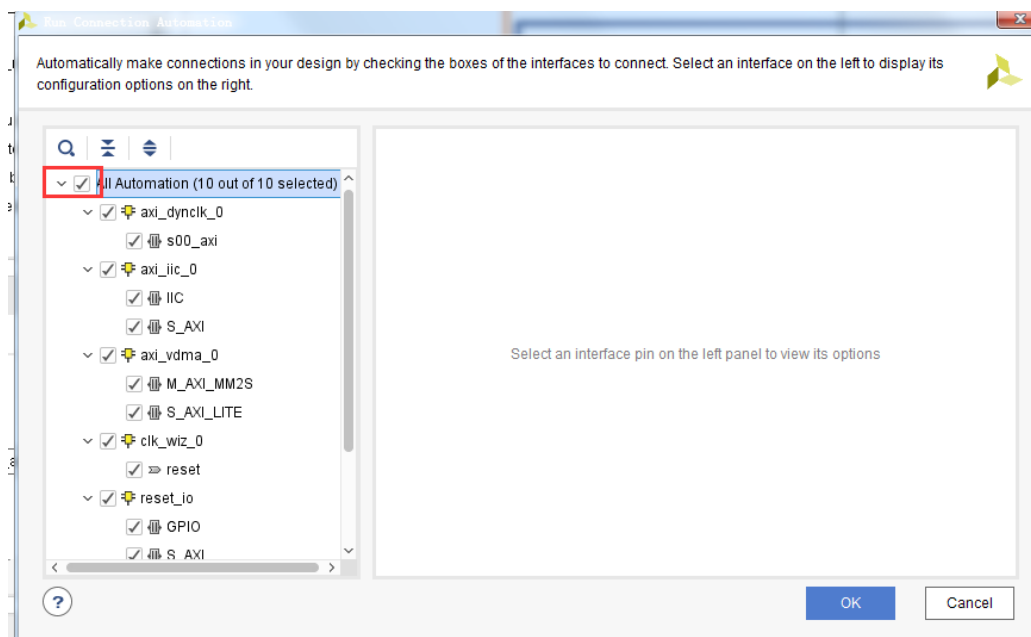
24) axi_dynclk 的输入时钟跟 clk_wiz 的输入时钟连接，频率都是 100mhz。



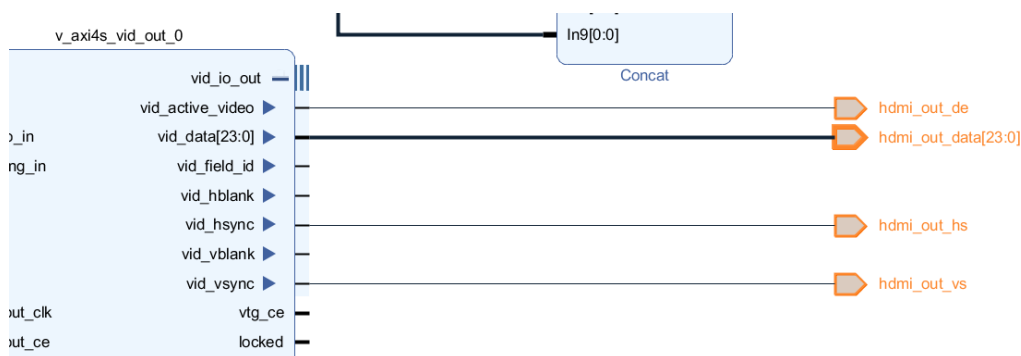
25) 运行 “Run Block Automation” 完成其它的端口连接



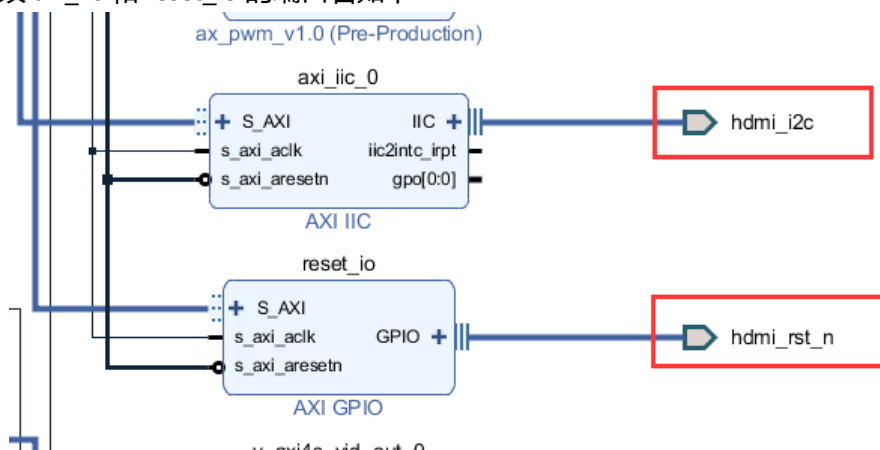
选择全部连接。



26) 设置 vid_out 的这 4 个为 “Make External” ,然后改端口名如下。



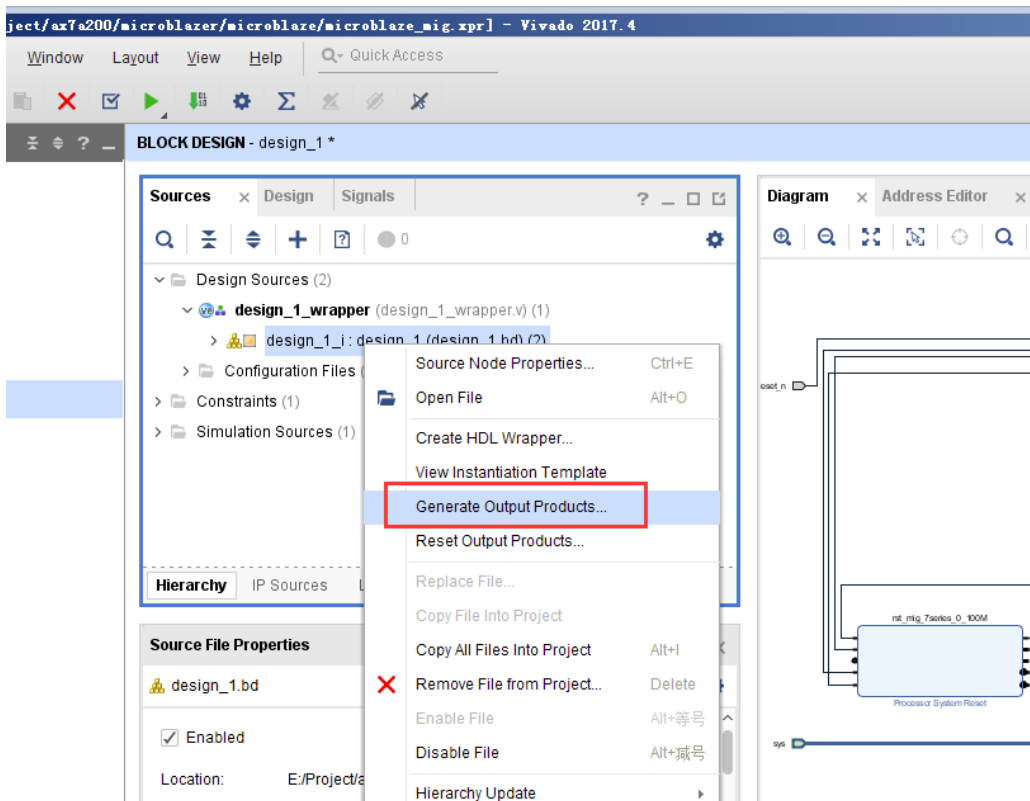
修改 axi_iic 和 reset_io 的端口名如下



9.1.9 保存并检查错误

27) 保存设计，然后按 “F6” 检查设计，没有错误的话，选择 “design_1.bd” 文件，右键选择

“Generate Output Products” ，把 IP 参数和连接信息 update 到 project 中



9.1.10 hdmi 的管脚约束 ,

28) xdc 文件添加 hdmi 输出的管脚分配 (以 ax7a200 为例)

```
set_property PACKAGE_PIN Y22 [get_ports hdmi_out_clk]
set_property PACKAGE_PIN V22 [get_ports {hdmi_out_data[0]}]
set_property PACKAGE_PIN Y18 [get_ports {hdmi_out_data[1]}]
set_property PACKAGE_PIN Y19 [get_ports {hdmi_out_data[2]}]
set_property PACKAGE_PIN W19 [get_ports {hdmi_out_data[3]}]
set_property PACKAGE_PIN W20 [get_ports {hdmi_out_data[4]}]
set_property PACKAGE_PIN Y21 [get_ports {hdmi_out_data[5]}]
set_property PACKAGE_PIN U21 [get_ports {hdmi_out_data[6]}]
set_property PACKAGE_PIN T21 [get_ports {hdmi_out_data[7]}]
set_property PACKAGE_PIN W21 [get_ports {hdmi_out_data[8]}]
set_property PACKAGE_PIN W22 [get_ports {hdmi_out_data[9]}]
set_property PACKAGE_PIN T20 [get_ports {hdmi_out_data[10]}]
set_property PACKAGE_PIN AB18 [get_ports {hdmi_out_data[11]}]
set_property PACKAGE_PIN AA18 [get_ports {hdmi_out_data[12]}]
set_property PACKAGE_PIN AA19 [get_ports {hdmi_out_data[13]}]
set_property PACKAGE_PIN AB20 [get_ports {hdmi_out_data[14]}]
set_property PACKAGE_PIN AA20 [get_ports {hdmi_out_data[15]}]
set_property PACKAGE_PIN AA21 [get_ports {hdmi_out_data[16]}]
set_property PACKAGE_PIN AB22 [get_ports {hdmi_out_data[17]}]
set_property PACKAGE_PIN AB21 [get_ports {hdmi_out_data[18]}]
set_property PACKAGE_PIN W17 [get_ports {hdmi_out_data[19]}]
set_property PACKAGE_PIN V17 [get_ports {hdmi_out_data[20]}]
set_property PACKAGE_PIN V20 [get_ports {hdmi_out_data[21]}]
```

```
set_property PACKAGE_PIN U20 [get_ports {hdmi_out_data[22]]}
set_property PACKAGE_PIN V19 [get_ports {hdmi_out_data[23]]}
set_property PACKAGE_PIN U22 [get_ports {hdmi_out_de}]
set_property PACKAGE_PIN T18 [get_ports {hdmi_out_hs}]
set_property PACKAGE_PIN R18 [get_ports {hdmi_out_vs}]

set_property PACKAGE_PIN H13 [get_ports {hdmi_i2c_scl_io}]
set_property PACKAGE_PIN G13 [get_ports {hdmi_i2c_sda_io}]
set_property PULLUP true [get_ports {hdmi_i2c_scl_io}]
set_property PULLUP true [get_ports {hdmi_i2c_sda_io}]
set_property PACKAGE_PIN L18 [get_ports {hdmi_rst_n_tri_o[0]]}

set_property IOSTANDARD LVCMOS33 [get_ports {hdmi_out_clk}]
set_property IOSTANDARD LVCMOS33 [get_ports {hdmi_out_data[*]]}
set_property IOSTANDARD LVCMOS33 [get_ports {hdmi_out_de}]
set_property IOSTANDARD LVCMOS33 [get_ports {hdmi_out_hs}]
set_property IOSTANDARD LVCMOS33 [get_ports {hdmi_out_vs}]
set_property IOSTANDARD LVCMOS33 [get_ports {hdmi_i2c_scl_io}]
set_property IOSTANDARD LVCMOS33 [get_ports {hdmi_i2c_sda_io}]
set_property IOSTANDARD LVCMOS33 [get_ports {hdmi_rst_n_tri_o[0]]}

set_property SLEW FAST [get_ports {hdmi_out_data[*]]}
set_property SLEW FAST [get_ports {hdmi_out_de}]
set_property SLEW FAST [get_ports {hdmi_out_hs}]
set_property SLEW FAST [get_ports {hdmi_out_vs}]

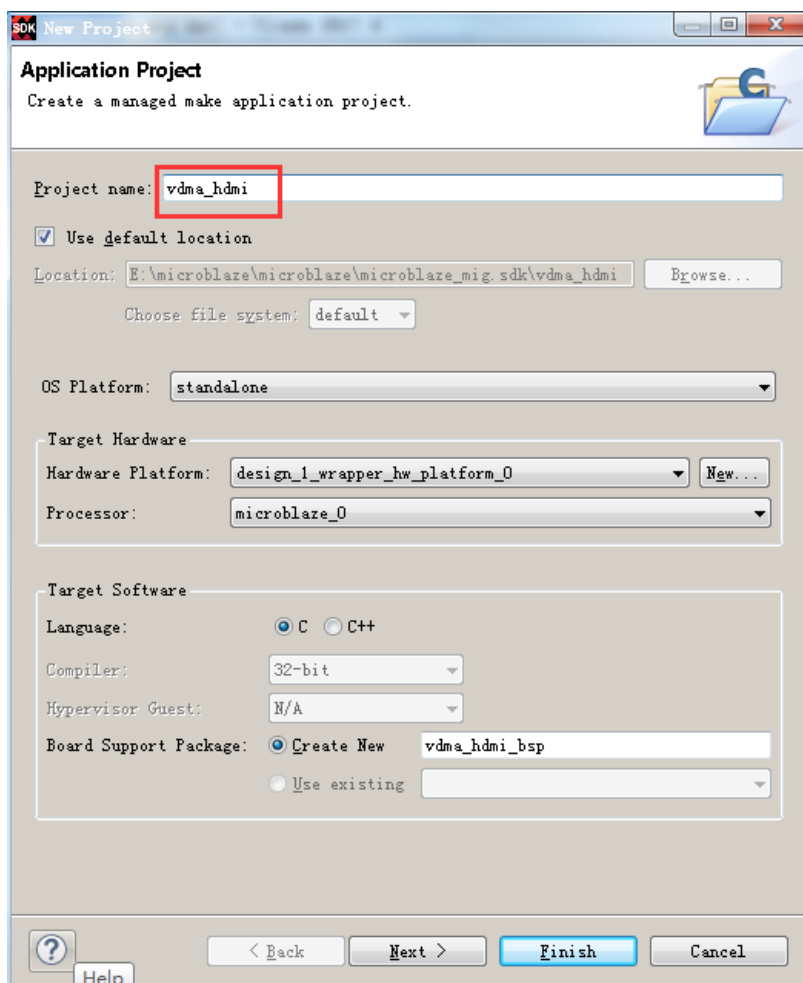
#set_property DRIVE 24 [get_ports {hdmi_out_data[*]]}
#set_property DRIVE 24 [get_ports {hdmi_out_de}]
#set_property DRIVE 24 [get_ports {hdmi_out_hs}]
#set_property DRIVE 24 [get_ports {hdmi_out_vs}]
#set_property DRIVE 24 [get_ports {hdmi_out_clk}]
#set_property SLEW FAST [get_ports {hdmi_out_clk}]
```

29) 编译生成 bit 文件，然后导出硬件信息，启动 SDK

9.2 SDK 软件编写调试

9.2.1 SDK 新建工程

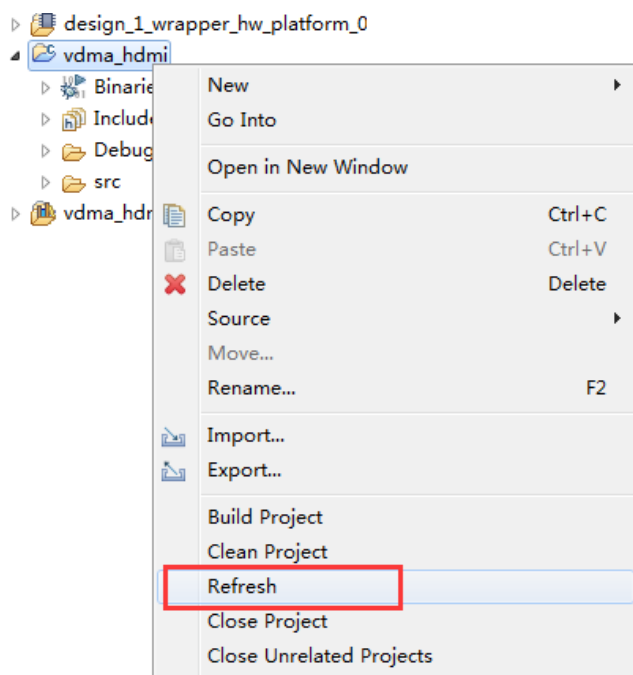
30) 新建 APP，工程名取名为"vdma_hdmi"，模板选择 "Hello World"，



- 31) 由于程序文件较多，不再具体介绍，直接复制例程的源代码。删除 src 目录下的文件，使用例程的 src 目录文件代替

croblaze ► microblaze ► microblaze_mig.sdk ► vdma_test ► src ►				
夹				
名称	修改日期	类型	大小	
display_ctrl	2020/9/26 10:50	文件夹		
dyncclk	2020/9/26 10:50	文件夹		
i2c	2020/9/26 10:50	文件夹		
display_demo.h	2020/9/26 9:28	C/C++ Header File	3 KB	
lscript.ld	2020/9/26 9:21	LD 文件	5 KB	
main.c	2020/9/26 10:09	C Source File	7 KB	
pic_800_600.h	2020/9/26 9:28	C/C++ Header File	7,208 KB	

- 32) 在 SDK 下刷新



9.2.2 程序说明

我们先来看一下 main.c 程序，在程序的前面，我们定义了一个 DisplayCtrl 结构体的变量 dispCtrl, DisplayCtrl 结构体是在 display_ctrl.h 文件里定义，DisplayCtrl 结构体包含 dyncclk IP 的地址，vdma 结构体，vtc 结构体，video mode，显示图像 buffer 指针等等这些信息。

首先 main 函数里初始化 GPIO，产生复位信号，让 HDMI 芯片复位一下。

```
XGpio_Initialize(&hdmi_rstn, XPAR_RESET_IO_DEVICE_ID); //initialize GPIO IP
XGpio_SetDataDirection(&hdmi_rstn, 1, 0x0);           //set GPIO as output
XGpio_DiscreteWrite(&hdmi_rstn, 1, 0x0);              //set GPIO output value to 0
usleep(50000);
XGpio_DiscreteWrite(&hdmi_rstn, 1, 0x1);
usleep(500);
/*
```

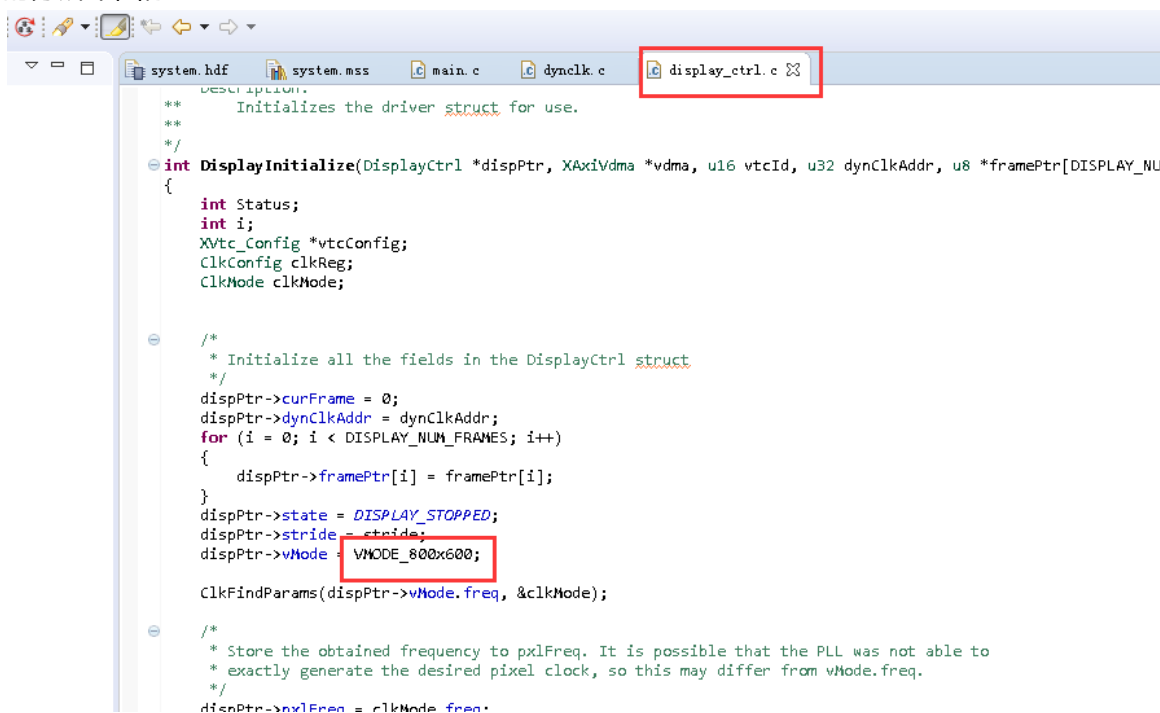
再调用 I2C 来配置 HDMI 芯片的寄存器。

```
usleep(500);
/*
 * Initialize hdmi
 */
i2c_reg8_write(XPAR_AXI_IIC_0_BASEADDR, 0x72>>1, 0x08, 0x35);
i2c_reg8_write(XPAR_AXI_IIC_0_BASEADDR, 0x7a>>1, 0x2f, 0x00);
/*
 */
```

然后初始化 vdma 设备，然后用 DisplayInitialize 函数初始化显示设备 dispCtrl 结构体，再调用 DemoPrintTest 函数显示 Video 的图像。DemoPrintTest 函数的最后的参数 pattern 用来指示显示什么图像。默认 DEMO_PATTERN_0 是显示 pic_800_600.h 文件里的图片。如果参数 pattern 为 DEMO_PATTERN_1 则显示方格，参数 pattern 为 DEMO_PATTERN_2 则显示水平灰度渐变色，参数 pattern 为 DEMO_PATTERN_3 则显示 8 种颜色的彩条，用户可以自己修改。

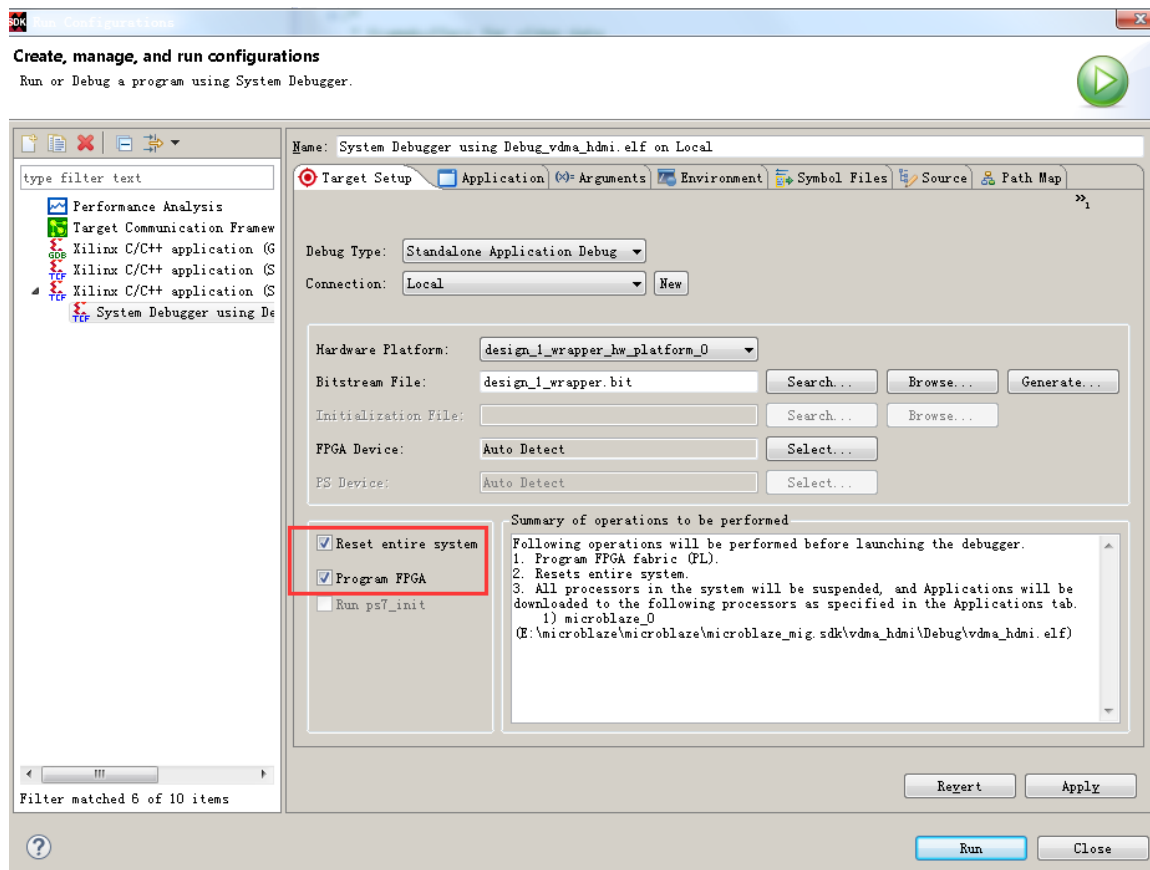
另外在 display_ctrl.c 文件里定义了输出视频的分辨率，用户可以修改这个参数来改变输出

不同的分辨率图像。

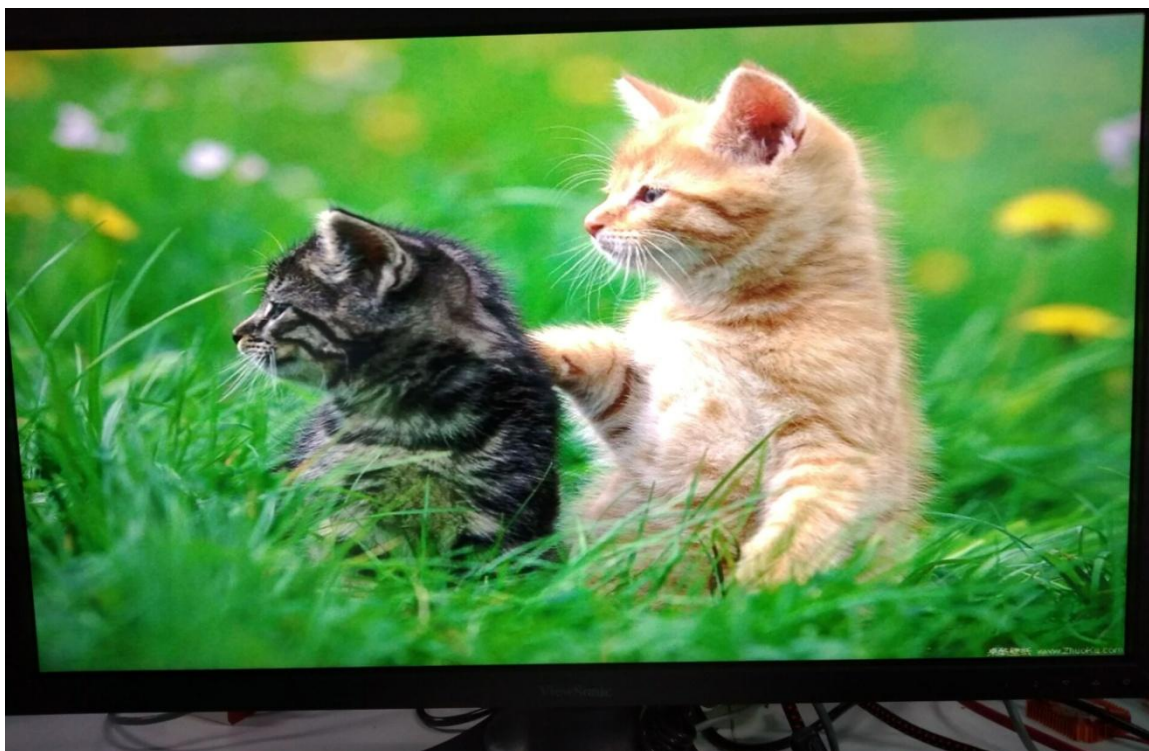


9.2.3 调试

33) 连接 HDMI 输出端口到显示器，编译运行



34) HDMI 显示器显示出一幅图片



9.3 实验总结

本实验使用的分辨率输出是 800*600 的图像，用户可以自己用工具（比如 Image2Lcd）生成其它分辨率的图像来显示。另外如果用户的开发板是 ax7035 或者 ax7050 的话，板上是没有 HDMI 芯片（Si9134）的，这样在 vivado 工程里需要添加一个 RGB 转 DVI 的 IP（rgb2dvi_v1_3），把 HDMI 配置的 I2C 和 GPIO 删除就可以了。

